UNIVERSITETI METROPOLITAN TIRANA

FACULTY OF COMPUTER SCIENCES AND IT

Software Engineering Program

Software Analysis and Design

AgriSoft

System Requirements Specification

Group

members:

Bajram Haxhiu

Megiljana Hidri

Ergi Adhami

Accepted by:

Dr. Igli Hakrama

Tirana, 20 May 2026

Table of Contents

1 Executive Summary

1.1 Project Overview;

1.2 Purpose of the Project;

1.3 Scope of this Specification

2 Service Description

2.1 Service Context;

2.2 User Characteristics;

2.3 Assumptions;

2.4 Constraints;

2.5 Dependencies

3 Requirements

3.1 Functional Requirements;

3.2 Functional Requirement Explanation;

3.3 Non-Functional Requirements;

3.4 Domain Requirements

4 Software Design

4.1 User Stories;

4.2 User Scenarios;

4.3 Stakeholders Identification Table;

4.4 Onion Diagram; 4.5 BPMN; 4.6 Use Cases;

4.7 Use Case Diagram;

4.8 Activity Diagrams;

4.9 State Diagrams;

4.10 Sequence Diagrams;

4.11 Collaboration Diagram;

4.12 DFD;

4.13 ERD;

4.14 Relational Schema;

4.15 Class Diagrams;

4.16 Object Diagrams;

4.17 Package Diagram;

4.18 Composite Structure Diagram;

4.19 Deployment Diagram

5 Design Patterns

5.1 Overview;

5.2 MVC;

5.3 Repository;

5.4 Facade;

5.5 State;

5.6 Observer

6 High Fidelity Prototype

6.1 Prototype Overview;

6.2 Homepage;

6.3 Restaurant;

6.4 Rooms;

6.5 Reservation;

6.6 Dashboard;

6.7 Reports;

6.8 Contact;
6.9 UI/UX Evaluation

7 Testing

7.1 Testing Strategy;

7.2 Functional Testing;

7.3 UI Testing;

7.4 Test Cases;

7.5 Summary

8 Project Management

8.1 Team Roles;

8.2 Methodology;

8.3 Timeline;

8.4 GitHub Workflow;

8.5 Challenges and Solutions

9 Future Improvements

9.1 Backend Integration;

9.2 Database Integration;

9.3 Authentication;

9.4 Notifications;

9.5 Cloud Deployment

10 Conclusion

10.1 Project Summary;

10.2 10.2 Achieved Objectives;

10.3 10.3 Final Reflection

11 References

12 Appendices

12.1 Additional Diagrams;

12.2 12.2 Screenshots;

12.3 12.3 Source Code Notes

1 Executive Summary

1.1 Project Overview

AgriSoft is a Software Analysis and Design project created to support the management needs of a modern agritourism business inspired by Mrizi i Zanave. The system is designed around the operational reality of an agritourism enterprise where restaurant services, room reservations, product sales, inventory control, and management reporting must work together as a coordinated digital environment.

The project focuses on organizing daily business workflows into structured software modules. These modules allow staff and managers to handle reservations, customer orders, traditional product sales, stock updates, and business reports. In its current implementation, AgriSoft is represented as a modern web application with a React and TanStack based frontend, reusable UI components, dashboard pages, mock data, and design assets. Backend and full database persistence are documented as proposed development areas for future versions.

1.2 Purpose of the Project

- Provide a centralized system concept for agritourism management.
- Reduce manual work in reservations, order handling, product sales, and inventory tracking.
- Support staff with clear screens and organized workflows.
- Give management access to performance data through dashboards and reports.
- Demonstrate Software Analysis and Design concepts through requirements, scenarios, use cases, UML diagrams, database design, and design patterns.

1.3 Scope of this Specification

This specification describes the AgriSoft system from the perspective of analysis and design. It includes functional and non-functional requirements, stakeholders, user stories, user scenarios, detailed use cases, UML diagrams, data modeling, UI/UX prototype documentation, testing, and project management elements. It is prepared as an academic final documentation package for the Software Analysis and Design course.

The scope covers the current frontend project and the full proposed system architecture. Where the current code does not implement a backend, permanent database, payment gateway, or production authentication, these elements are clearly described as proposed future improvements and are not presented as already completed implementation.

2 Service Description

2.1 Service Context

AgriSoft is positioned in the context of a hospitality and agriculture business where restaurant operations, guesthouse bookings, local product sales, farm inventory, and management reporting must be coordinated. The system supports the digital transformation of daily operations by creating a single software environment for staff and managers.

2.2 User Characteristics

The expected users include reception staff, waiters, inventory managers, business managers, administrators, and customers. Staff users are expected to have basic digital skills and to use the system during daily operations. Managers need dashboards and reports for decision-making. Customers interact mainly with public pages, reservations, room information, restaurant information, and product displays.

2.3 Assumptions

The project assumes internet access, modern browsers, trained internal staff, and a future backend/database environment. The current frontend prototype assumes that sample data can represent the main workflows until database integration is completed.

2.4 Constraints

The current system is constrained by frontend-only implementation, mock data usage, and absence of production authentication. Some diagrams and models therefore describe the intended complete system rather than the current deployable backend. Time, academic scope, and available project resources also influence the depth of implementation.

2.5 Dependencies

AgriSoft depends on modern web technologies including React, TypeScript, TanStack Router/Start, Vite, TailwindCSS, shadcn style UI components, and supporting libraries such as React Hook Form, Zod, Recharts, and Radix UI components. Proposed future deployment may depend on a database server, API server, cloud hosting, and secure authentication service.

Table 1: User Characteristics Summary

User Type	Main Goal	Main Interaction
Customer	Discover agritourism services	Browse restaurant, rooms, shop, contact, and reservation pages
Reception Staff	Manage reservations	Create, modify, search, and confirm reservations
Waiter	Manage orders	Record orders, associate orders with tables, and send orders to kitchen
Inventory Manager	Control stock	Update products, monitor thresholds, and receive low-stock alerts
Business Manager	Analyze performance	View reports, dashboards, sales, reservations, and inventory status
Developer	Maintain system	Manage frontend components, routing, mock data, and future backend integration

3 Requirements

3.1 Functional Requirements

Functional requirements define the services that AgriSoft must provide to its users. The requirements below are grouped around the project modules found in the documentation and diagrams: Reservation Management, Restaurant Management, Product Sales Management, Inventory Management, Reporting and Analytics, Authentication, and User Interface navigation.

Table 2: Functional Requirements Imported from AgriSoft Project

ID	Module	Requirement Description	Priority	Stakeholders
FR-01	Reservation Management	The system shall allow staff to create restaurant reservations.	High	Staff, Manager
FR-02	Reservation Management	The system shall allow staff to create guesthouse reservations.	High	Staff, Manager
FR-03	Reservation Management	The system shall display real-time availability of tables and rooms.	High	Staff
FR-04	Reservation Management	The system shall prevent double booking of tables or rooms.	High	Staff, Manager
FR-05	Reservation Management	The system shall allow modification and cancellation of reservations.	Medium	Staff
FR-06	Reservation Management	The system shall store reservation history.	Medium	Manager
FR-07	Reservation Management	The system shall allow searching reservations.	Medium	Staff
FR-08	Reservation Management	The system shall allow filtering reservations.	Medium	Staff
FR-09	Restaurant Management	The system shall allow management of menu items.	High	Manager
FR-10	Restaurant Management	The system shall allow staff to record customer orders.	High	Staff
FR-11	Restaurant Management	The system shall link orders to tables or reservations.	High	Staff
FR-12	Restaurant Management	The system shall calculate bills automatically.	High	Staff
FR-13	Restaurant Management	The system shall store completed orders.	Medium	Manager
FR-14	Restaurant Management	The system shall allow bill splitting.	Medium	Staff
FR-15	Restaurant Management	The system shall allow printing receipts.	Medium	Staff
FR-16	Restaurant Management	The system shall track most popular dishes.	Low	Manager
FR-17	Product Sales Management	The system shall register traditional products for sale.	High	Staff
FR-18	Product Sales Management	The system shall update product price and quantity.	High	Staff
FR-19	Product Sales Management	The system shall record product sales transactions.	High	Staff, Manager
FR-20	Product Sales Management	The system shall maintain product purchase history.	Medium	Manager
FR-21	Product Sales Management	The system shall allow applying discounts.	Medium	Staff
FR-22	Product Sales Management	The system shall generate invoices.	Medium	Staff
FR-23	Inventory Management	The system shall track all ingredients and product stock.	High	Inventory Manager
FR-24	Inventory Management	The system shall update inventory automatically after sales.	High	Inventory Manager
FR-25	Inventory Management	The system shall allow adding new stock.	High	Inventory Manager
FR-26	Inventory Management	The system shall notify low stock levels.	High	Inventory Manager

FR-27	Inventory Management	The system shall display current stock levels.	High	Inventory Manager
FR-28	Inventory Management	The system shall track supplier information.	Medium	Inventory Manager
FR-29	Inventory Management	The system shall record damaged goods.	Low	Inventory Manager
FR-30	Inventory Management	The system shall track product expiration dates.	High	Inventory Manager
FR-31	Reporting & Analytics	The system shall generate restaurant sales reports.	Medium	Manager
FR-32	Reporting & Analytics	The system shall generate product sales reports.	Medium	Manager
FR-33	Reporting & Analytics	The system shall generate reservation statistics.	Medium	Manager
FR-34	Reporting & Analytics	The system shall generate revenue reports by service type.	High	Manager
FR-35	Reporting & Analytics	The system shall display historical performance data.	Medium	Manager
FR-36	Reporting & Analytics	The system shall allow exporting reports.	Medium	Manager
FR-37	Reporting & Analytics	The system shall provide a dashboard view.	High	Manager
FR-38	User Management	The system shall allow user login.	High	All Users
FR-39	User Management	The system shall allow user logout.	High	All Users
FR-40	User Management	The system shall allow administrators to create user accounts.	High	Admin
FR-41	User Management	The system shall allow role assignment to users.	High	Admin
FR-42	User Management	The system shall allow updating user information.	Medium	Admin
FR-43	System	The system shall perform data backups.	High	Admin
FR-44	System	The system shall restore data from backups.	High	Admin
FR-45	System	The system shall maintain audit logs.	Medium	Admin
FR-46	System	The system shall send notifications.	Medium	All Users
FR-47	System	The system shall send email confirmations.	Medium	Customers
FR-48	System	The system shall support multiple languages.	Low	All Users
FR-49	System	The system shall support mobile access.	Medium	All Users
FR-50	System	The system shall support API integration.	Low	Developers

3.2 Functional Requirement Explanation

Reservation Management

The Reservation Management requirements describe how AgriSoft supports a specific business workflow. In the analysis model, every requirement is connected to one or more user roles, one or more use cases, and at least one diagram category. This traceability helps the development team understand how business needs are translated into software behavior and user interface actions.

- The requirement group contributes directly to the operational efficiency of the AgriSoft system.
- Validation, clear user feedback, and structured data storage are expected for reliable operation.
- In the current prototype, some behavior may be represented through frontend screens, mock data, or design diagrams; production persistence is a future improvement.

Restaurant Management

The Restaurant Management requirements describe how AgriSoft supports a specific business workflow. In the analysis model, every requirement is connected to one or more user roles, one or more use cases, and at least one diagram category. This traceability helps the development team understand how business needs are translated into software behavior and user interface actions.

- The requirement group contributes directly to the operational efficiency of the AgriSoft system.
- Validation, clear user feedback, and structured data storage are expected for reliable operation.
- In the current prototype, some behavior may be represented through frontend screens, mock data, or design diagrams; production persistence is a future improvement.

Product Sales Management

The Product Sales Management requirements describe how AgriSoft supports a specific business workflow. In the analysis model, every requirement is connected to one or more user roles, one or more use cases, and at least one diagram category. This traceability helps the development team understand how business needs are translated into software behavior and user interface actions.

- The requirement group contributes directly to the operational efficiency of the AgriSoft system.
- Validation, clear user feedback, and structured data storage are expected for reliable operation.
- In the current prototype, some behavior may be represented through frontend screens, mock data, or design diagrams; production persistence is a future improvement.

Inventory Management

The Inventory Management requirements describe how AgriSoft supports a specific business workflow. In the analysis model, every requirement is connected to one or more user roles, one or more use cases, and at least one diagram category. This traceability helps the development team understand how business needs are translated into software behavior and user interface actions.

- The requirement group contributes directly to the operational efficiency of the AgriSoft system.
- Validation, clear user feedback, and structured data storage are expected for reliable operation.
- In the current prototype, some behavior may be represented through frontend screens, mock data, or design diagrams; production persistence is a future improvement.

Reporting and Analytics

The Reporting and Analytics requirements describe how AgriSoft supports a specific business workflow. In the analysis model, every requirement is connected to one or more user roles, one or more use cases, and at least one diagram category. This traceability helps the development team understand how business needs are translated into

software behavior and user interface actions.

- The requirement group contributes directly to the operational efficiency of the AgriSoft system.
- Validation, clear user feedback, and structured data storage are expected for reliable operation.
- In the current prototype, some behavior may be represented through frontend screens, mock data, or design diagrams; production persistence is a future improvement.

Authentication and Account Access

The Authentication and Account Access requirements describe how AgriSoft supports a specific business workflow. In the analysis model, every requirement is connected to one or more user roles, one or more use cases, and at least one

diagram category. This traceability helps the development team understand how business needs are translated into software behavior and user interface actions.

- The requirement group contributes directly to the operational efficiency of the AgriSoft system.
- Validation, clear user feedback, and structured data storage are expected for reliable operation.
- In the current prototype, some behavior may be represented through frontend screens, mock data, or design diagrams; production persistence is a future improvement.

UI Navigation and Information Pages

The UI Navigation and Information Pages requirements describe how AgriSoft supports a specific business workflow. In the analysis model, every requirement is connected to one or more user roles, one or more use cases, and at least one diagram category. This traceability helps the development team understand how business needs are translated into software behavior and user interface actions.

- The requirement group contributes directly to the operational efficiency of the AgriSoft system.
- Validation, clear user feedback, and structured data storage are expected for reliable operation.
- In the current prototype, some behavior may be represented through frontend screens, mock data, or design diagrams; production persistence is a future improvement.

3.3 Non-Functional Requirements

Non-functional requirements specify quality attributes of the system. These requirements determine how well AgriSoft performs, how secure it is, how usable it feels to staff and customers, and how maintainable it remains as the project grows.

Table 3: Non-Functional Requirements Imported from AgriSoft Project

ID	Main Category	Subcategory	Requirement Description	Priority	Stakeholders
NFR-01	Product Requirements	Efficiency (Performance)	The system shall support multiple users simultaneously.	High	All Users
NFR-02	Product Requirements	Efficiency (Performance)	The system shall process transactions within 2 seconds.	High	Staff
NFR-03	Product Requirements	Efficiency (Performance)	The system shall handle at least 100 concurrent users.	Medium	Manager
NFR-04	Product Requirements	Efficiency (Performance)	The system shall load pages within 3 seconds.	Medium	All Users
NFR-05	Product Requirements	Efficiency (Performance)	The system shall handle peak-hour traffic efficiently.	High	All Users
NFR-06	Product Requirements	Efficiency (Performance)	The system shall minimize downtime during updates.	Medium	Manager
NFR-07	Product Requirements	Efficiency (Performance)	The system shall optimize database queries for efficiency.	High	Developer
NFR-08	Product Requirements	Dependability	The system shall ensure 99% uptime during business hours.	High	All Users
NFR-09	Product Requirements	Dependability	The system shall recover data after failures.	High	Manager
NFR-10	Product Requirements	Dependability	The system shall prevent data loss.	High	Manager
NFR-11	Product Requirements	Dependability	The system shall perform automatic backups daily.	High	Manager
NFR-12	Product Requirements	Dependability	The system shall log system errors.	Medium	Developer
NFR-13	Product Requirements	Dependability	The system shall ensure fault tolerance.	Medium	Manager
NFR-14	Product Requirements	Security	The system shall require authentication.	High	All Users
NFR-15	Product Requirements	Security	The system shall enforce role-based access control.	High	Manager
NFR-16	Product Requirements	Security	The system shall protect sensitive data.	High	Manager

NFR-17	Product Requirements	Security	The system shall log	Medium	Manager

			user activities.		
NFR-18	Product Requirements	Security	The system shall encrypt sensitive data.	High	Manager
NFR-19	Product Requirements	Security	The system shall enforce secure password policies.	High	All Users
NFR-20	Product Requirements	Security	The system shall prevent unauthorized access attempts.	High	All Users
NFR-21	Product Requirements	Security	The system shall support session timeout.	Medium	All Users
NFR-22	Product Requirements	Security	The system shall use HTTPS for communication.	High	All Users
NFR-23	Product Requirements	Usability	The system shall provide an intuitive interface.	High	Staff
NFR-24	Product Requirements	Usability	The system shall minimize steps to complete tasks.	High	Staff
NFR-25	Product Requirements	Usability	The system shall provide feedback messages.	Medium	Staff
NFR-26	Product Requirements	Usability	The system shall support multiple languages.	Medium	All Users
NFR-27	Product Requirements	Usability	The system shall be accessible to non-technical users.	High	Staff
NFR-28	Product Requirements	Usability	The system shall include help/documentation for users.	Medium	Staff
NFR-29	Product Requirements	Usability	The system shall provide error guidance messages.	Medium	Staff
NFR-30	Organizational Requirements	Operational	The system shall support business growth.	Medium	Manager
NFR-31	Organizational Requirements	Development	The system shall allow addition of new modules.	Medium	Developer
NFR-32	Organizational Requirements	Operational	The system shall support increased data volume over time.	High	Manager
NFR-33	Organizational Requirements	Operational	The system shall support additional users without redesign.	High	Manager
NFR-34	Organizational Requirements	Environmental	The system shall allow cloud deployment.	Medium	Developer
NFR-35	Organizational Requirements	Development	The system shall work on modern browsers.	High	All Users
NFR-36	Organizational Requirements	Operational	The system shall support integration with external systems.	Medium	Manager
NFR-37	Organizational Requirements	Development	The system shall allow easy debugging.	Medium	Developer
NFR-38	Organizational Requirements	Development	The system shall support modular updates.	High	Developer
NFR-39	Organizational Requirements	Development	The system shall support version control.	High	Developer
NFR-40	Product Requirements	Efficiency (Space)	The system shall store data in a structured database.	High	Developer
NFR-41	Product Requirements	Dependability	The system shall perform regular backups.	High	Manager
NFR-42	Product Requirements	Dependability	The system shall ensure data consistency.	High	All Users
NFR-43	Product Requirements	Dependability	The system shall ensure data integrity.	High	All Users
NFR-44	Product Requirements	Efficiency (Performance)	The system shall support data export.	Medium	Manager

NFR-45	Product Requirements	Dependability	The system shall support data backup recovery within minimal time.	High	Manager
NFR-46	External Requirements	Legislative	The system shall comply with data protection regulations.	High	Manager
NFR-47	External Requirements	Ethical	The system shall ensure ethical use of customer data.	Medium	All Users
NFR-48	External Requirements	Regulatory	The system shall comply with food safety reporting standards.	Medium	Manager
NFR-49	External Requirements	Regulatory	The system shall meet accounting and financial reporting standards.	High	Manager
NFR-50	External Requirements	Safety/Security	The system shall ensure secure handling of operational and safety data. (As stated in IEC 61511 Standard)	High	All Users

3.3.1 Product Requirements

Performance

Performance requirements ensure that AgriSoft is practical for real use in an agritourism business. The system should be responsive, predictable, secure, easy to learn, and simple to extend. For a business with restaurant and hospitality operations, quality attributes are as important as the visible features because delays or data loss can affect customer service.

Dependability

Dependability requirements ensure that AgriSoft is practical for real use in an agritourism business. The system should be responsive, predictable, secure, easy to learn, and simple to extend. For a business with restaurant and hospitality operations, quality attributes are as important as the visible features because delays or data loss can affect customer service.

Security

Security requirements ensure that AgriSoft is practical for real use in an agritourism business. The system should be responsive, predictable, secure, easy to learn, and simple to extend. For a business with restaurant and hospitality operations, quality attributes are as important as the visible features because delays or data loss can affect customer service.

Usability

Usability requirements ensure that AgriSoft is practical for real use in an agritourism business. The system should be responsive, predictable, secure, easy to learn, and simple to extend. For a business with restaurant and hospitality operations, quality attributes are as important as the visible features because delays or data loss can affect customer service.

Maintainability

Maintainability requirements ensure that AgriSoft is practical for real use in an agritourism business. The system should be responsive, predictable, secure, easy to learn, and simple to extend. For a business with restaurant and hospitality operations, quality attributes are as important as the visible features because delays or data loss can affect customer service.

Compatibility

Compatibility requirements ensure that AgriSoft is practical for real use in an agritourism business. The system should be responsive, predictable, secure, easy to learn, and simple to extend. For a business with restaurant and hospitality operations, quality attributes are as important as the visible features because delays or data loss can affect customer service.

3.3.2 Organizational Requirements

Organizational requirements address how the system fits the working environment of the agritourism business. Staff training, workflow alignment, role distribution, and project documentation are needed to ensure that the system can be adopted by non-technical users without disrupting daily operations.

3.3.3 External Requirements

External requirements include future integration with payment services, email notifications, external hosting, secure communication protocols, and possible compliance with privacy and data protection requirements. Since the current version is mainly a frontend prototype, these requirements are included as design objectives for the proposed full-stack system.

3.4 Domain Requirements

AgriSoft operates in the agritourism domain. Domain requirements include support for restaurant reservations, guesthouse room bookings, farm-to-table product tracking, local product sales, ingredient stock monitoring, and reporting for business decisions. The system must reflect the relationship between agriculture, hospitality, and gastronomy.

Table 4: Domain Requirements

Domain Area	Requirement	Explanation
Agritourism Reservations	Tables and rooms must not be double booked	Availability must be checked before confirmation.

Restaurant Operations	Orders must be linked to tables or reservations	This supports service coordination and billing.
Local Product Sales	Products must include price and quantity	Sales update stock and support reporting.
Inventory	Low stock must be visible to managers	Stock monitoring prevents shortages.
Reporting	Sales, reservations, and inventory data must be summarized	Management uses reports to evaluate performance.

4 Software Design

4.1 User Stories

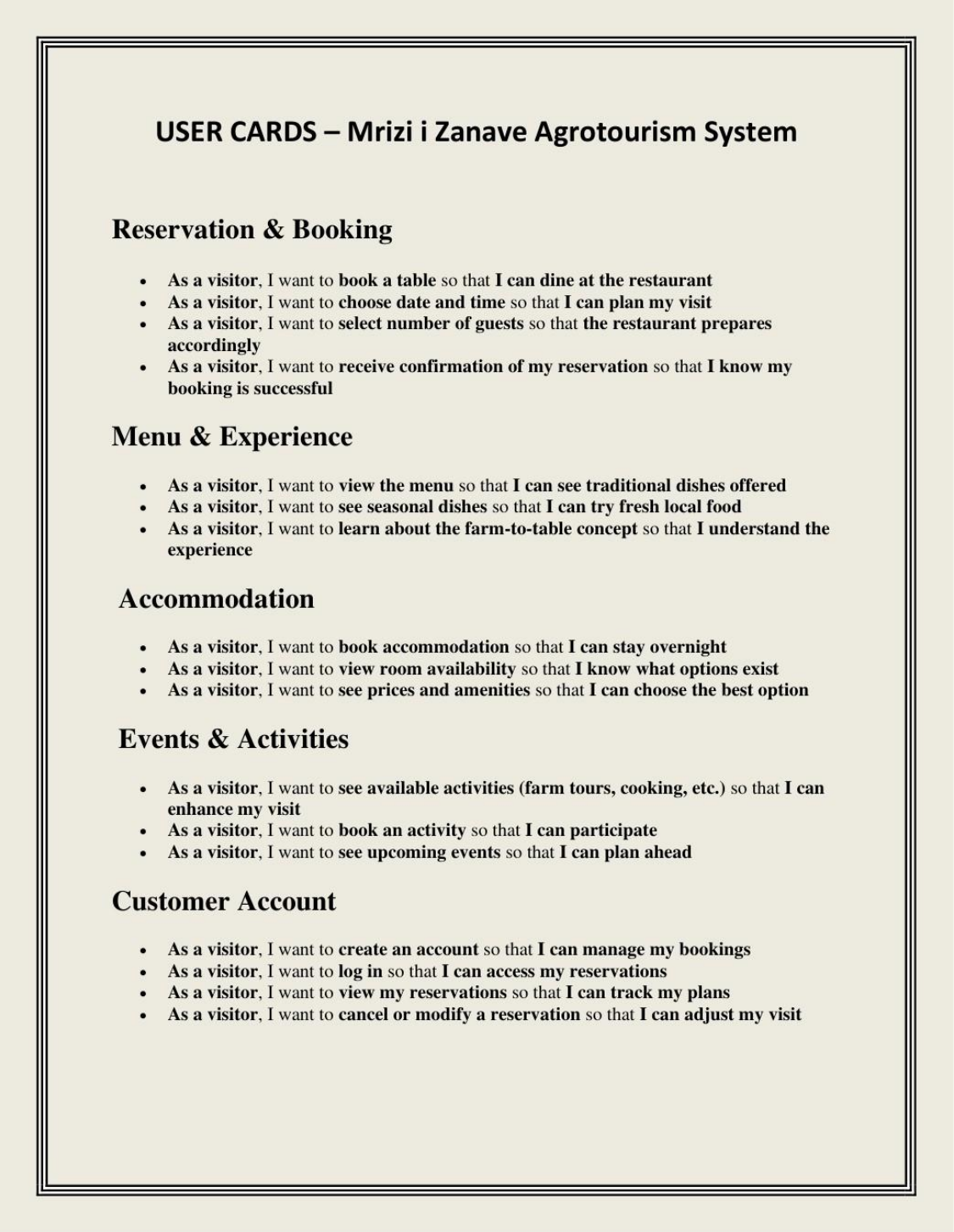
User stories express requirements from the perspective of system users. They help connect business needs with software behavior. The following stories summarize the major interactions expected from AgriSoft.

Table 5: User Stories

ID	As a	I want to	So that
US-01	Reception Staff	create a restaurant reservation	customers can reserve tables accurately
US-02	Reception Staff	create a guesthouse room booking	customers can stay overnight at the agritourism complex
US-03	Waiter	record a customer order	the kitchen and billing process are organized
US-04	Inventory Manager	update stock quantities	the business knows available ingredients and products
US-05	Inventory Manager	receive low-stock alerts	shortages can be avoided
US-06	Manager	view sales and reservation reports	business performance can be evaluated
US-07	Customer	view restaurant and room information	they can decide what service to use
US-08	Customer	contact the business	they can request additional information
US-09	Administrator	manage product and menu data	the system remains accurate
US-10	Business Owner	review dashboard analytics	strategic decisions can be based on data

The existing project also includes a user stories PDF. The rendered pages are included in the appendices and can be edited or replaced with updated diagrams if the team changes the analysis.

User Stories Mrirzi I Zanave Agrotourism Page 01



USER CARDS – Mrizi i Zanave Agrotourism System

Reservation & Booking

- **As a visitor**, I want to **book a table** so that **I can dine at the restaurant**
- **As a visitor**, I want to **choose date and time** so that **I can plan my visit**
- **As a visitor**, I want to **select number of guests** so that **the restaurant prepares accordingly**
- **As a visitor**, I want to **receive confirmation of my reservation** so that **I know my booking is successful**

Menu & Experience

- **As a visitor**, I want to **view the menu** so that **I can see traditional dishes offered**
- **As a visitor**, I want to **see seasonal dishes** so that **I can try fresh local food**
- **As a visitor**, I want to **learn about the farm-to-table concept** so that **I understand the experience**

Accommodation

- **As a visitor**, I want to **book accommodation** so that **I can stay overnight**
- **As a visitor**, I want to **view room availability** so that **I know what options exist**
- **As a visitor**, I want to **see prices and amenities** so that **I can choose the best option**

Events & Activities

- **As a visitor**, I want to **see available activities (farm tours, cooking, etc.)** so that **I can enhance my visit**
- **As a visitor**, I want to **book an activity** so that **I can participate**
- **As a visitor**, I want to **see upcoming events** so that **I can plan ahead**

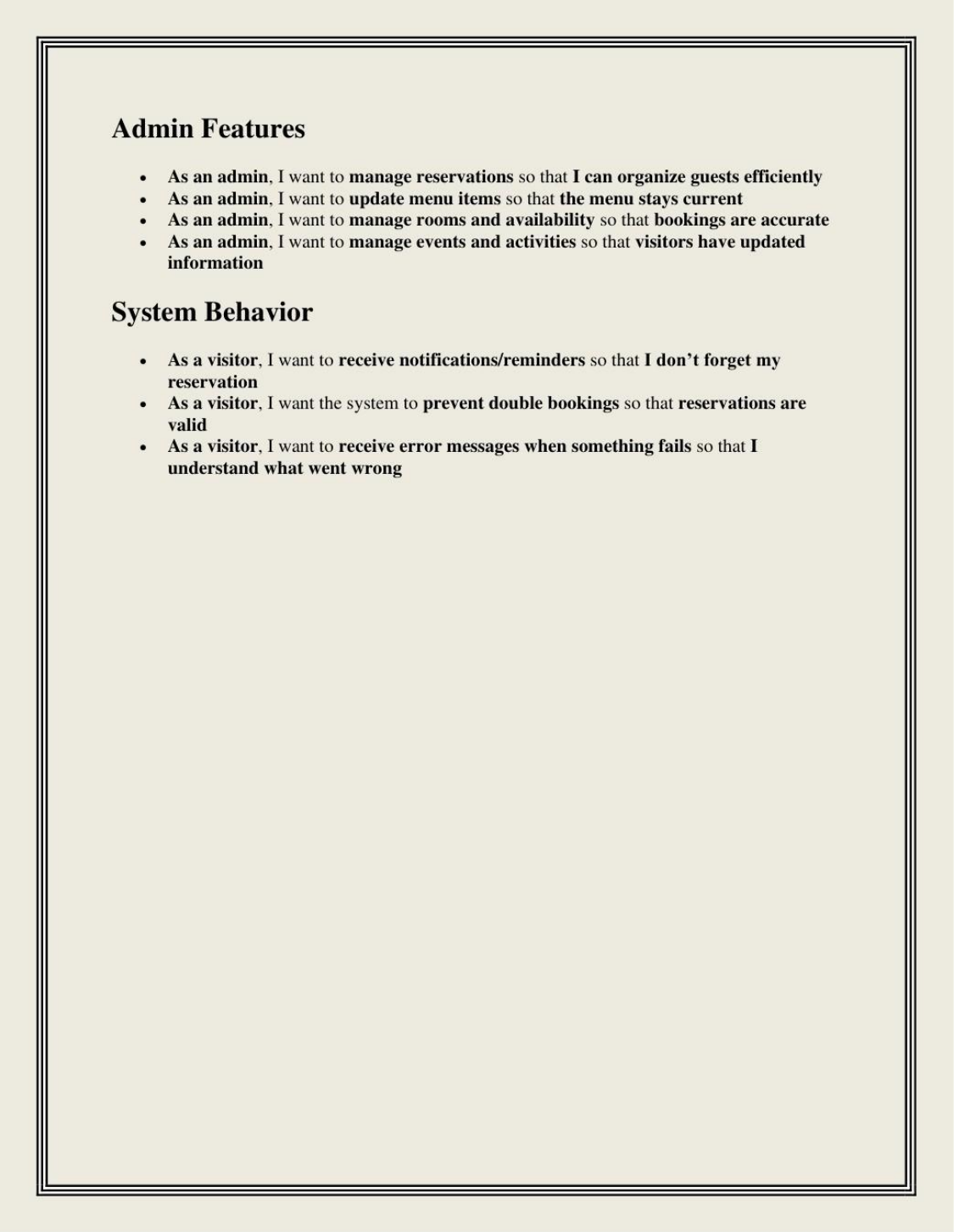
Customer Account

- **As a visitor**, I want to **create an account** so that **I can manage my bookings**
- **As a visitor**, I want to **log in** so that **I can access my reservations**
- **As a visitor**, I want to **view my reservations** so that **I can track my plans**
- **As a visitor**, I want to **cancel or modify a reservation** so that **I can adjust my visit**

Figure: User Stories Mrirzi I Zanave Agrotourism Page 01

This page is included from the original user stories material found in the project ZIP. It documents the scenario-oriented thinking behind the AgriSoft modules.

User Stories Mrirzi I Zanave Agrotourism Page 02



Admin Features

- **As an admin**, I want to **manage reservations** so that **I can organize guests efficiently**
- **As an admin**, I want to **update menu items** so that **the menu stays current**
- **As an admin**, I want to **manage rooms and availability** so that **bookings are accurate**
- **As an admin**, I want to **manage events and activities** so that **visitors have updated information**

System Behavior

- **As a visitor**, I want to **receive notifications/reminders** so that **I don't forget my reservation**
- **As a visitor**, I want the system to **prevent double bookings** so that **reservations are valid**
- **As a visitor**, I want to **receive error messages when something fails** so that **I understand what went wrong**

Figure: User Stories Mrirzi I Zanave Agrotourism Page 02

This page is included from the original user stories material found in the project ZIP. It documents the scenario-oriented thinking behind the AgriSoft modules.

4.2 User Scenarios

Scenario 1: Staff receives reservation request

Scenario: Staff receives reservation request. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

1. The responsible user opens the appropriate AgriSoft module.
2. The user performs the action: staff receives reservation request.
3. The system validates the entered information and checks related data.
4. If the operation is valid, the system stores or updates the information.
5. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 2: Staff enters reservation details

Scenario: Staff enters reservation details. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

6. The responsible user opens the appropriate AgriSoft module.
7. The user performs the action: staff enters reservation details.
8. The system validates the entered information and checks related data.
9. If the operation is valid, the system stores or updates the information.
10. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 3: System checks availability in real time

Scenario: System checks availability in real time. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

11. The responsible user opens the appropriate AgriSoft module.
12. The user performs the action: system checks availability in real time.
13. The system validates the entered information and checks related data.
14. If the operation is valid, the system stores or updates the information.
15. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 4: System confirms reservation

Scenario: System confirms reservation. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

16. The responsible user opens the appropriate AgriSoft module.
17. The user performs the action: system confirms reservation.
18. The system validates the entered information and checks related data.
19. If the operation is valid, the system stores or updates the information.
20. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 5: System suggests alternative if not available

Scenario: System suggests alternative if not available. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

21. The responsible user opens the appropriate AgriSoft module.
22. The user performs the action: system suggests alternative if not available.
23. The system validates the entered information and checks related data.
24. If the operation is valid, the system stores or updates the information.
25. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 6: Staff modifies or cancels reservation

Scenario: Staff modifies or cancels reservation. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

26. The responsible user opens the appropriate AgriSoft module.
27. The user performs the action: staff modifies or cancels reservation.
28. The system validates the entered information and checks related data.
29. If the operation is valid, the system stores or updates the information.
30. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 7: Waiter takes customer order

Scenario: Waiter takes customer order. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

31. The responsible user opens the appropriate AgriSoft module.
32. The user performs the action: waiter takes customer order.
33. The system validates the entered information and checks related data.
34. If the operation is valid, the system stores or updates the information.
35. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 8: System records order

Scenario: System records order. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

36. The responsible user opens the appropriate AgriSoft module.
37. The user performs the action: system records order.
38. The system validates the entered information and checks related data.
39. If the operation is valid, the system stores or updates the information.
40. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 9: Order is associated with table or reservation

Scenario: Order is associated with table or reservation. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

41. The responsible user opens the appropriate AgriSoft module.
42. The user performs the action: order is associated with table or reservation.
43. The system validates the entered information and checks related data.
44. If the operation is valid, the system stores or updates the information.
45. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 10: Order is sent to kitchen

Scenario: Order is sent to kitchen. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

46. The responsible user opens the appropriate AgriSoft module.

47. The user performs the action: order is sent to kitchen.
48. The system validates the entered information and checks related data.
49. If the operation is valid, the system stores or updates the information.
50. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 11: Food is prepared and served

Scenario: Food is prepared and served. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

51. The responsible user opens the appropriate AgriSoft module.
52. The user performs the action: food is prepared and served.
53. The system validates the entered information and checks related data.
54. If the operation is valid, the system stores or updates the information.
55. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 12: Stock is updated after usage or sale

Scenario: Stock is updated after usage or sale. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

56. The responsible user opens the appropriate AgriSoft module.
57. The user performs the action: stock is updated after usage or sale.
58. The system validates the entered information and checks related data.
59. If the operation is valid, the system stores or updates the information.
60. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 13: System checks stock level

Scenario: System checks stock level. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

61. The responsible user opens the appropriate AgriSoft module.
62. The user performs the action: system checks stock level.
63. The system validates the entered information and checks related data.
64. If the operation is valid, the system stores or updates the information.
65. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 14: System notifies manager if stock is low

Scenario: System notifies manager if stock is low. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

66. The responsible user opens the appropriate AgriSoft module.
67. The user performs the action: system notifies manager if stock is low.
68. The system validates the entered information and checks related data.
69. If the operation is valid, the system stores or updates the information.
70. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 15: Manager updates stock manually

Scenario: Manager updates stock manually. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

71. The responsible user opens the appropriate AgriSoft module.

72. The user performs the action: manager updates stock manually.
73. The system validates the entered information and checks related data.
74. If the operation is valid, the system stores or updates the information.
75. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 16: Staff selects traditional product

Scenario: Staff selects traditional product. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

76. The responsible user opens the appropriate AgriSoft module.
77. The user performs the action: staff selects traditional product.
78. The system validates the entered information and checks related data.
79. If the operation is valid, the system stores or updates the information.
80. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 17: System shows product information

Scenario: System shows product information. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

81. The responsible user opens the appropriate AgriSoft module.
82. The user performs the action: system shows product information.
83. The system validates the entered information and checks related data.
84. If the operation is valid, the system stores or updates the information.
85. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 18: System processes sale transaction

Scenario: System processes sale transaction. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

86. The responsible user opens the appropriate AgriSoft module.
87. The user performs the action: system processes sale transaction.
88. The system validates the entered information and checks related data.
89. If the operation is valid, the system stores or updates the information.
90. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 19: System records transaction and updates quantity

Scenario: System records transaction and updates quantity. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

91. The responsible user opens the appropriate AgriSoft module.
92. The user performs the action: system records transaction and updates quantity.
93. The system validates the entered information and checks related data.
94. If the operation is valid, the system stores or updates the information.
95. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 20: System blocks sale if product unavailable

Scenario: System blocks sale if product unavailable. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

96. The responsible user opens the appropriate AgriSoft module.

97. The user performs the action: system blocks sale if product unavailable.
98. The system validates the entered information and checks related data.
99. If the operation is valid, the system stores or updates the information.
100. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 21: System collects sales, reservation, and inventory data

Scenario: System collects sales, reservation, and inventory data. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

101. The responsible user opens the appropriate AgriSoft module.
102. The user performs the action: system collects sales, reservation, and inventory data.
103. The system validates the entered information and checks related data.
104. If the operation is valid, the system stores or updates the information.
105. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 22: System generates reports and analytics

Scenario: System generates reports and analytics. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

106. The responsible user opens the appropriate AgriSoft module.
107. The user performs the action: system generates reports and analytics.
108. The system validates the entered information and checks related data.
109. If the operation is valid, the system stores or updates the information.
110. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

Scenario 23: Management reviews system performance

Scenario: Management reviews system performance. This scenario describes one operational step in the AgriSoft workflow. It is connected to the activity diagrams and sequence diagrams included later in the document.

111. The responsible user opens the appropriate AgriSoft module.
112. The user performs the action: management reviews system performance.
113. The system validates the entered information and checks related data.
114. If the operation is valid, the system stores or updates the information.
115. The system displays a confirmation message or updated dashboard state.

- Alternative Sequence: if required information is missing, the system requests correction.
- Exception Sequence: if data is unavailable or invalid, the system prevents unsafe completion.
- Post-condition: the related business record is created, updated, reviewed, or blocked according to the workflow.

4.3 Stakeholders Identification Table

Stakeholders are individuals or groups that influence or are affected by AgriSoft. The table below identifies operational and external stakeholders, their responsibilities, and their concerns.

Table 6: Stakeholder Identification Table

Stakeholder	Role/Responsibility	Importance	Influence	Positive Impact	Concern
Business Owner	Oversees agritourism operations and approves strategic decisions	High	High	Better control and data-driven decisions	Cost and adoption
Manager	Monitors reservations, sales, rooms, inventory, and reports	High	High	Operational efficiency	Data accuracy
Reception Staff	Creates and updates reservations	High	Medium	Faster customer service	Availability errors
Waiters	Record customer orders and table information	Medium	Medium	Better restaurant workflow	Usability under pressure
Inventory Manager	Maintains stock and product availability	High	High	Reduced stock shortages	Manual update mistakes
	Browse services and				

Customers	request reservations or information	High	Low	Improved experience	Privacy and clarity

Developer Team	Maintains and extends the software	Medium	High	Maintainable product	Technical debt
Suppliers	Provide local ingredients and products	Medium	Low	Traceability and collaboration	Data sharing
University Evaluator	Reviews academic quality of analysis and design	High	Medium	Clear documentation and diagrams	Completeness of SAD deliverables

Stakeholders Identification Table Page 01

4) Stakeholders Identification Table

✓Operational Stakeholders (Internal)

Stakeholders	Role/Resopnsibility	Importance	Influence	Interests/Positive Impacts	Concerns
Business Owner	Strategic decision-making, oversees entire system	High	High	Business growth, efficiency, strong brand reputation	System cost, ROI, implementation risks
Business Managers	Operational and strategic management	High	High	Performance improvement, data-driven decisions	System complexity, adoption challenges
Restaurant Manager	Manages reservations and service flow	High	High	Smooth operations, customer satisfaction	Overbooking, system reliability
Kitchen Staff	Prepare food based on orders/reservations	Medium	Medium	Clear orders, better coordination	Miscommunication, timing issues
Restaurant Service Staff	Serve customers and manage tables	Medium	Medium	Efficient service, fewer errors	System usability, training needs

Figure: Stakeholders Identification Table Page 01

This rendered page shows the stakeholder table material included in the original ZIP package.

Stakeholders Identification Table Page 02

Hotel Manager	Manages accommodation bookings	High	High	Higher occupancy, better planning	Integration with booking system
Hospitality Staff	Handle guest services and rooms	Medium	Medium	Organized workflow, improved guest experience	Workload, system learning curve
Farm Manager	Oversees agricultural production	Medium	Medium	Better planning and resource management	Data accuracy, coordination issues
Farm Workers	Execute farming and livestock tasks	Low	Low	Clear schedules and instructions	Resistance to new systems
Food Processing Staff	Produce and manage food products	Medium	Medium	Improved production tracking, quality control	Process disruptions
Administrative and Finance Staff	Manage finances, reports, records	High	High	Accurate reporting, automation	Data security, errors
Inventory / Supply Manager	Tracks stock and supplies	High	High	Reduced waste, better stock control	Inventory mismatches

Figure: Stakeholders Identification Table Page 02

This rendered page shows the stakeholder table material included in the original ZIP package.

Stakeholders Identification Table Page 03

✓External Stakeholders

Stakeholders	Role/Resopnsibility	Importance	Influence	Interests/Positive Impacts	Concerns
Local Farmers and Product Suppliers	Provide raw materials/products	Medium	Medium	Stable demand, long-term partnerships	Payment delays, demand fluctuation
Logistics and Delivery Partners	Handle transportation of goods	Medium	Medium	Efficient coordination	Scheduling conflicts
Customers (restaurant visitors)	Use system for reservations/services	High	High	Easy booking, better experience	Privacy, usability issues
Tourists (guesthouse)	Book accommodation and experiences	Medium	Medium	Convenience, integrated services	Booking errors
Agritourism Visitors	Participate in experiences	Medium	Medium	Unique experience, accessibility	Poor organization
Government and Regulatory Authorities	Ensure legal compliance	High	High	Compliance, safety standards	Non-compliance risks
Local Community / Families	Collaborate and are locally impacted	Low	Low	Economic growth, local engagement	Environmental/social impact

Figure: Stakeholders Identification Table Page 03

This rendered page shows the stakeholder table material included in the original ZIP package.

4.4 Onion Diagram

The onion diagram places AgriSoft at the center and organizes stakeholders in layers according to proximity and influence. The innermost layers contain users that directly operate the system, while the outer layers contain external participants such as suppliers, customers, and regulatory or academic evaluators.

Onion Diagram

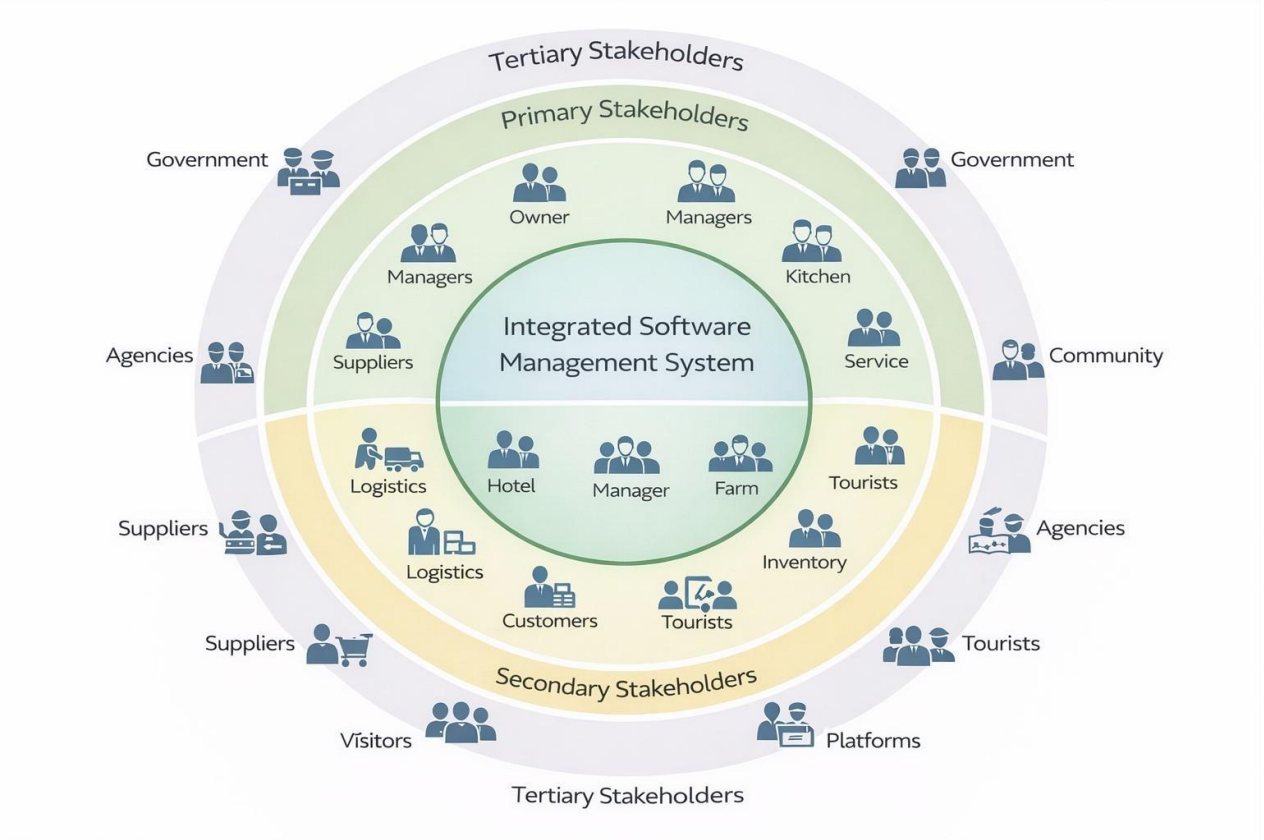
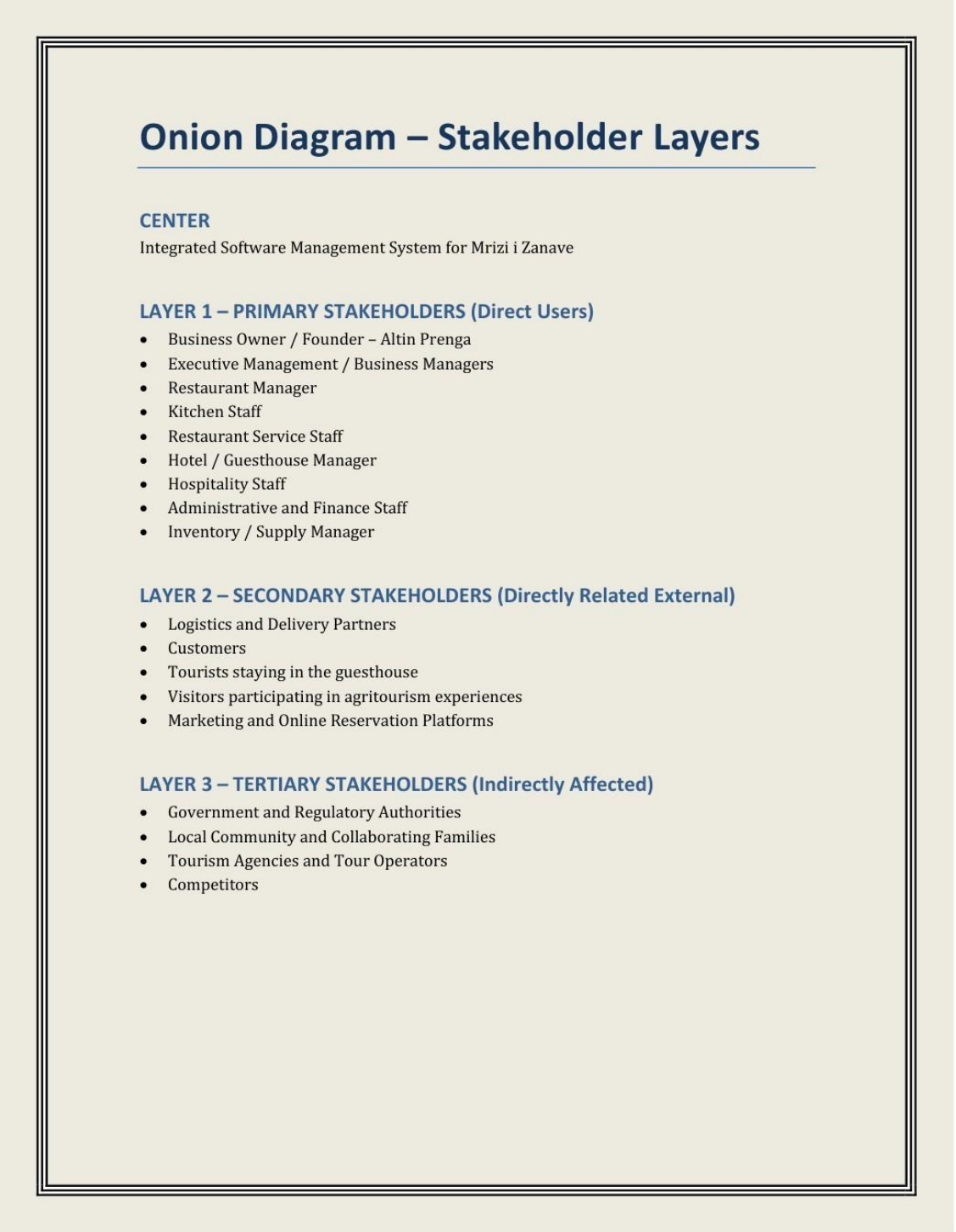


Figure: Onion Diagram

The onion diagram visually maps stakeholder influence around AgriSoft.

Onion Diagram Layers Page 01



Onion Diagram – Stakeholder Layers

CENTER

Integrated Software Management System for Mrizi i Zanave

LAYER 1 – PRIMARY STAKEHOLDERS (Direct Users)

- Business Owner / Founder – Altin Prenga
- Executive Management / Business Managers
- Restaurant Manager
- Kitchen Staff
- Restaurant Service Staff
- Hotel / Guesthouse Manager
- Hospitality Staff
- Administrative and Finance Staff
- Inventory / Supply Manager

LAYER 2 – SECONDARY STAKEHOLDERS (Directly Related External)

- Logistics and Delivery Partners
- Customers
- Tourists staying in the guesthouse
- Visitors participating in agritourism experiences
- Marketing and Online Reservation Platforms

LAYER 3 – TERTIARY STAKEHOLDERS (Indirectly Affected)

- Government and Regulatory Authorities
- Local Community and Collaborating Families
- Tourism Agencies and Tour Operators
- Competitors

Figure: Onion Diagram Layers Page 01

This page describes the onion diagram layers and their interpretation.

4.5 Business Process Model and Notation (BPMN)

BPMN diagrams describe how business processes flow across actors and system responsibilities. In AgriSoft, BPMN is important because restaurant reservations, room bookings, inventory updates, and reporting involve several roles and decision points.

Bpmn 1

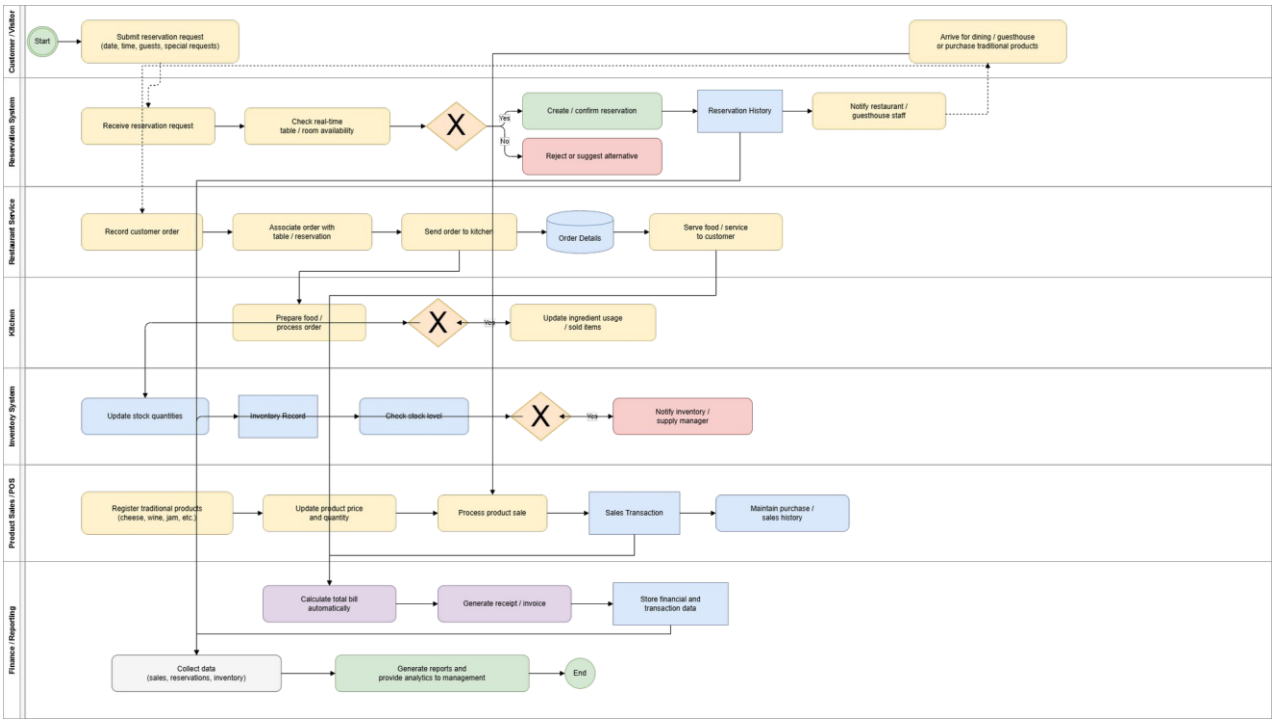


Figure: Bpmn 1

The BPMN model illustrates a business process flow, showing activities, decisions, and handoffs between staff and system components.

Bpmn 2

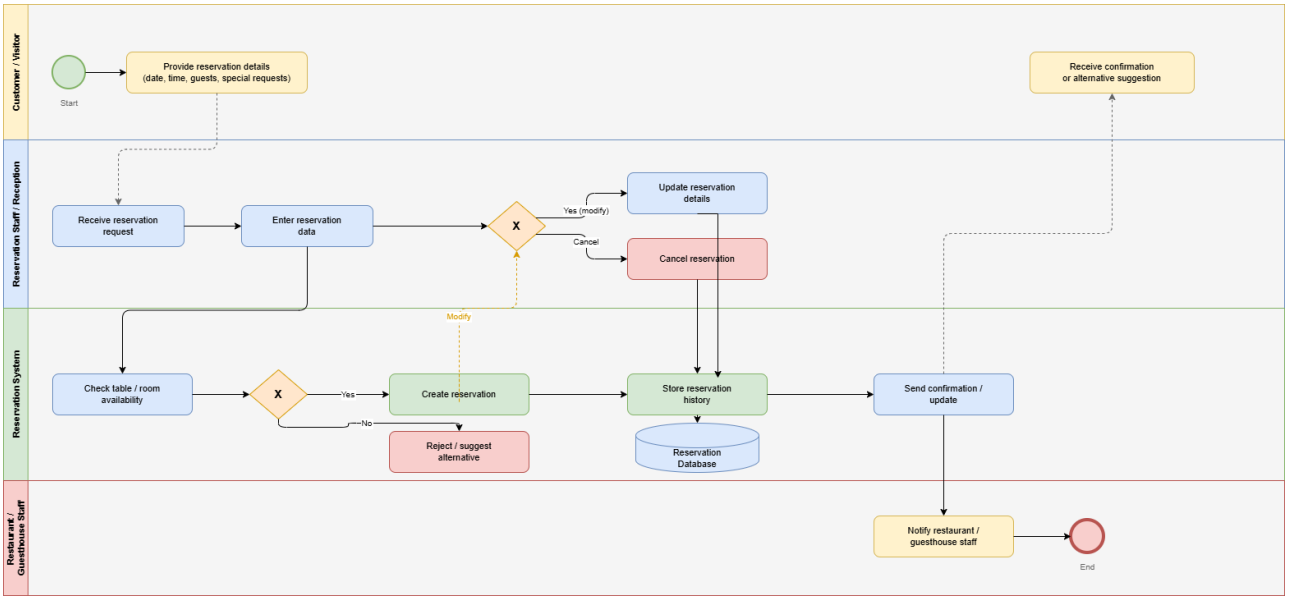


Figure: Bpmn 2

The BPMN model illustrates a business process flow, showing activities, decisions, and handoffs between staff and system components.

Bpmn 3

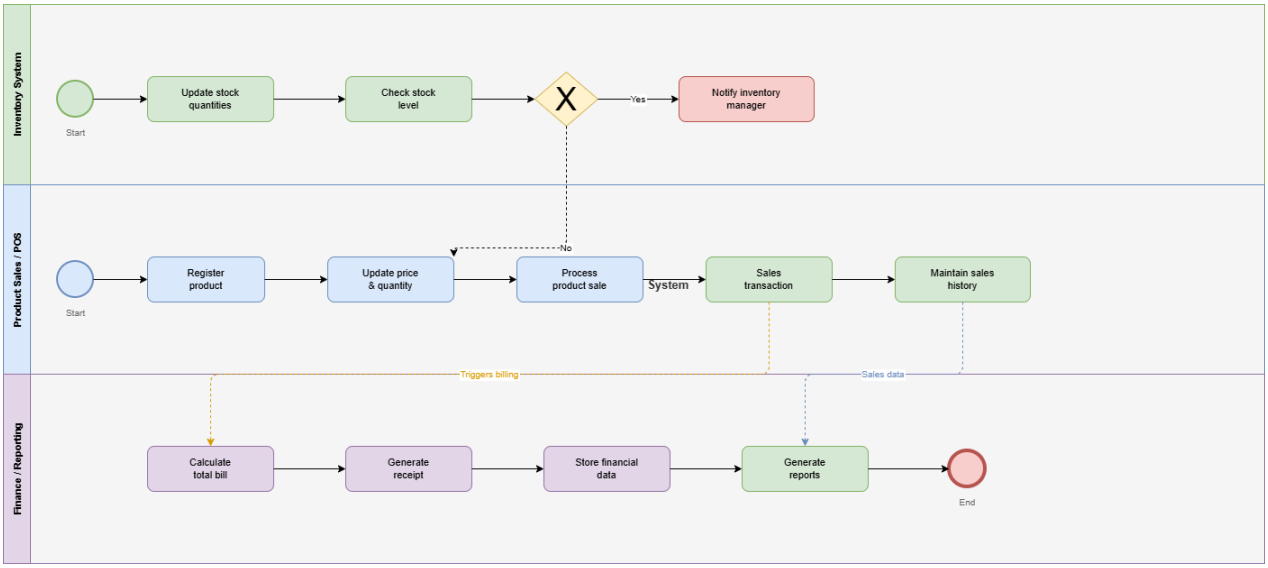


Figure: Bpmn 3

The BPMN model illustrates a business process flow, showing activities, decisions, and handoffs between staff and system components.

Bpmndiagram

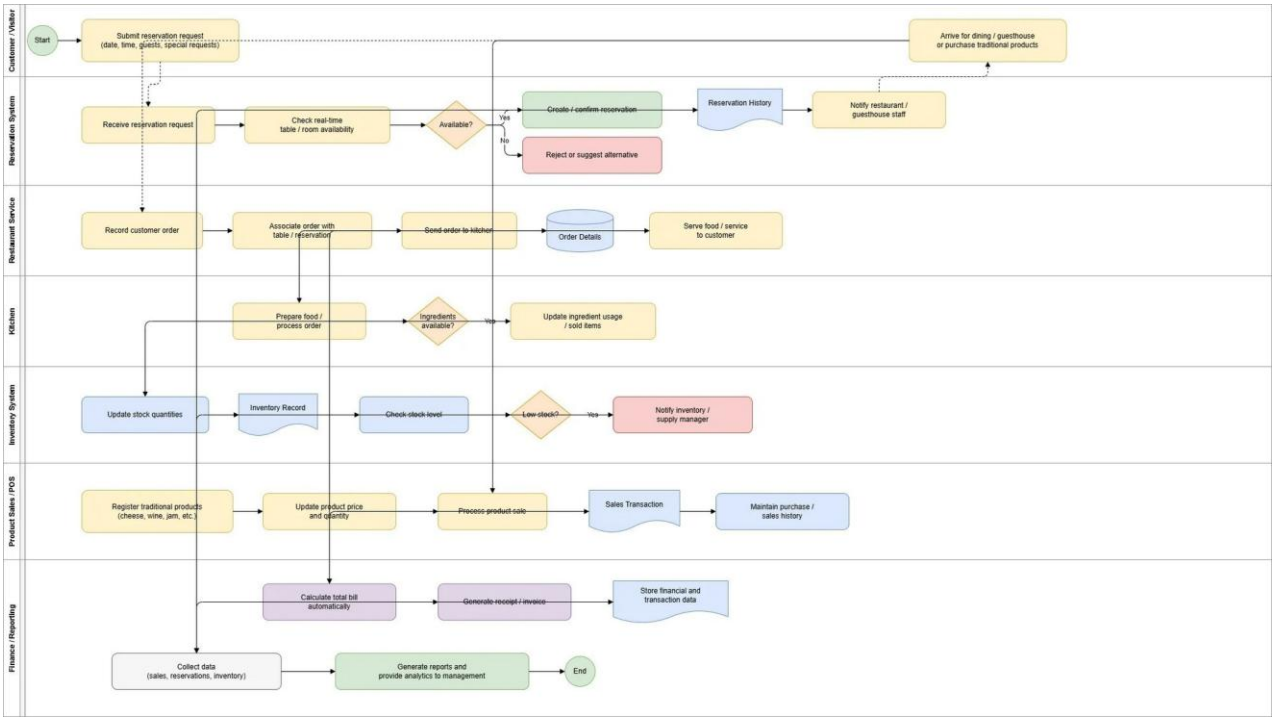


Figure: Bpmndiagram

The BPMN model illustrates a business process flow, showing activities, decisions, and handoffs between staff and system components.

4.6 Use Cases

Use cases describe structured interactions between actors and the AgriSoft system. Each use case has a triggering event, preconditions, postconditions, normal flow, and exception conditions. This format follows the academic model used in the reference documentation and supports traceability between requirements and diagrams.

Table 7: Main Use Case Summary

Use Case	Actor	Trigger	Brief Description	Postcondition
UC-01 Create Restaurant Reservation	Reception Staff	Customer requests table	Staff records reservation details and checks availability	Reservation is stored or alternative is proposed
UC-02 Create Room Reservation	Reception Staff	Customer requests room	Staff selects room, dates, and customer information	Room booking is confirmed
UC-03 Modify or Cancel Reservation	Reception Staff	Customer changes plan	Reservation is updated or cancelled	Availability is recalculated
UC-04 Record Restaurant Order	Waiter	Customer orders food	Waiter creates an order linked to table or reservation	Order is sent to kitchen
UC-05 Process Product Sale	Staff	Customer buys product	System records product and payment details	Sale is stored and stock updated
UC-06 Update Inventory	Inventory Manager	Stock changes	Manager updates quantities and thresholds	Inventory reflects current stock
UC-07 Generate Reports	Manager	Manager opens reports	System aggregates reservation, sales, and inventory data	Report is displayed
UC-08 View Dashboard	Manager/Admin	User logs in	Dashboard shows key business information	User can navigate modules

Use Case Tables Fr01 Fr25 Page 01

Use Case Tables

Functional Requirements FR-01 to FR-25

FR-01 - Create Restaurant Reservation

Use Case Name:	Create Restaurant Reservation	
Scenario:	A new restaurant reservation is created.	
Triggering Event:	Customer requests a table reservation.	
Brief Description:	Staff enters guest details, reservation date and time, and the system books an available table.	
Actors:	Staff	
Stakeholders:	Staff, Manager	
Preconditions:	Restaurant tables and schedule exist in the system.	
Postconditions:	Restaurant reservation is created and linked to a table.	
Flow of Activities:		
	Actor	**System**
1.	Staff starts a new restaurant reservation.	System opens the reservation form.
2.	Staff enters customer name, contact details, date, time, and number of guests.	System validates the entered reservation information.
3.	Staff requests availability.	System checks real-time table availability for the selected time slot.
4.	Staff selects an available table and confirms the booking.	System saves the reservation and marks the table as reserved.
5.	Staff informs the customer that the reservation is confirmed.	System displays the reservation reference.

Figure: Use Case Tables Fr01 Fr25 Page 01

This page contains detailed use case tables from the original AgriSoft documentation package.

Use Case Tables Fr01 Fr25 Page 02

FR-02 - Create Guesthouse Reservation

Use Case Name:	Create Guesthouse Reservation	
Scenario:	A new guesthouse reservation is created.	
Triggering Event:	Customer requests a room reservation.	
Brief Description:	Staff enters guest stay details and the system books an available room for the requested dates.	
Actors:	Staff	
Stakeholders:	Staff, Manager	
Preconditions:	Guesthouse rooms and room schedule exist in the system.	
Postconditions:	Guesthouse reservation is created and linked to a room.	
Flow of Activities:		
	Actor	**System**
1.	Staff starts a new guesthouse reservation.	System opens the room reservation form.
2.	Staff enters guest identity and stay details (check-in, check-out, guests).	System validates the entered reservation information.
3.	Staff requests room availability.	System checks real-time room availability for the selected dates.
4.	Staff selects an available room and confirms the booking.	System saves the reservation and marks the room as reserved.
5.	Staff provides the booking confirmation to the customer.	System displays the reservation reference.

Figure: Use Case Tables Fr01 Fr25 Page 02

This page contains detailed use case tables from the original AgriSoft documentation package.

Use Case Tables Fr01 Fr25 Page 03

FR-03 - Display Real-Time Availability

Use Case Name:	Display Real-Time Availability	
Scenario:	Availability of tables and rooms is displayed.	
Triggering Event:	Staff needs to check available resources.	
Brief Description:	Staff requests an overview of free tables or rooms for a selected date and time.	
Actors:	Staff	
Stakeholders:	Staff	
Preconditions:	Tables, rooms, and reservation data exist in the system.	
Postconditions:	Current availability is displayed to staff.	
Flow of Activities:		
	Actor	**System**
1.	Staff opens the availability view.	System displays search fields for date, time, and resource type.
2.	Staff selects date, time, and whether to view tables or rooms.	System retrieves current reservation data.
3.	Staff submits the request.	System displays available and unavailable tables or rooms in real time.

Figure: Use Case Tables Fr01 Fr25 Page 03

This page contains detailed use case tables from the original AgriSoft documentation package.

Use Case Tables Fr01 Fr25 Page 04

FR-04 - Prevent Double Booking

Use Case Name:	Prevent Double Booking	
Scenario:	System blocks a conflicting booking.	
Triggering Event:	Staff attempts to book a table or room already reserved.	
Brief Description:	Before saving a reservation, the system checks for conflicts and rejects any double booking.	
Actors:	Staff	
Stakeholders:	Staff, Manager	
Preconditions:	Reservation details have been entered.	
Postconditions:	No conflicting reservation is created.	
Flow of Activities:		
	Actor	**System**
1.	Staff enters reservation details and selects a table or room.	System checks existing reservations for overlaps.
2.	Staff attempts to confirm the reservation.	System detects a conflict if the resource is already booked.
3.	Staff reviews the warning.	System blocks the reservation and prompts staff to choose another table or room.

Figure: Use Case Tables Fr01 Fr25 Page 04

This page contains detailed use case tables from the original AgriSoft documentation package.

Use Case Tables Fr01 Fr25 Page 05

FR-05 - Modify or Cancel Reservation

Use Case Name:	Modify or Cancel Reservation	
Scenario:	An existing reservation is changed or canceled.	
Triggering Event:	Customer requests a reservation update or cancellation.	
Brief Description:	Staff locates an existing reservation and updates or cancels it.	
Actors:	Staff	
Stakeholders:	Staff	
Preconditions:	Reservation already exists.	
Postconditions:	Reservation is updated or canceled and availability is adjusted.	
Flow of Activities:		
	Actor	**System**
1.	Staff searches for the reservation.	System displays matching reservations.
2.	Staff opens the selected reservation.	System loads the reservation details.
3.	Staff edits reservation details or chooses cancel.	System validates the changes or cancellation request.
4.	Staff confirms the action.	System updates or cancels the reservation and refreshes availability.

Figure: Use Case Tables Fr01 Fr25 Page 05

This page contains detailed use case tables from the original AgriSoft documentation package.

Use Case Tables Fr01 Fr25 Page 06

FR-06 - Store Reservation History

Use Case Name:	Store Reservation History	
Scenario:	Past reservation records are stored.	
Triggering Event:	A reservation is completed, modified, or canceled.	
Brief Description:	The system keeps historical records of reservations for later review by management.	
Actors:	Manager	
Stakeholders:	Manager	
Preconditions:	Reservations have been created and processed.	
Postconditions:	Reservation history is stored and can be reviewed.	
Flow of Activities:		
	Actor	**System**
1.	Reservation activity occurs in the system.	System records the reservation status, timestamps, and related details.
2.	Manager opens reservation history.	System retrieves historical reservation records.
3.	Manager reviews past reservations.	System displays stored reservation history.

Figure: Use Case Tables Fr01 Fr25 Page 06

This page contains detailed use case tables from the original AgriSoft documentation package.

Use Case Tables Fr01 Fr25 Page 07

FR-07 - Search Reservations

Use Case Name:	Search Reservations	
Scenario:	Reservations are searched.	
Triggering Event:	Staff needs to find a specific reservation.	
Brief Description:	Staff searches reservations by guest name, phone number, date, or reference.	
Actors:	Staff	
Stakeholders:	Staff	
Preconditions:	Reservations exist in the system.	
Postconditions:	Matching reservations are displayed.	
Flow of Activities:		
	Actor	**System**
1.	Staff opens the reservation search screen.	System displays search fields.
2.	Staff enters a search term or criteria.	System searches reservation records.
3.	Staff submits the search.	System displays matching reservations.

Figure: Use Case Tables Fr01 Fr25 Page 07

This page contains detailed use case tables from the original AgriSoft documentation package.

Use Case Tables Fr01 Fr25 Page 08

FR-08 - Filter Reservations

Use Case Name:	Filter Reservations	
Scenario:	Reservation list is filtered.	
Triggering Event:	Staff wants to narrow the reservation list.	
Brief Description:	Staff applies filters such as date, status, or reservation type to view relevant bookings.	
Actors:	Staff	
Stakeholders:	Staff	
Preconditions:	Reservations exist in the system.	
Postconditions:	Filtered reservation list is displayed.	
Flow of Activities:		
	Actor	**System**
1.	Staff opens the reservations list.	System displays all available reservations.
2.	Staff selects filter criteria such as date, status, or type.	System applies the selected filters.
3.	Staff confirms the filter.	System displays only reservations that match the selected criteria.

Figure: Use Case Tables Fr01 Fr25 Page 08

This page contains detailed use case tables from the original AgriSoft documentation package.

Use Case Tables 26 50 Page 01

FR-26 - Notify Low Stock Levels

Use Case Name:	Notify Low Stock Levels	
Scenario:	A notify low stock levels process.	
Triggering Event:	User initiates the action.	
Brief Description:	The system handles notify low stock levels based on user input.	
Actors:	Inventory Manager	
Stakeholders:	Inventory Manager	
Preconditions:	System data is available.	
Postconditions:	Operation completed successfully.	
Flow of Activities:		
	Actor	System
1.	Inventory Manager starts the process.	System opens the interface.
2.	Inventory Manager enters required data.	System validates data.
3.	Inventory Manager requests action.	System processes request.
4.	Inventory Manager confirms action.	System saves data.
5.	Inventory Manager completes process.	System displays result.

FR-27 - Display Current Stock Levels

Use Case Name:	Display Current Stock Levels
Scenario:	A display current stock levels process.
Triggering Event:	User initiates the action.

Figure: Use Case Tables 26 50 Page 01

This page contains detailed use case tables from the original AgriSoft documentation package.

Use Case Tables 26 50 Page 02

Brief Description:	The system handles display current stock levels based on user input.	
Actors:	Inventory Manager	
Stakeholders:	Inventory Manager	
Preconditions:	System data is available.	
Postconditions:	Operation completed successfully.	
Flow of Activities:		
	Actor	System
1.	Inventory Manager starts the process.	System opens the interface.
2.	Inventory Manager enters required data.	System validates data.
3.	Inventory Manager requests action.	System processes request.
4.	Inventory Manager confirms action.	System saves data.
5.	Inventory Manager completes process.	System displays result.

FR-28 - Track Supplier Information

Use Case Name:	Track Supplier Information
Scenario:	A track supplier information process.
Triggering Event:	User initiates the action.
Brief Description:	The system handles track supplier information based on user input.
Actors:	Inventory Manager
Stakeholders:	Inventory Manager
Preconditions:	System data is available.

Figure: Use Case Tables 26 50 Page 02

This page contains detailed use case tables from the original AgriSoft documentation package.

Use Case Tables 26 50 Page 03

Postconditions:	Operation completed successfully.	
Flow of Activities:		
	Actor	System
1.	Inventory Manager starts the process.	System opens the interface.
2.	Inventory Manager enters required data.	System validates data.
3.	Inventory Manager requests action.	System processes request.
4.	Inventory Manager confirms action.	System saves data.
5.	Inventory Manager completes process.	System displays result.

FR-29 - Record Damaged Goods

Use Case Name:	Record Damaged Goods	
Scenario:	A record damaged goods process.	
Triggering Event:	User initiates the action.	
Brief Description:	The system handles record damaged goods based on user input.	
Actors:	Inventory Manager	
Stakeholders:	Inventory Manager	
Preconditions:	System data is available.	
Postconditions:	Operation completed successfully.	
Flow of Activities:		
	Actor	System
1.	Inventory Manager starts the process.	System opens the interface.

Figure: Use Case Tables 26 50 Page 03

This page contains detailed use case tables from the original AgriSoft documentation package.

Use Case Tables 26 50 Page 04

2.	Inventory Manager enters required data.	System validates data.
3.	Inventory Manager requests action.	System processes request.
4.	Inventory Manager confirms action.	System saves data.
5.	Inventory Manager completes process.	System displays result.

FR-30 - Track Product Expiration Dates

Use Case Name:	Track Product Expiration Dates	
Scenario:	A track product expiration dates process.	
Triggering Event:	User initiates the action.	
Brief Description:	The system handles track product expiration dates based on user input.	
Actors:	Inventory Manager	
Stakeholders:	Inventory Manager	
Preconditions:	System data is available.	
Postconditions:	Operation completed successfully.	
Flow of Activities:		
	Actor	System
1.	Inventory Manager starts the process.	System opens the interface.
2.	Inventory Manager enters required data.	System validates data.
3.	Inventory Manager requests action.	System processes request.
4.	Inventory Manager confirms action.	System saves data.

Figure: Use Case Tables 26 50 Page 04

This page contains detailed use case tables from the original AgriSoft documentation package.

4.7 Use Case Diagram

The use case diagrams show the main system functions and the actors that interact with them. AgriSoft uses module-level use case diagrams for reservation, restaurant, inventory, product sales, and reporting.

Uml Use Case Inventory

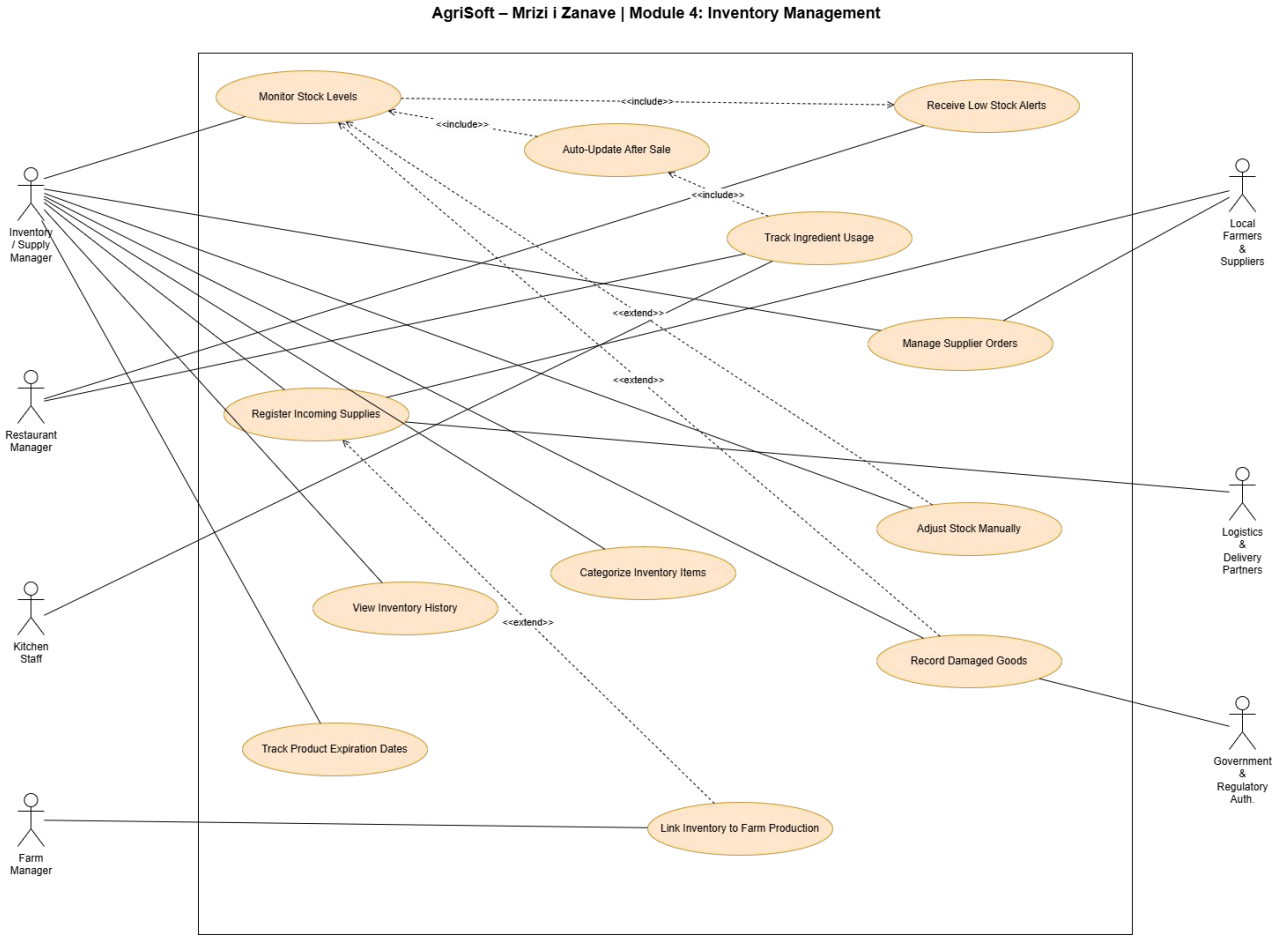


Figure: Uml Use Case Inventory

This use case diagram connects an AgriSoft actor with the functional services provided by the selected module.

Uml Use Case Product Sales

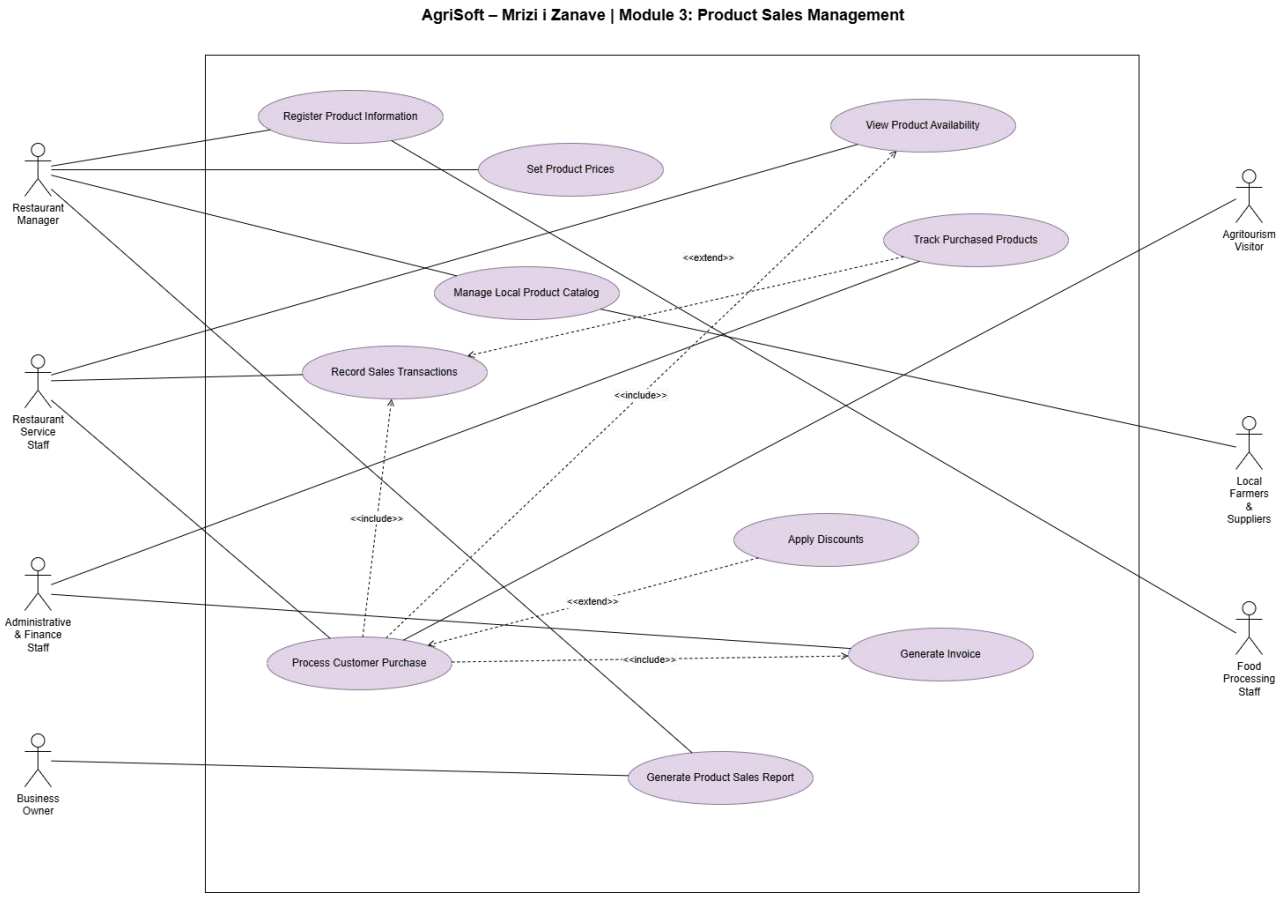


Figure: Uml Use Case Product Sales

This use case diagram connects an AgriSoft actor with the functional services provided by the selected module.

Uml Use Case Reporting

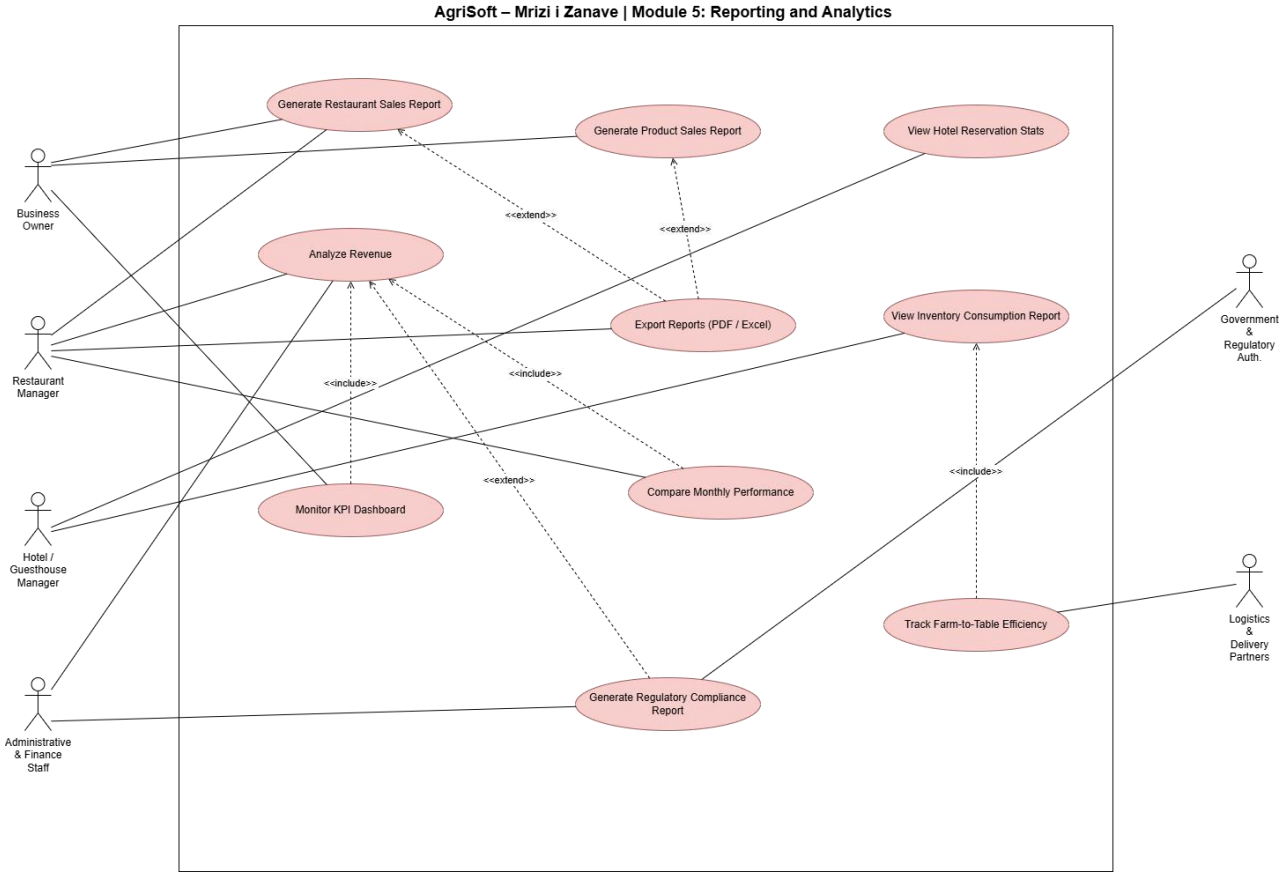


Figure: Uml Use Case Reporting

This use case diagram connects an AgriSoft actor with the functional services provided by the selected module.

Uml Use Case Reservation

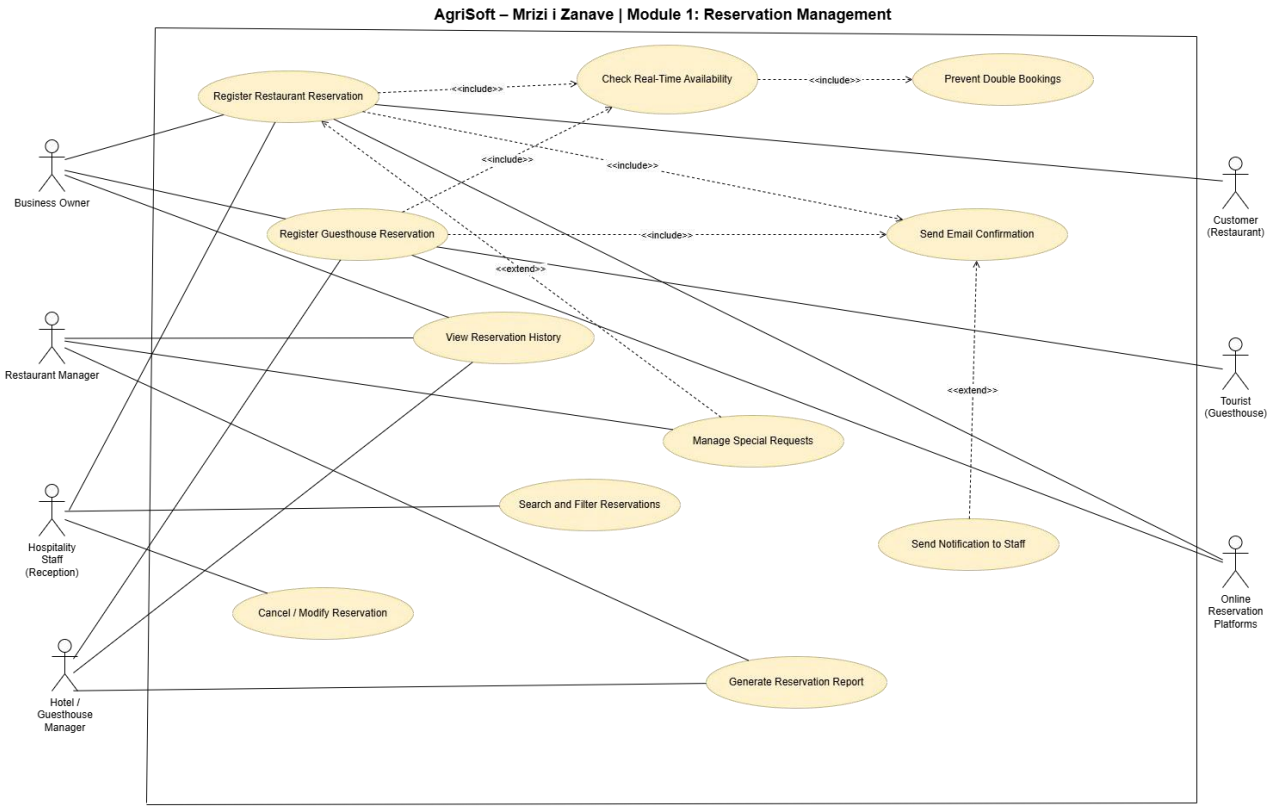


Figure: Uml Use Case Reservation

This use case diagram connects an AgriSoft actor with the functional services provided by the selected module.

Uml Use Case Restaurant

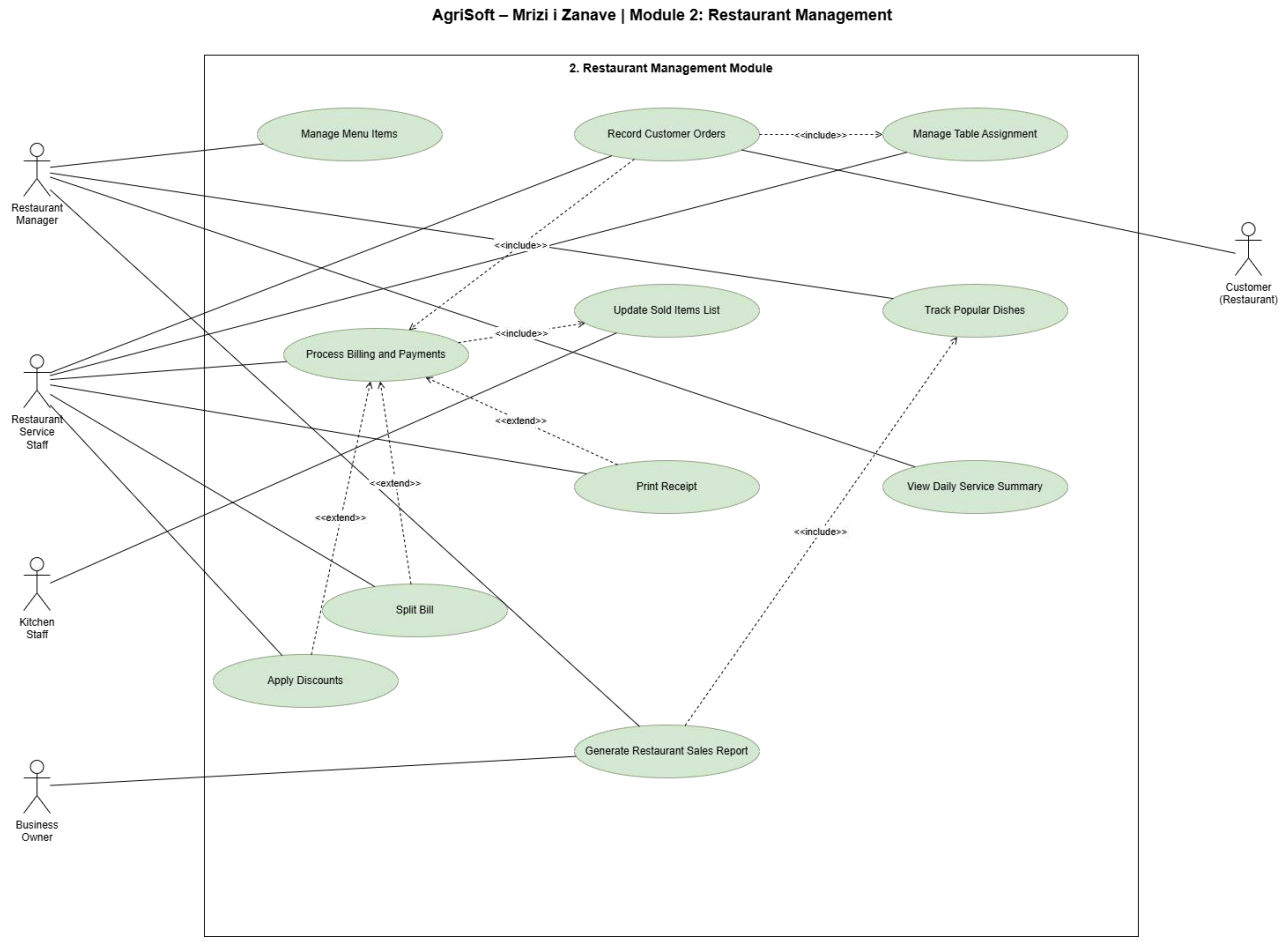


Figure: Uml Use Case Restaurant

This use case diagram connects an AgriSoft actor with the functional services provided by the selected module.

4.8 Activity Diagrams

Activity diagrams show step-by-step workflows. The AgriSoft project includes 23 scenario-based activity diagrams, matching the scenario list documented earlier. Each diagram represents a clear operational process from initiation to completion, including decision points where the system validates or blocks an action.

01 Staff Receives Reservation Request

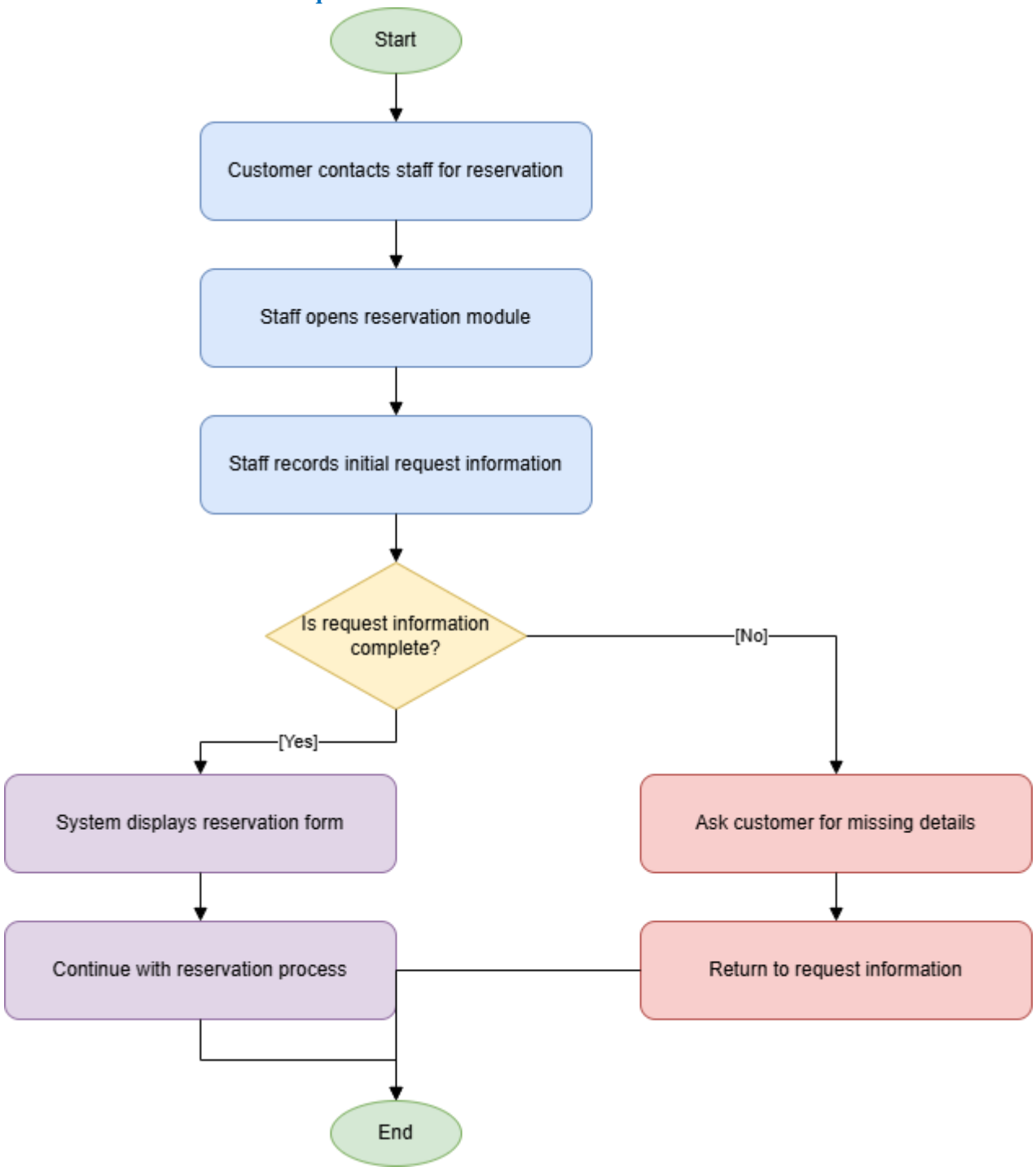


Figure: 01 Staff Receives Reservation Request

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

02 Enter Reservation Details

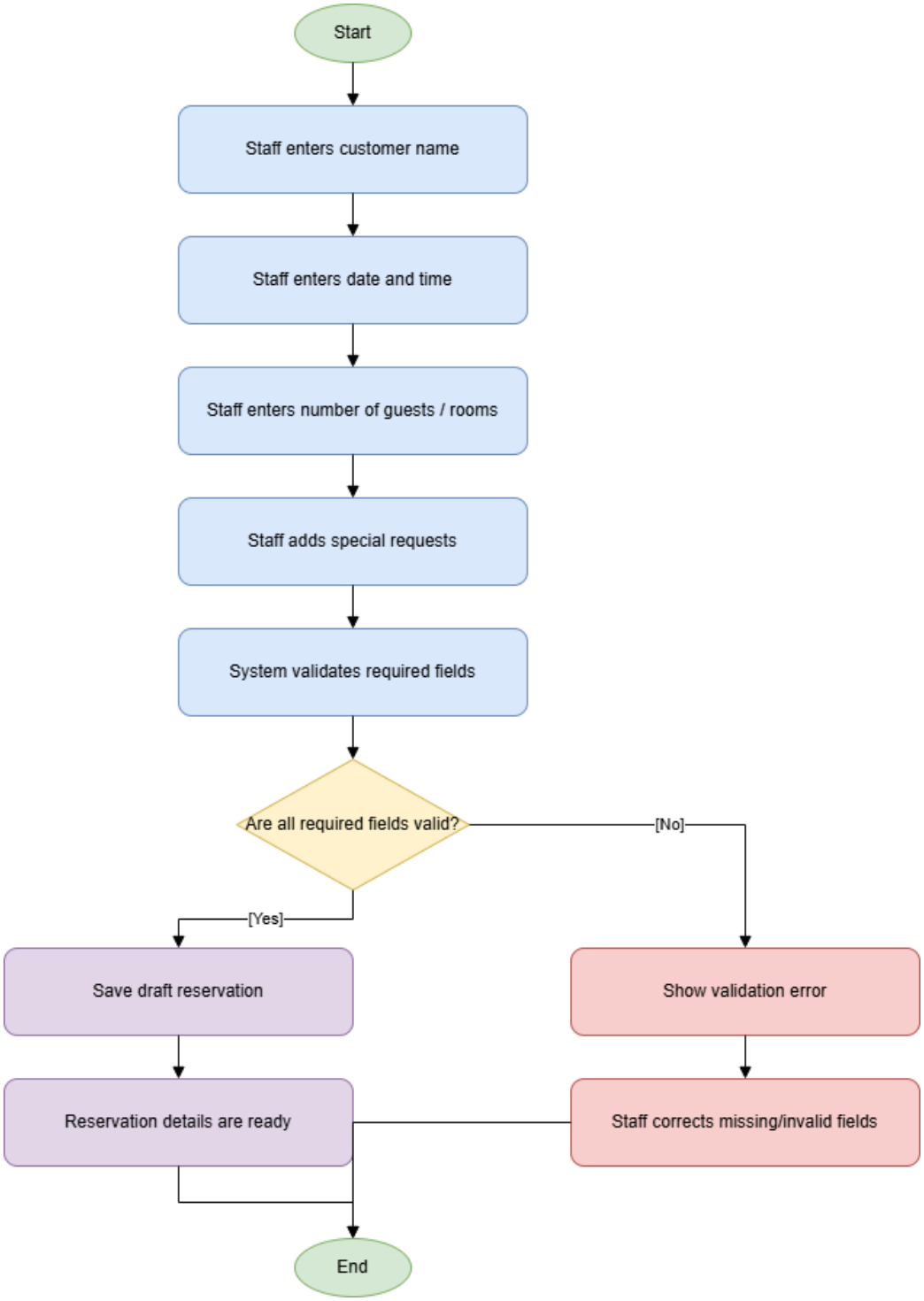


Figure: 02 Enter Reservation Details

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

03 Check Availability In Real Time

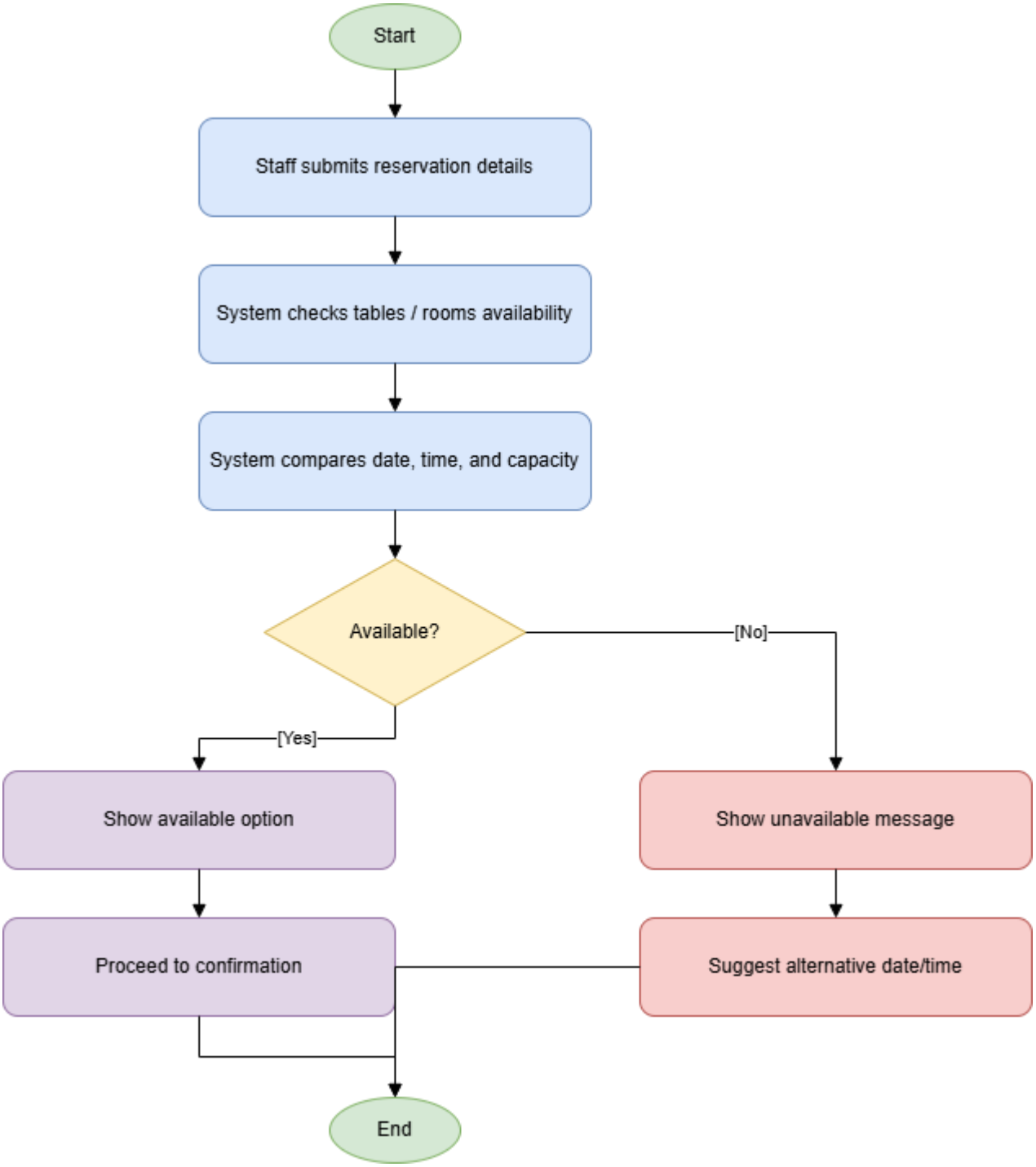


Figure: 03 Check Availability In Real Time

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

04 Confirm Reservation

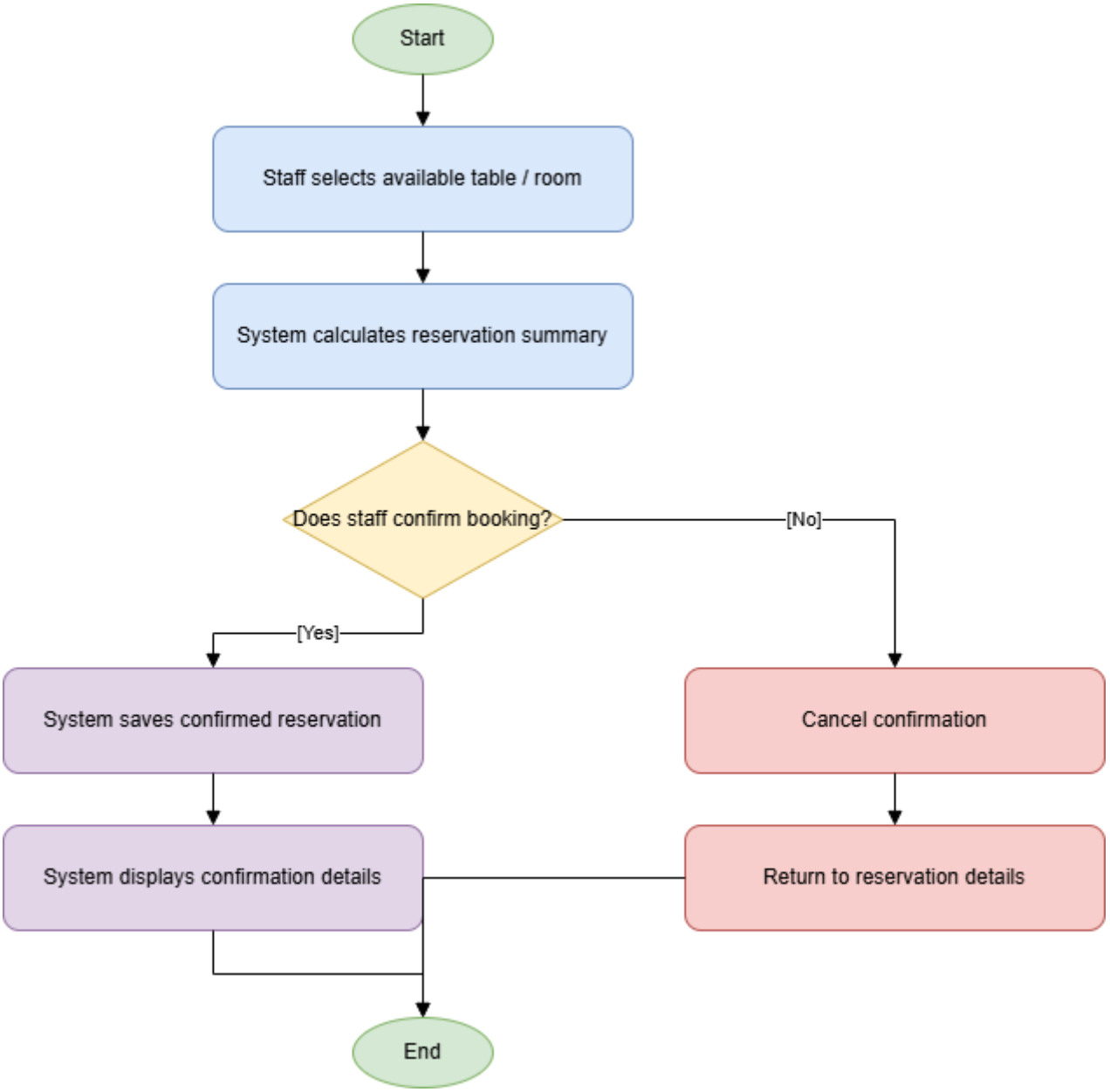


Figure: 04 Confirm Reservation

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

05 Suggest Alternative If Not Available

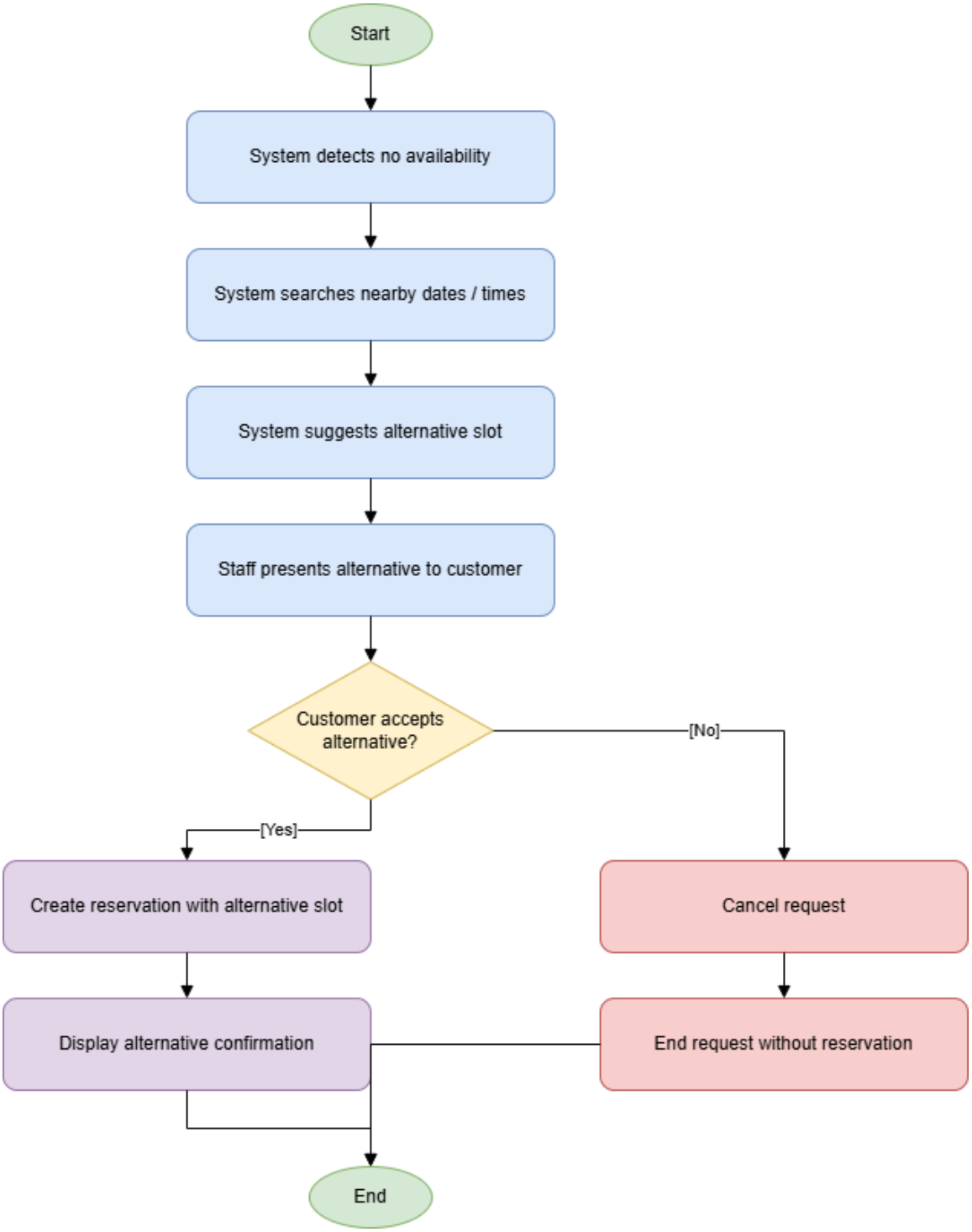


Figure: 05 Suggest Alternative If Not Available

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

06 Modify Or Cancel Reservation If Needed

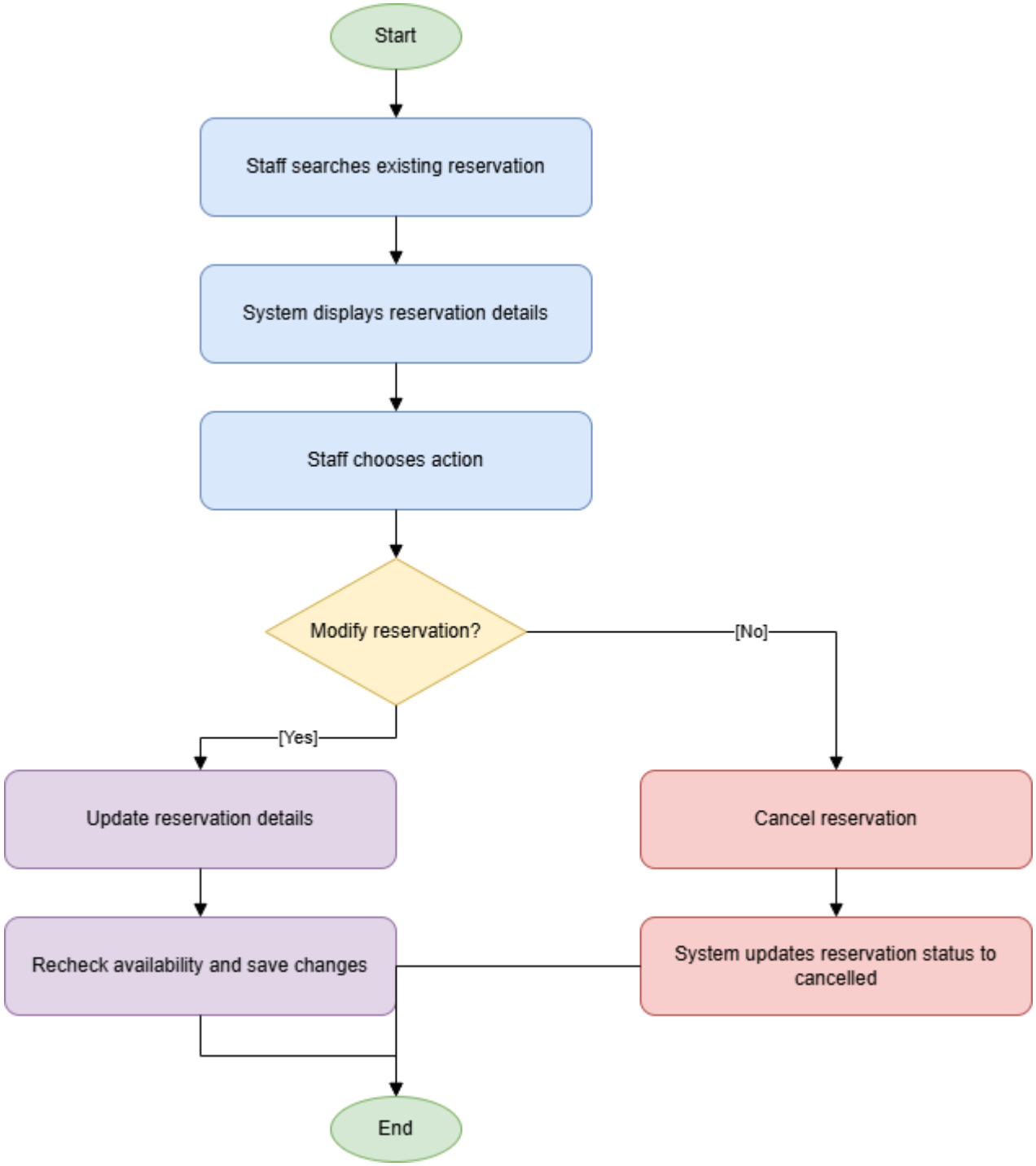


Figure: 06 Modify Or Cancel Reservation If Needed

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

07 Waiter Takes Customer Order

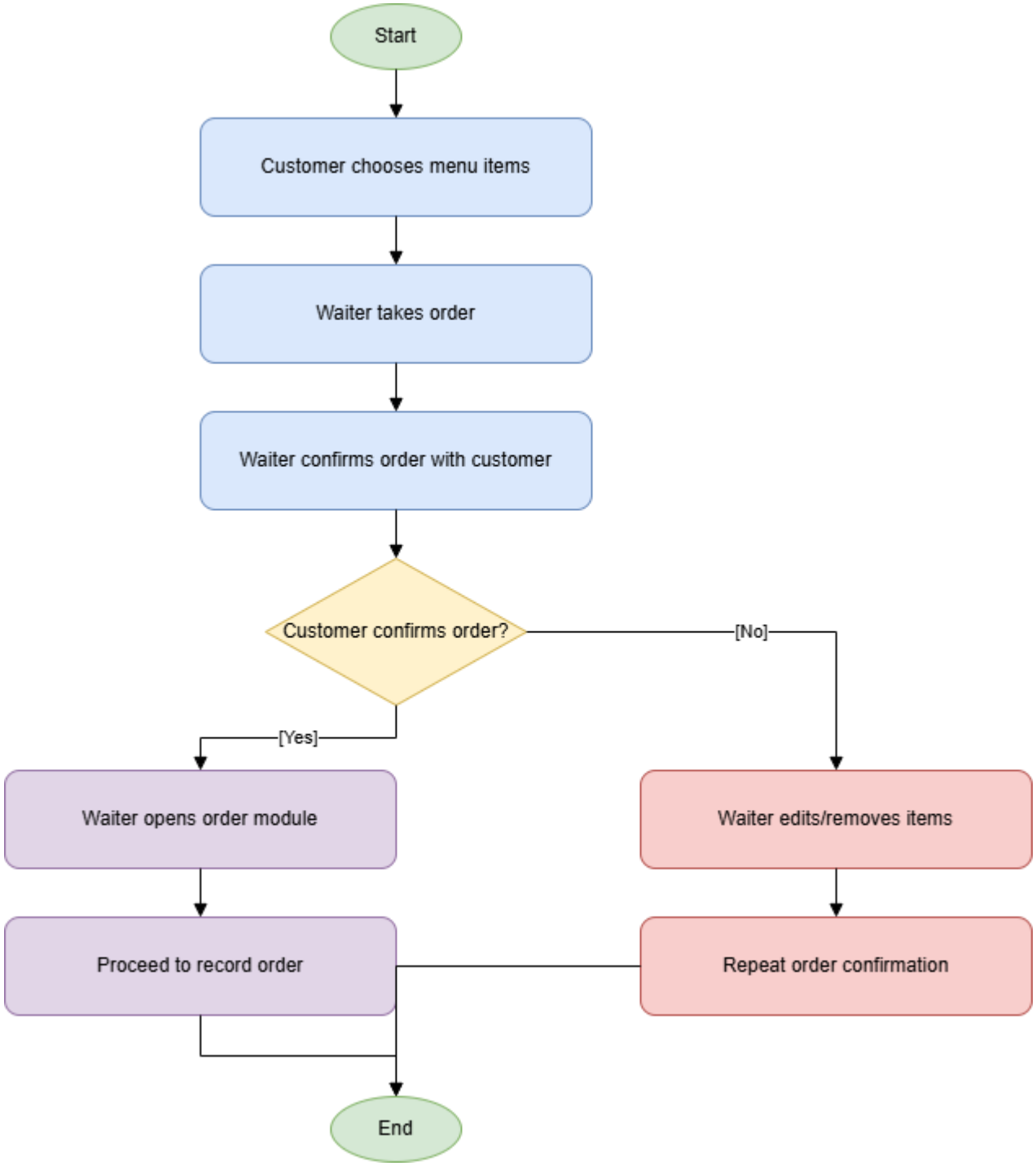


Figure: 07 Waiter Takes Customer Order

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

08 Record Order In System

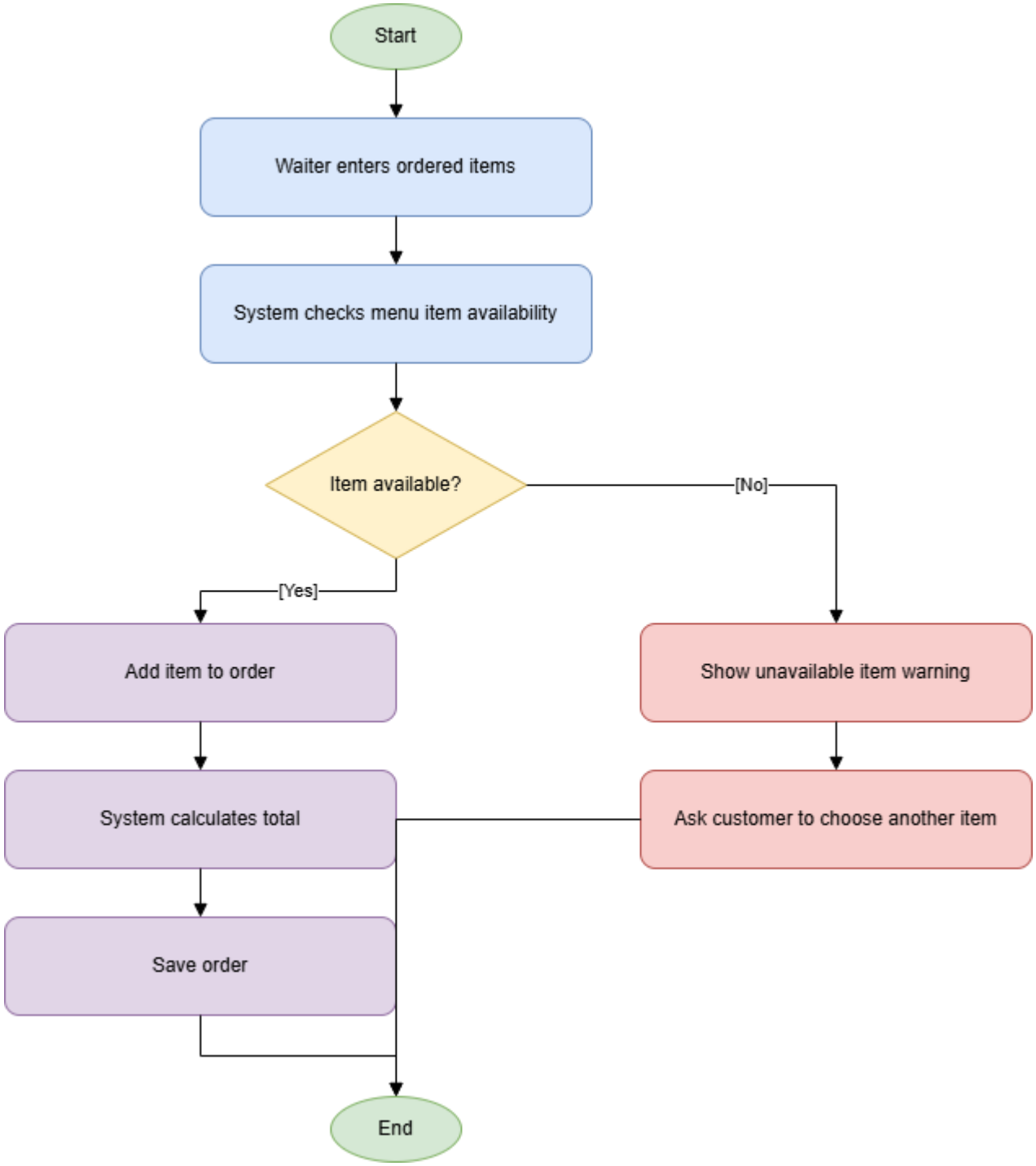


Figure: 08 Record Order In System

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

09 Associate Order With Table Or Reservation

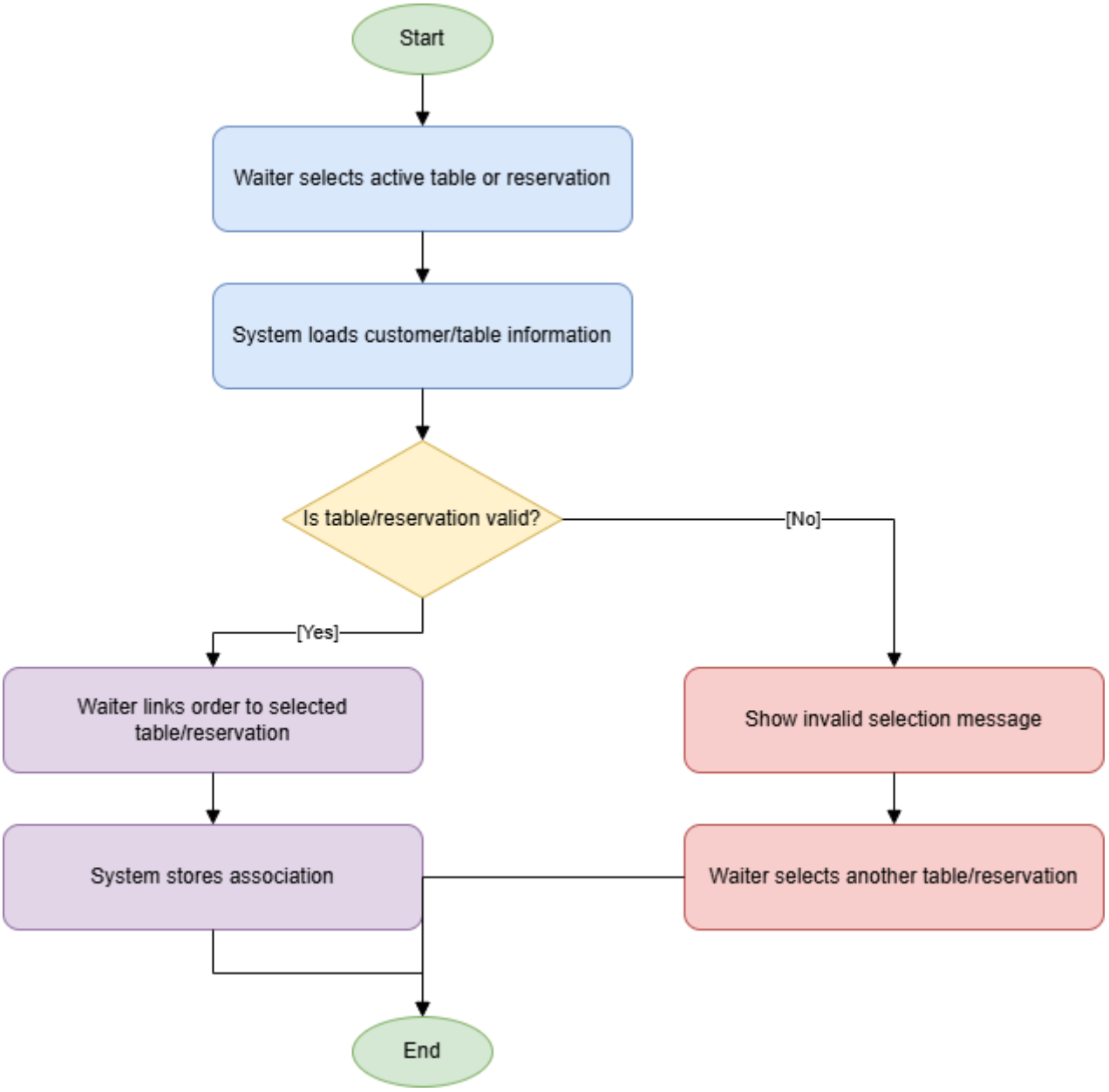


Figure: 09 Associate Order With Table Or Reservation

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

10 Send Order To Kitchen

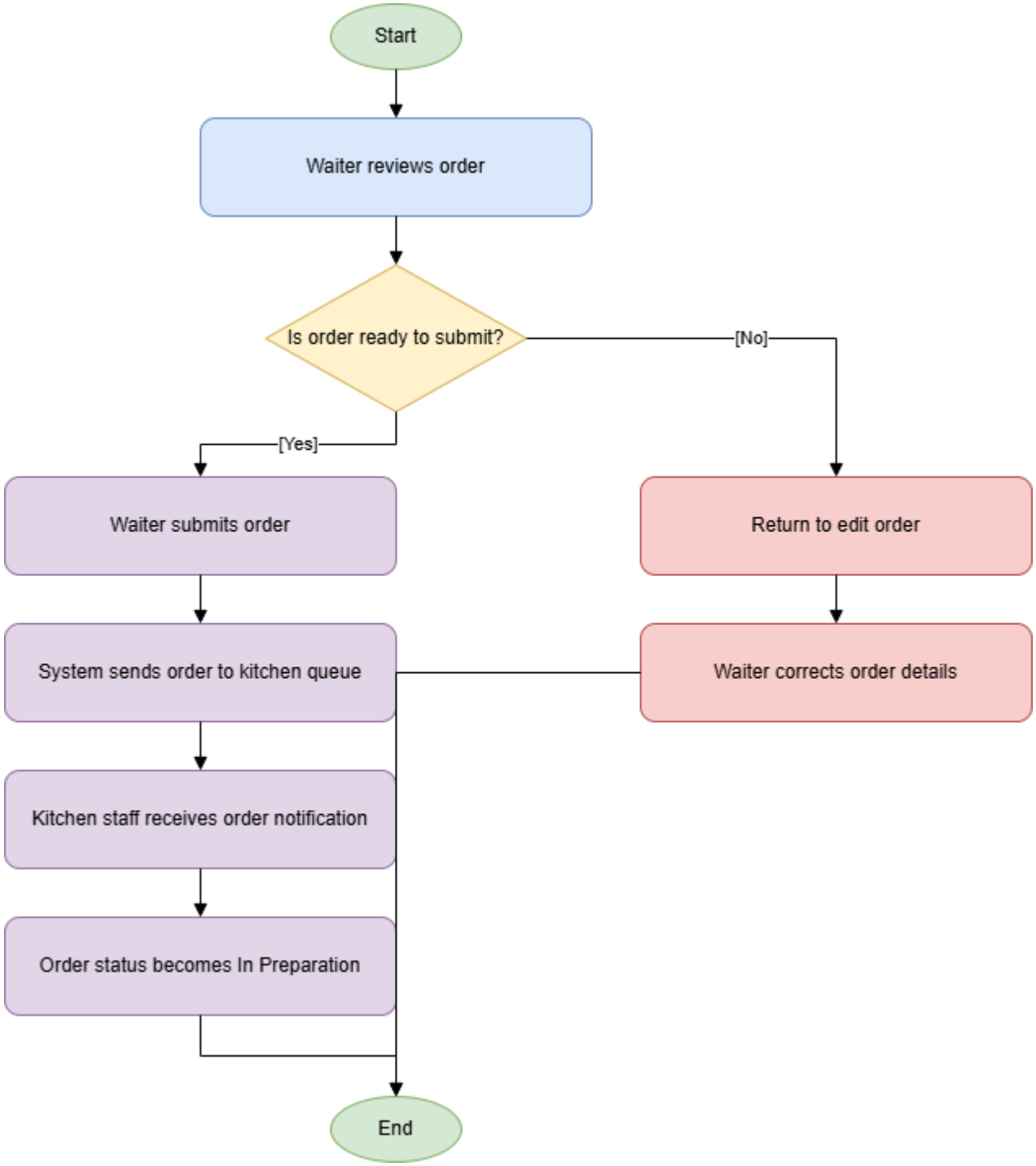


Figure: 10 Send Order To Kitchen

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

11 Prepare Food And Serve Customer

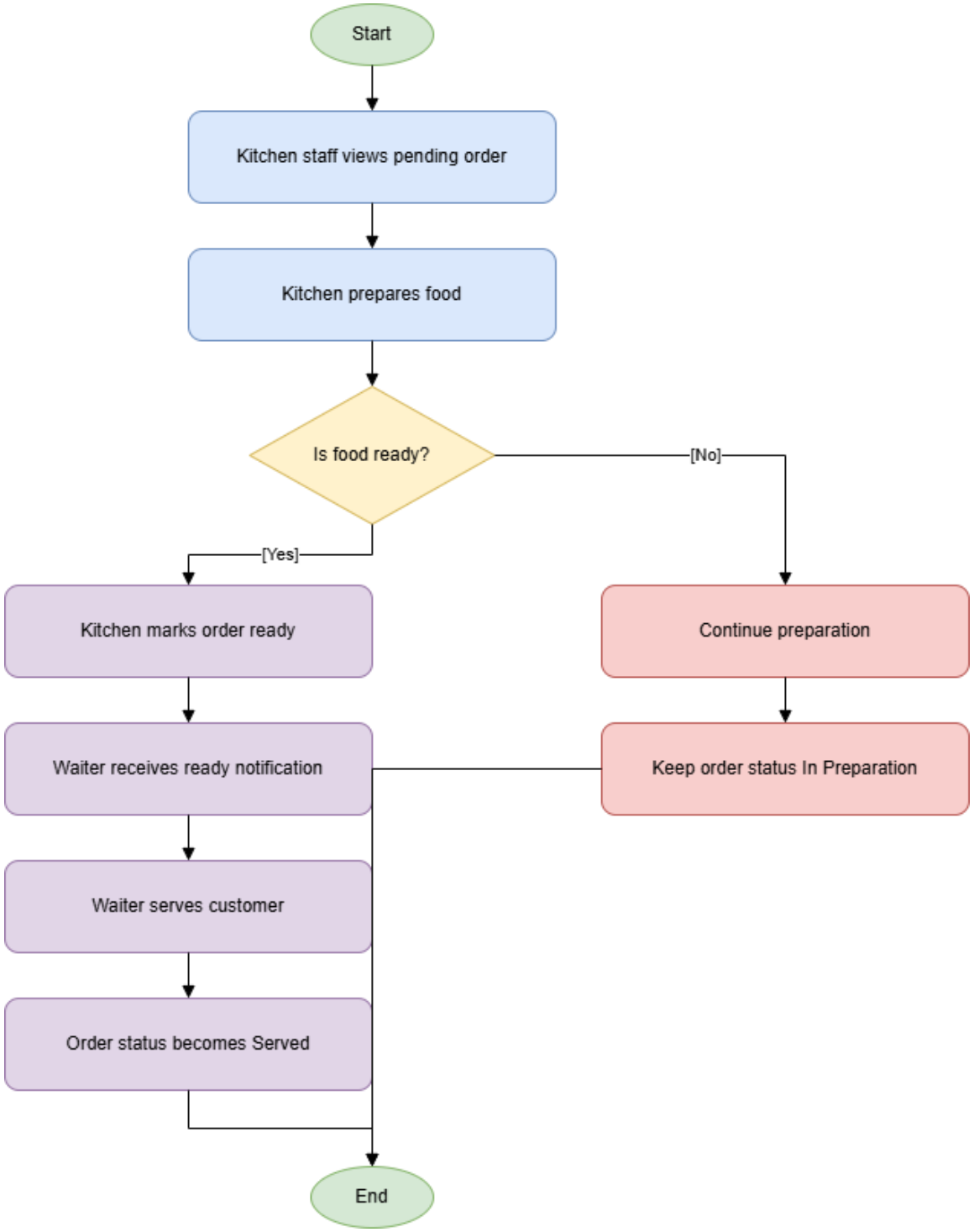


Figure: 11 Prepare Food And Serve Customer

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

12 Update Stock After Usage Or Sale

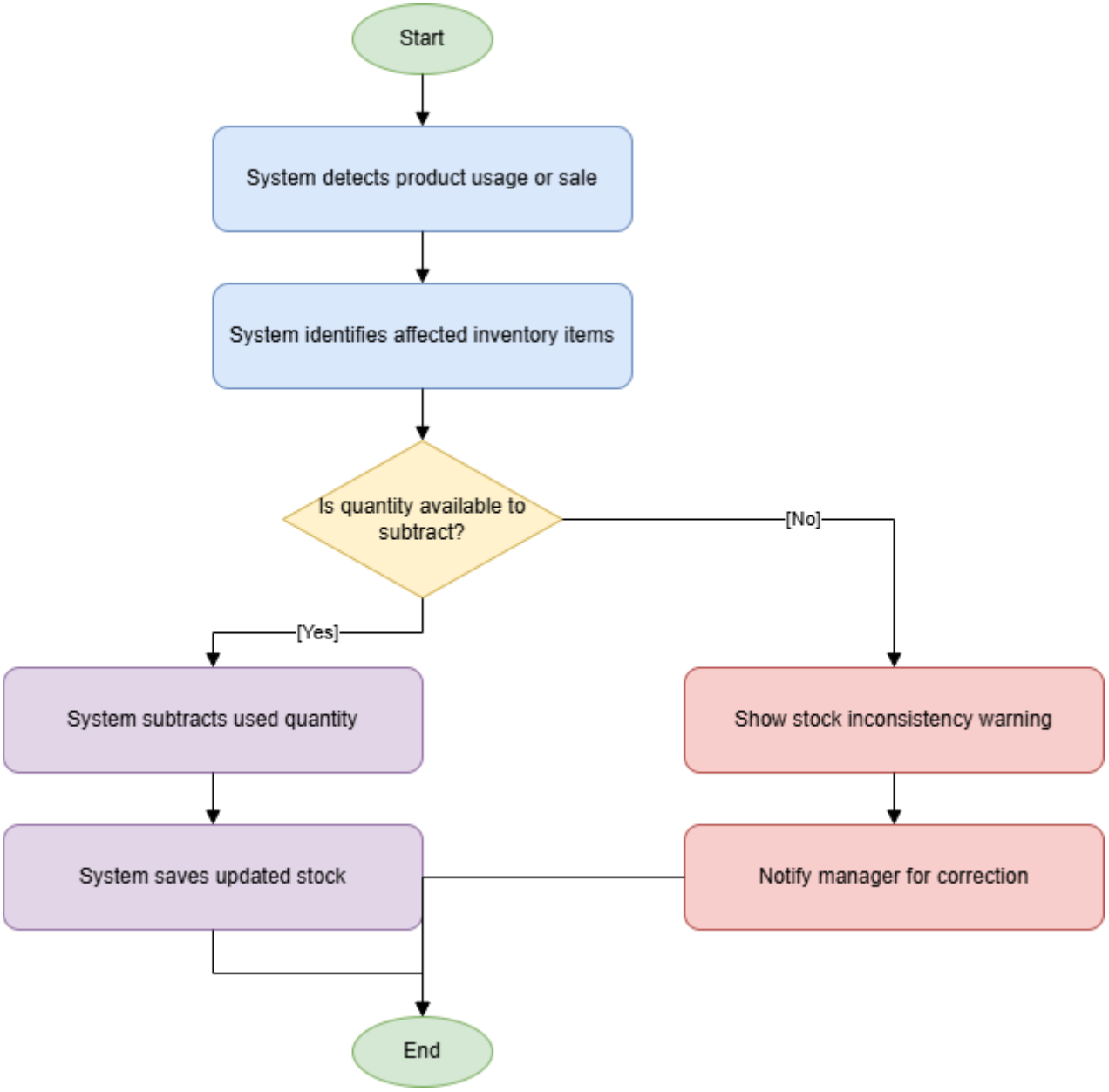


Figure: 12 Update Stock After Usage Or Sale

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

13 Check Stock Level

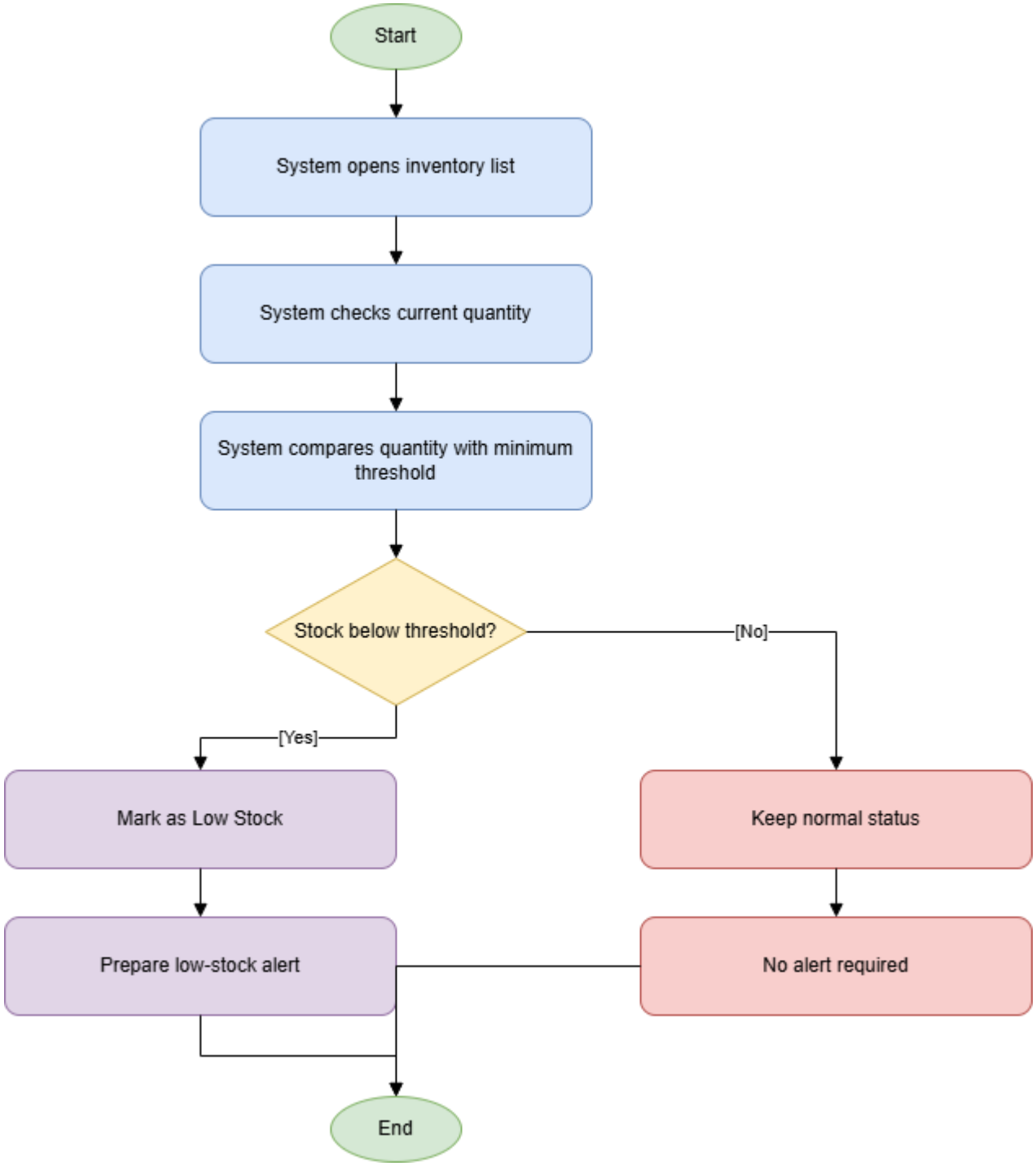


Figure: 13 Check Stock Level

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

14 Notify Manager If Stock Is Low

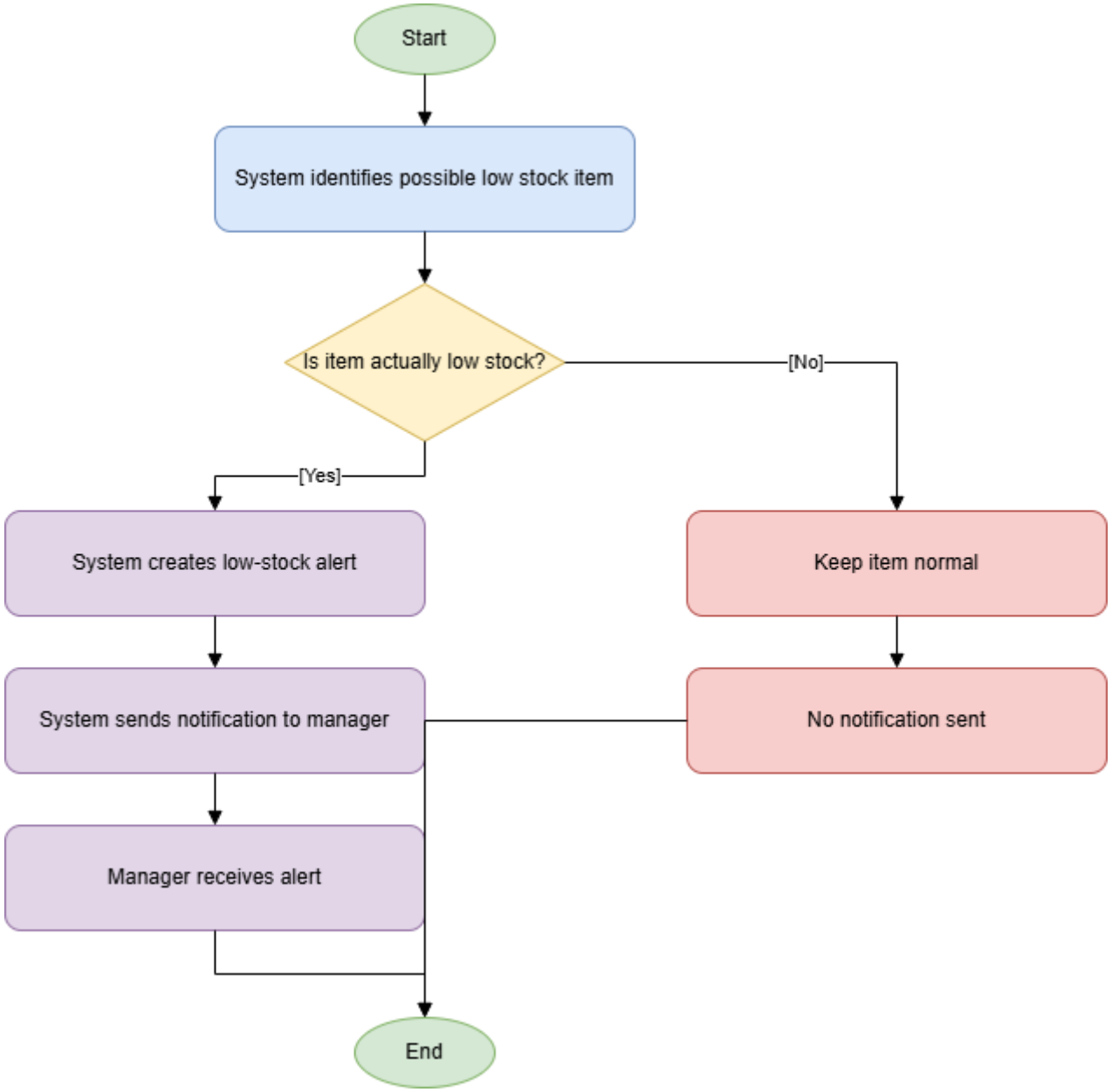


Figure: 14 Notify Manager If Stock Is Low

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

15 Manager Updates Stock Manually If Needed

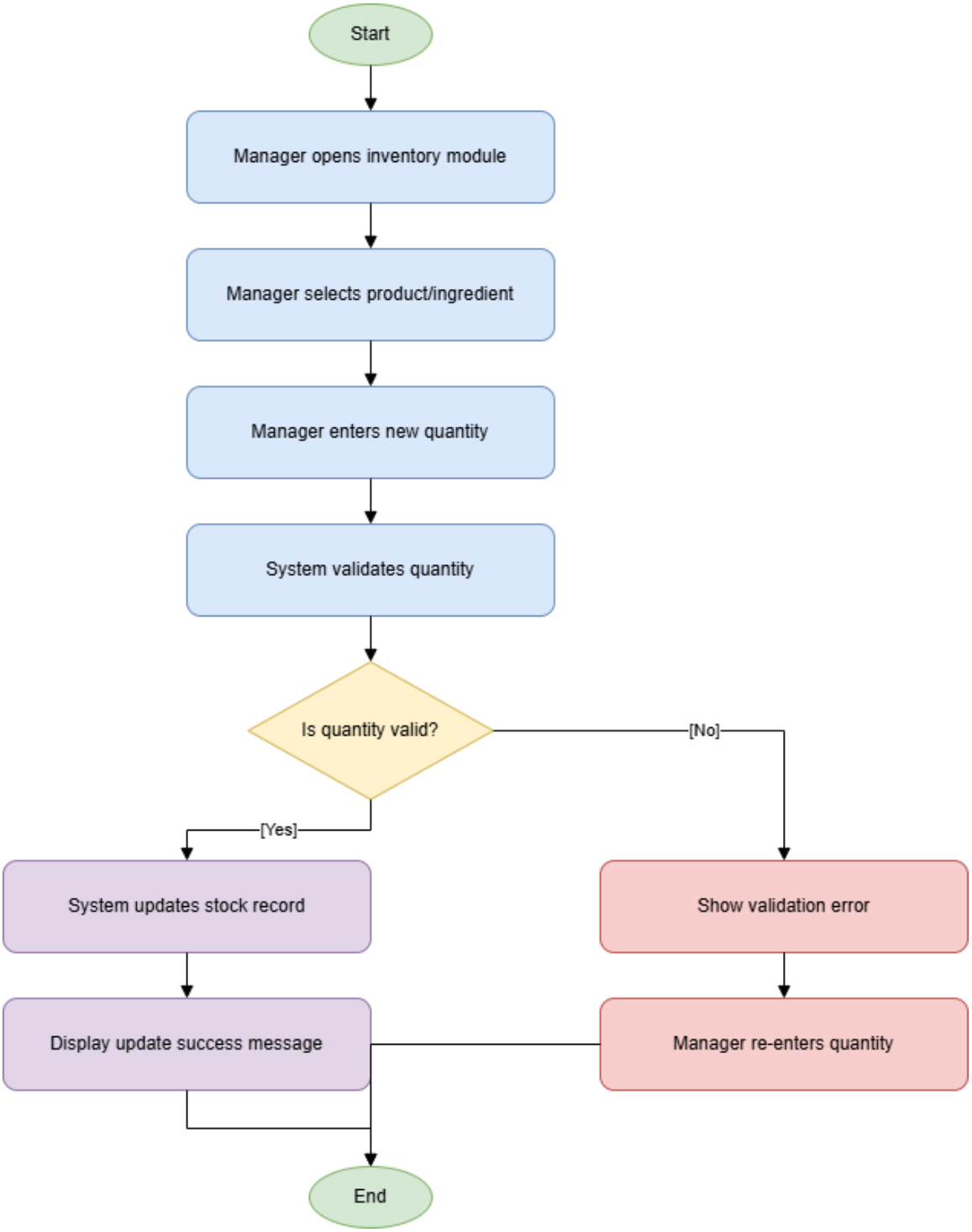


Figure: 15 Manager Updates Stock Manually If Needed

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

16 Staff Selects Traditional Product

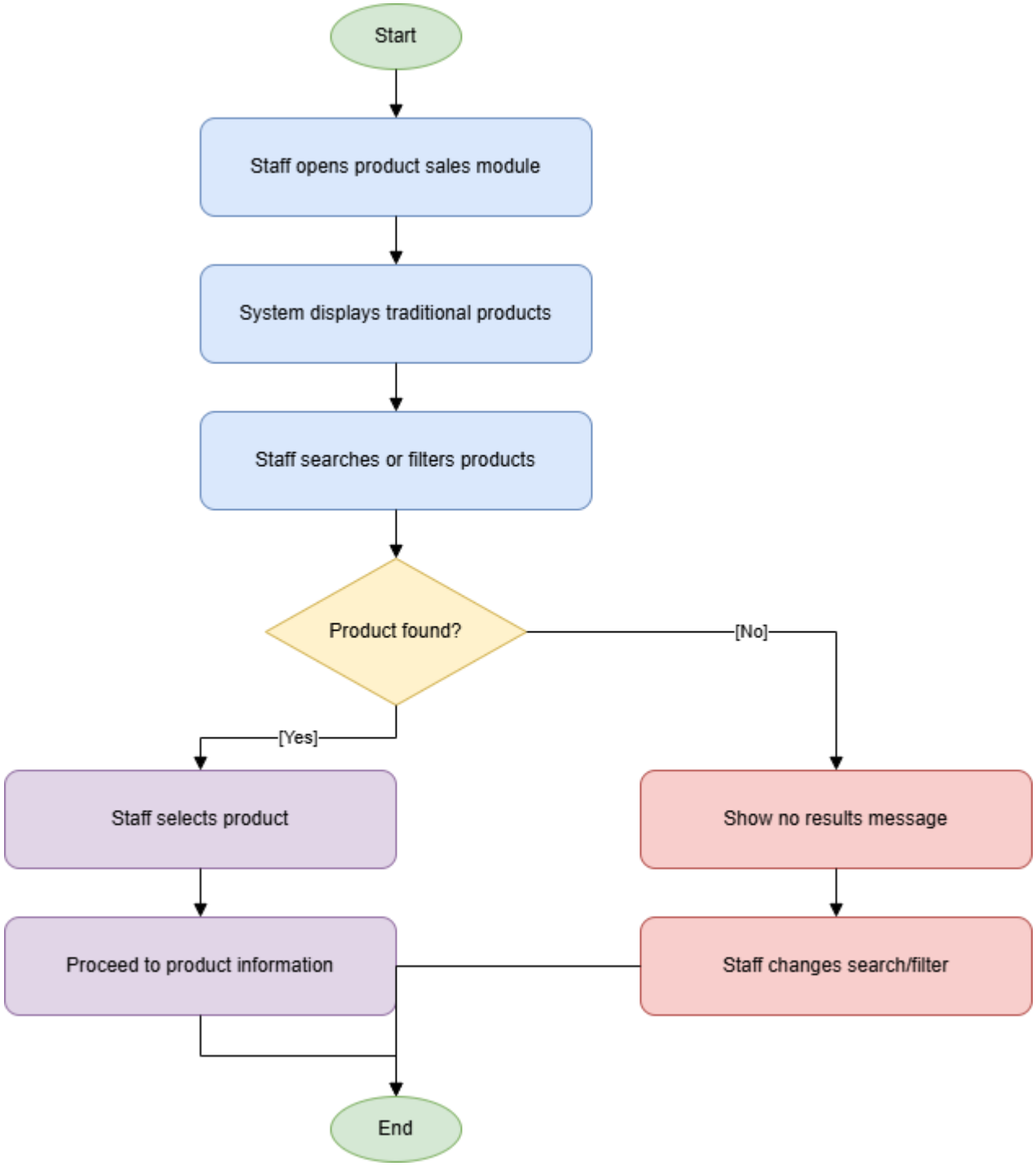


Figure: 16 Staff Selects Traditional Product

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

17 System Shows Product Information

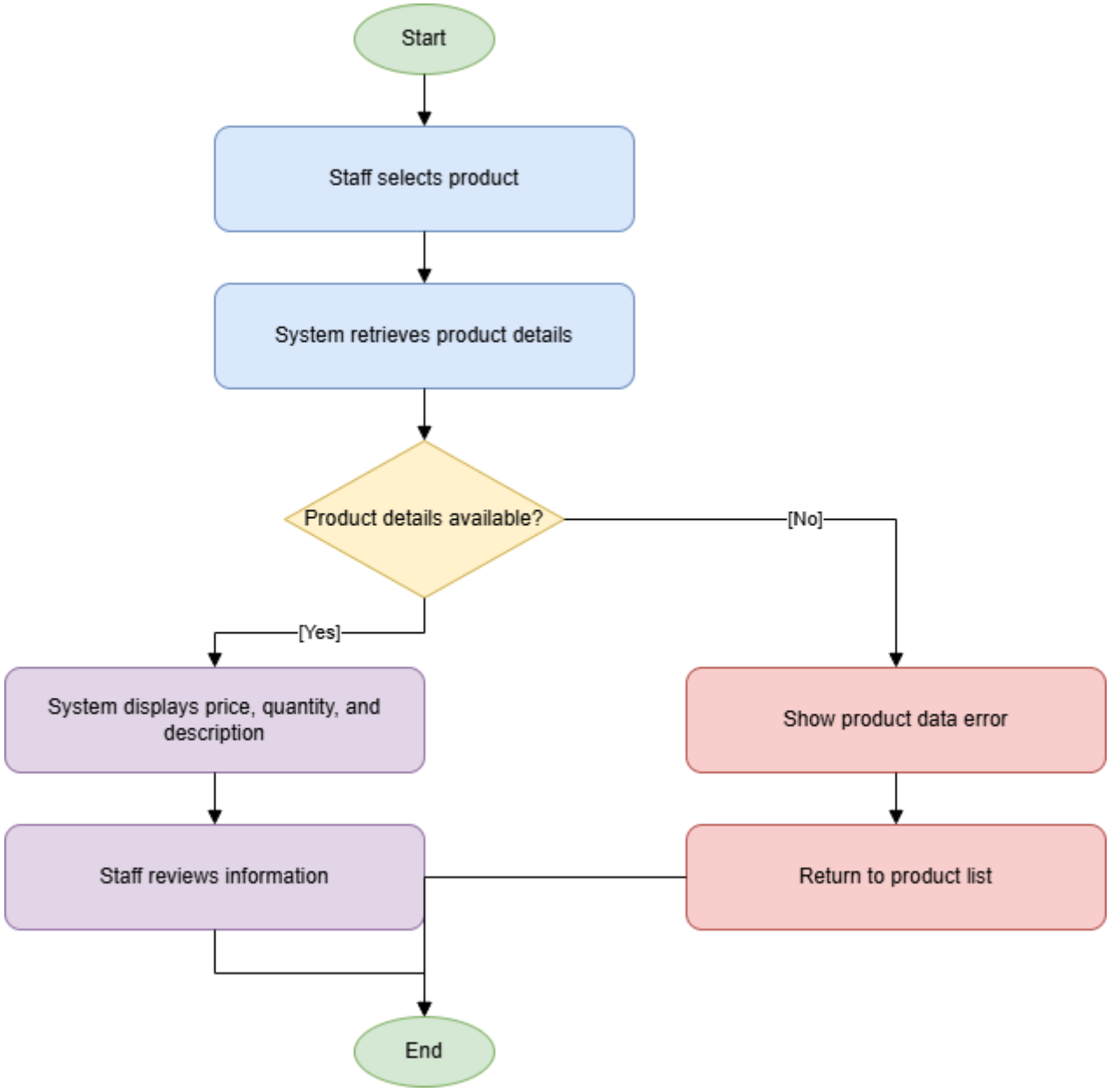


Figure: 17 System Shows Product Information

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

18 Process Sale Transaction

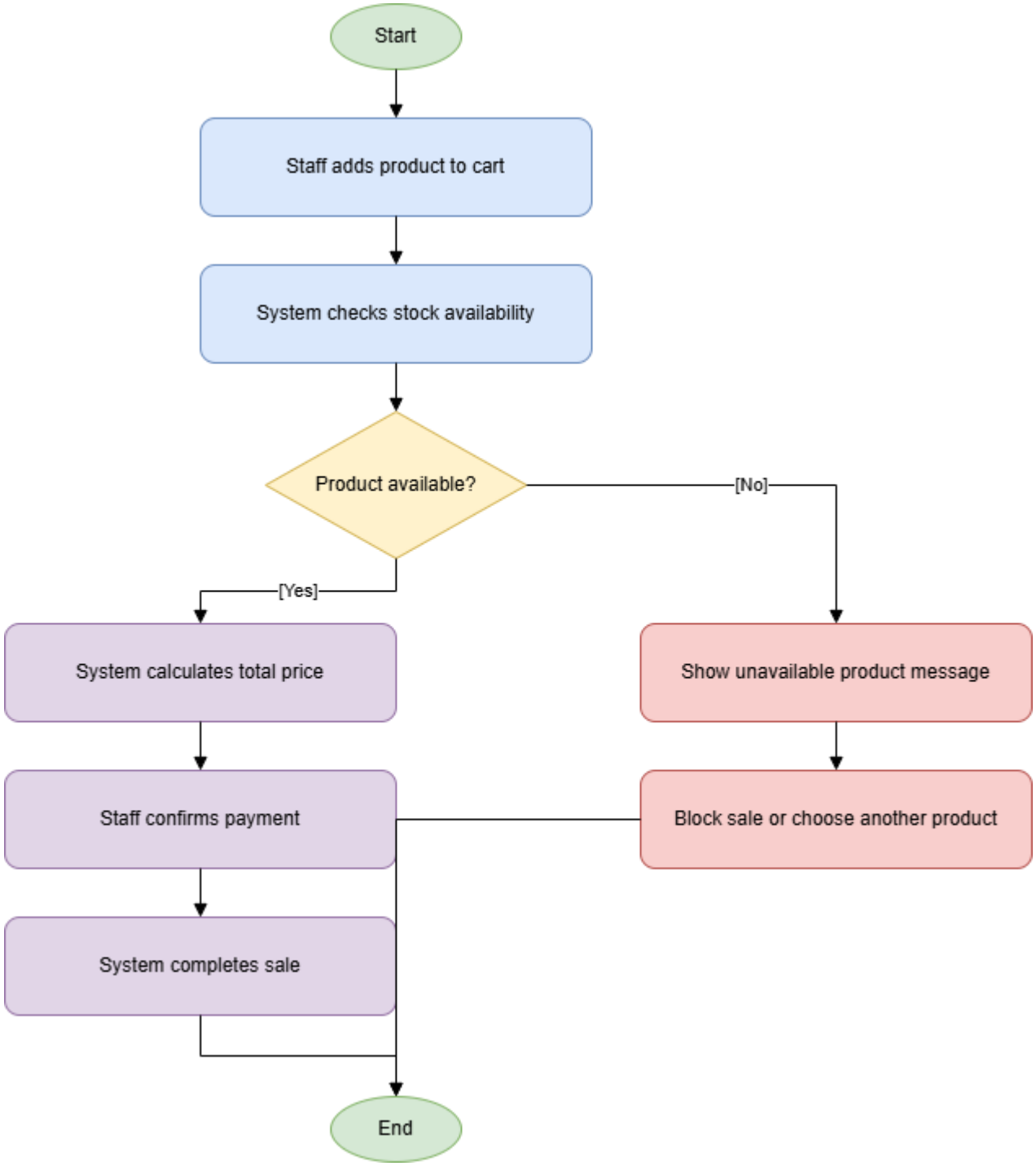


Figure: 18 Process Sale Transaction

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

19 Record Transaction And Update Quantity

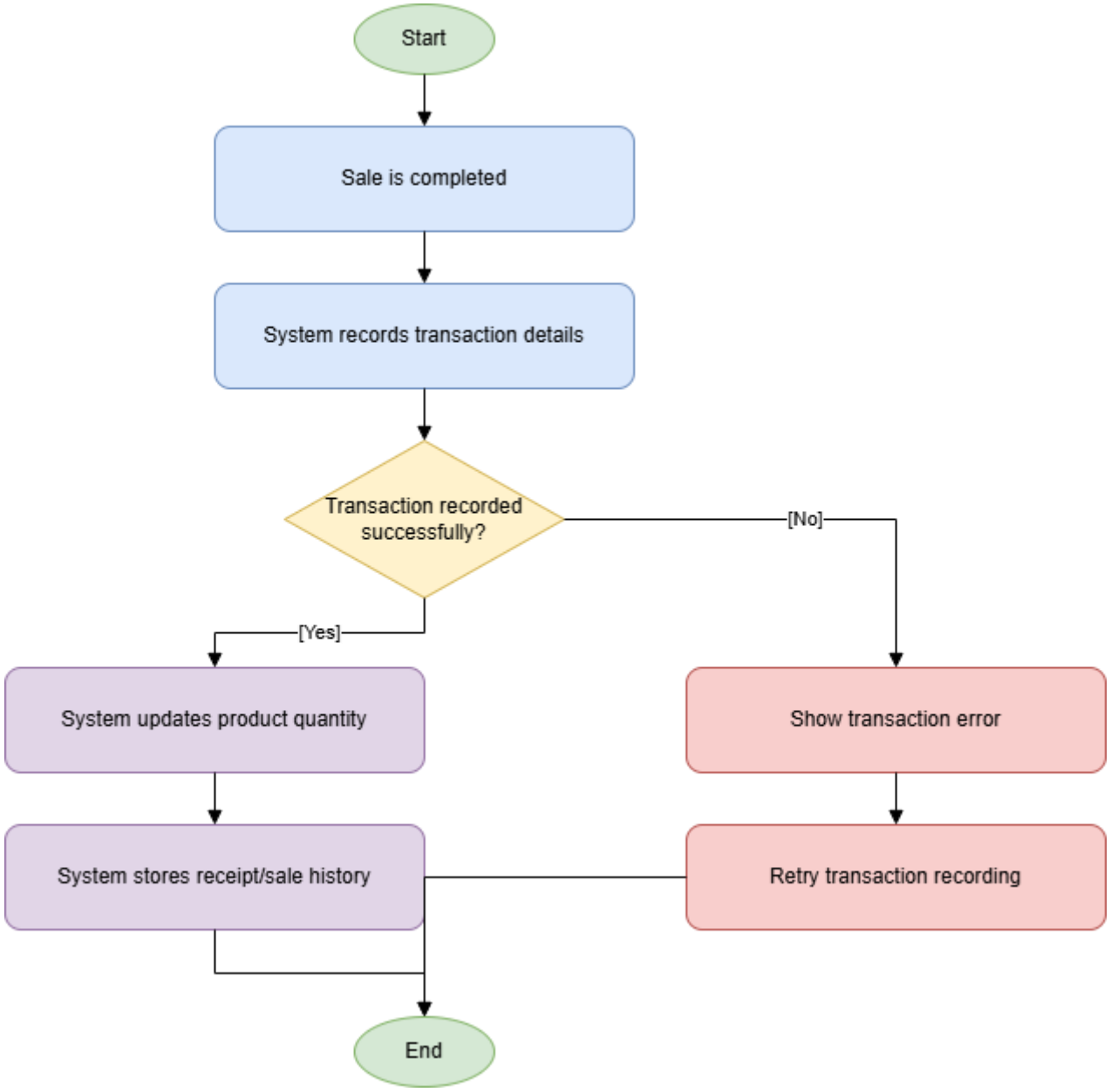


Figure: 19 Record Transaction And Update Quantity

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

21 Collect Sales Reservation And Inventory Data

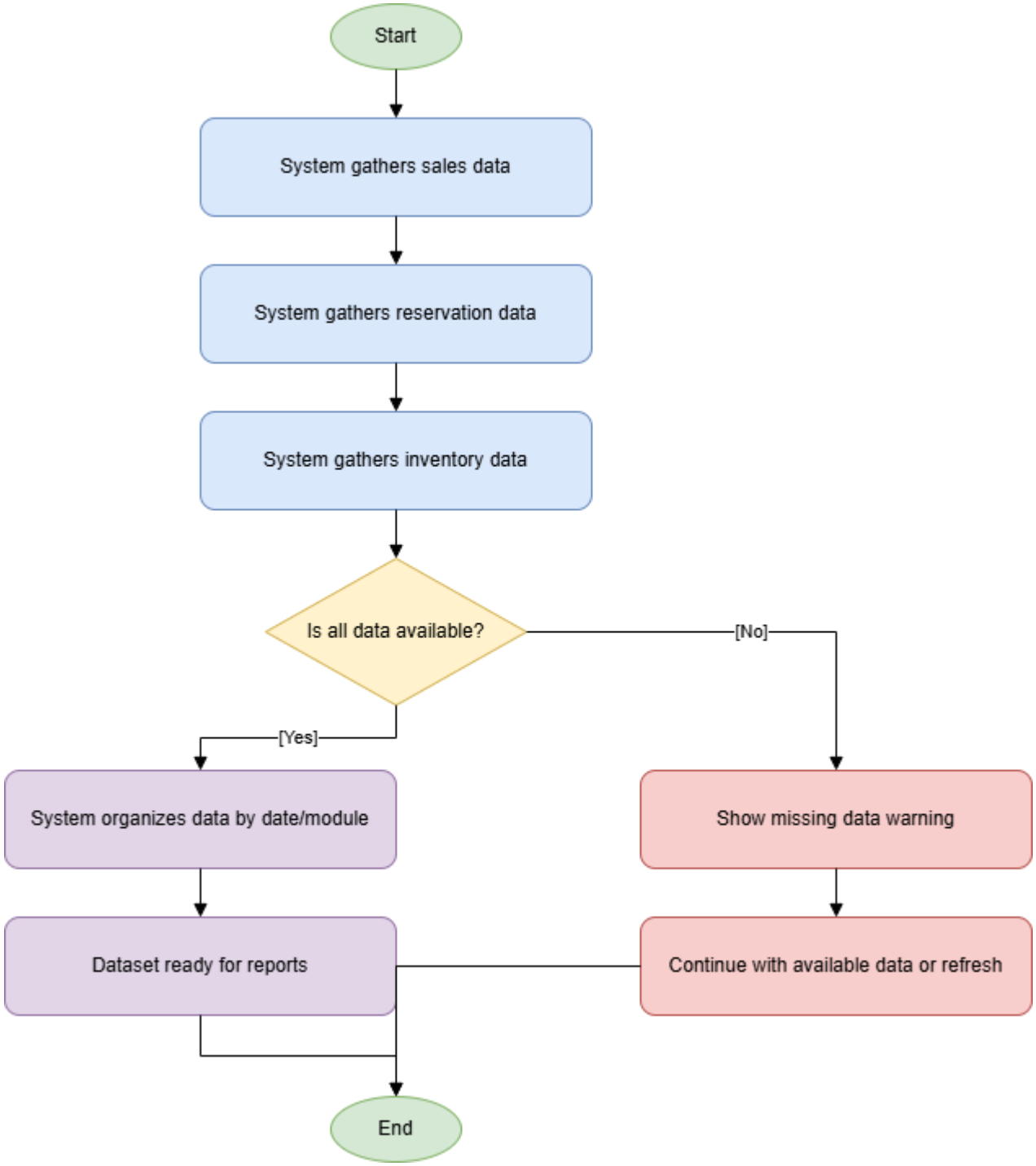


Figure: 21 Collect Sales Reservation And Inventory Data

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

22 Generate Reports And Analytics

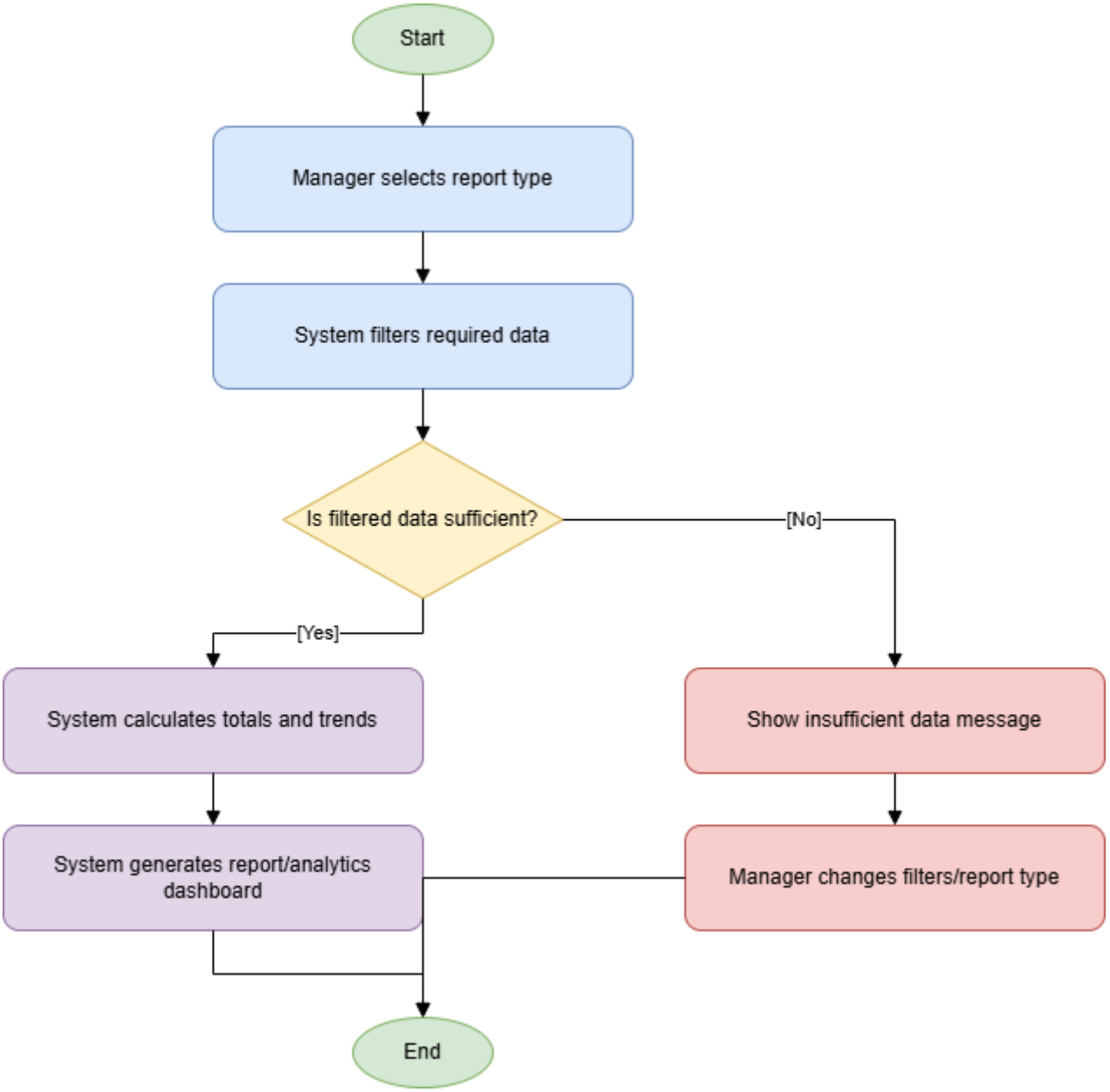


Figure: 22 Generate Reports And Analytics

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

23 Management Reviews System Performance

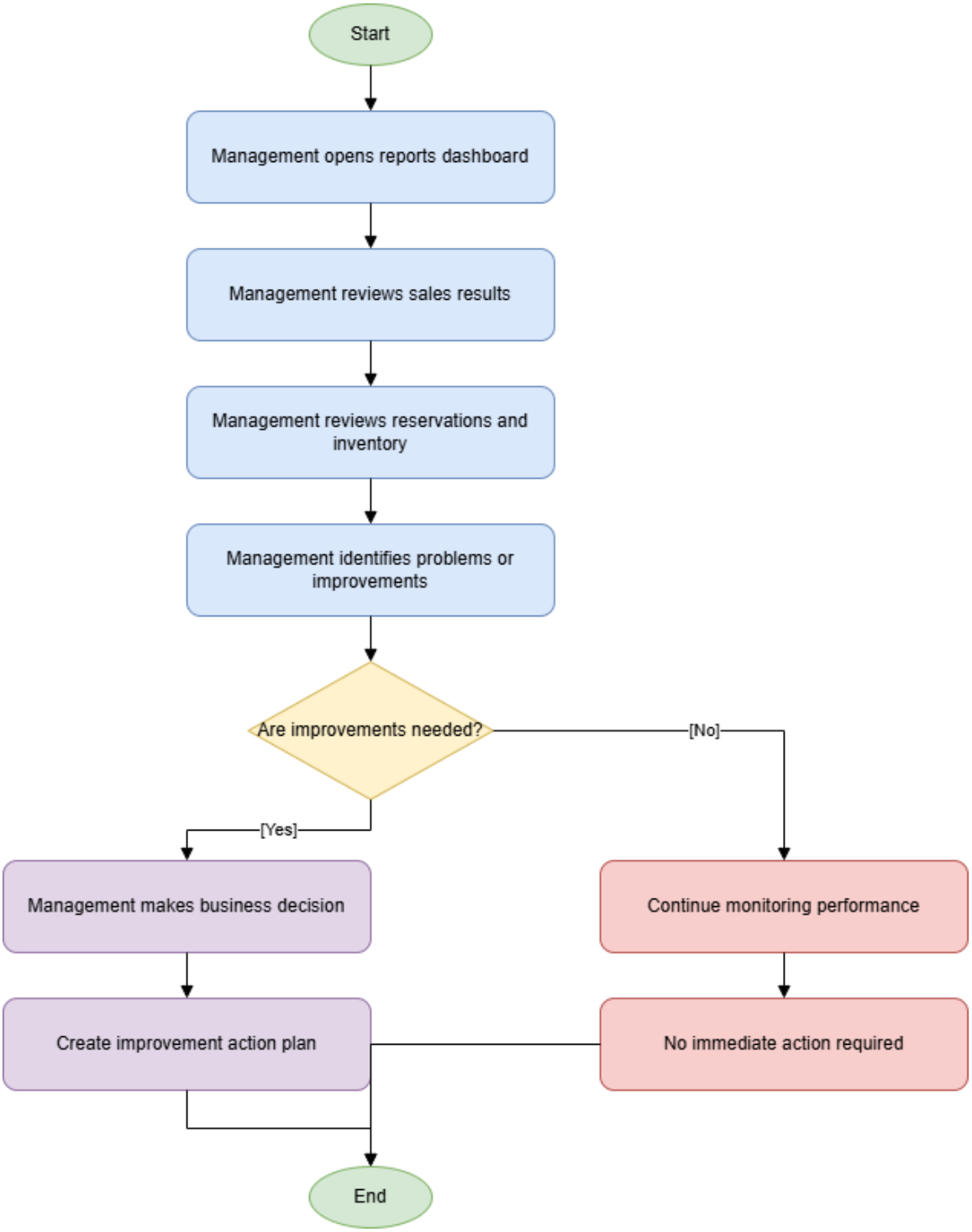


Figure: 23 Management Reviews System Performance

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

Agrisoft 23 Activity Diagrams All Pages

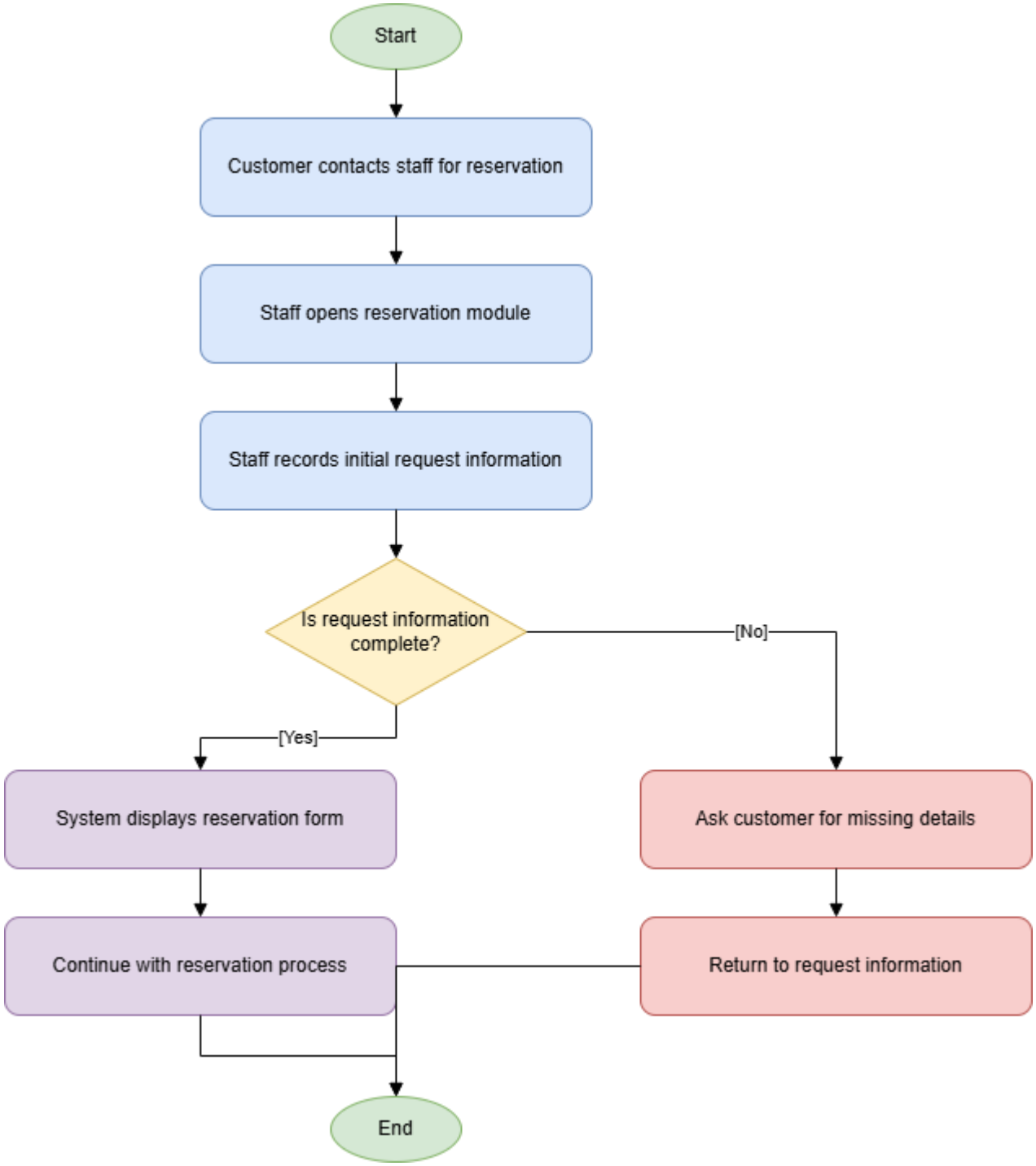


Figure: Agrisoft 23 Activity Diagrams All Pages

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

Inventory Activity Diagram

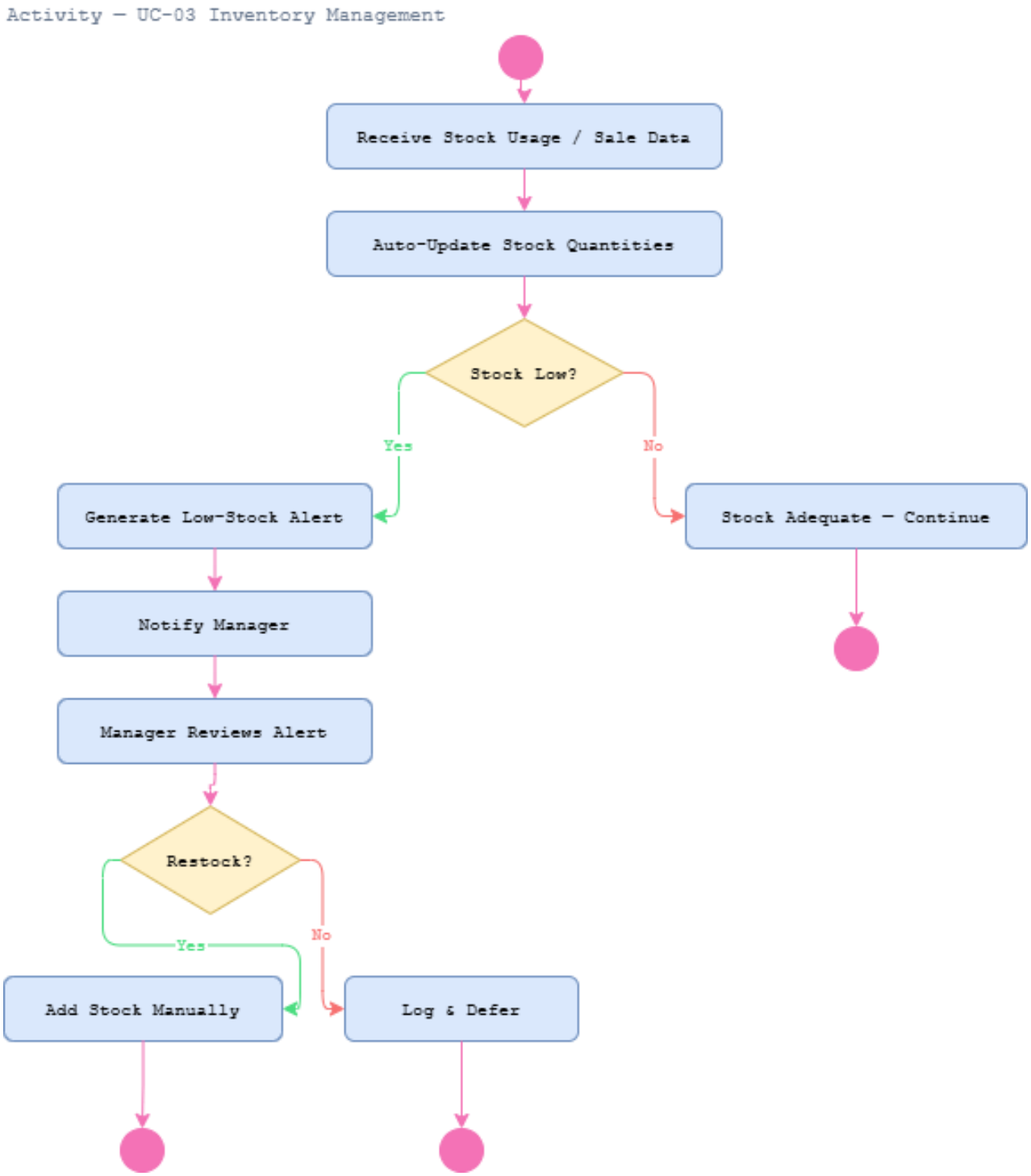


Figure: Inventory Activity Diagram

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

Order Flow Activity Diagram

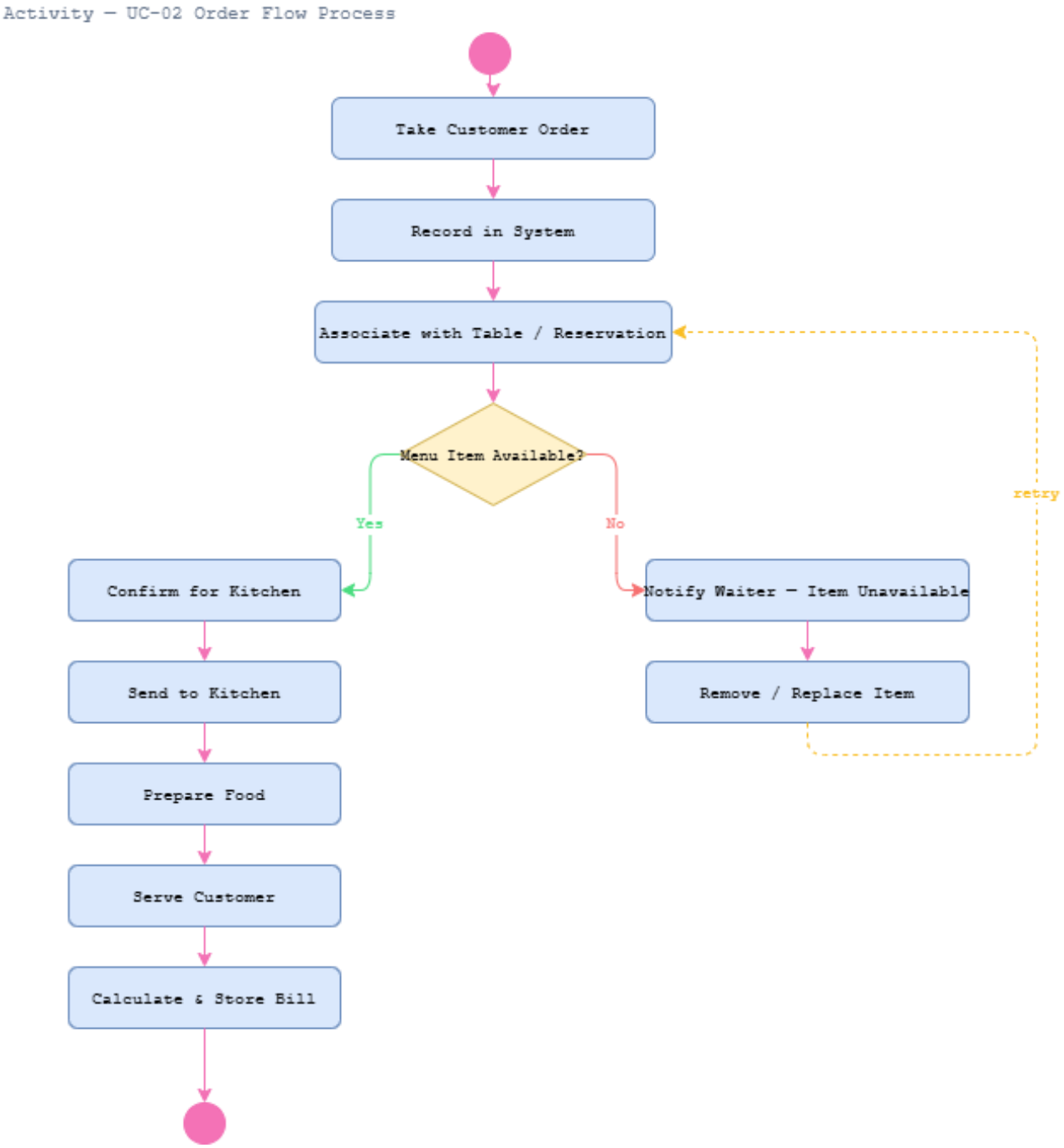


Figure: Order Flow Activity Diagram

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

Product Sale Activity Diagram

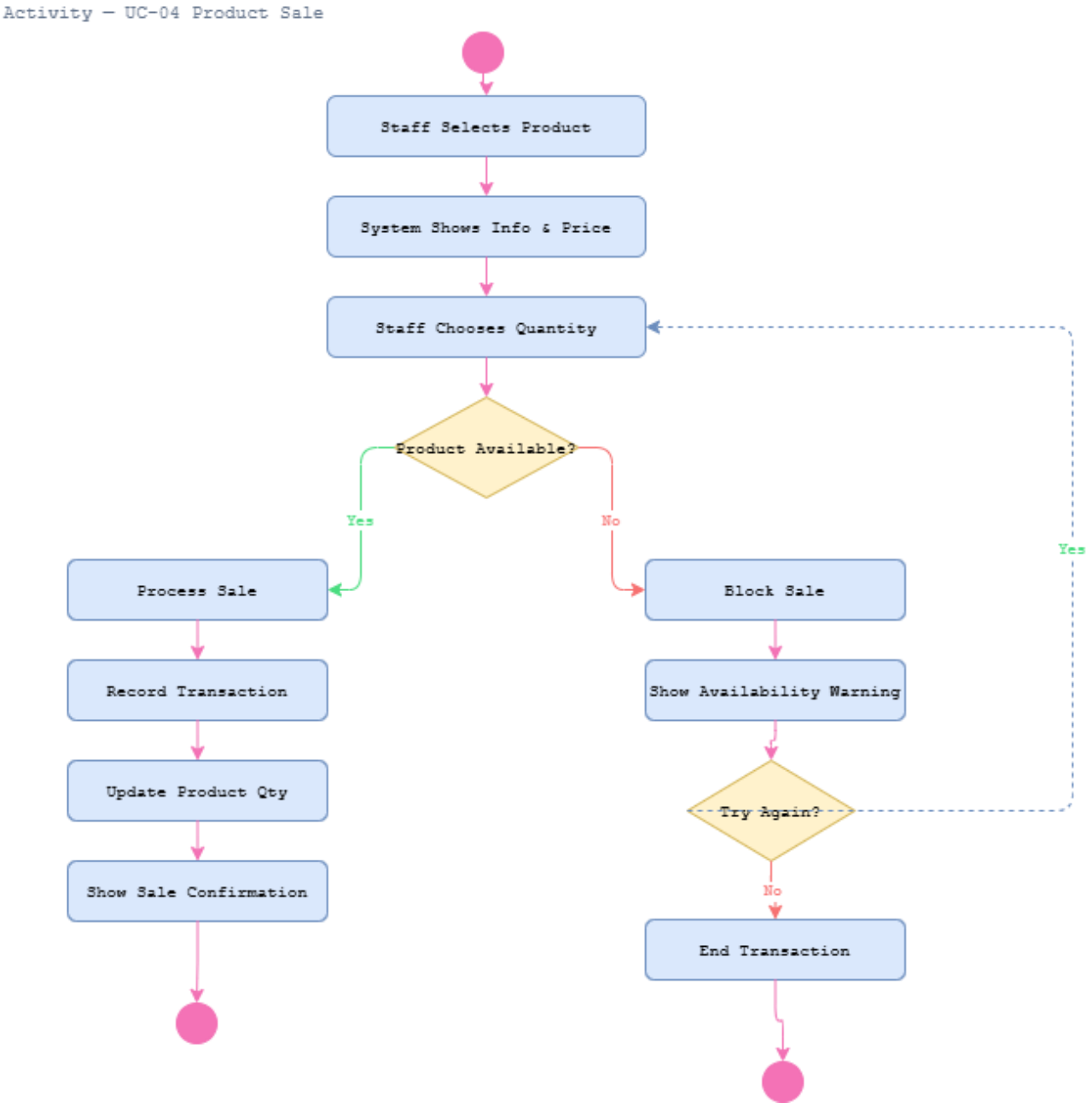


Figure: Product Sale Activity Diagram

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

Reporting Activity Diagram

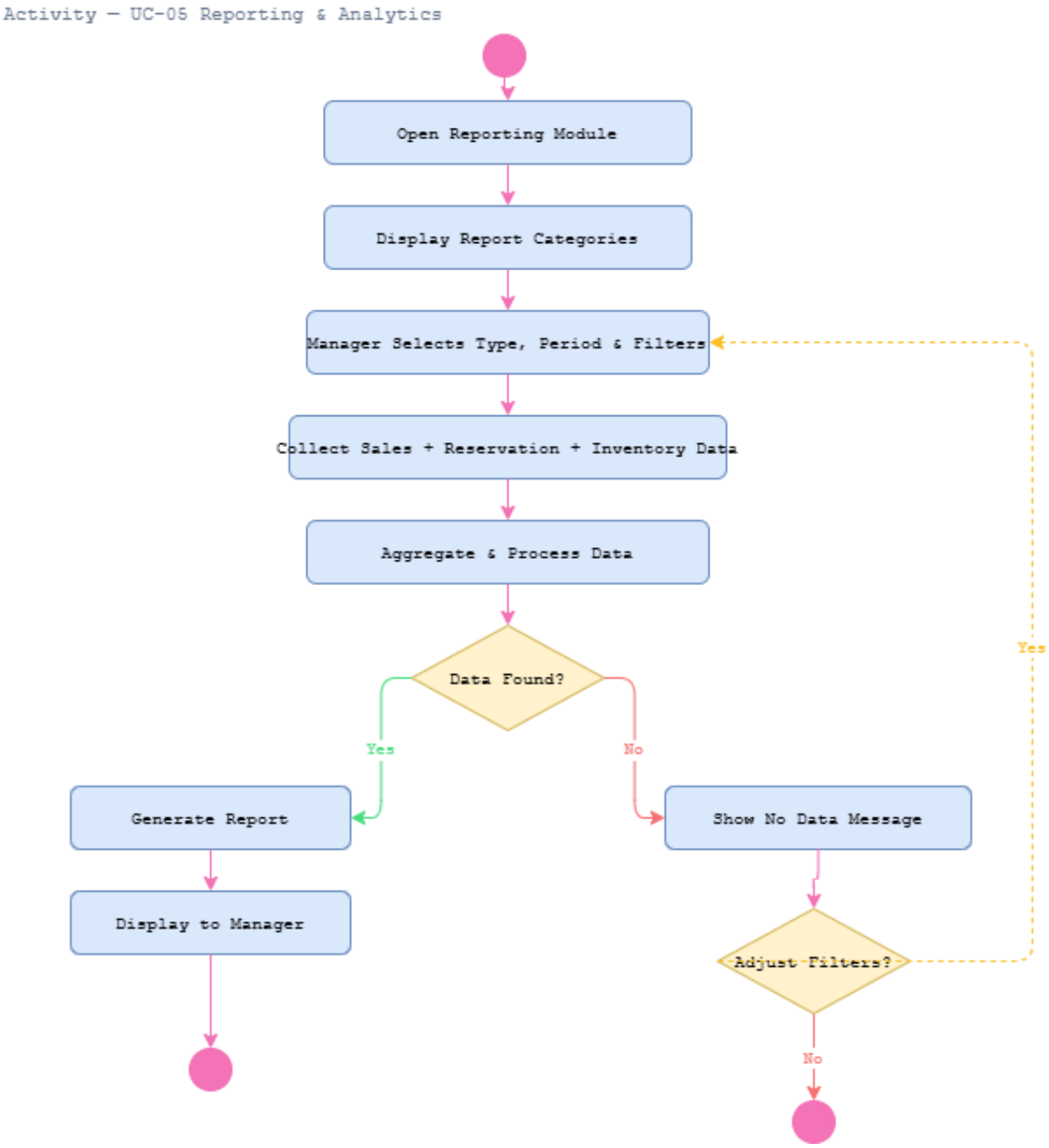


Figure: Reporting Activity Diagram

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

Reservation Activity Diagram

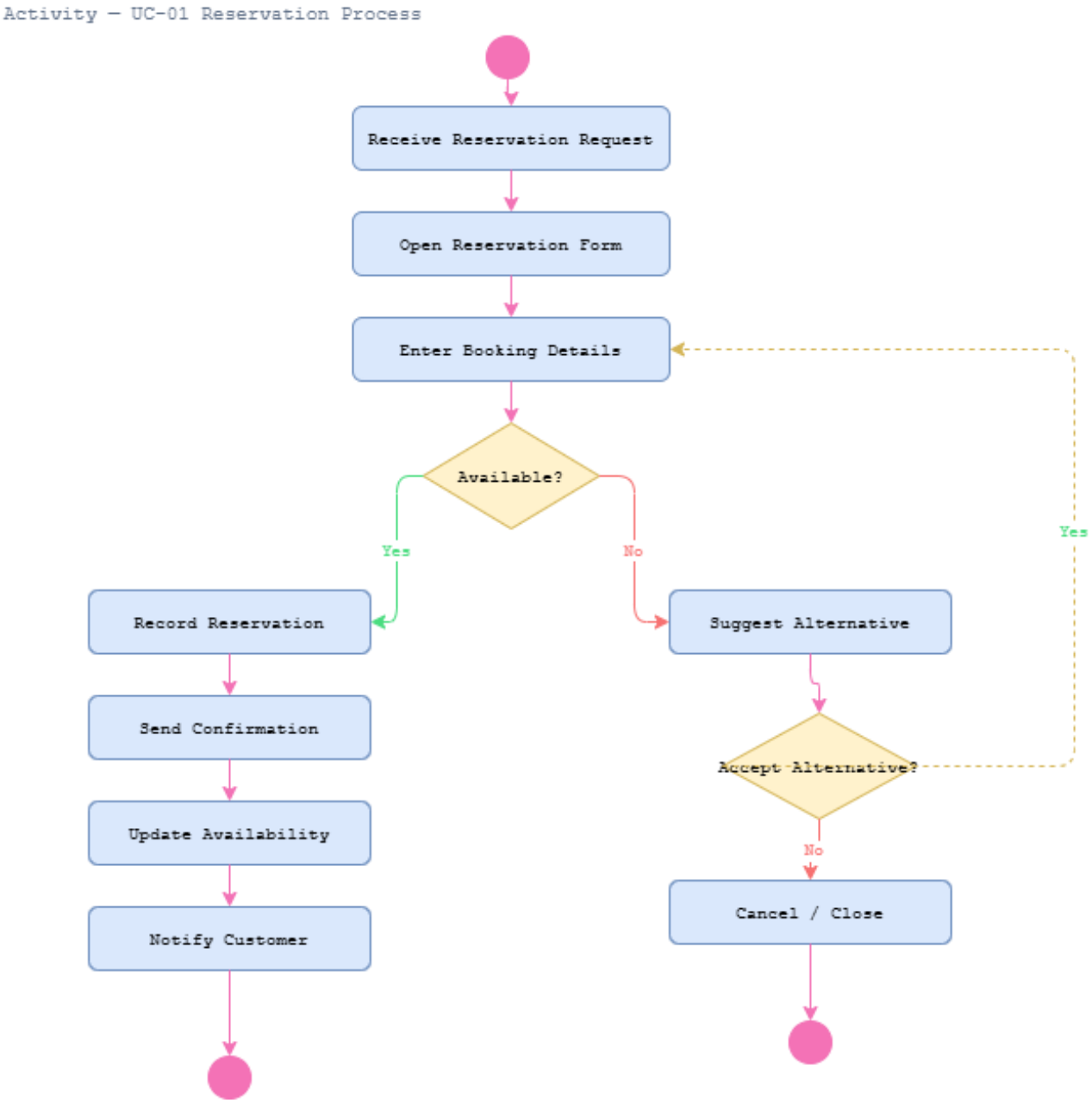


Figure: Reservation Activity Diagram

This activity diagram documents the control flow for one AgriSoft scenario, including actions, validations, and final states.

4.9 State Diagrams

State diagrams describe how important business objects change state during their lifecycle. For AgriSoft, important states include reservation status, order processing status, inventory stock status, product sale availability, and report generation status.

Inventory State Diagram

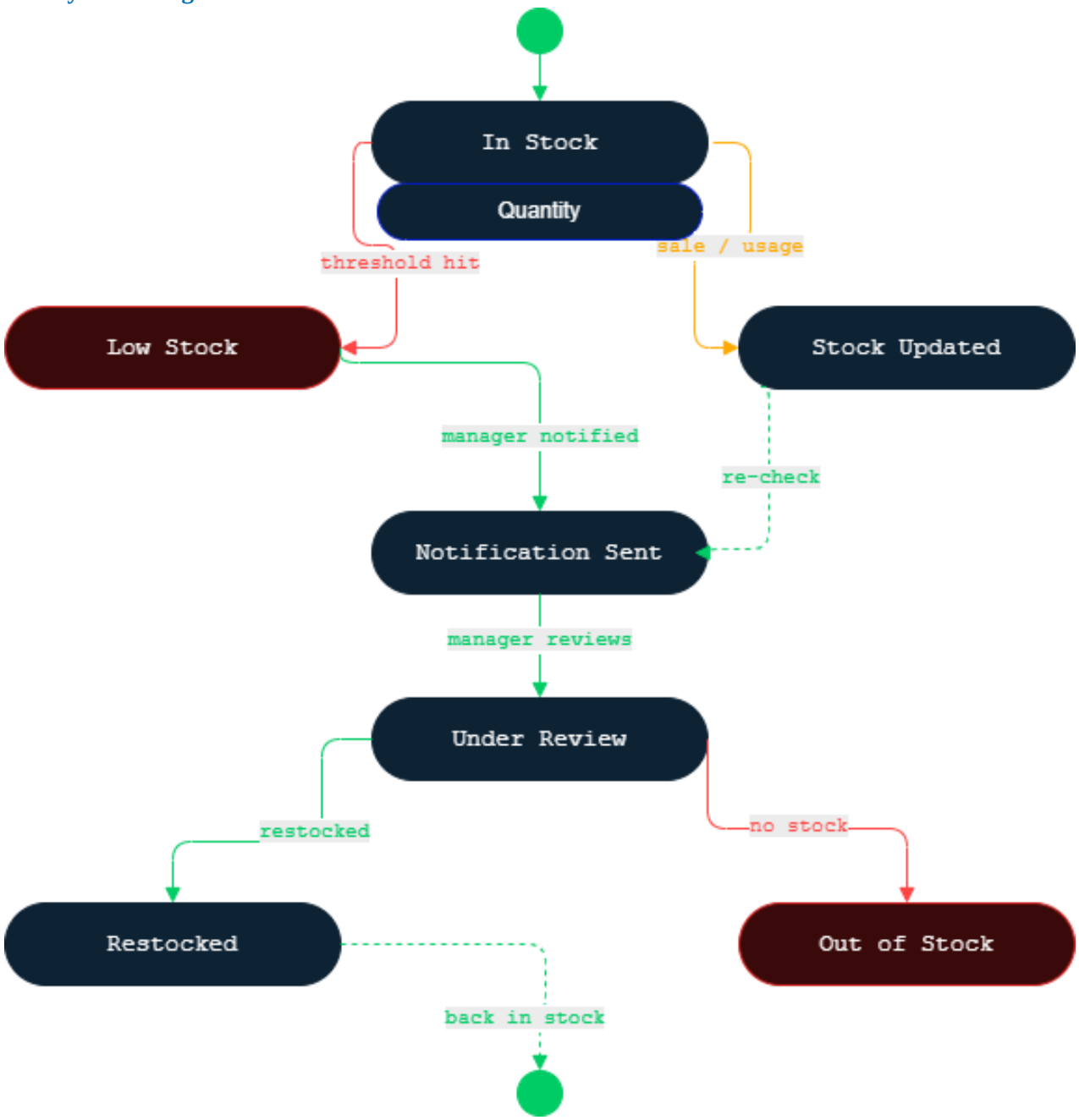


Figure: Inventory State Diagram

This state diagram shows lifecycle transitions for a main AgriSoft object or process.

Order State Diagram Page 16

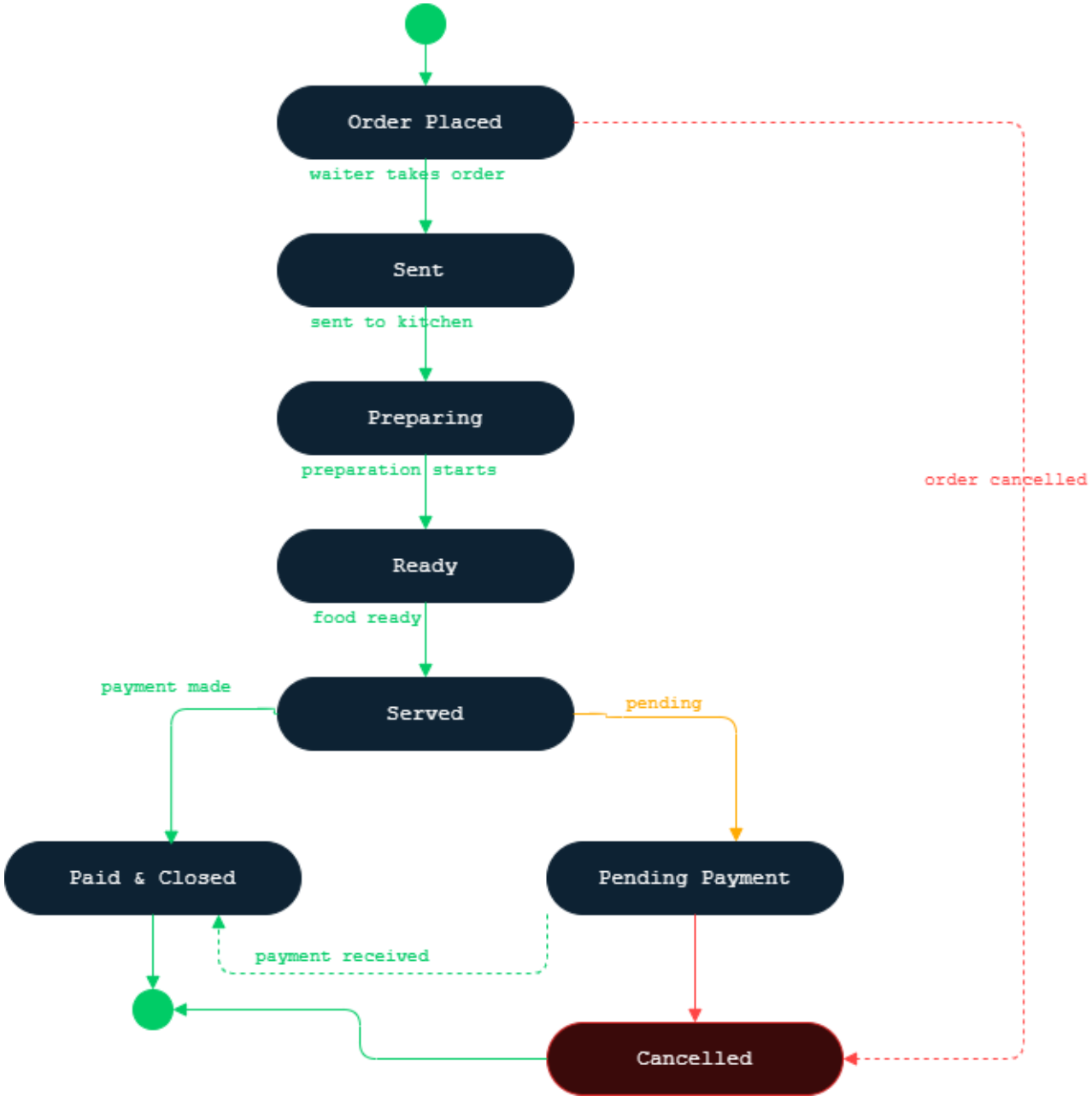


Figure: Order State Diagram Page 16

This state diagram shows lifecycle transitions for a main AgriSoft object or process.

Product Sale State Diagram

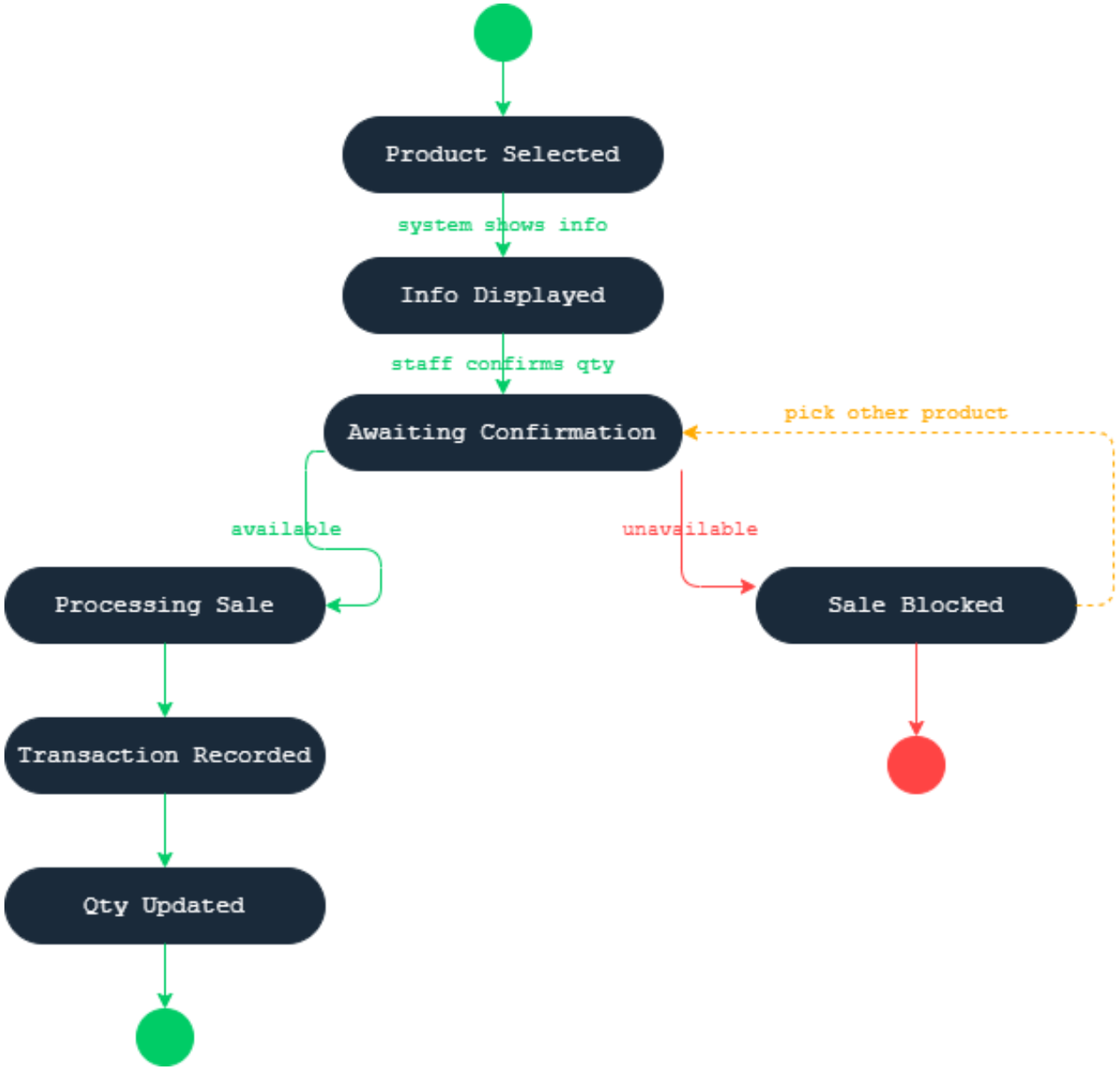


Figure: Product Sale State Diagram

This state diagram shows lifecycle transitions for a main AgriSoft object or process.

Report State Diagram

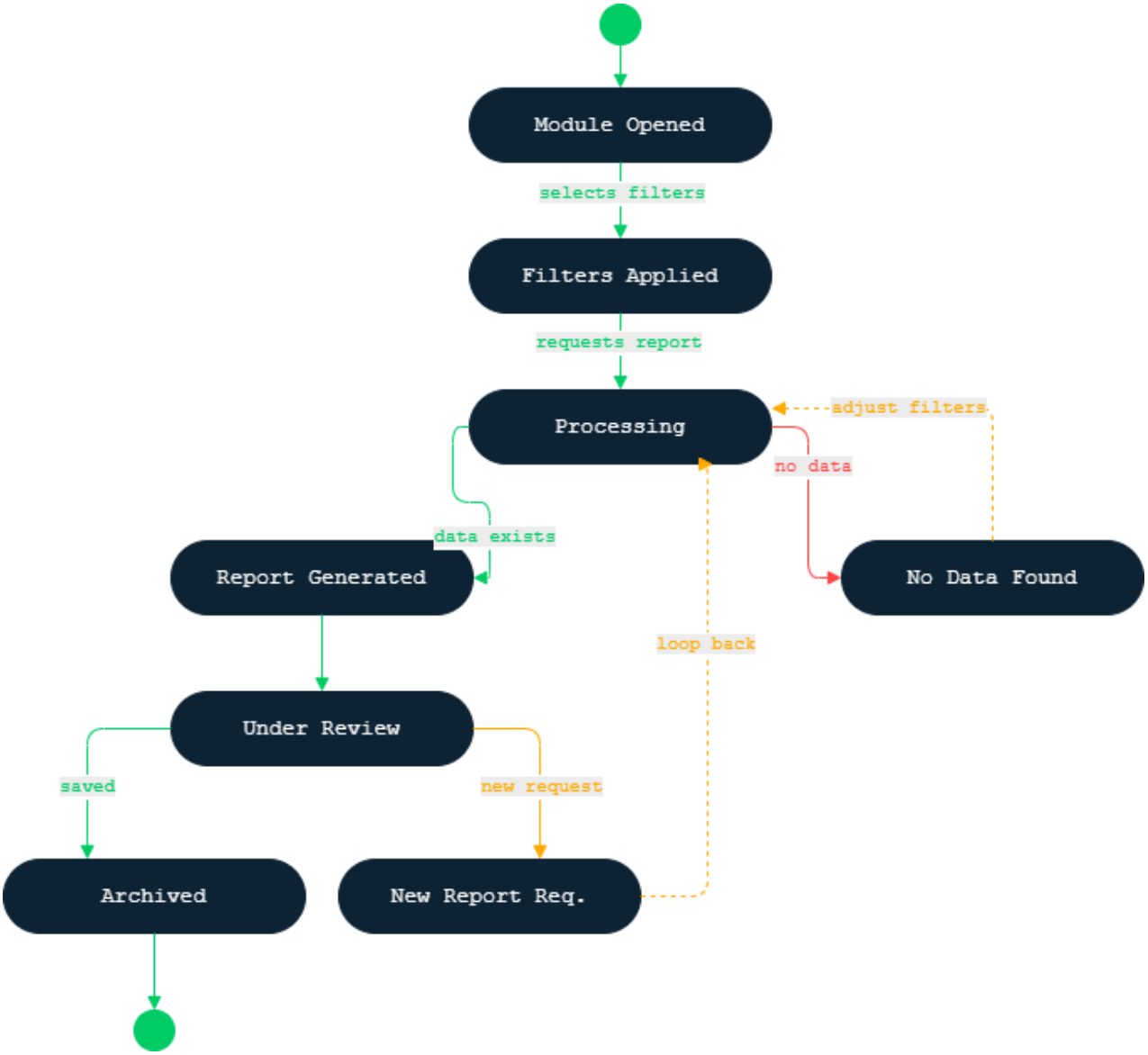


Figure: Report State Diagram

This state diagram shows lifecycle transitions for a main AgriSoft object or process.

Reservation State Diagram

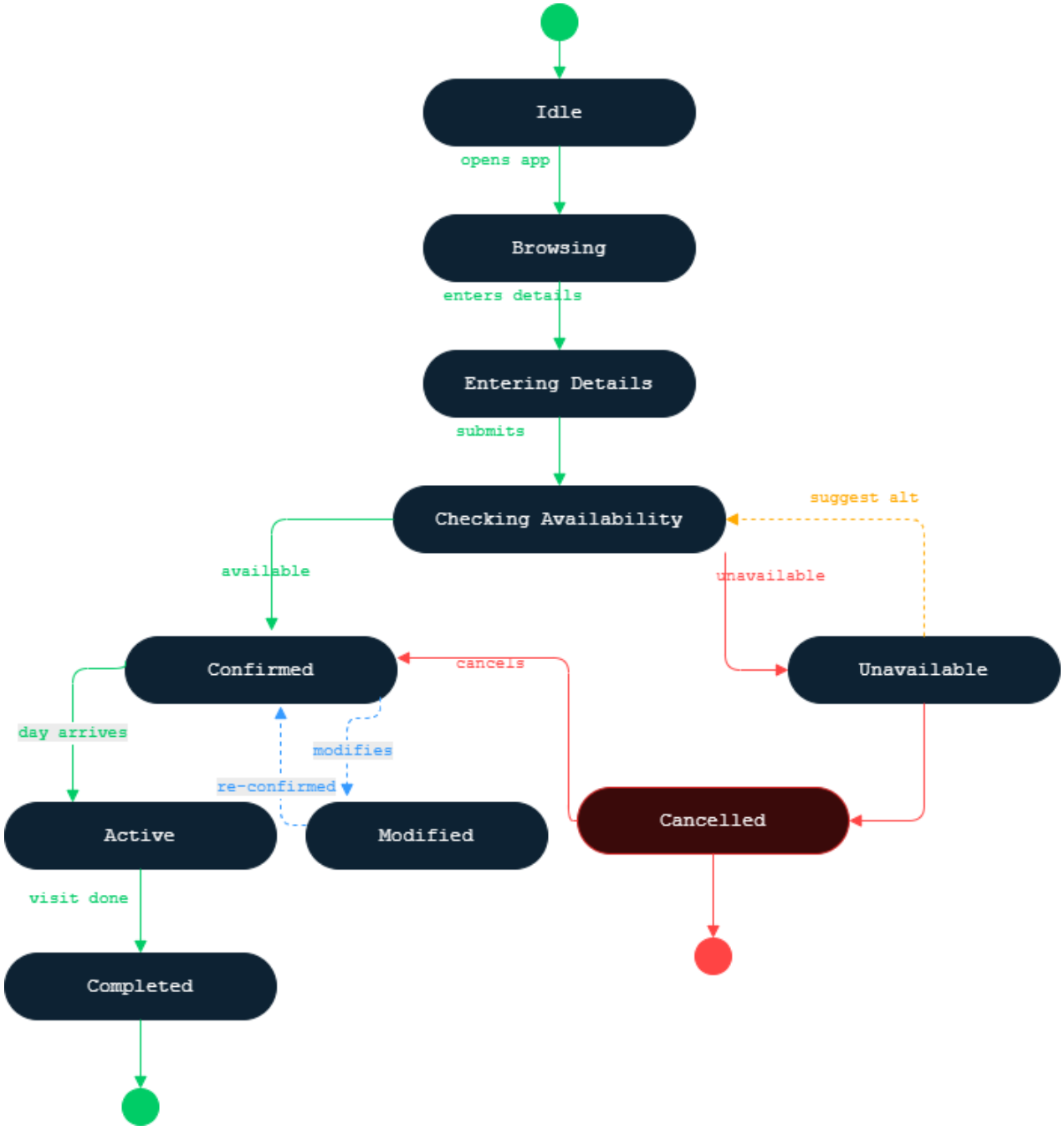


Figure: Reservation State Diagram

This state diagram shows lifecycle transitions for a main AgriSoft object or process.

4.10 Sequence Diagrams

Sequence diagrams represent time-ordered communication between actors, interface components, system services, and data objects. They are used to demonstrate how an action is processed from the user interface to business logic and data update.

Availabilitycheck

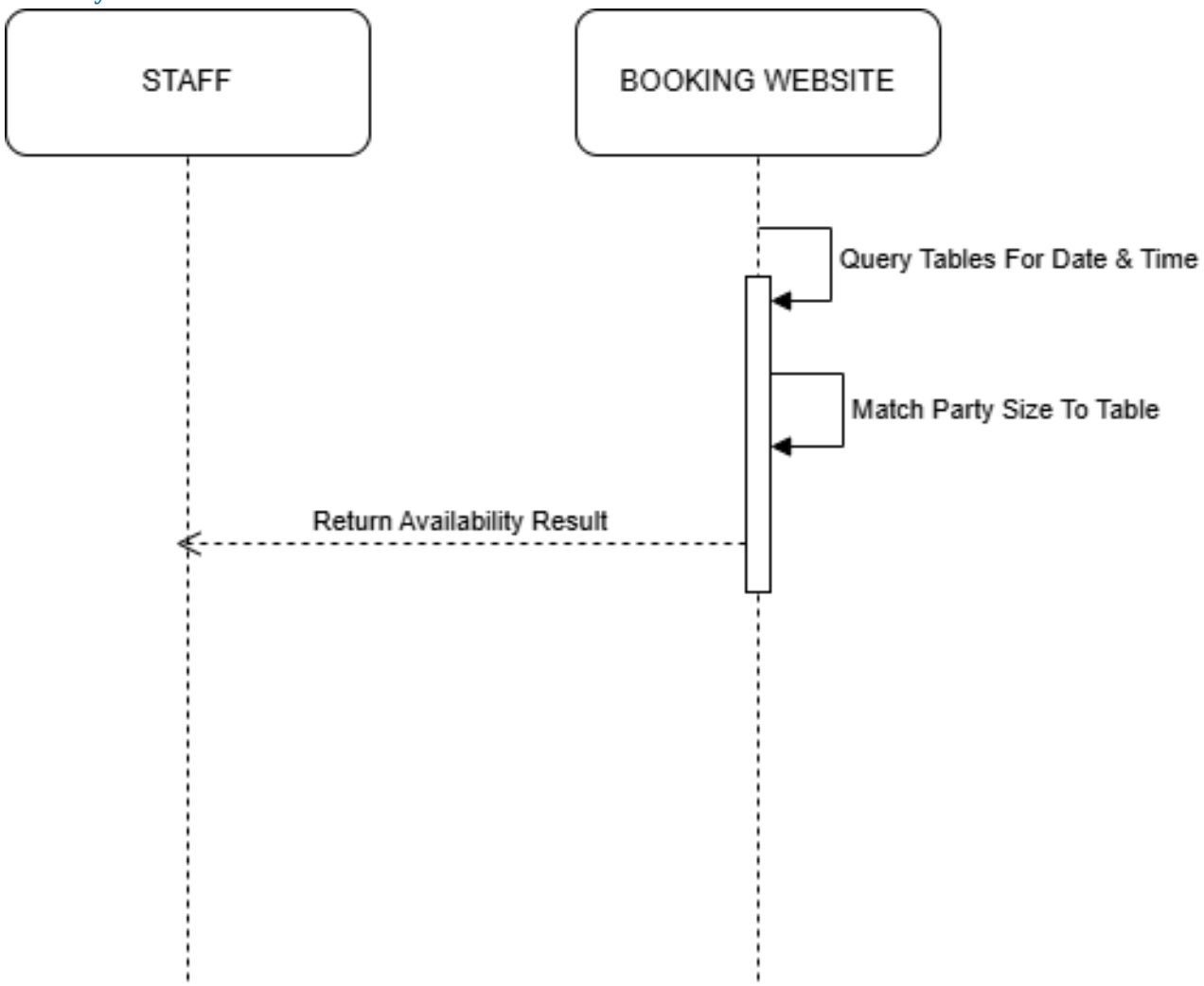


Figure: Availabilitycheck

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Blockifunavailable

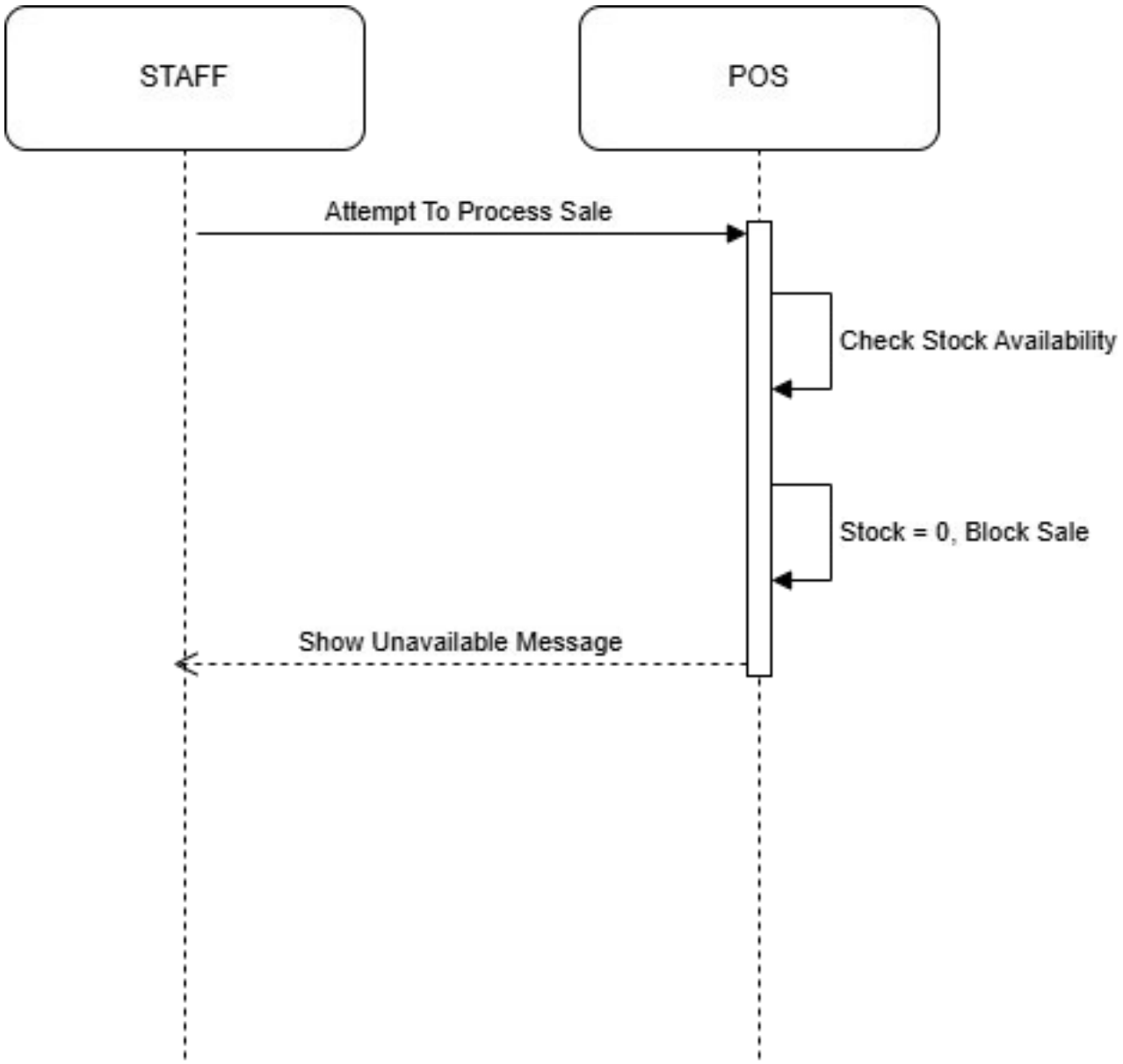


Figure: Blockifunavailable

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Checkstock

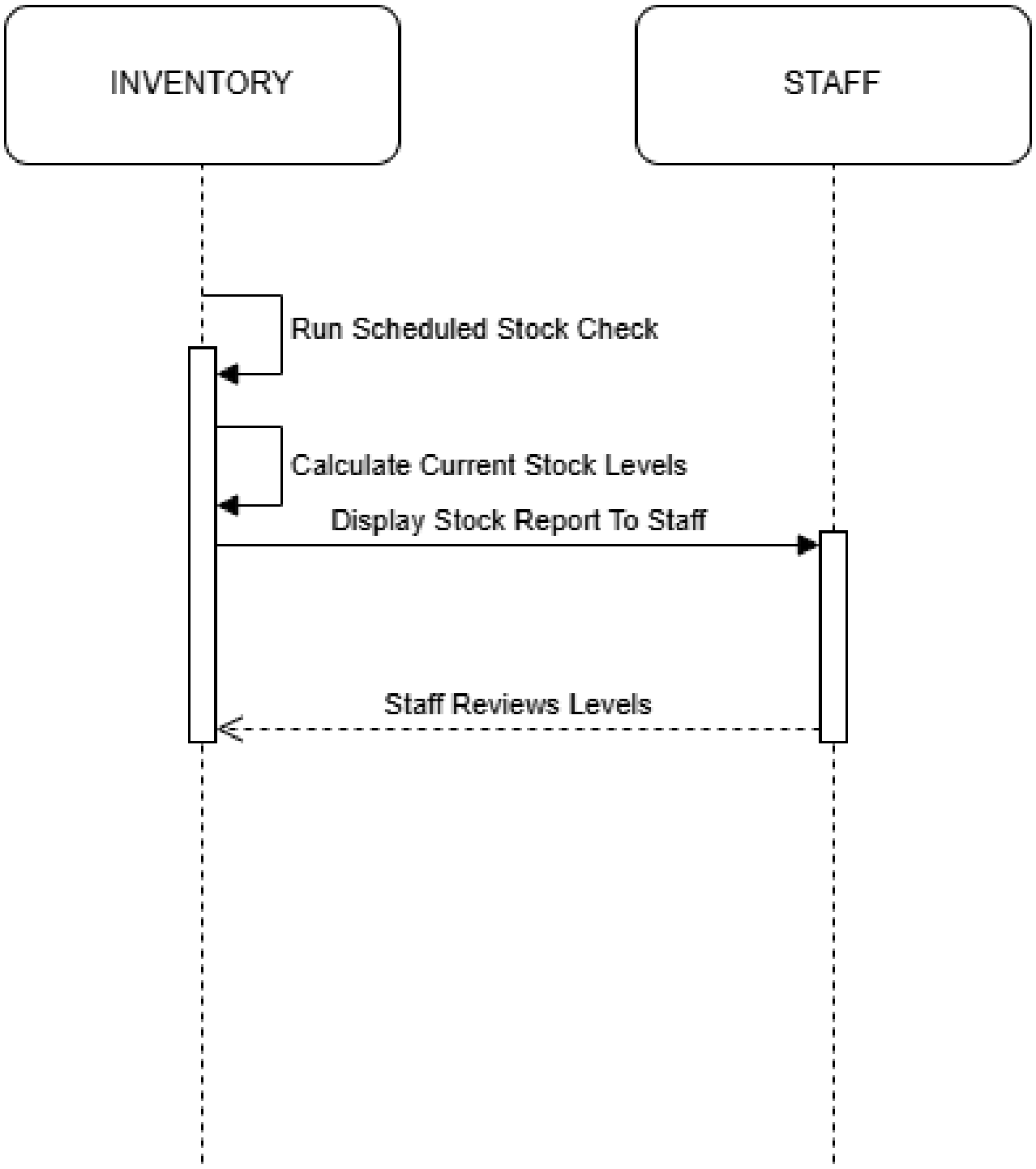


Figure: Checkstock

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Collectdata

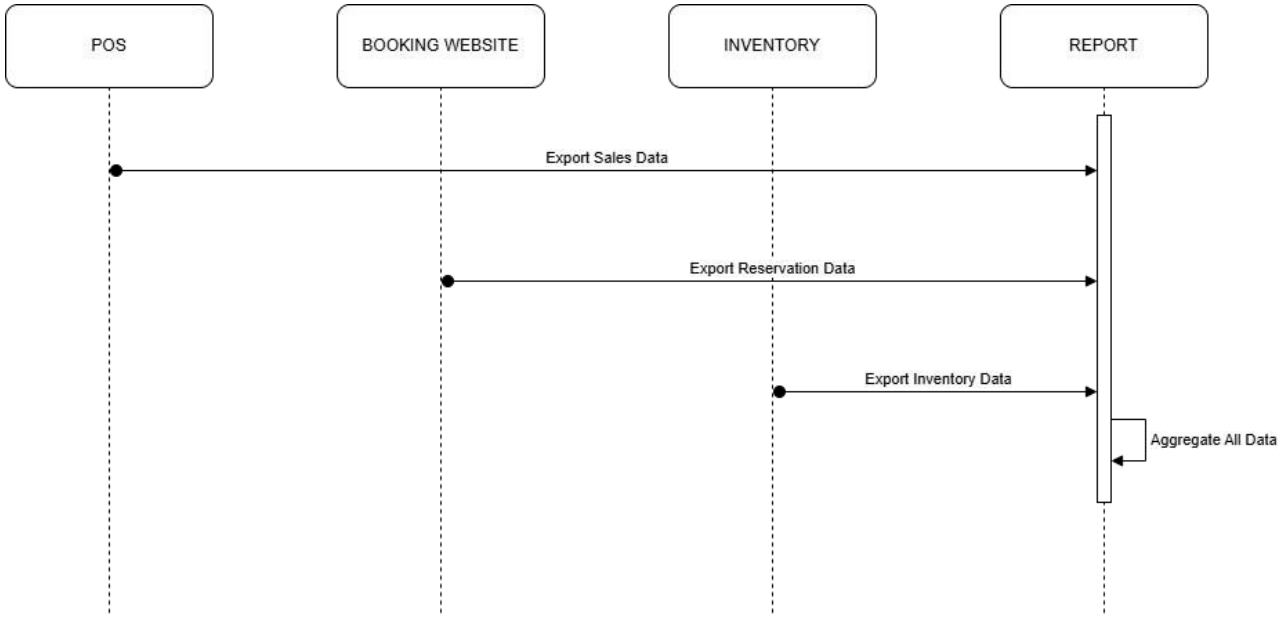


Figure: Collectdata

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Confirmationofreservation

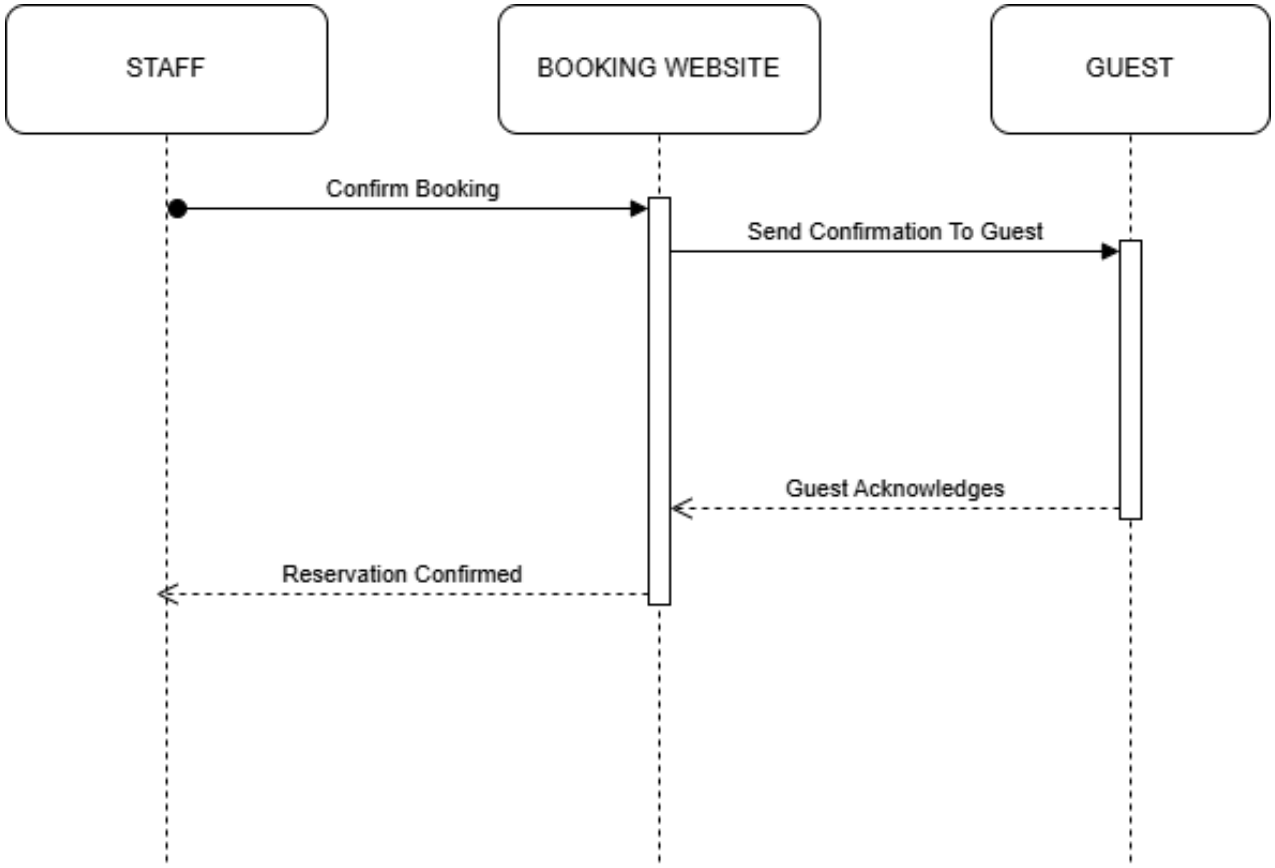


Figure: Confirmationofreservation

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Enterdetails

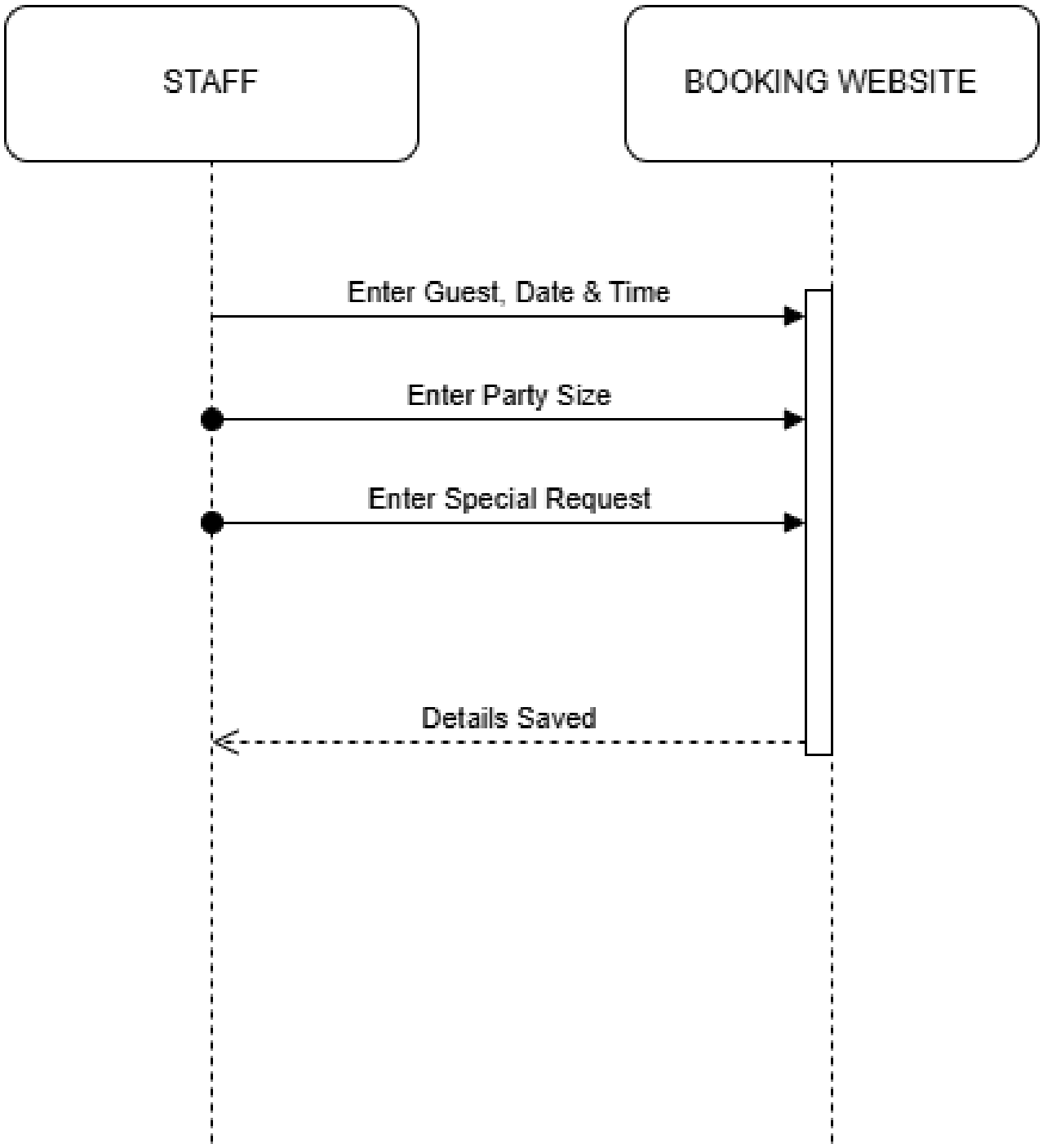


Figure: Enterdetails

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Generatereports

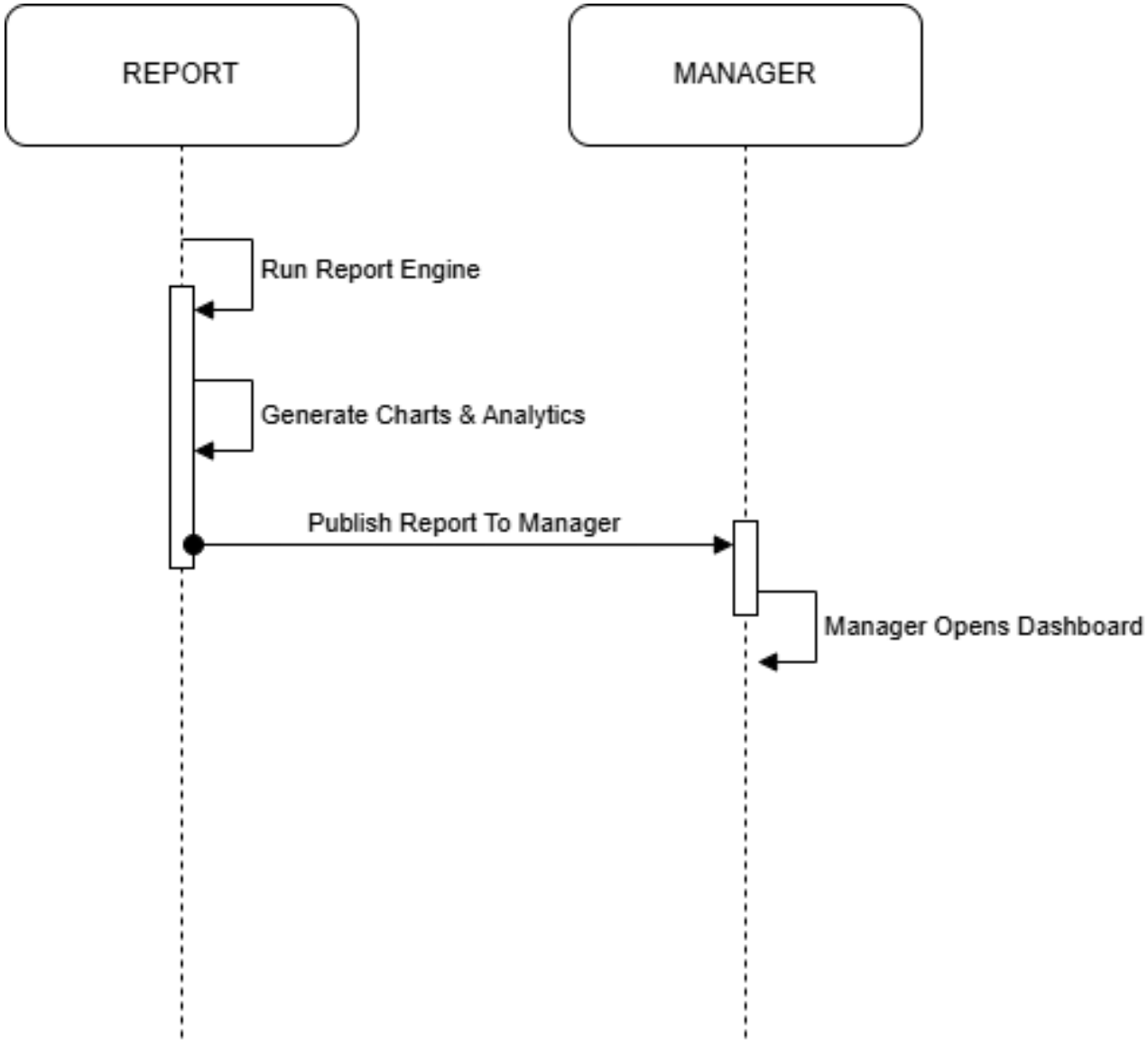


Figure: Generatereports

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Managerupdates

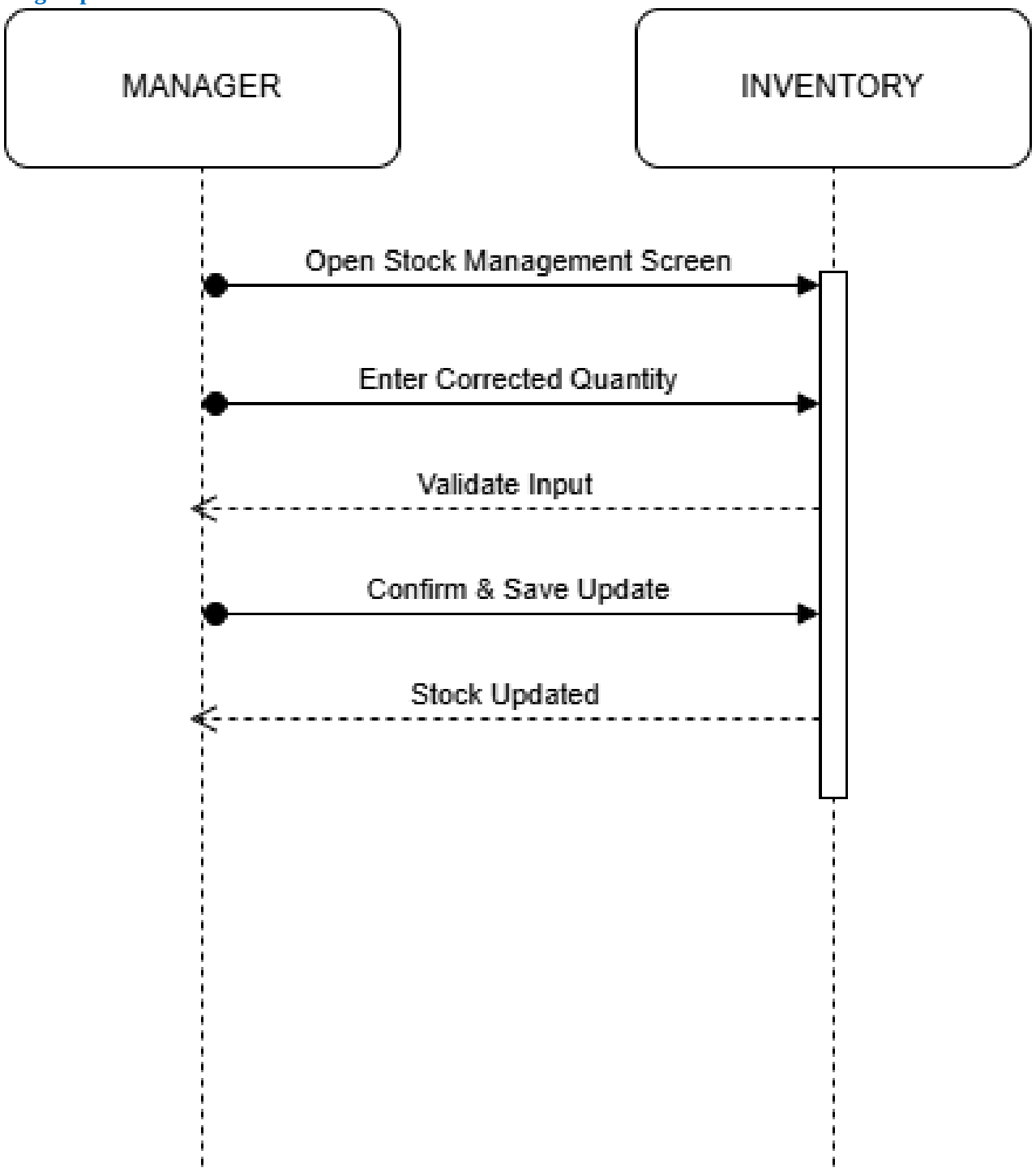


Figure: Managerupdates

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Modifycancel

Figure: Modifycancel

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Notifymanager

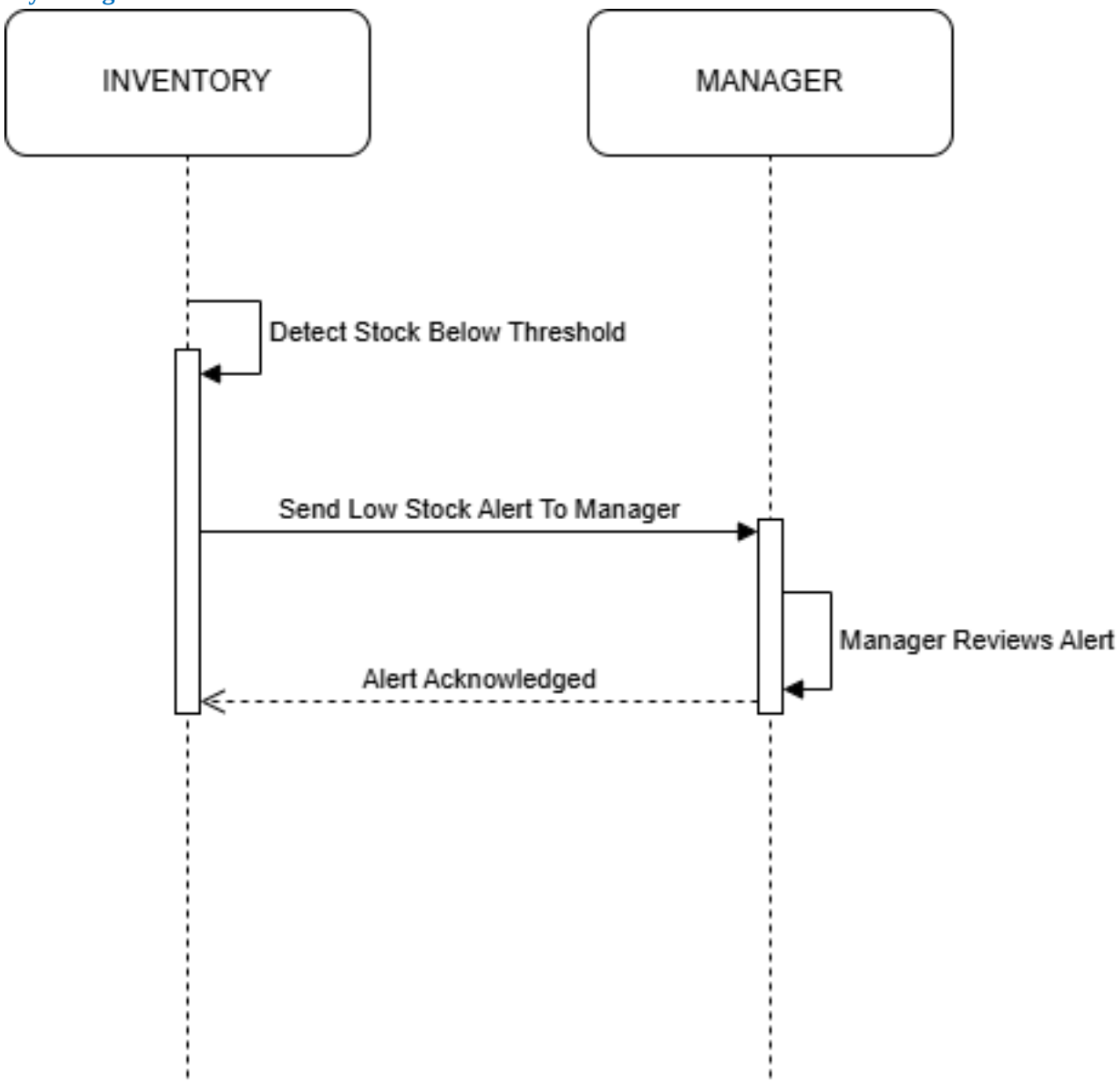


Figure: Notifymanager

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Prepareandserve

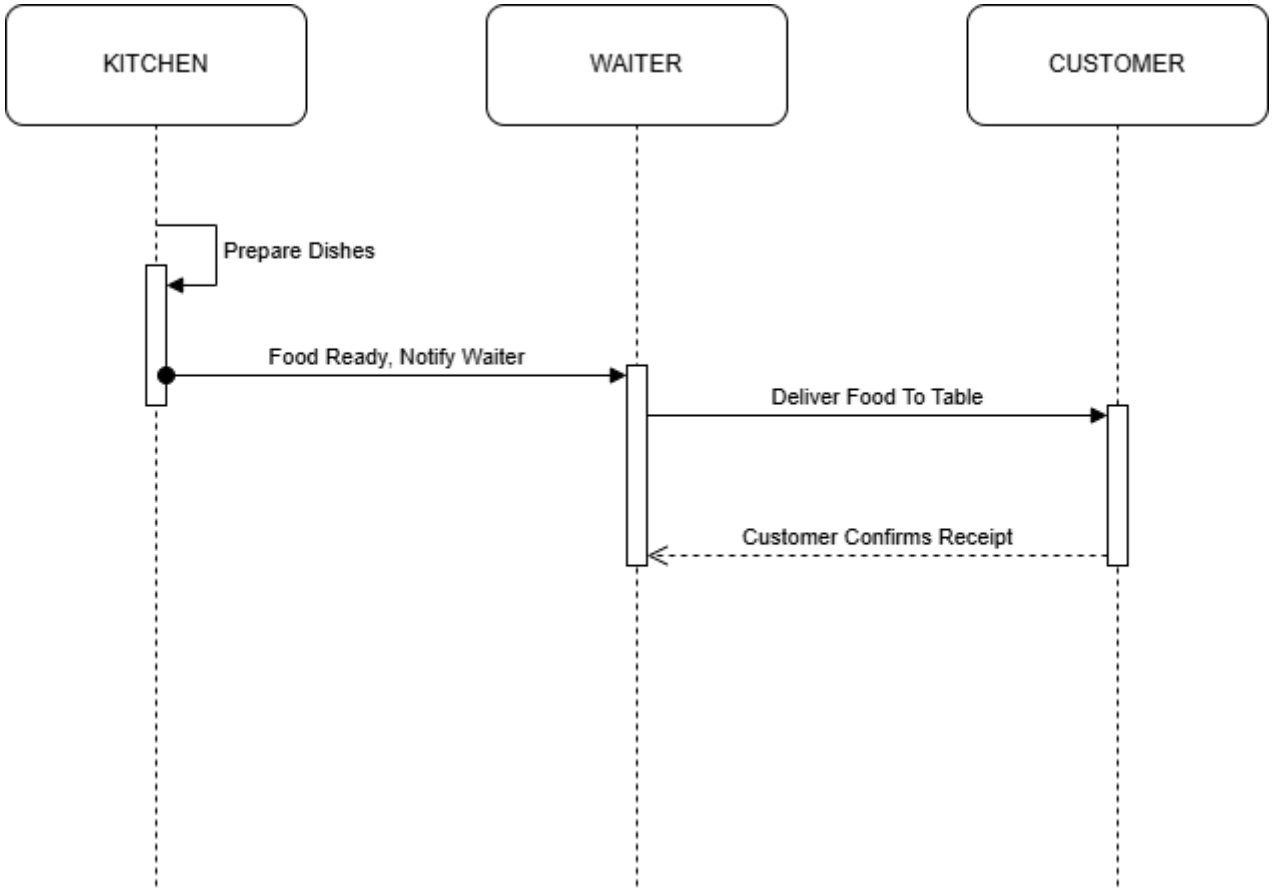


Figure: Prepareandserve

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Processsale

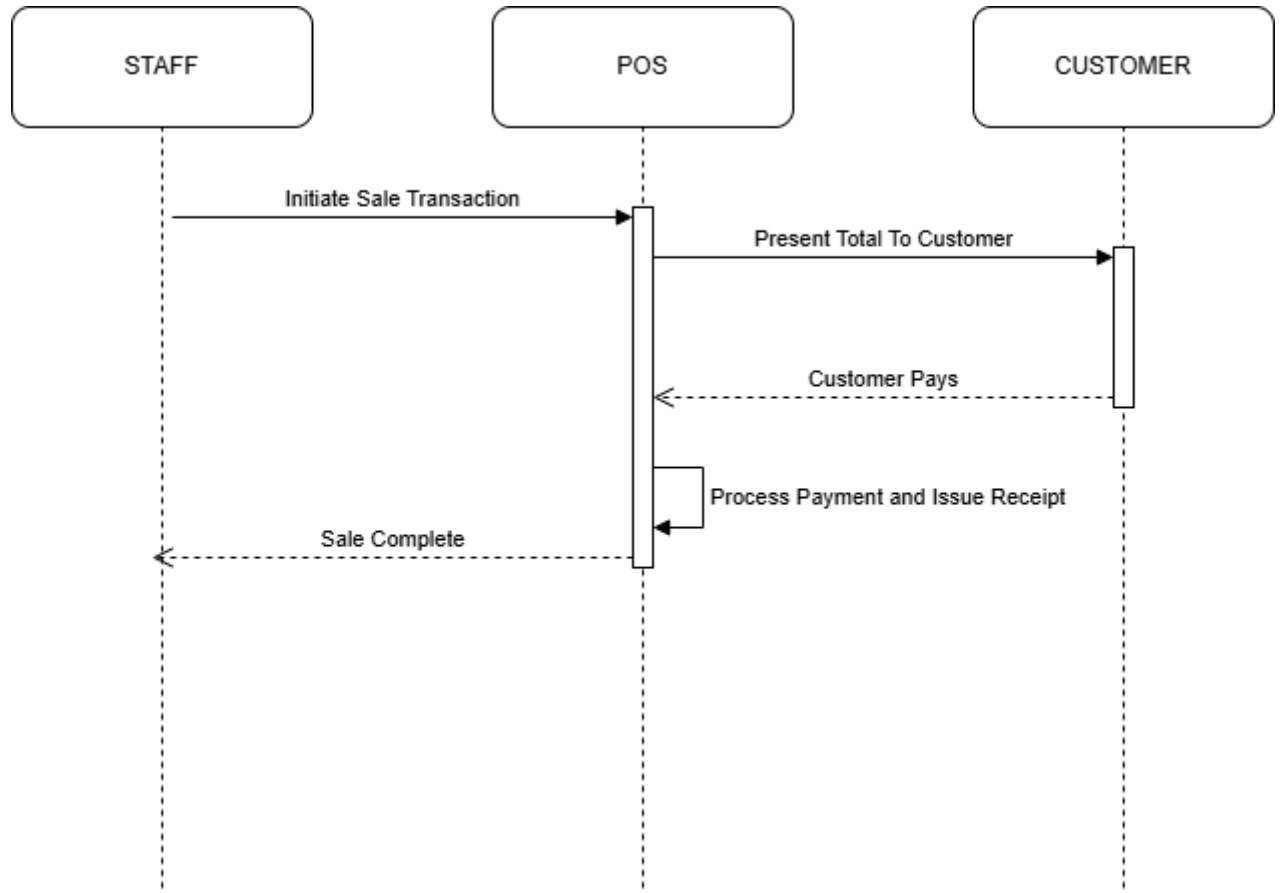


Figure: Processsale

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Recieverequest

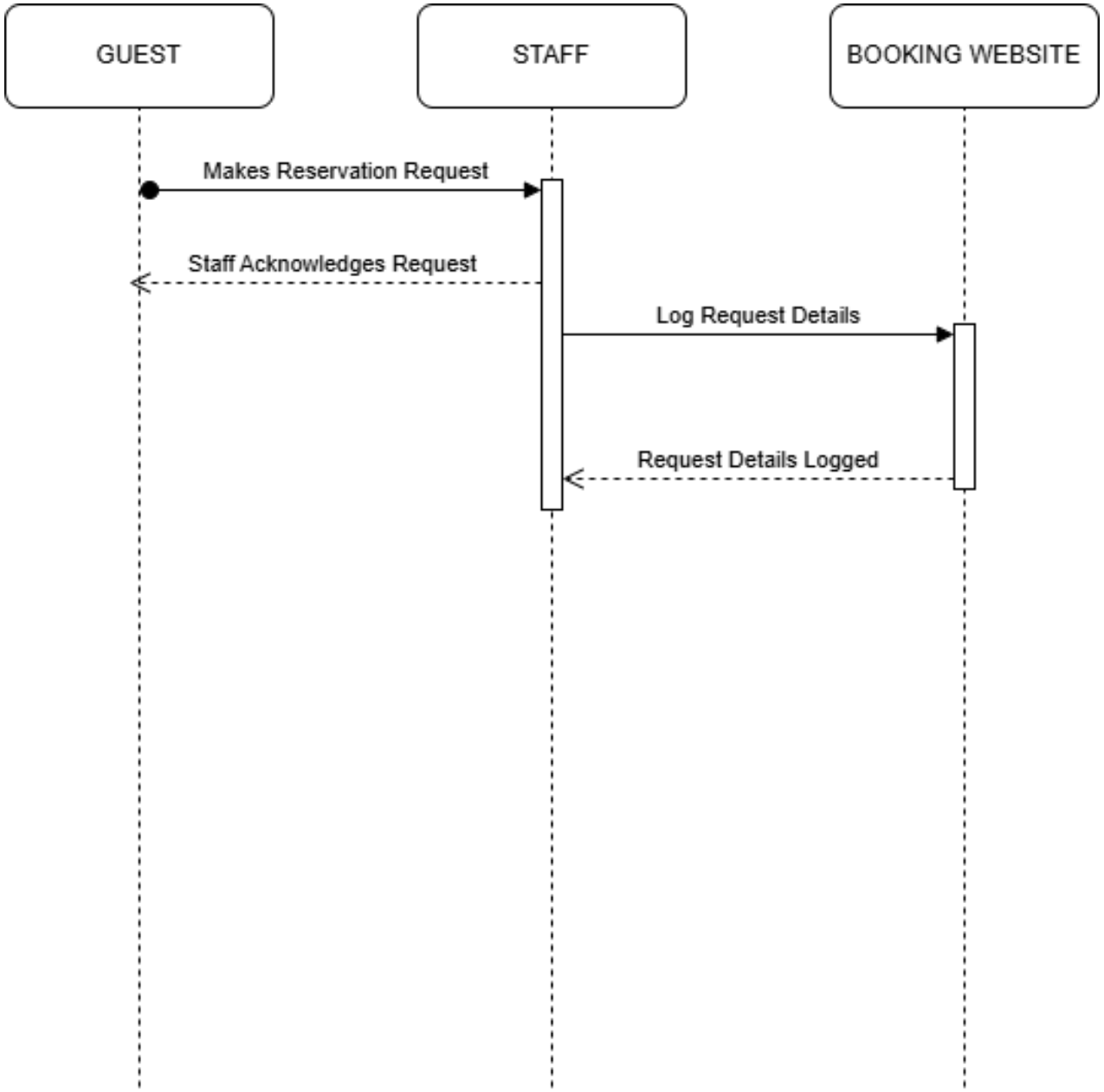


Figure: Recieverequest

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Recordinsystem

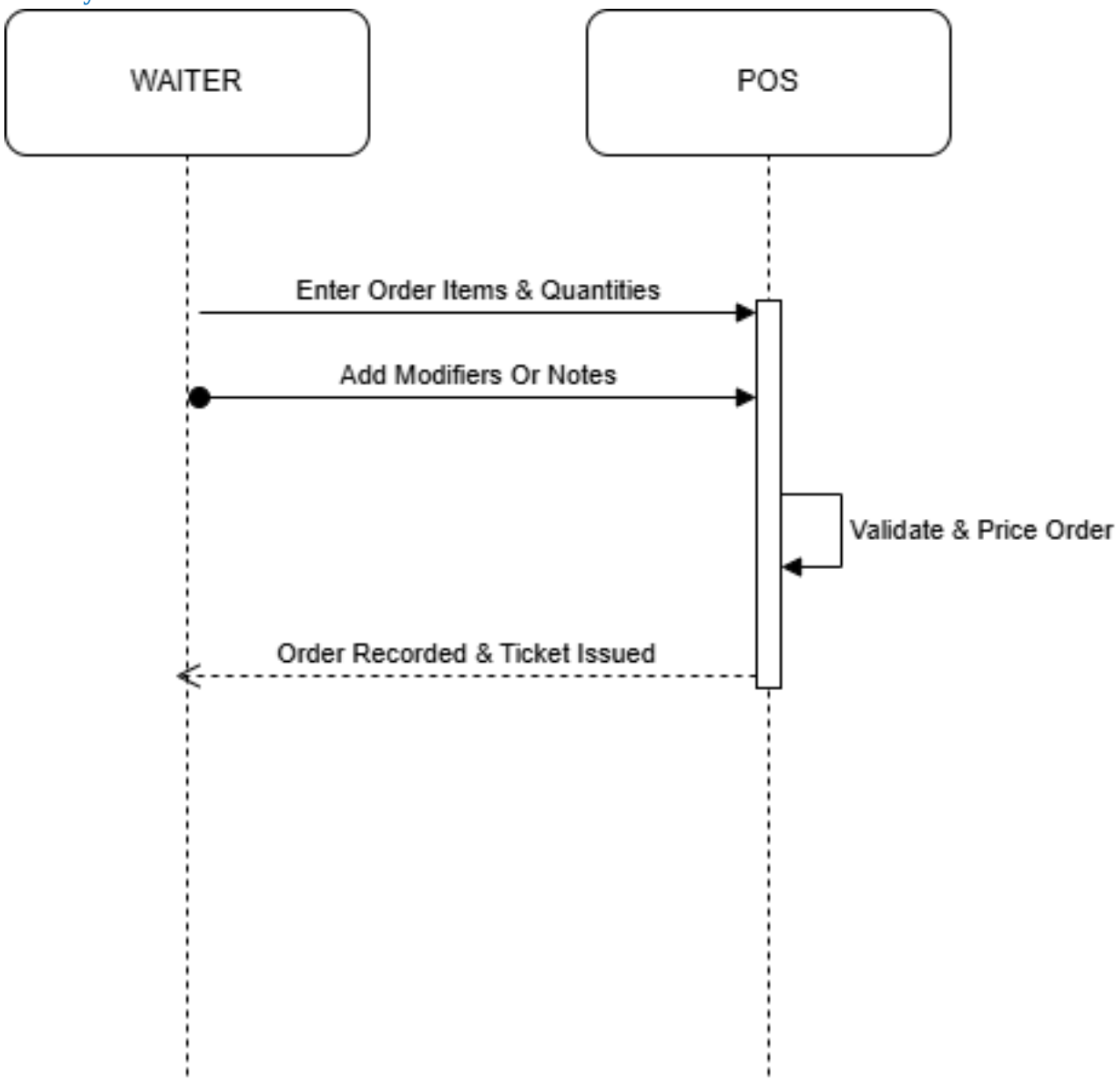


Figure: Recordinsystem

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Recordtransaction

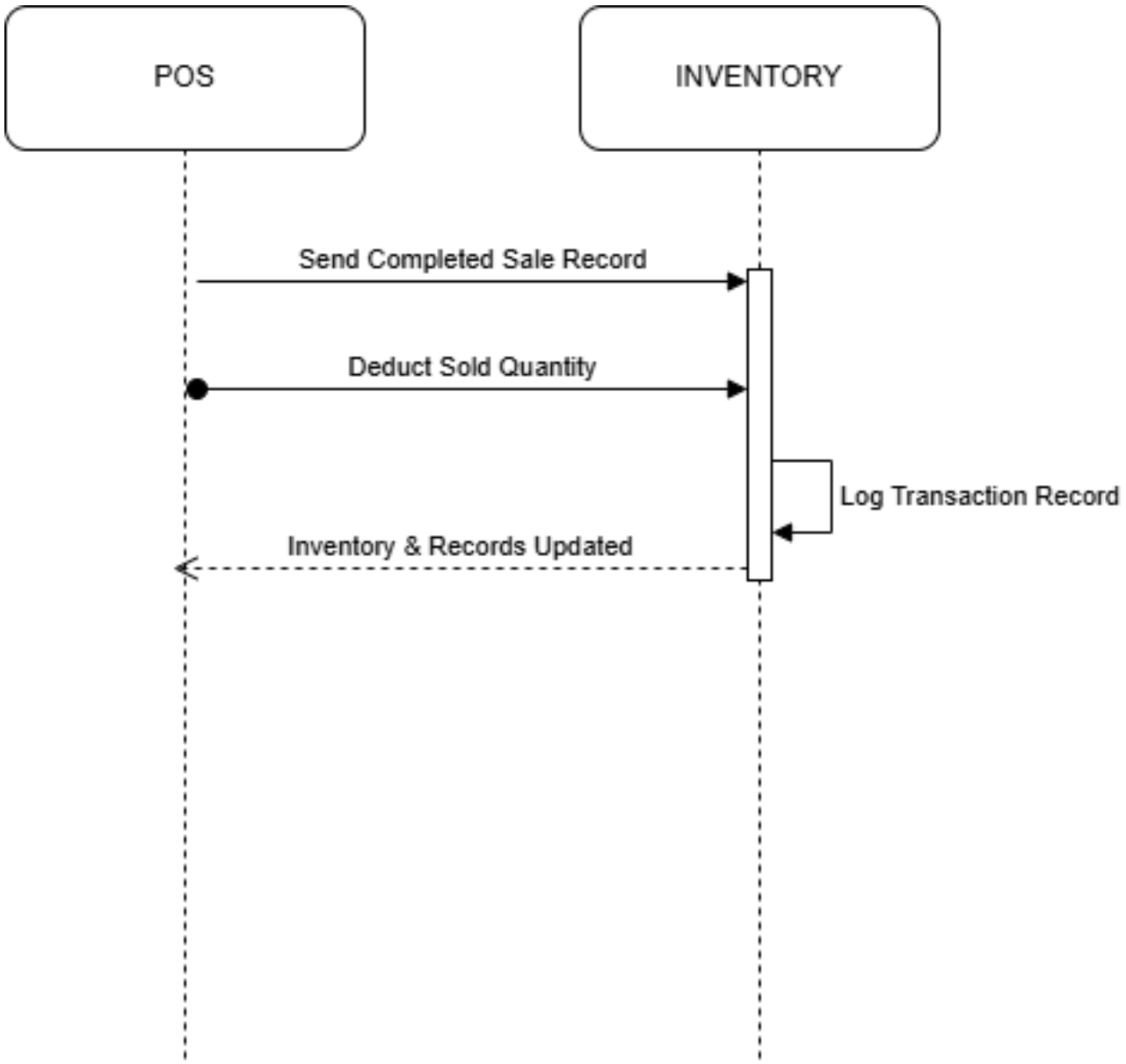


Figure: Recordtransaction

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Reviewperformance

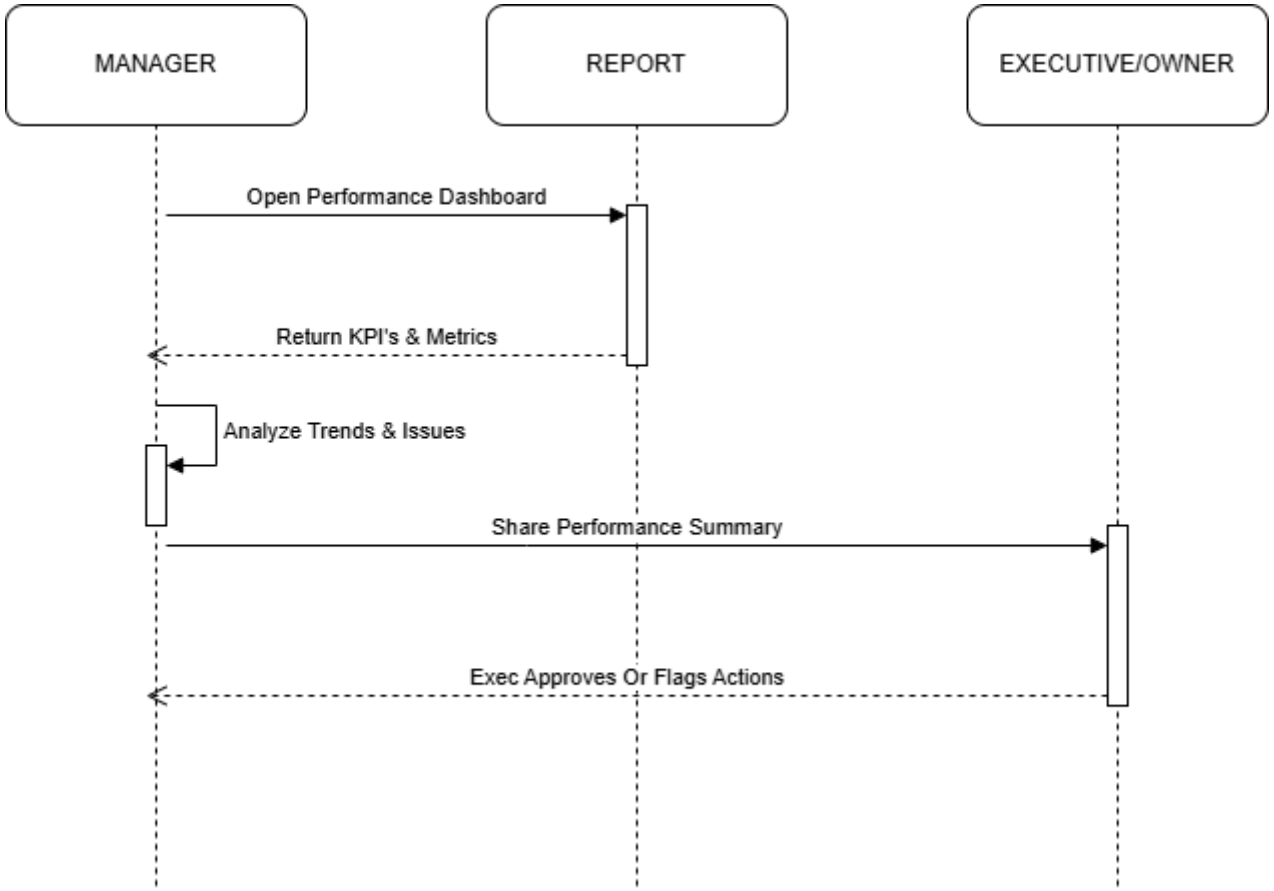


Figure: Reviewperformance

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Selectproduct

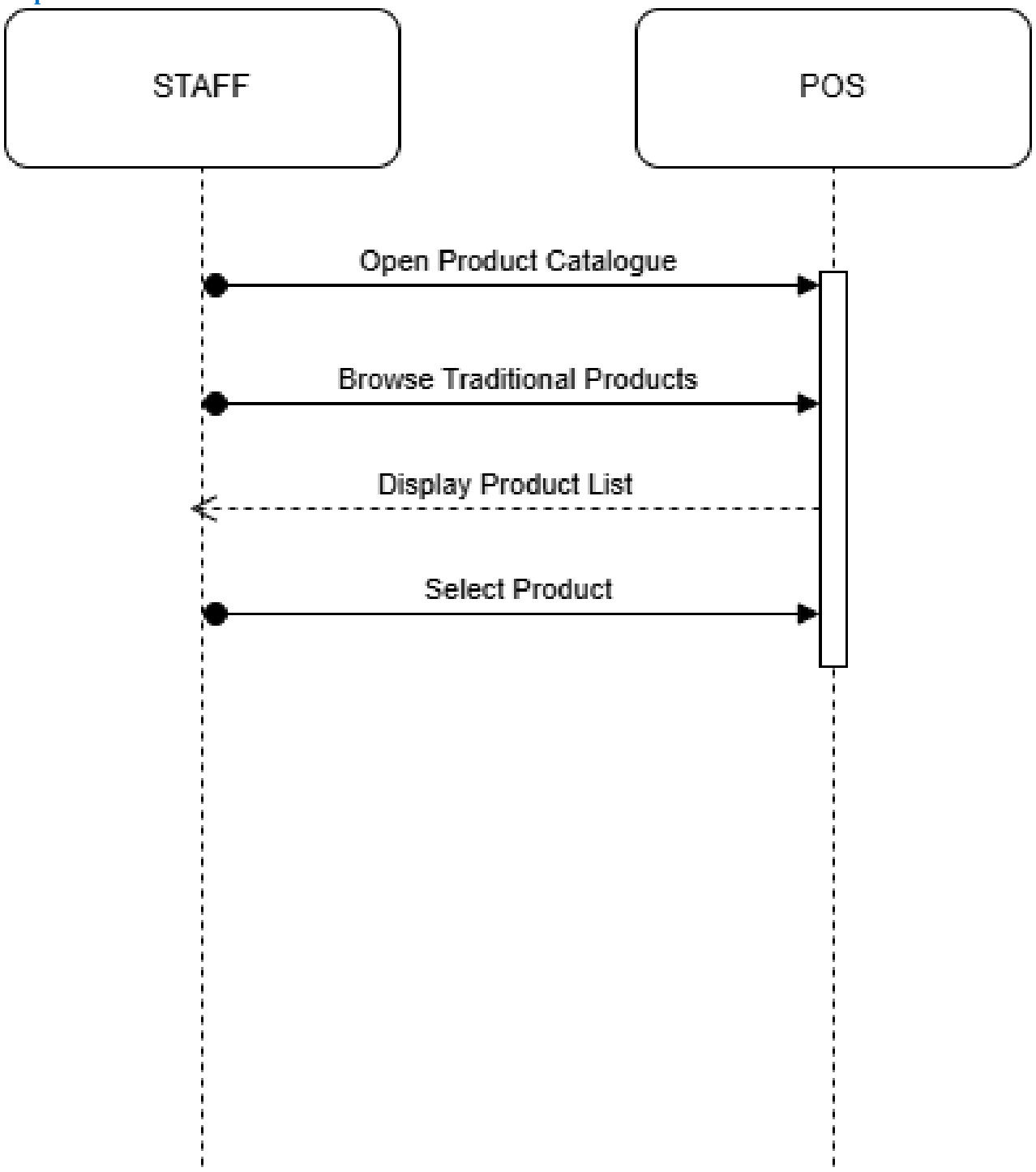


Figure: Selectproduct

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Sendtokitchen

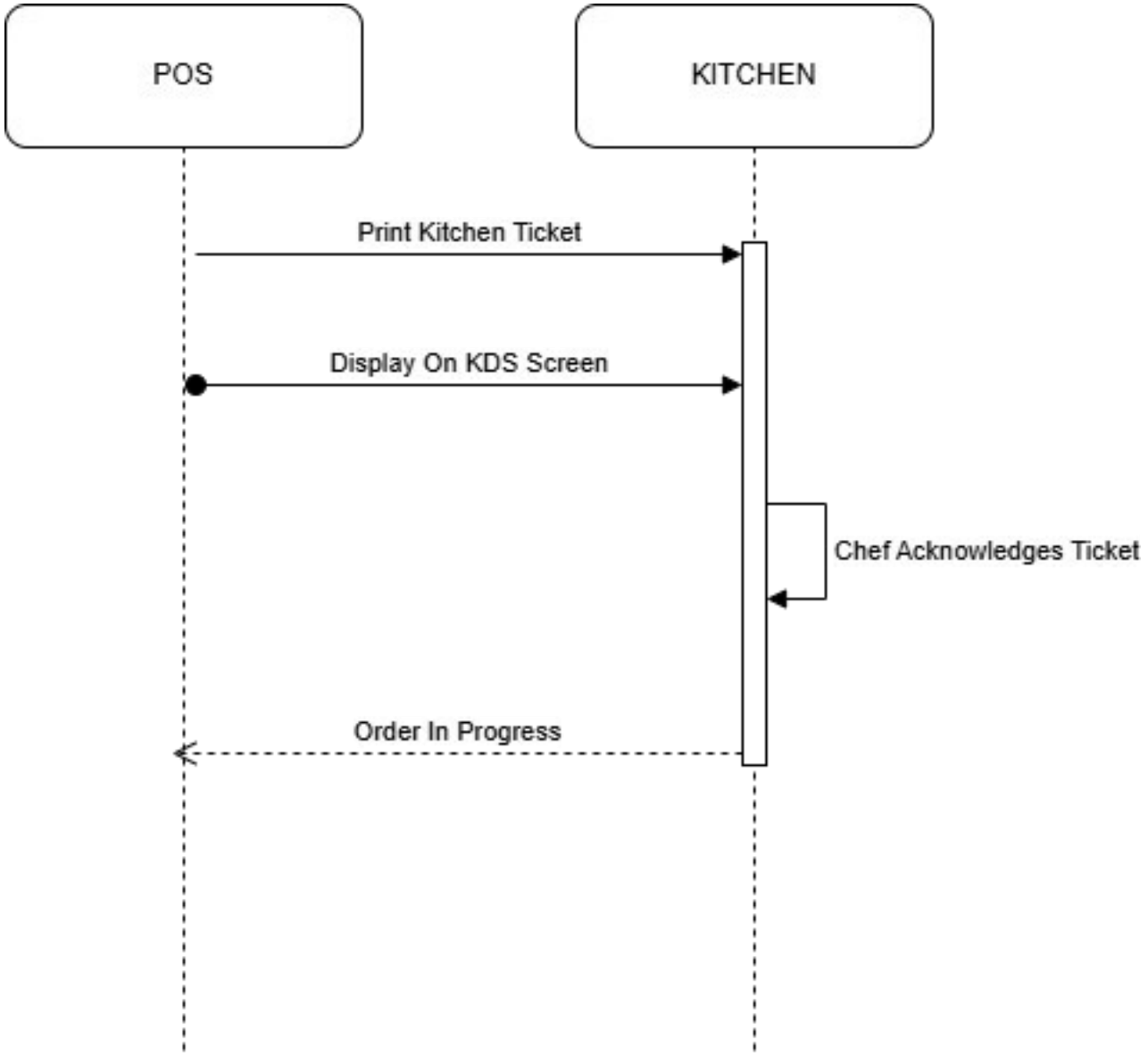


Figure: Sendtokitchen

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Showproductinfo

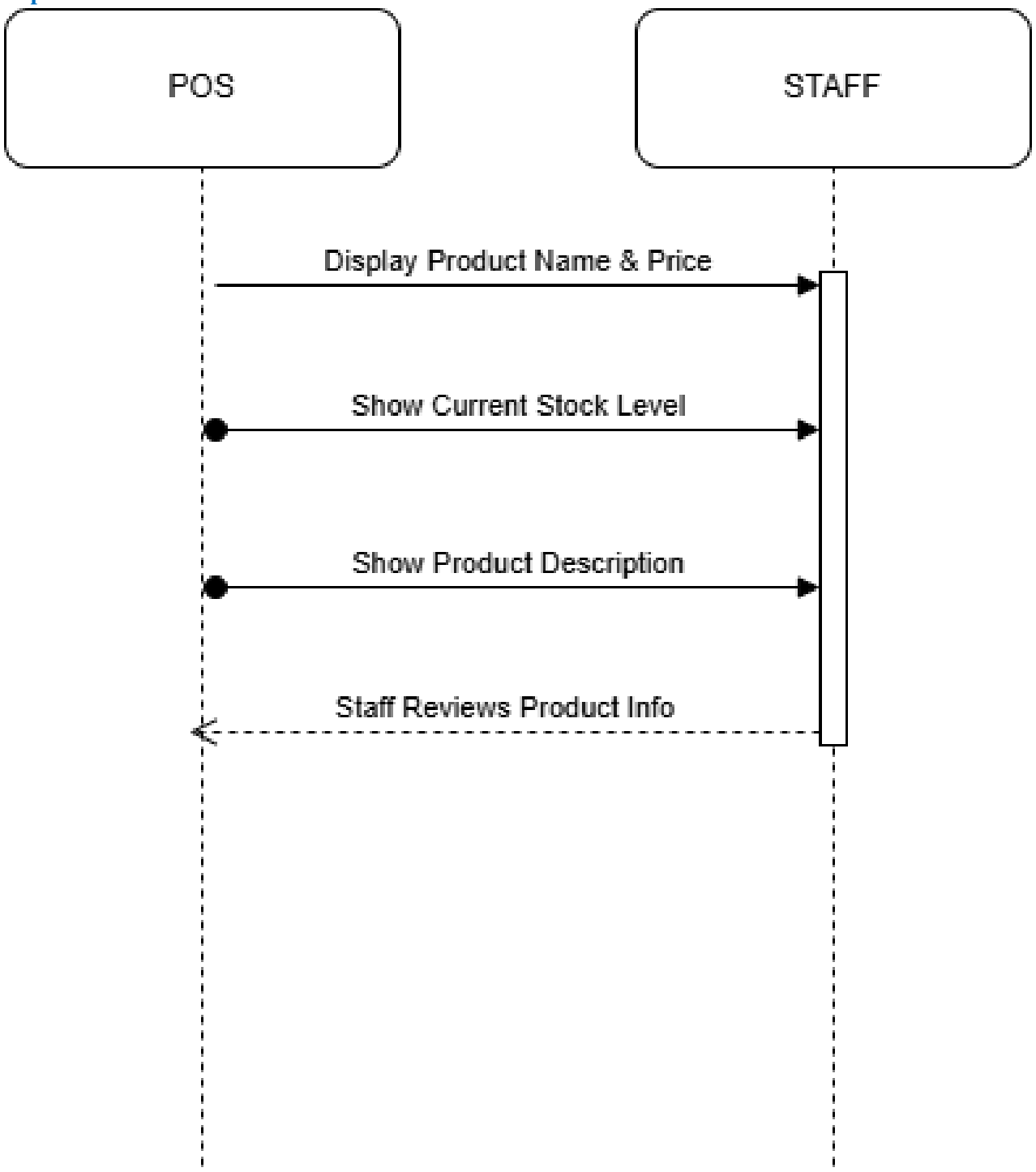


Figure: Showproductinfo

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Stockupdate

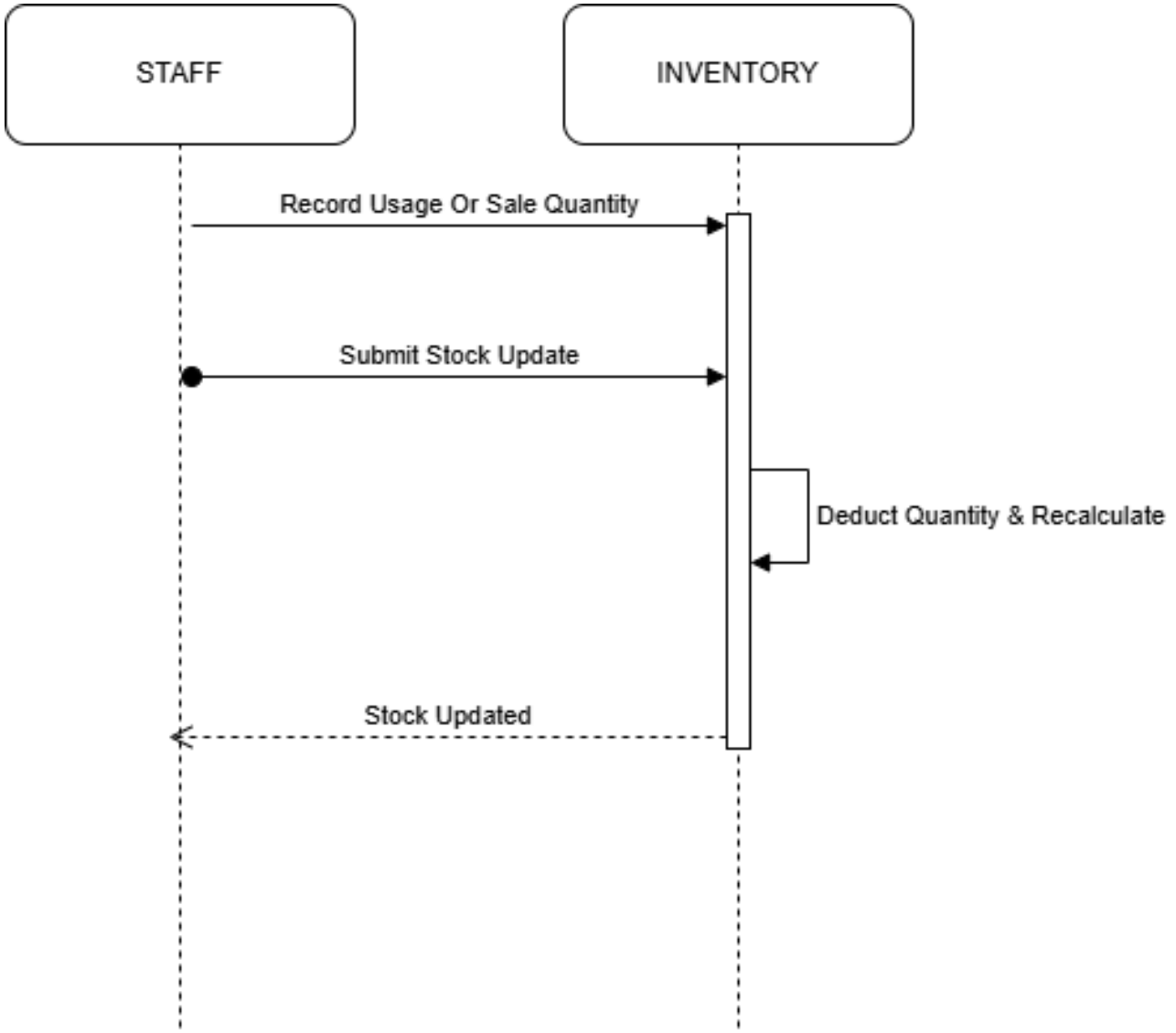


Figure: Stockupdate

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Suggestalternative

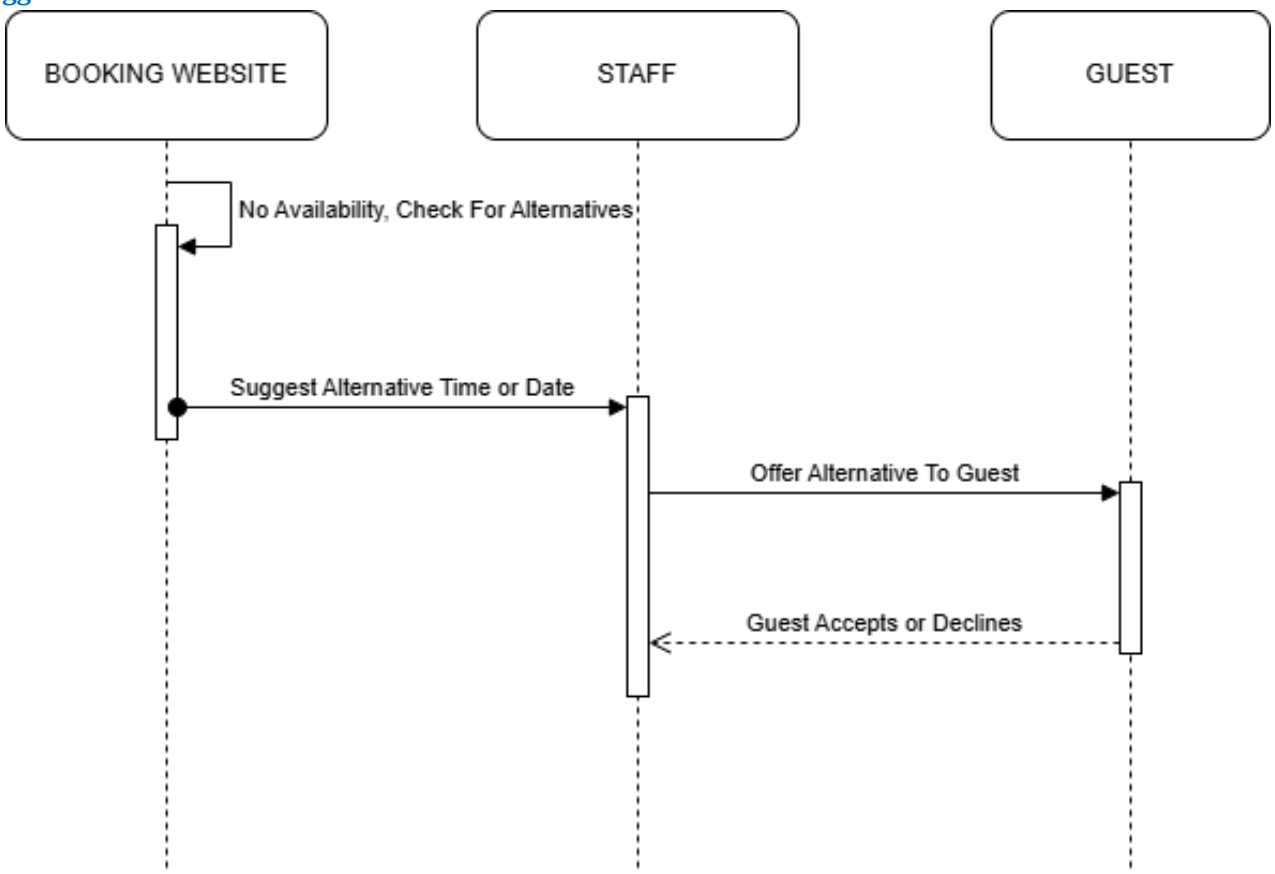


Figure: Suggestalternative

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Table Association

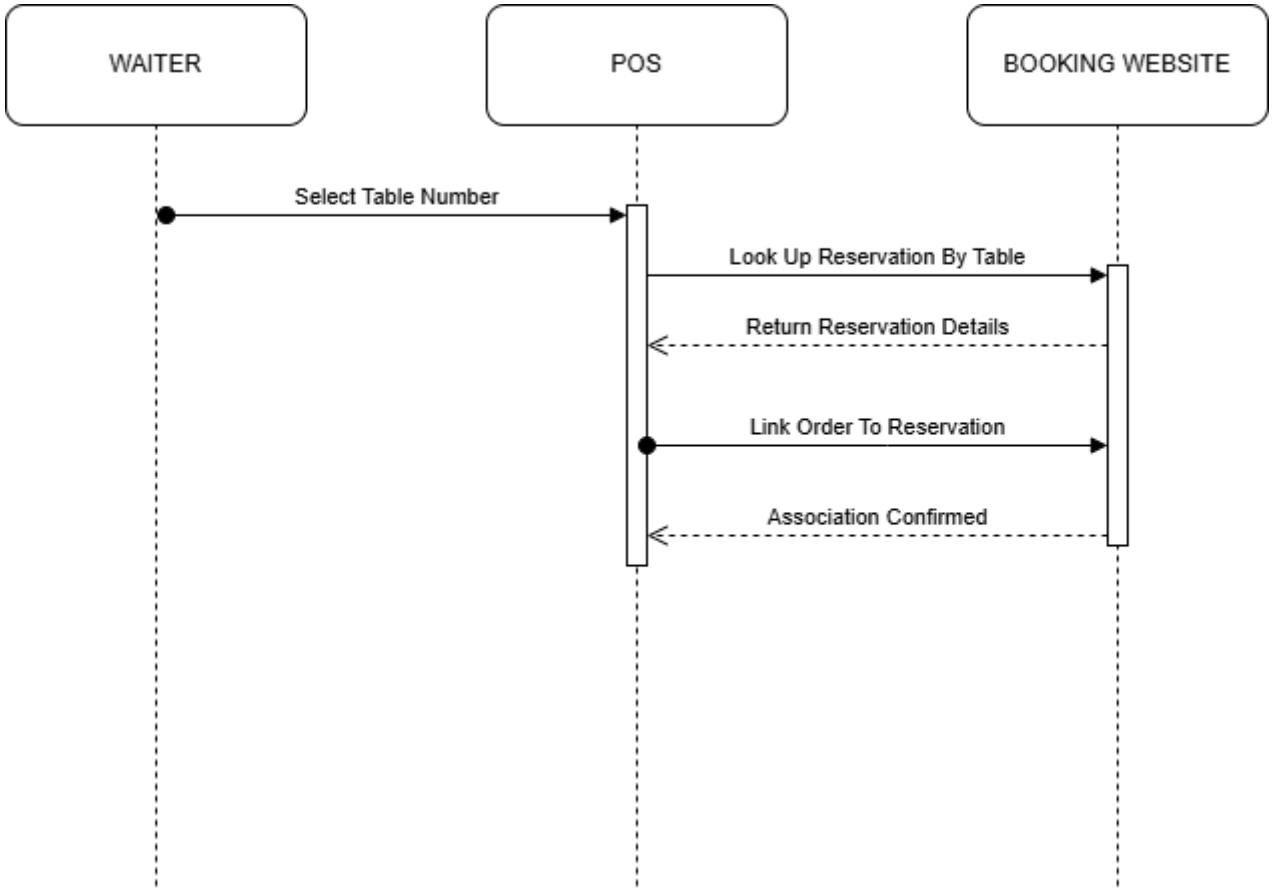


Figure: Table Association

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

Takeorder

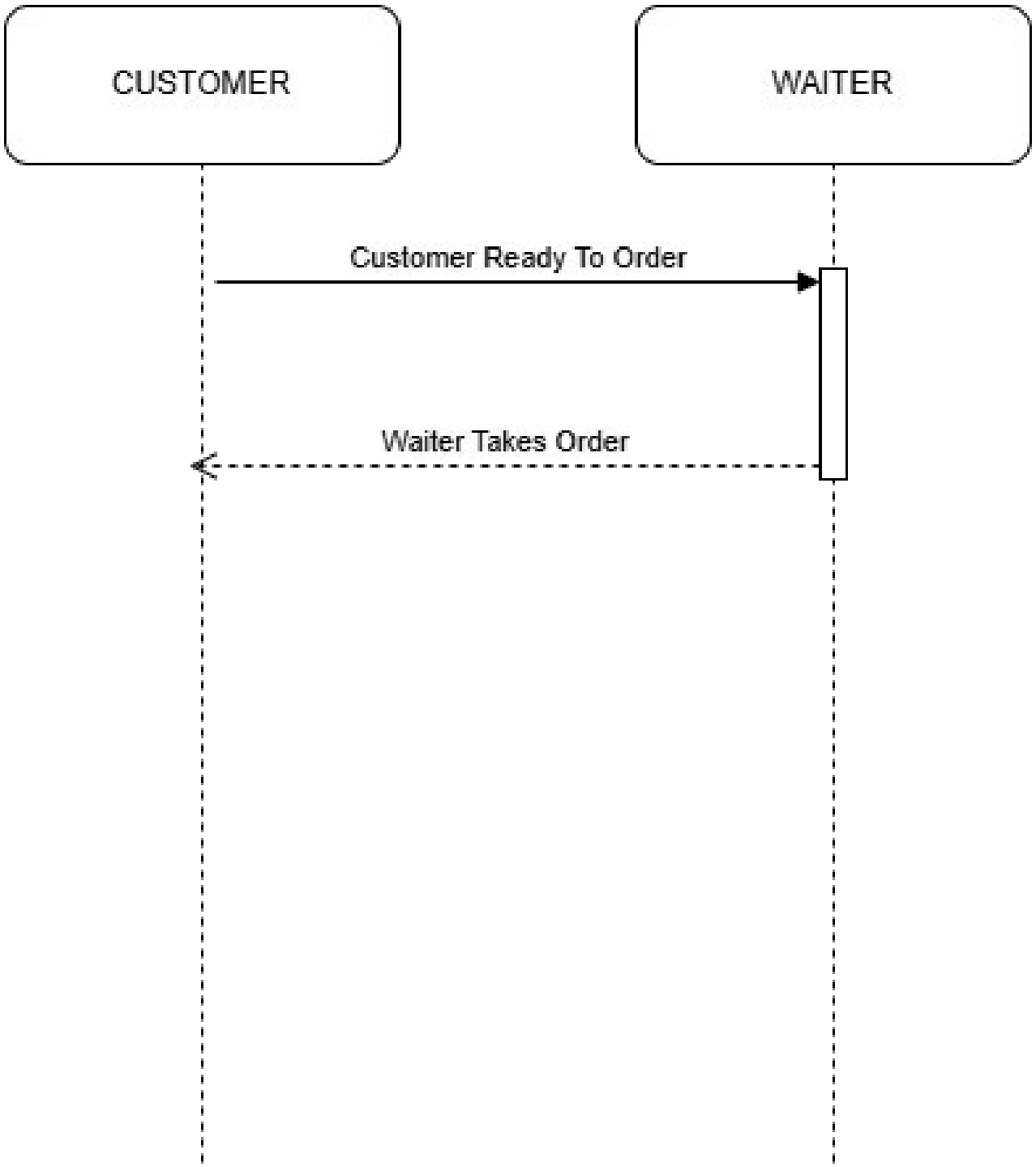


Figure: Takeorder

This sequence diagram shows the message flow for an AgriSoft operation. It helps clarify the order of interaction between staff, interface, service, and data layer.

4.11 Collaboration Diagram

Collaboration diagrams focus on object relationships and communication links. They complement sequence diagrams by emphasizing which objects cooperate in each module.

01 Order Flow Diagram

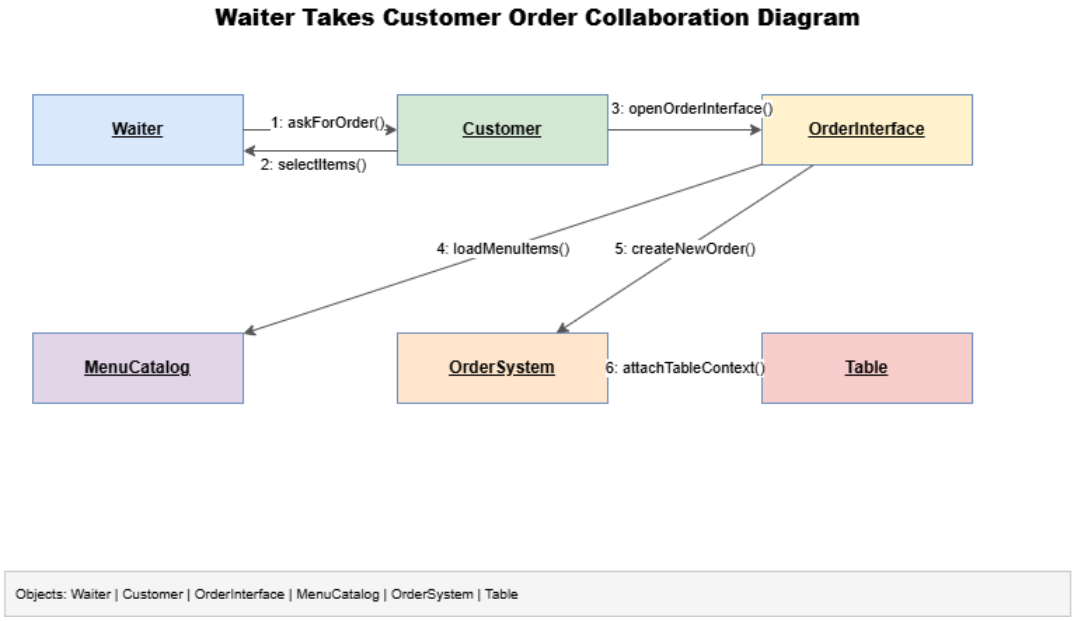


Figure: 01 Order Flow Diagram

This collaboration diagram shows how objects and components work together in one AgriSoft module.

01 Product Sales Module

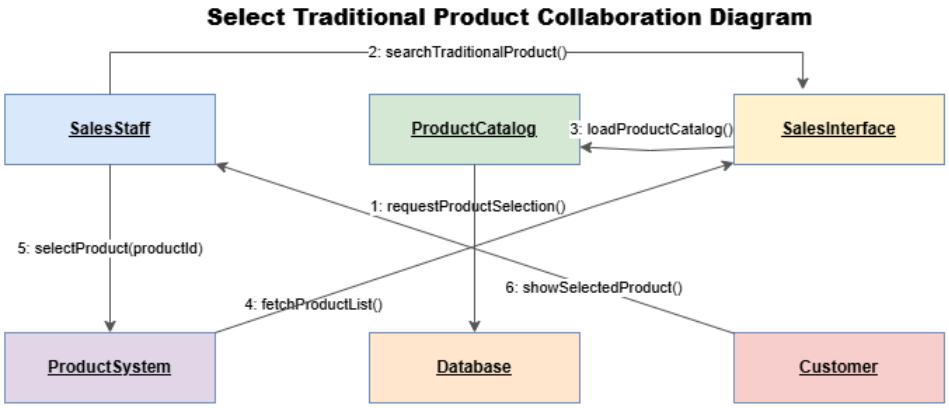


Figure: 01 Product Sales Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

01 Reporting Module

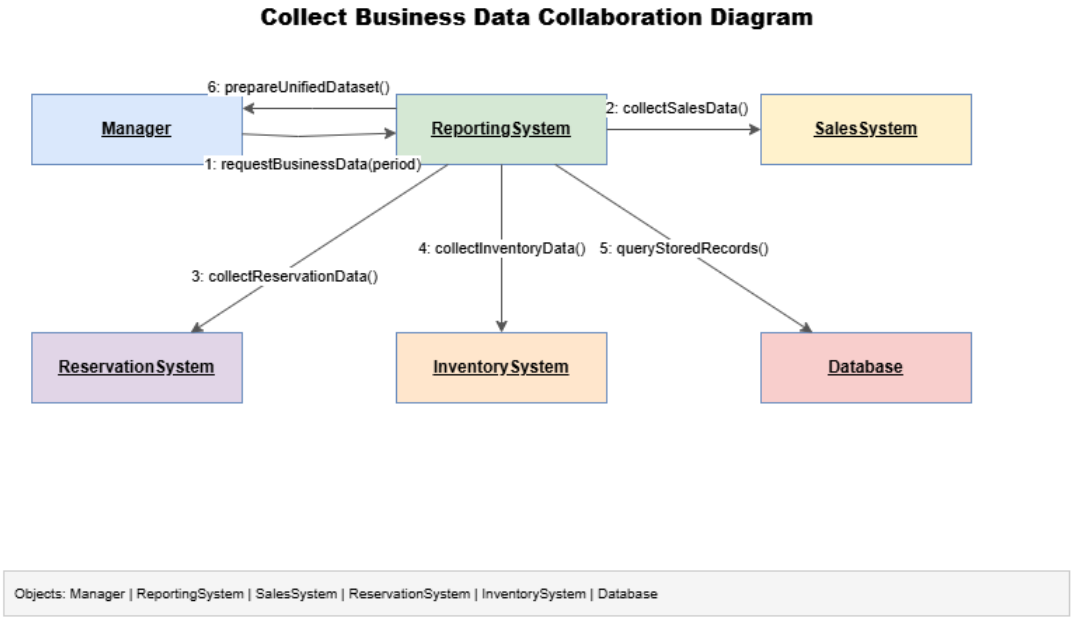


Figure: 01 Reporting Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

01 Reservation Module

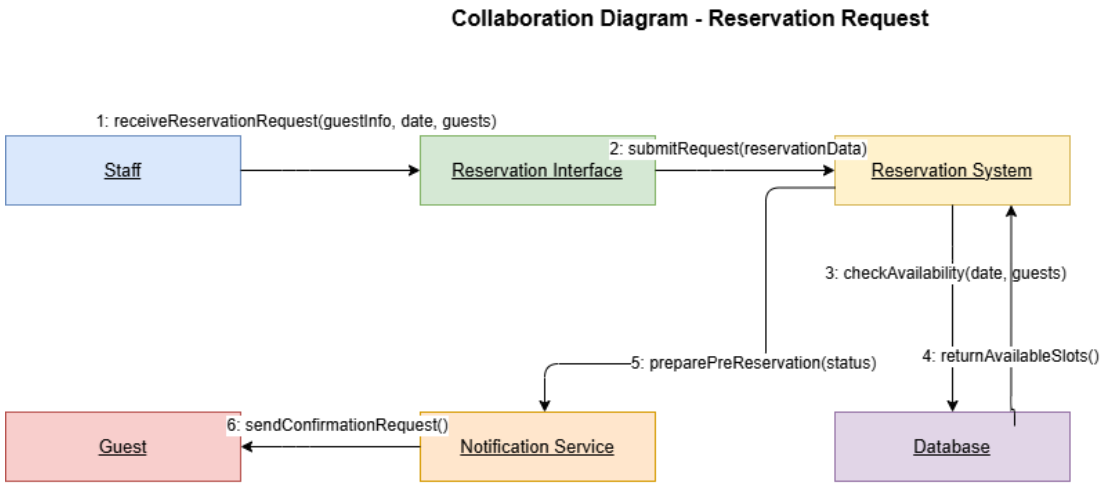
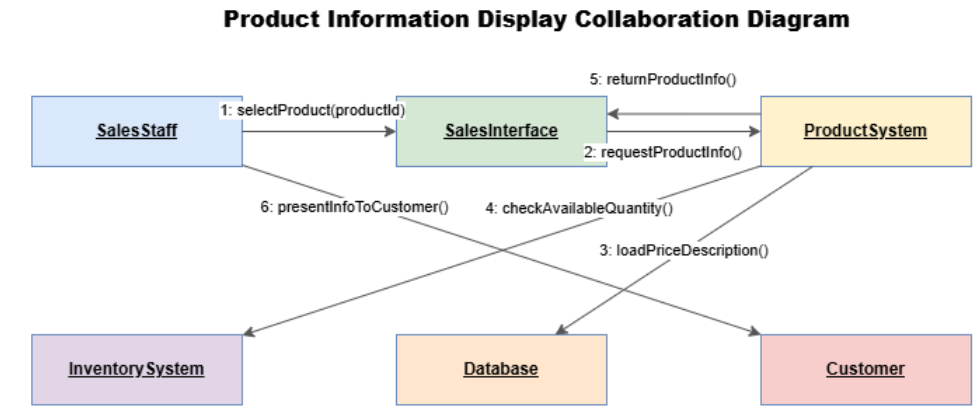


Figure: 01 Reservation Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

02 Product Sales Module



Objects: SalesStaff | SalesInterface | ProductSystem | InventorySystem | Database | Customer

Figure: 02 Product Sales Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

02 Reporting Module

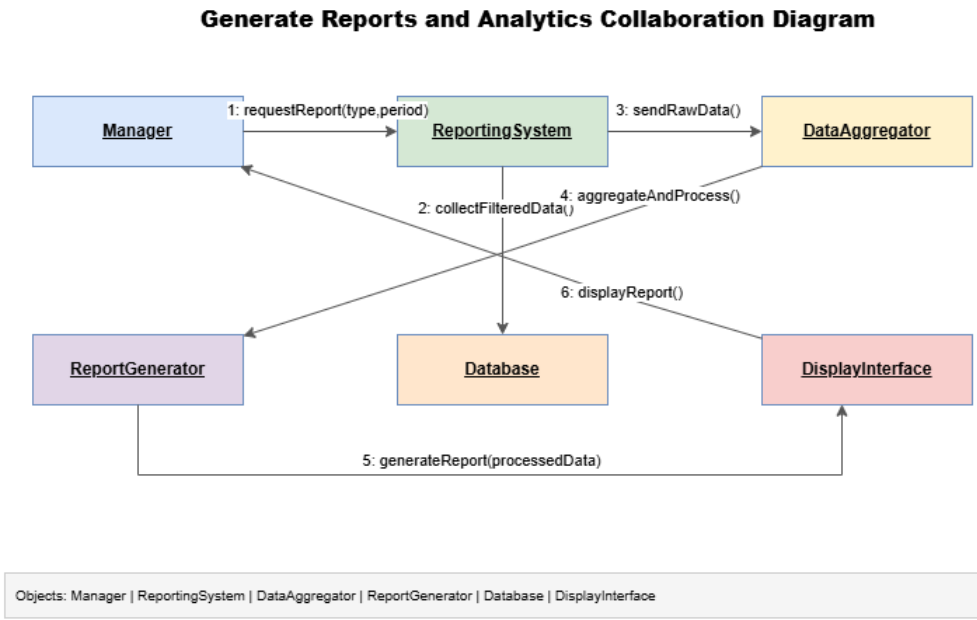


Figure: 02 Reporting Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

02 Reservation Module

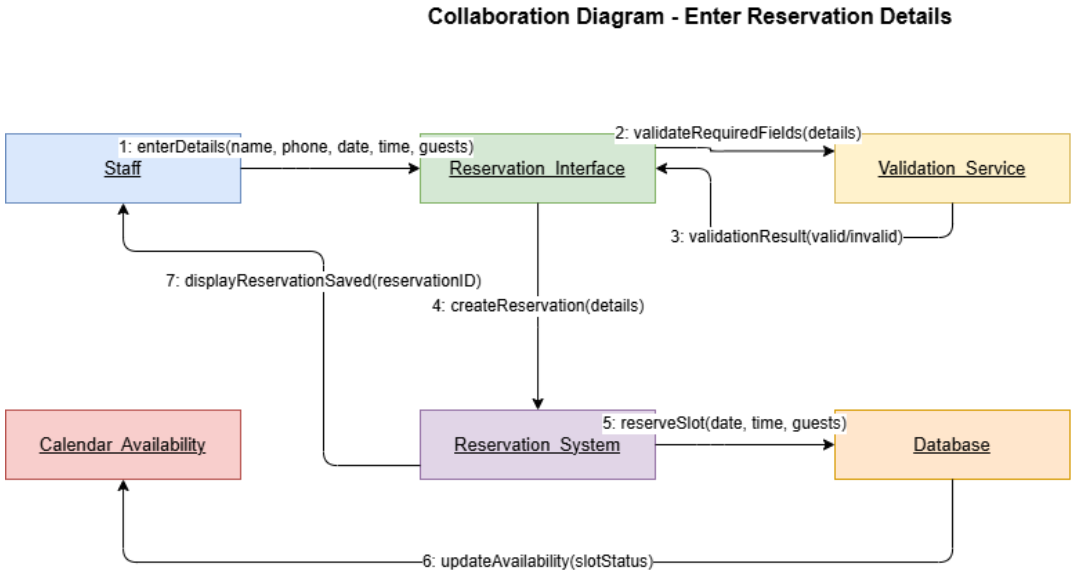


Figure: 02 Reservation Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

03 Inventory Module

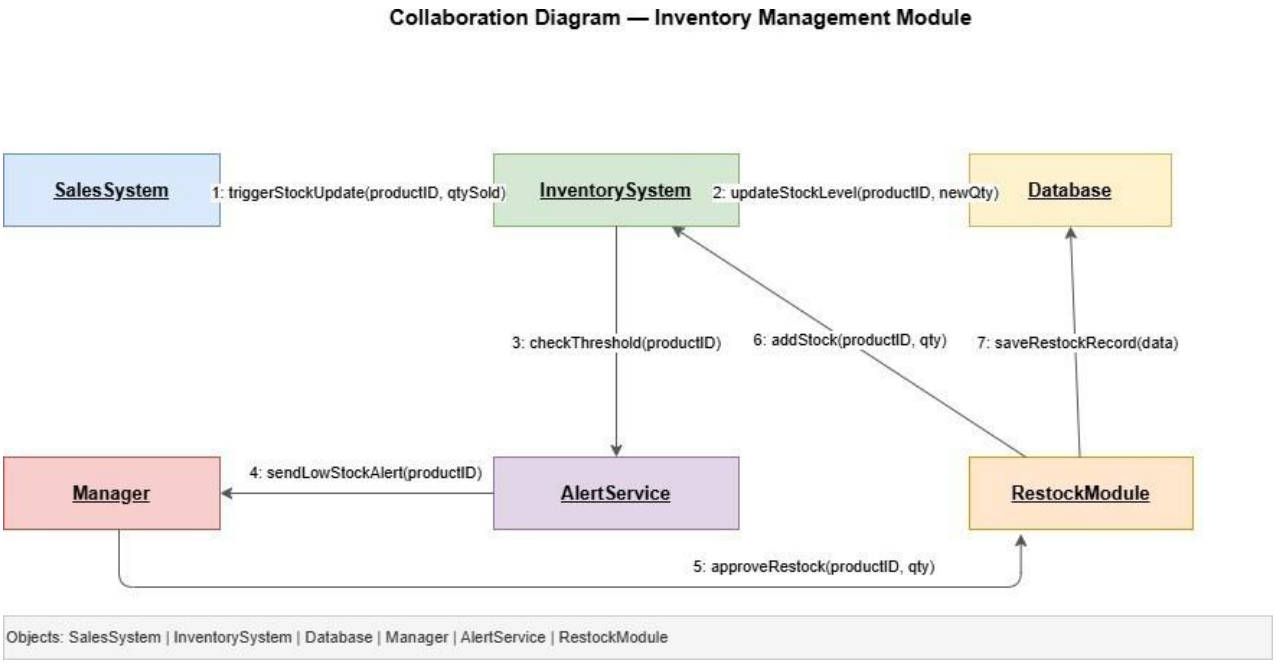


Figure: 03 Inventory Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

03 Product Sales Module

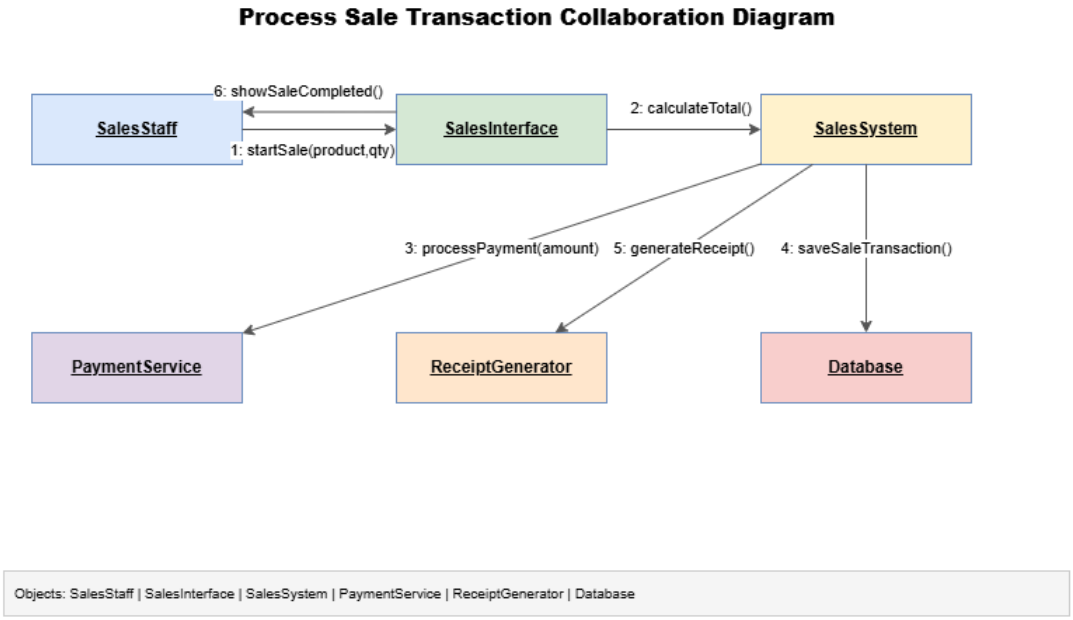


Figure: 03 Product Sales Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

03 Reservation Module

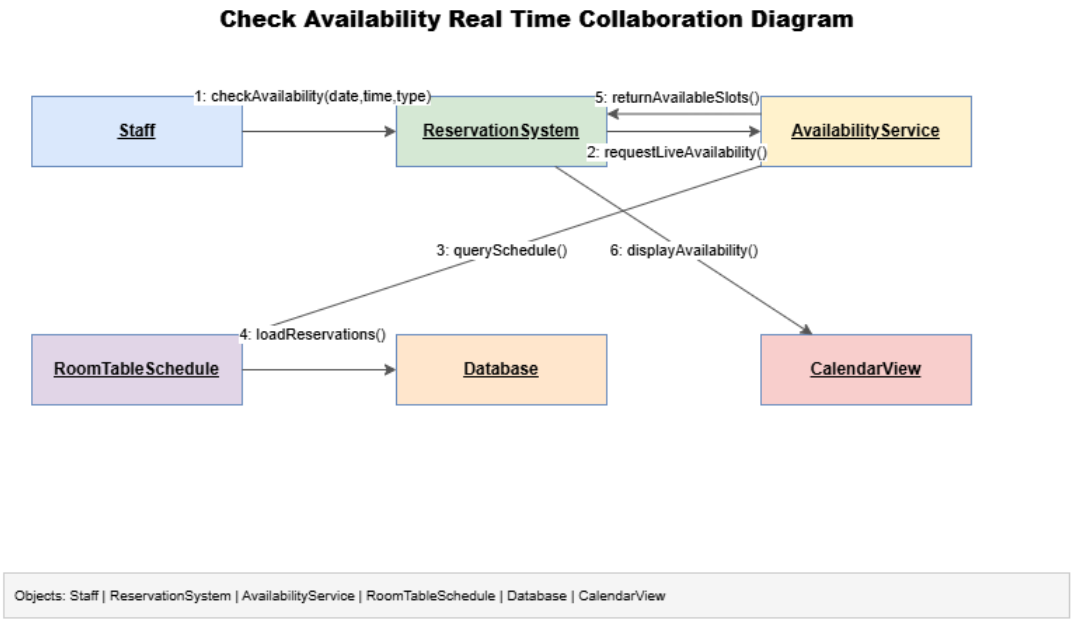


Figure: 03 Reservation Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

04 Order Flow Module

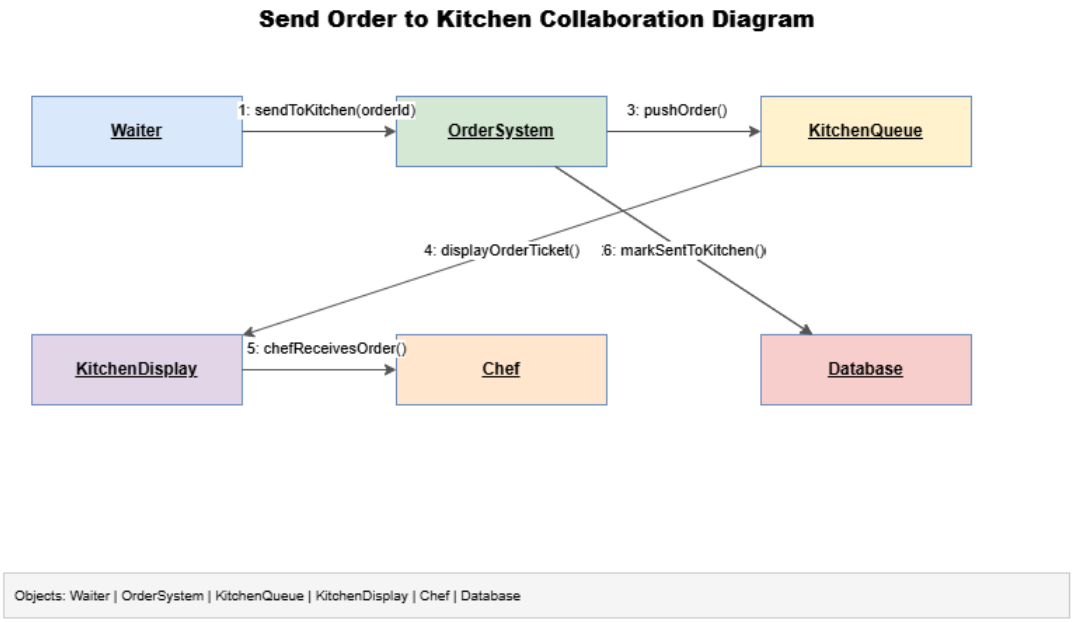


Figure: 04 Order Flow Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

04 Product Sales Module

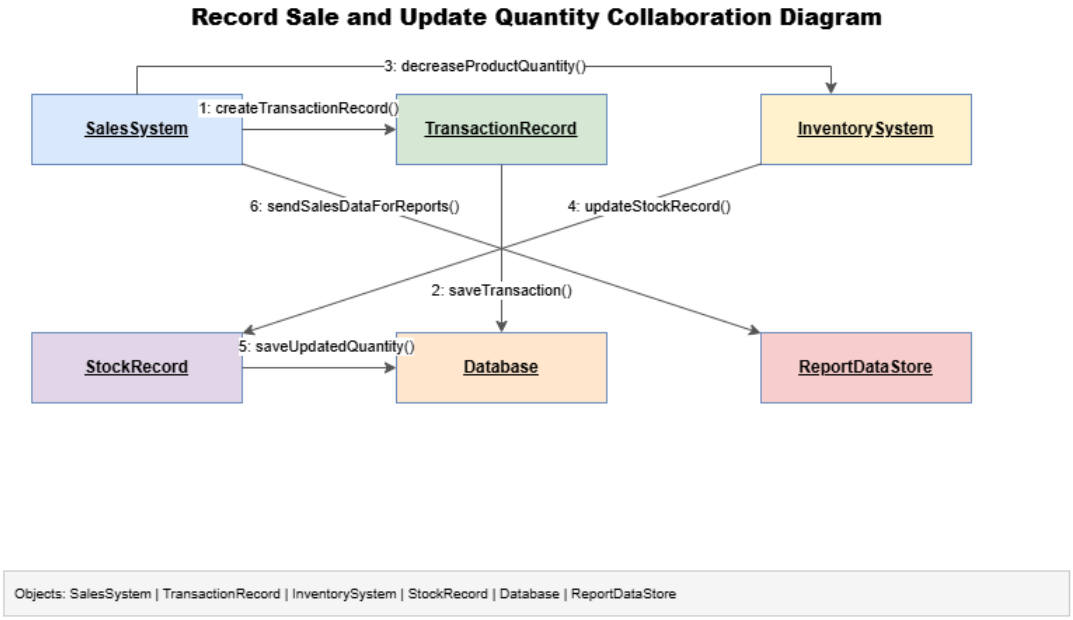


Figure: 04 Product Sales Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

04 Reservation Module

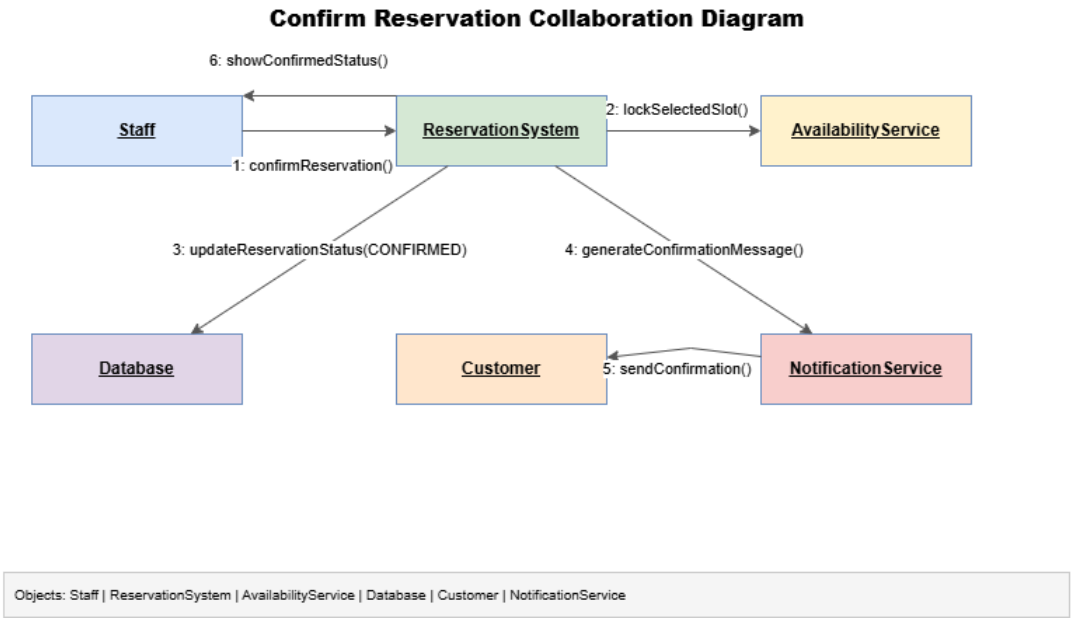


Figure: 04 Reservation Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

05 Order Flow Module

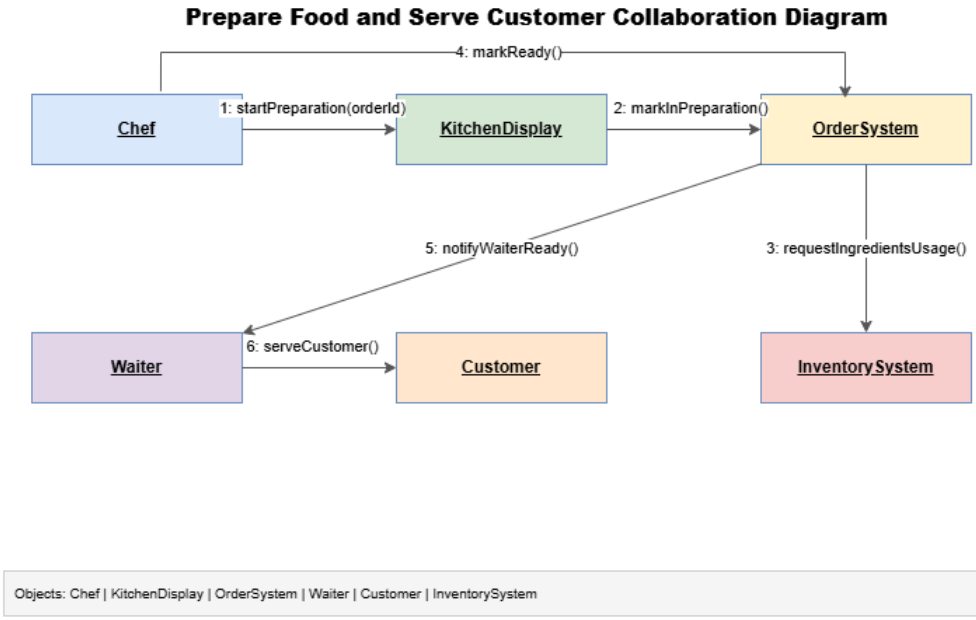


Figure: 05 Order Flow Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

05 Product Sales Module

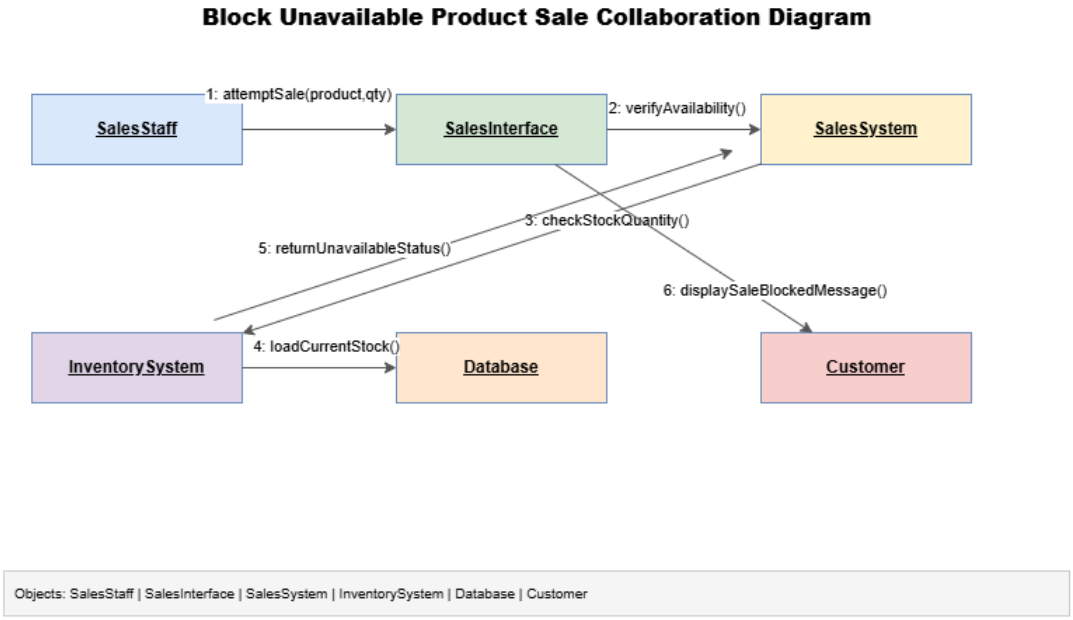


Figure: 05 Product Sales Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

05 Reservation Module

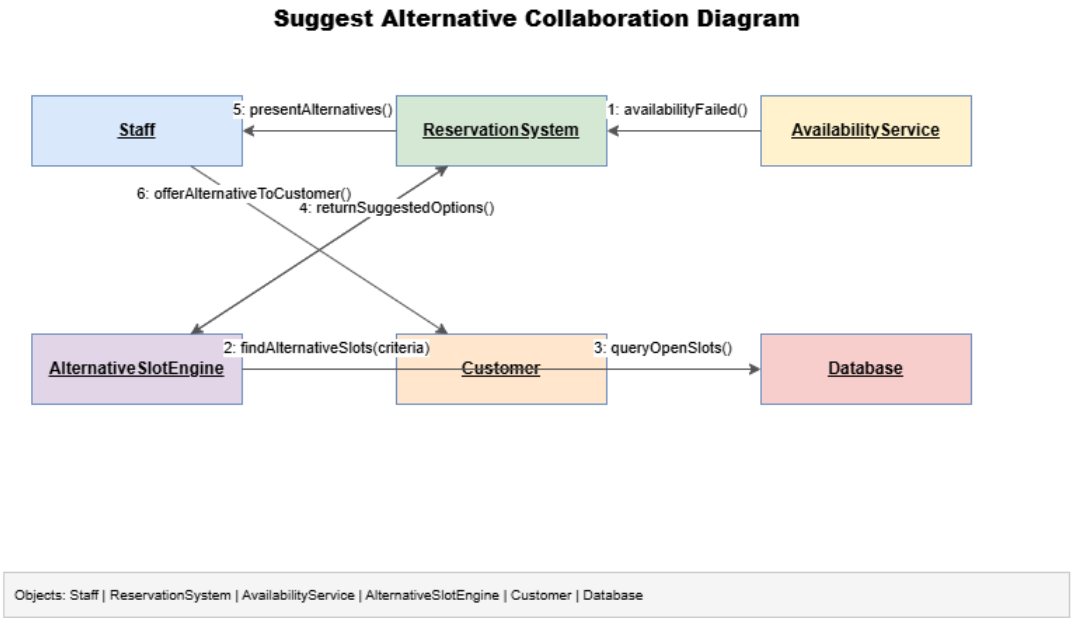


Figure: 05 Reservation Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

06 Reservation Module

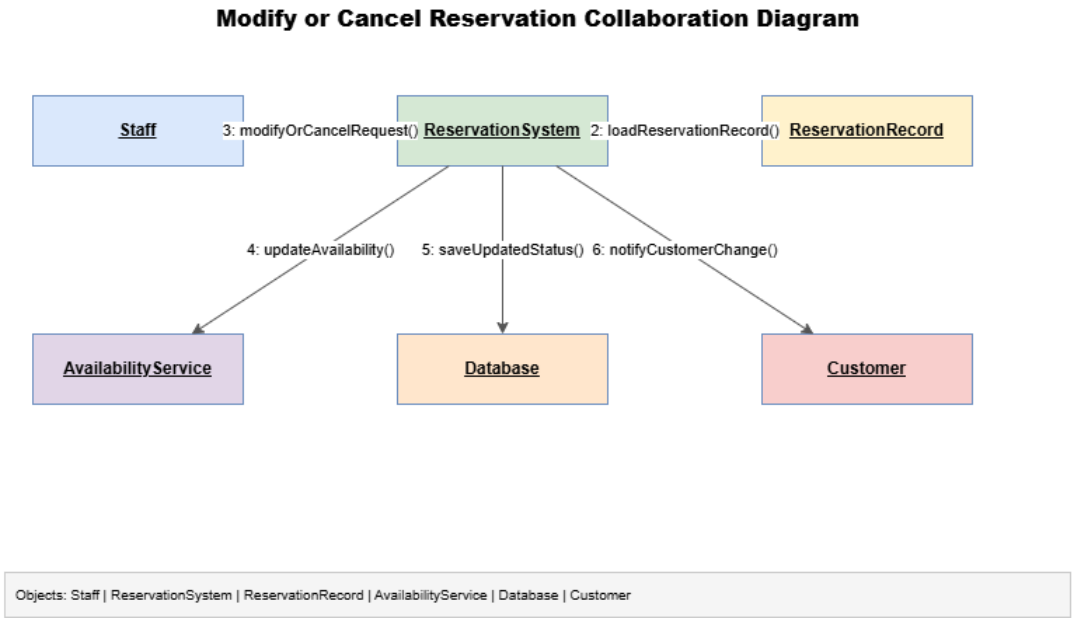


Figure: 06 Reservation Module

This collaboration diagram shows how objects and components work together in one AgriSoft module.

4.12 Data Flow Diagrams (DFD)

DFD diagrams describe how information enters, moves through, and exits the AgriSoft system. They help identify external entities, processes, data stores, and data flows.

Dfd Level 0 Diagram

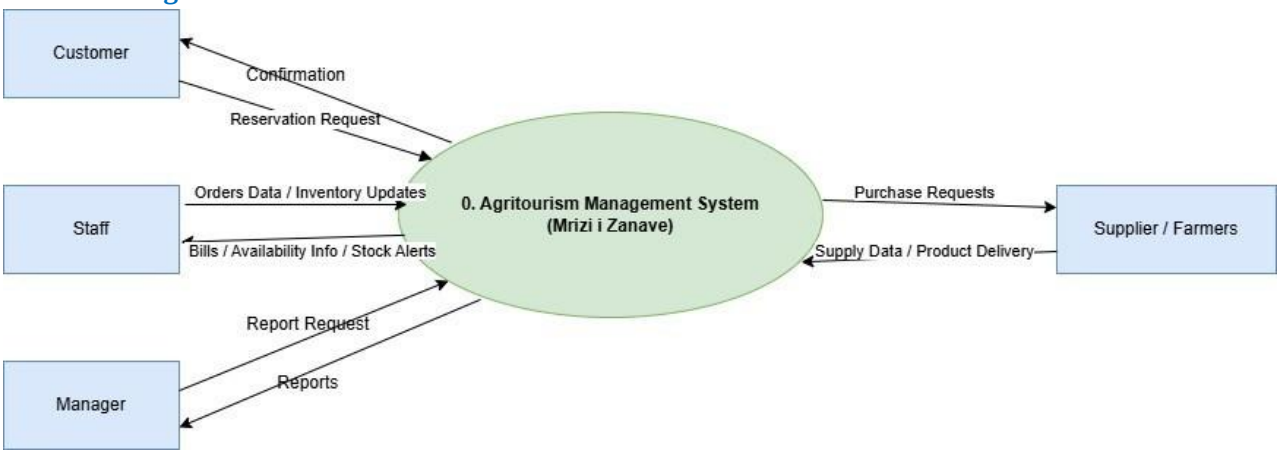


Figure: Dfd Level 0 Diagram

This DFD illustrates how data flows through AgriSoft at a specific level of detail.

Dfd Level 1 Diagram

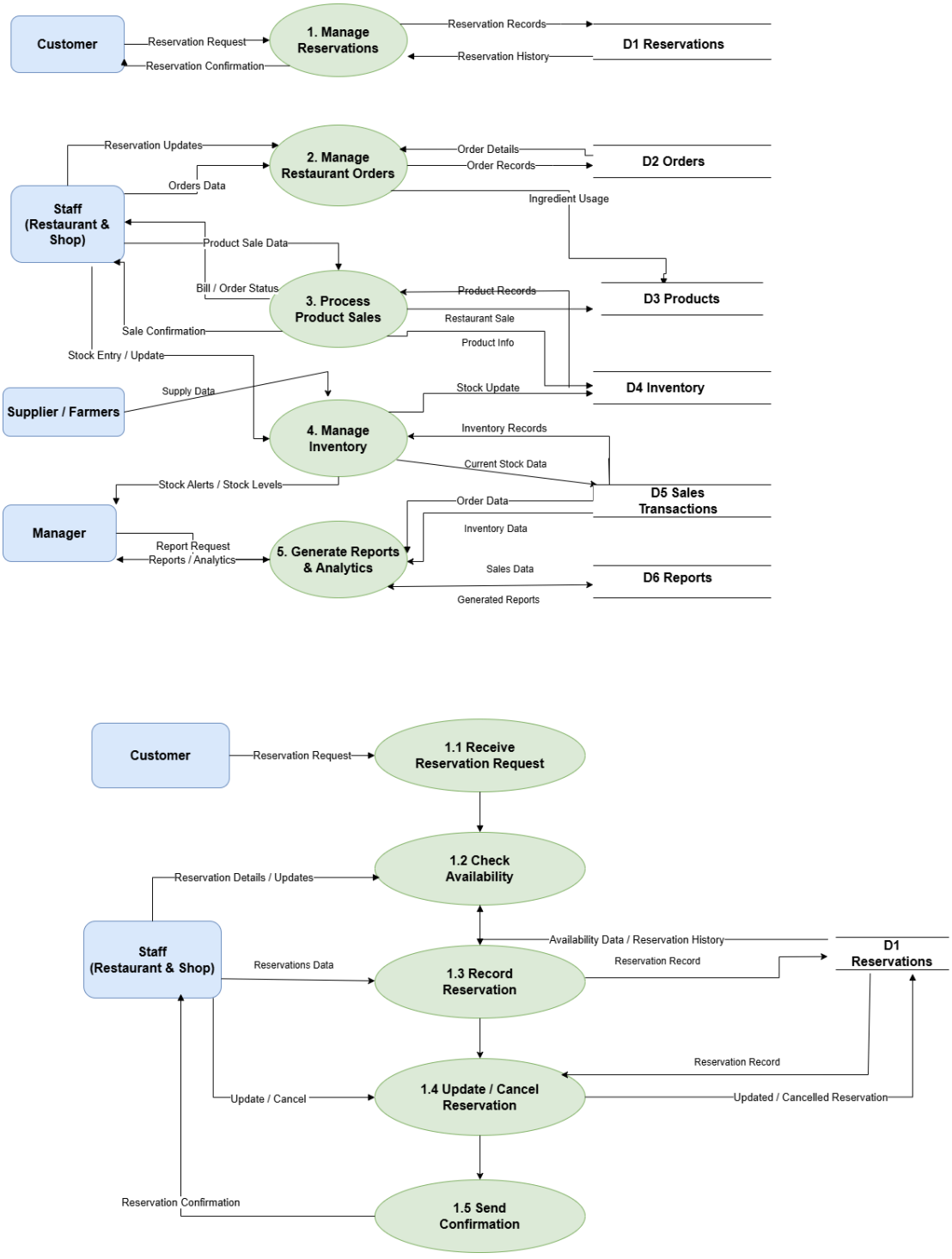


Figure: Dfd Level 1 Diagram

This DFD illustrates how data flows through AgriSoft at a specific level of detail.

Dfd Level 2 Diagram

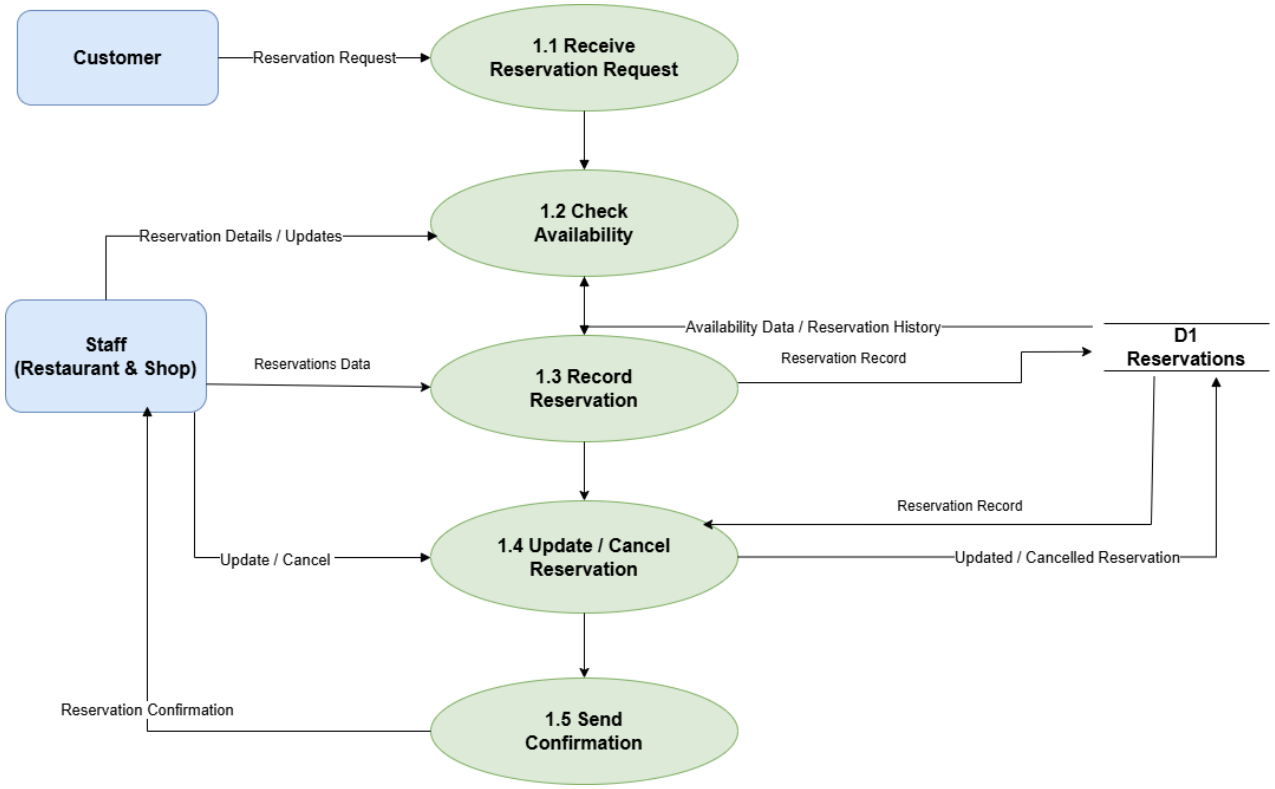


Figure: Dfd Level 2 Diagram

This DFD illustrates how data flows through AgriSoft at a specific level of detail.

4.13 Entity Relationship Diagram

The ERD models the persistent data required for the proposed full system. It includes customers, reservations, rooms/tables, products, suppliers, employees, departments, orders, and relationships between these entities.

Entity Relationship Diagram

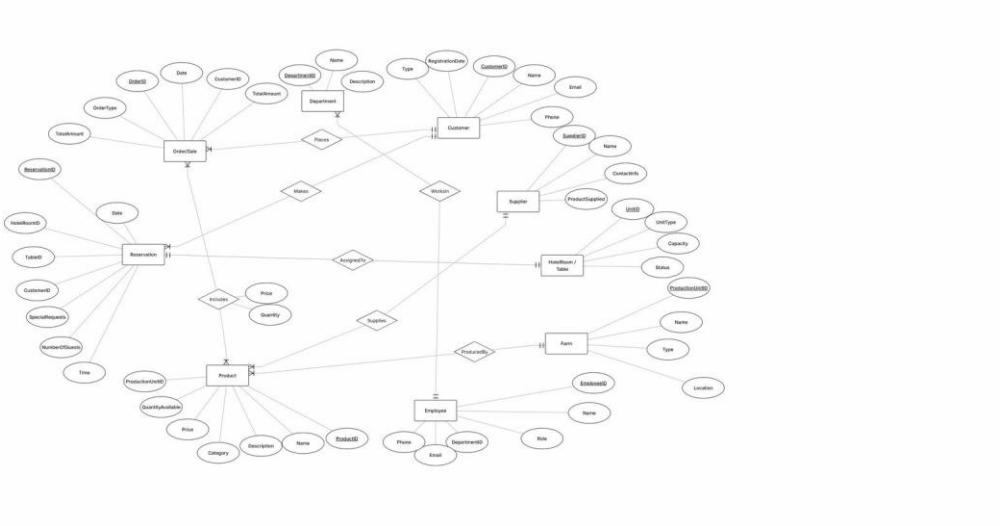


Figure: Entity Relationship Diagram

The ERD defines the main entities and relationships required for the proposed AgriSoft database.

4.14 Relational Schema

Relational Schema

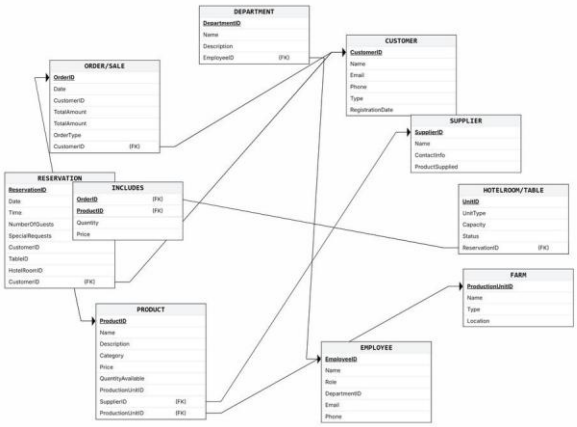


Figure: Relational Schema

The relational schema translates the ERD into tables, primary keys, foreign keys, and relationships.

The SQL file included in the project package provides an initial proposed schema. Some table and column names should be refined before implementation, for example avoiding special characters in table names and using appropriate text/date/decimal data types rather than INT for all fields.

4.15 Class Diagrams

Class diagrams define the static structure of the software model. They identify classes, attributes, operations, and associations. AgriSoft provides module-level class diagrams for authentication, reservation, restaurant, inventory, product sales, and reporting.

Authentication Class Diagram

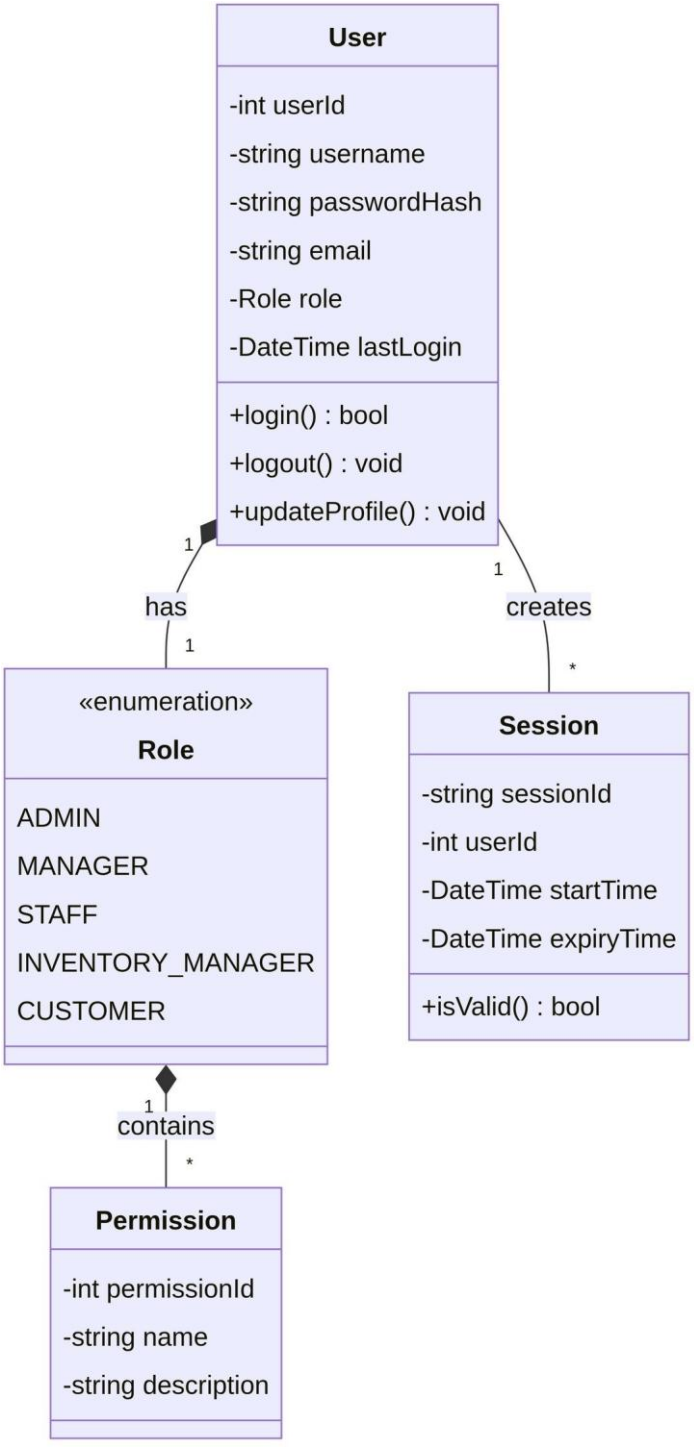


Figure: Authentication Class Diagram

This class diagram presents the structural design for one AgriSoft module.

Inventory Class Diagram

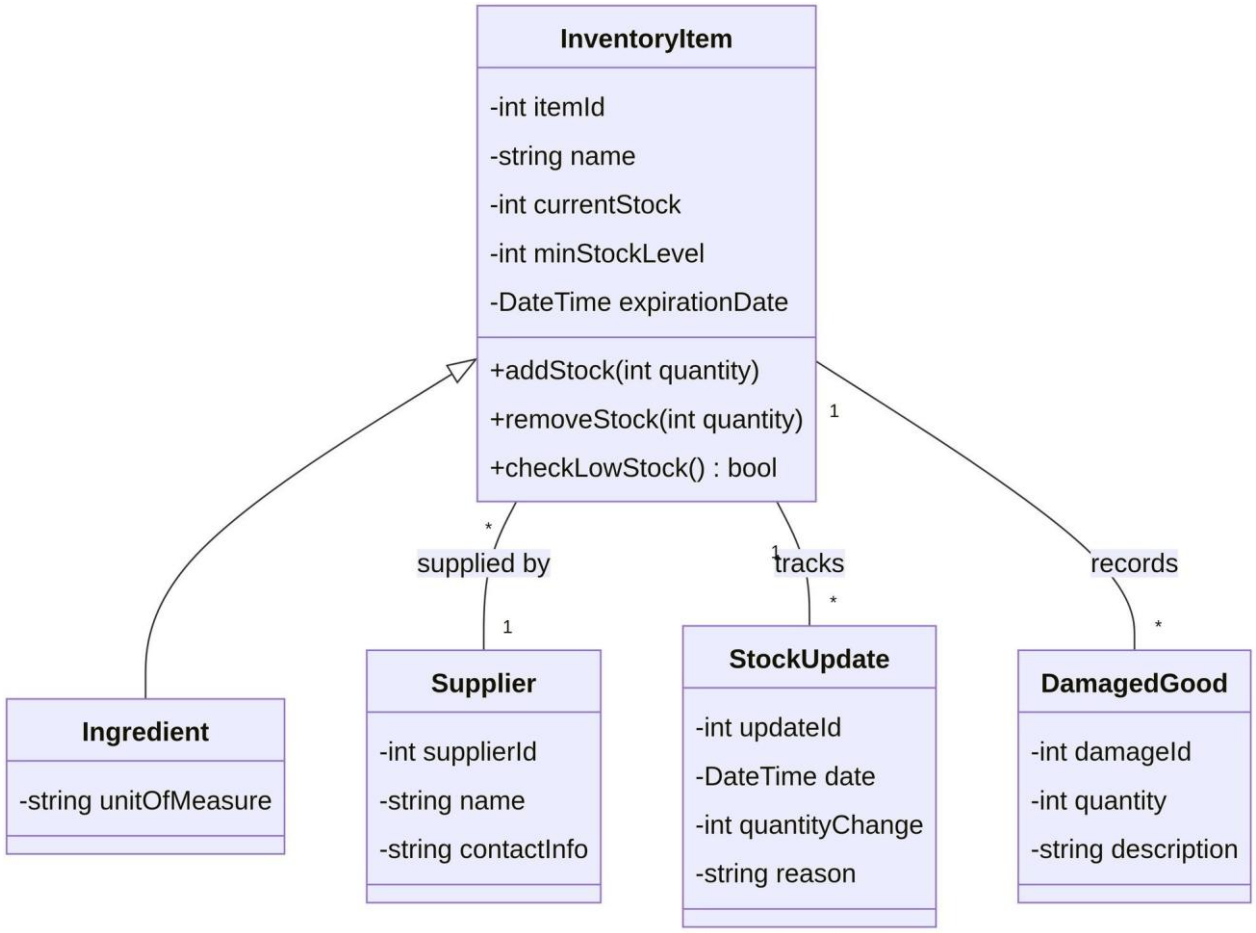


Figure: Inventory Class Diagram

This class diagram presents the structural design for one AgriSoft module.

Productsales Class Diagram

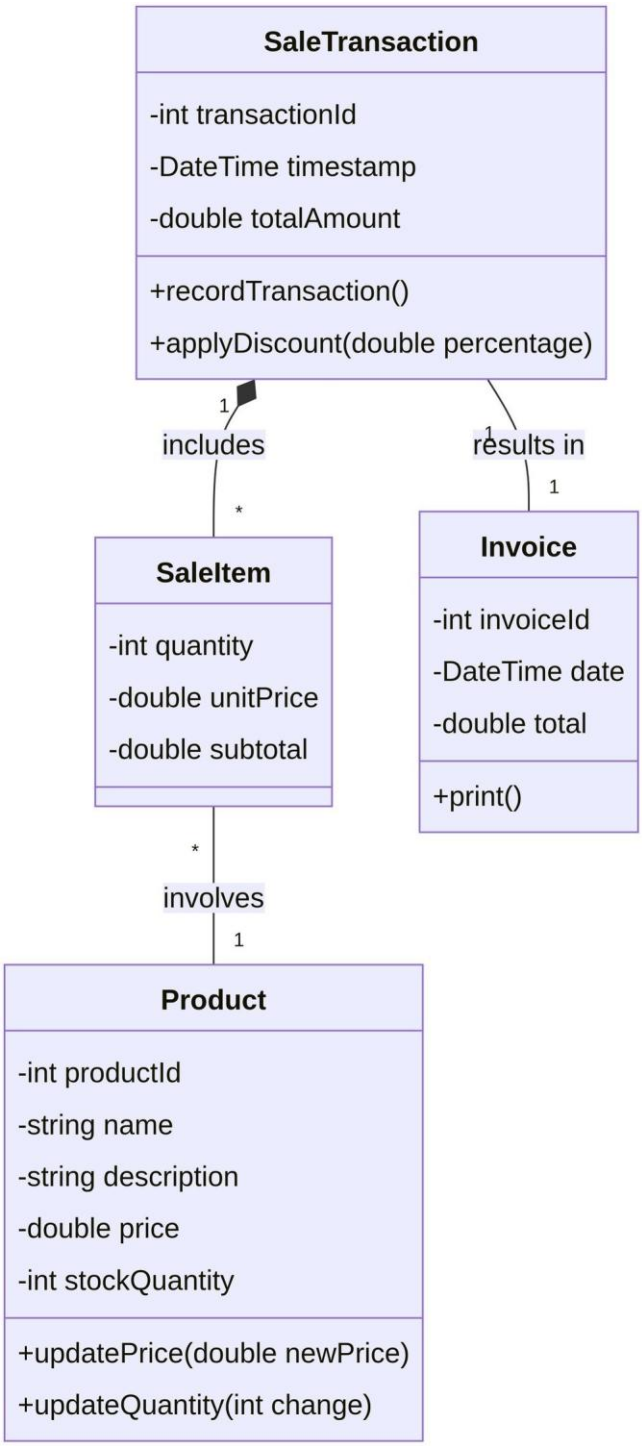


Figure: Productsales Class Diagram

This class diagram presents the structural design for one AgriSoft module.

Reporting Class Diagram

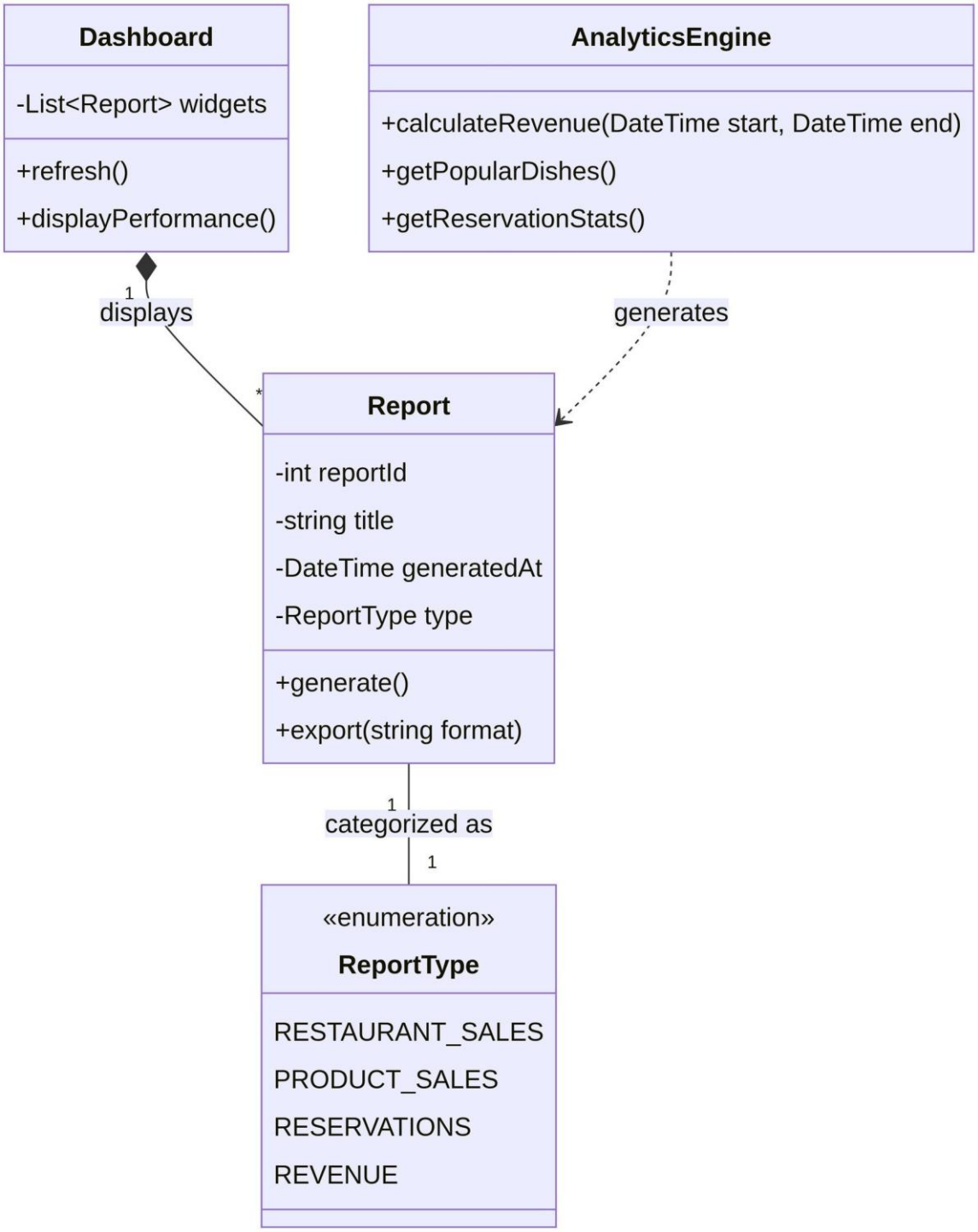


Figure: Reporting Class Diagram

This class diagram presents the structural design for one AgriSoft module.

Reservation Class Diagram

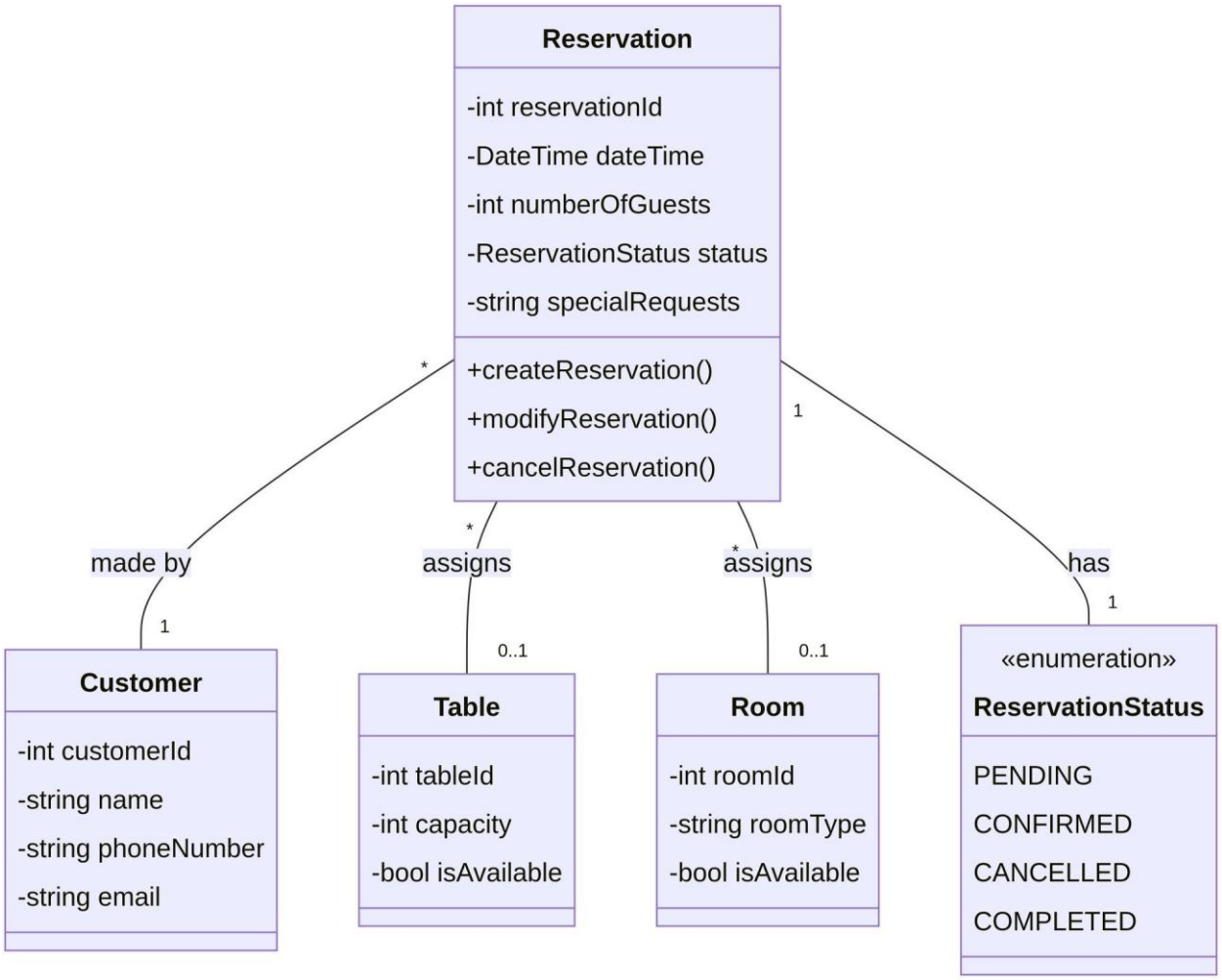


Figure: Reservation Class Diagram

This class diagram presents the structural design for one AgriSoft module.

Restaurant Class Diagram

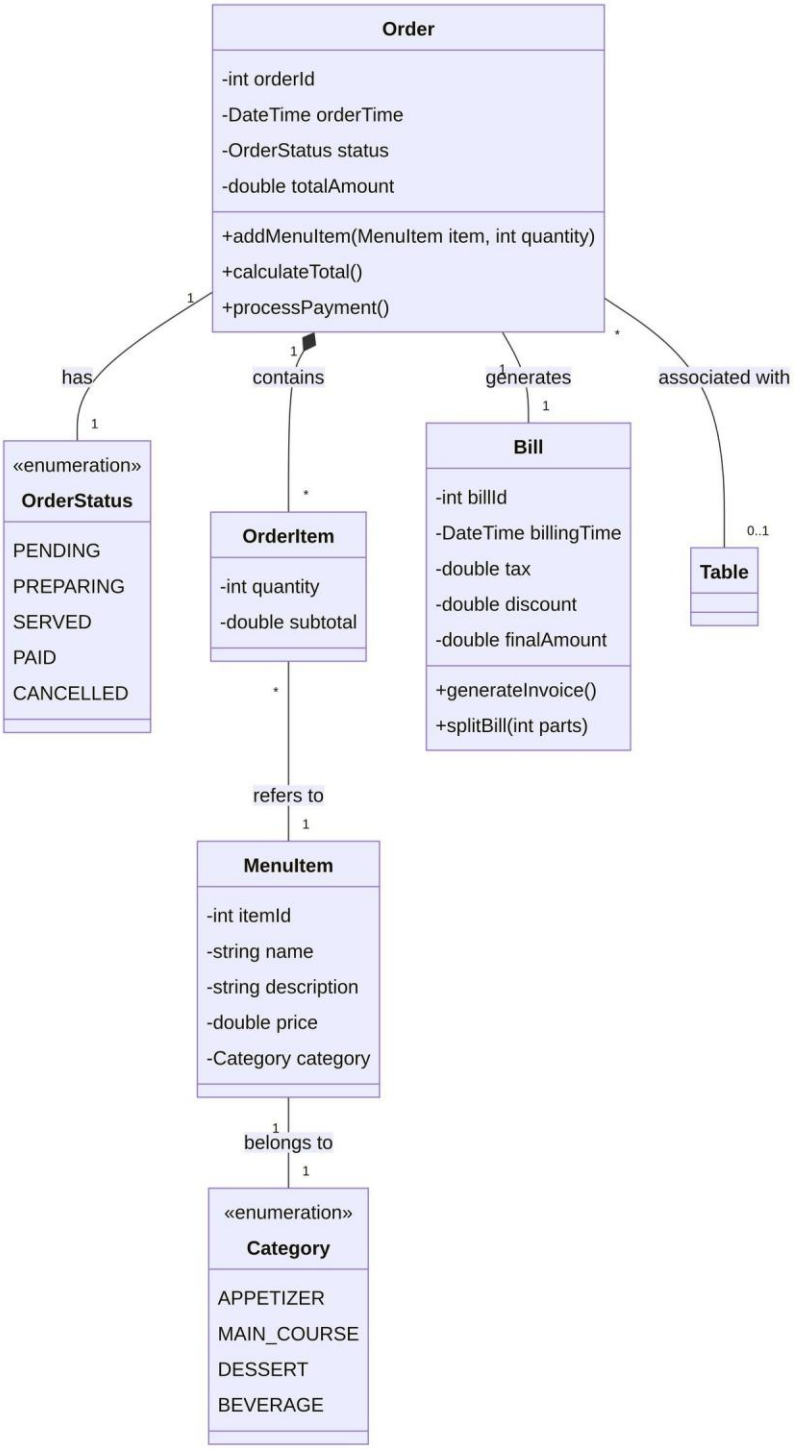


Figure: Restaurant Class Diagram

This class diagram presents the structural design for one AgriSoft module.

4.16 Object Diagrams

Object diagrams show concrete runtime examples of the class structure. They are useful for validating whether the modeled classes can represent real use cases. The following object diagrams were generated from the AgriSoft scenarios and are included as editable-image based visual documentation.

Customer Reservation Object Diagram

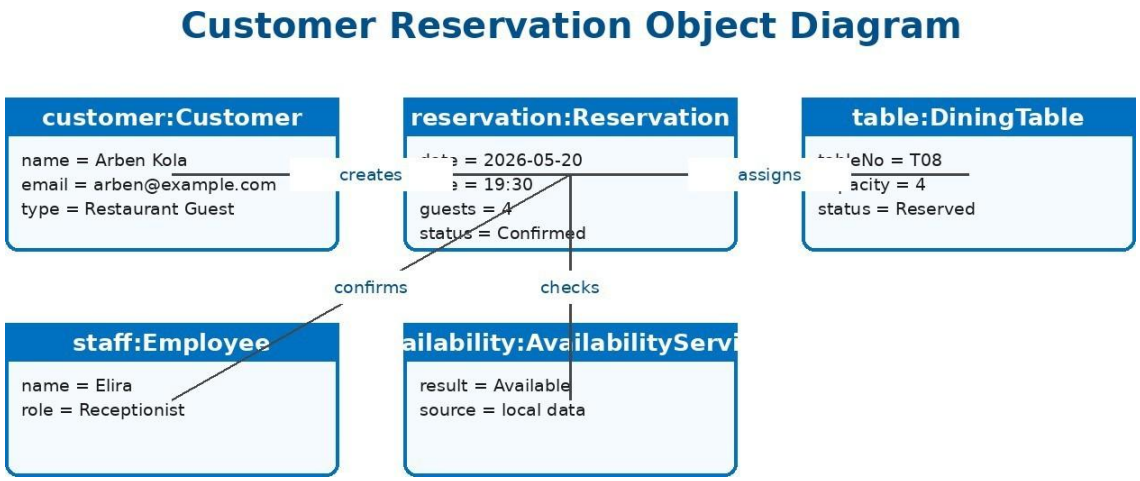


Figure: Customer Reservation Object Diagram

This object diagram provides a concrete example of objects, attributes, and links for a realistic AgriSoft scenario.

Hotel Room Booking Object Diagram

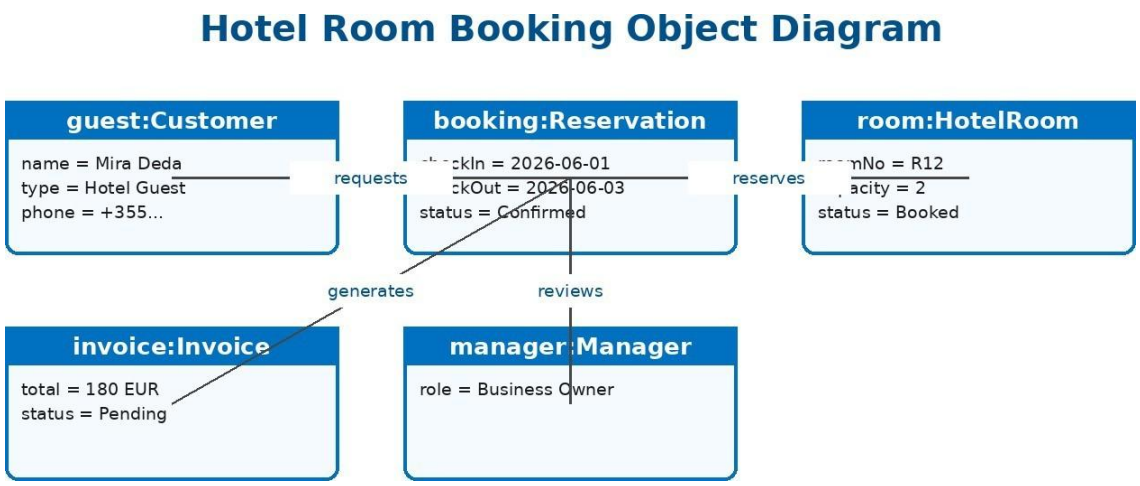


Figure: Hotel Room Booking Object Diagram

This object diagram provides a concrete example of objects, attributes, and links for a realistic AgriSoft scenario.

Inventory Stock Object Diagram

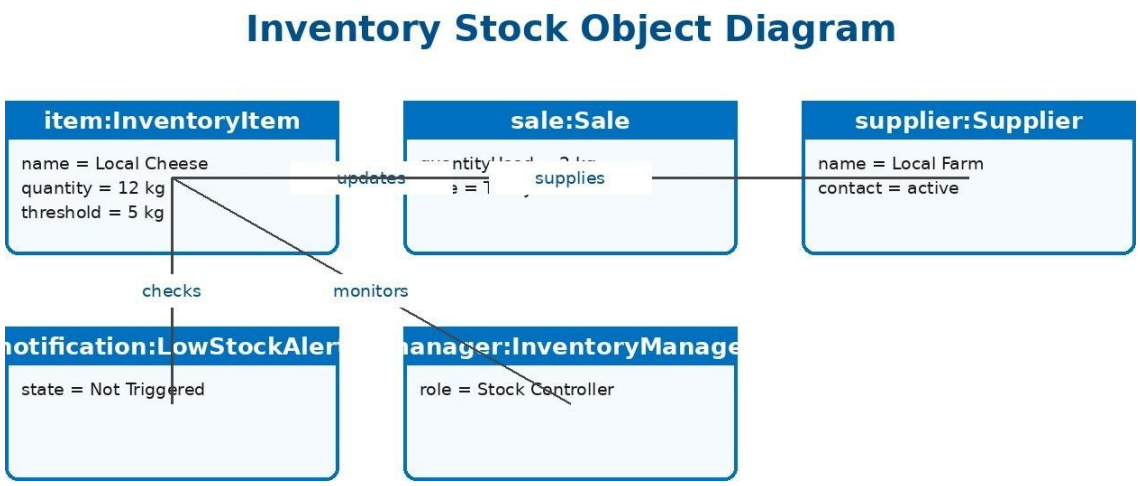


Figure: Inventory Stock Object Diagram

This object diagram provides a concrete example of objects, attributes, and links for a realistic AgriSoft scenario.

Product Sale Object Diagram

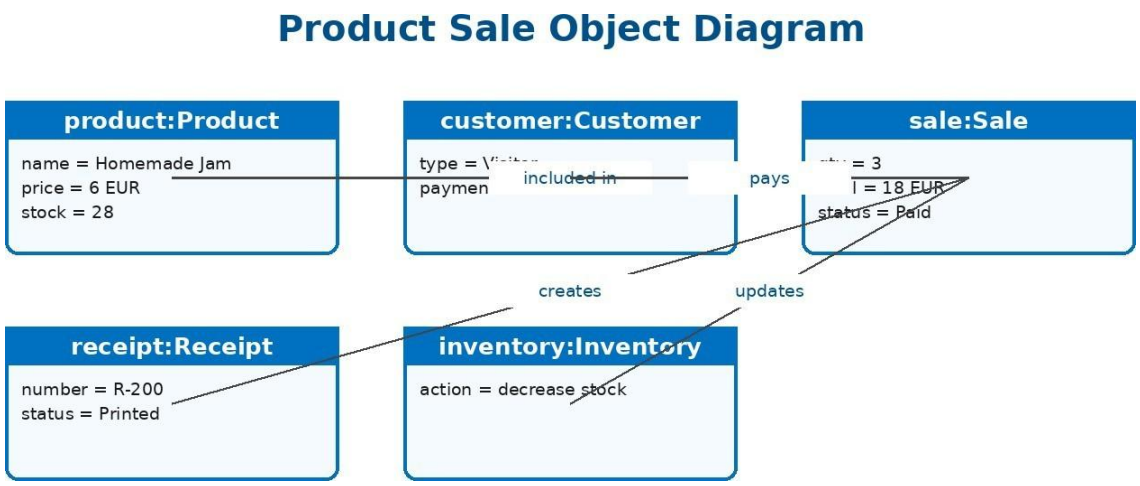


Figure: Product Sale Object Diagram

This object diagram provides a concrete example of objects, attributes, and links for a realistic AgriSoft scenario.

Reporting Object Diagram

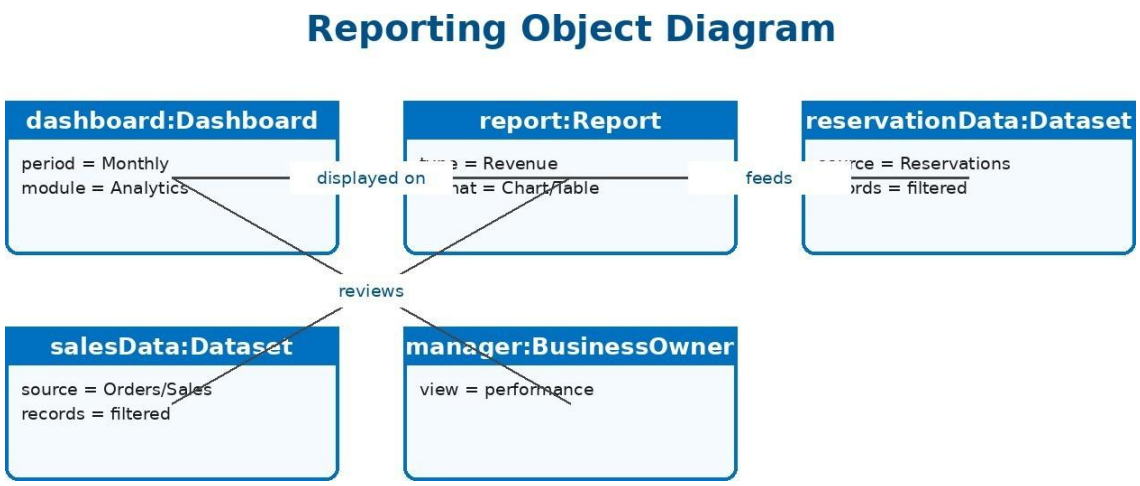


Figure: Reporting Object Diagram

This object diagram provides a concrete example of objects, attributes, and links for a realistic AgriSoft scenario.

Restaurant Order Object Diagram

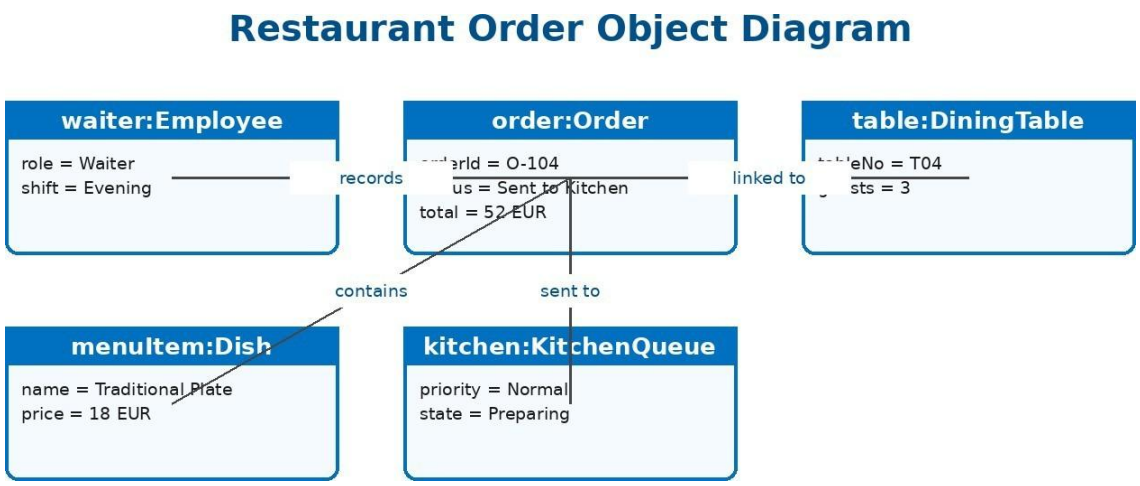


Figure: Restaurant Order Object Diagram

This object diagram provides a concrete example of objects, attributes, and links for a realistic AgriSoft scenario.

4.17 Package Diagram

Package diagrams organize the system into logical groups. AgriSoft includes module-level package diagrams that show dependencies between reservation, order flow, product sales, inventory, and reporting packages.

Inventory

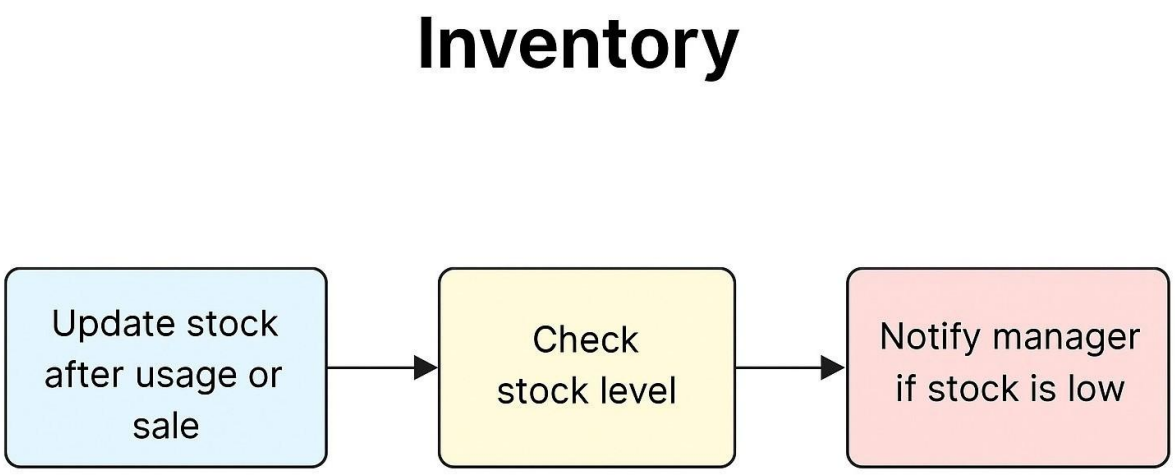


Figure: Inventory

This package diagram organizes related classes and responsibilities into a module-level structure.

Orderflow

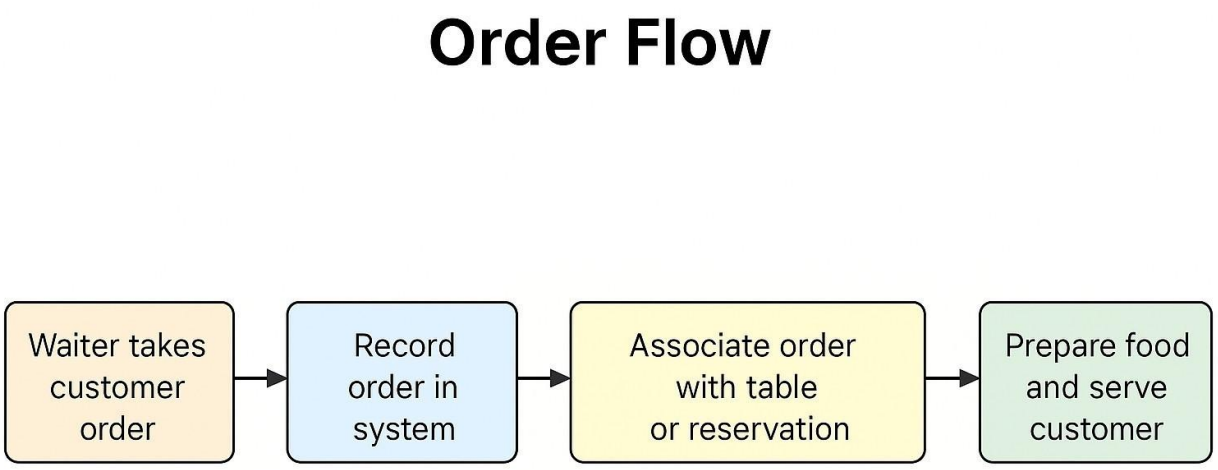


Figure: Orderflow

This package diagram organizes related classes and responsibilities into a module-level structure.

Productsales

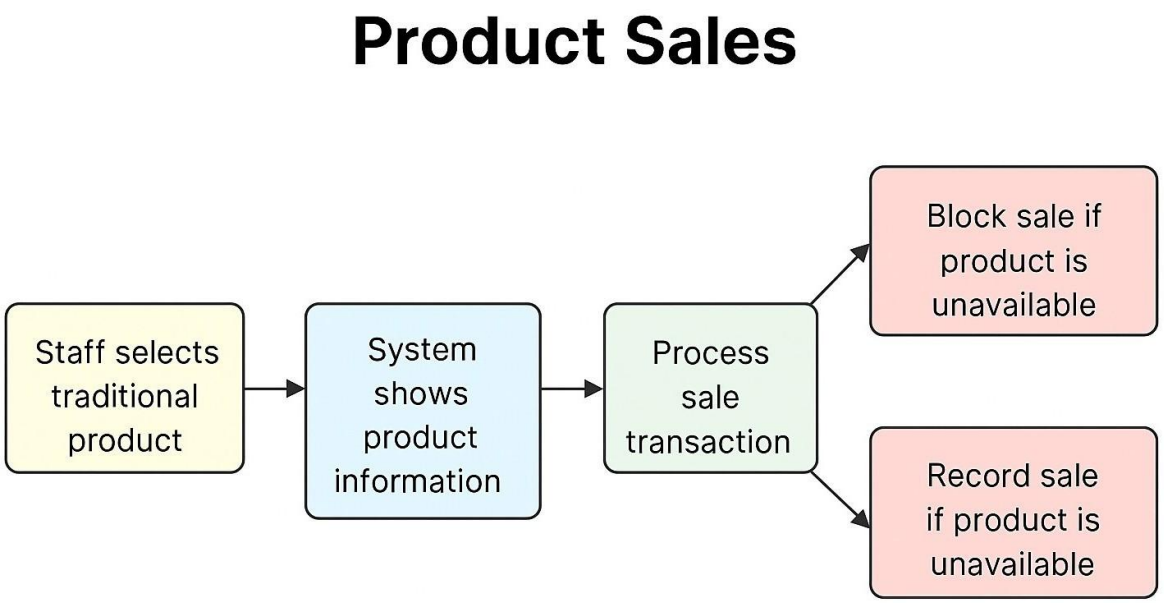


Figure: Productsales

This package diagram organizes related classes and responsibilities into a module-level structure.

Reporting

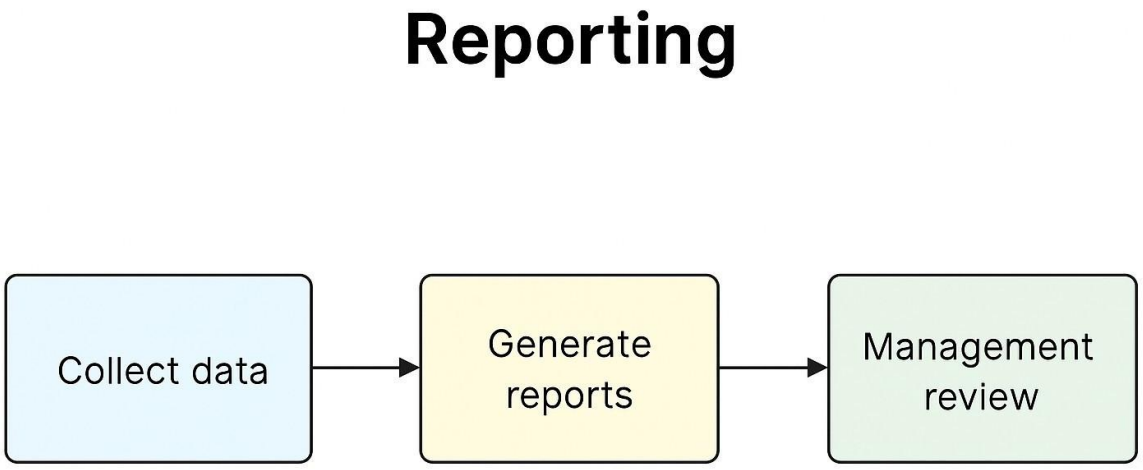


Figure: Reporting

This package diagram organizes related classes and responsibilities into a module-level structure.

Reservation

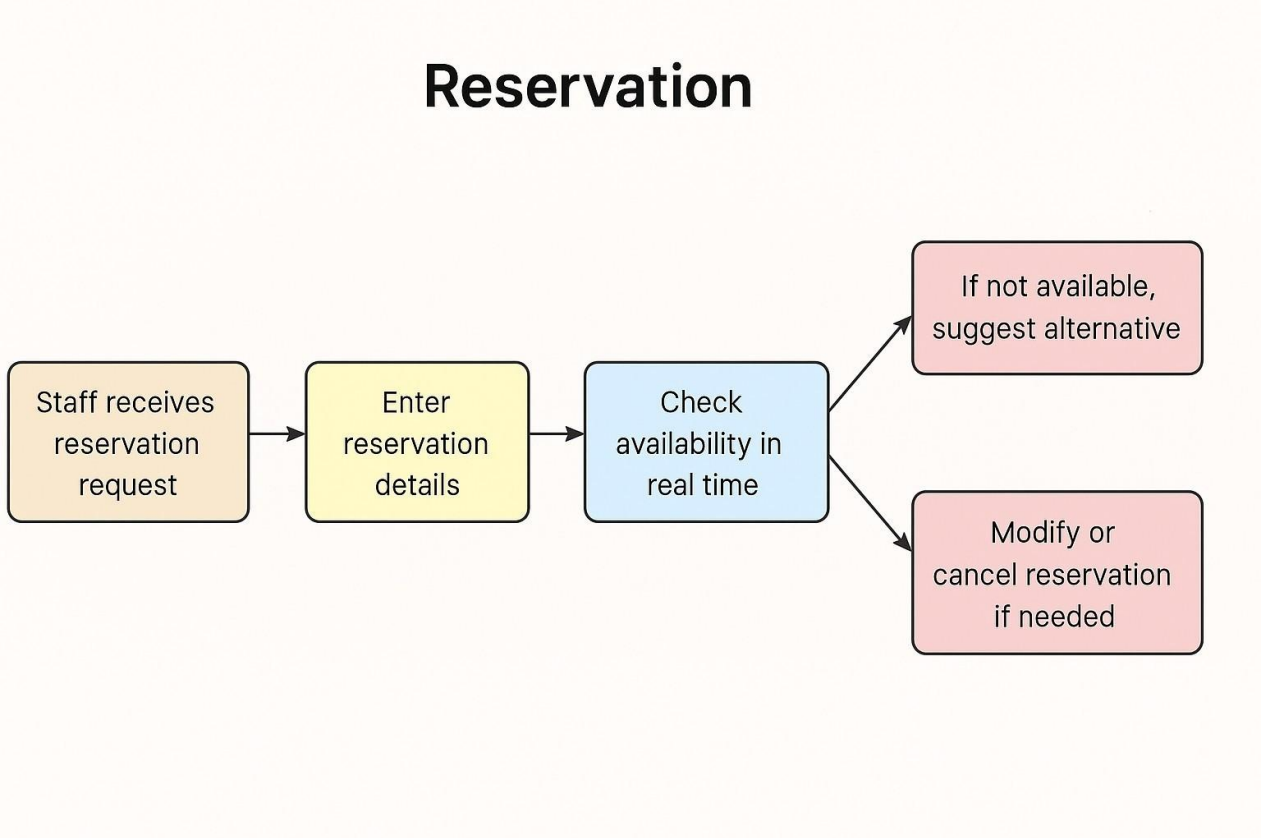


Figure: Reservation

This package diagram organizes related classes and responsibilities into a module-level structure.

4.18 Composite Structure Diagram

Composite structure diagrams show the internal parts of a structured classifier and how they collaborate. In AgriSoft, they are useful for explaining how each major module is internally composed of UI, service, validation, and data parts.

Inventory

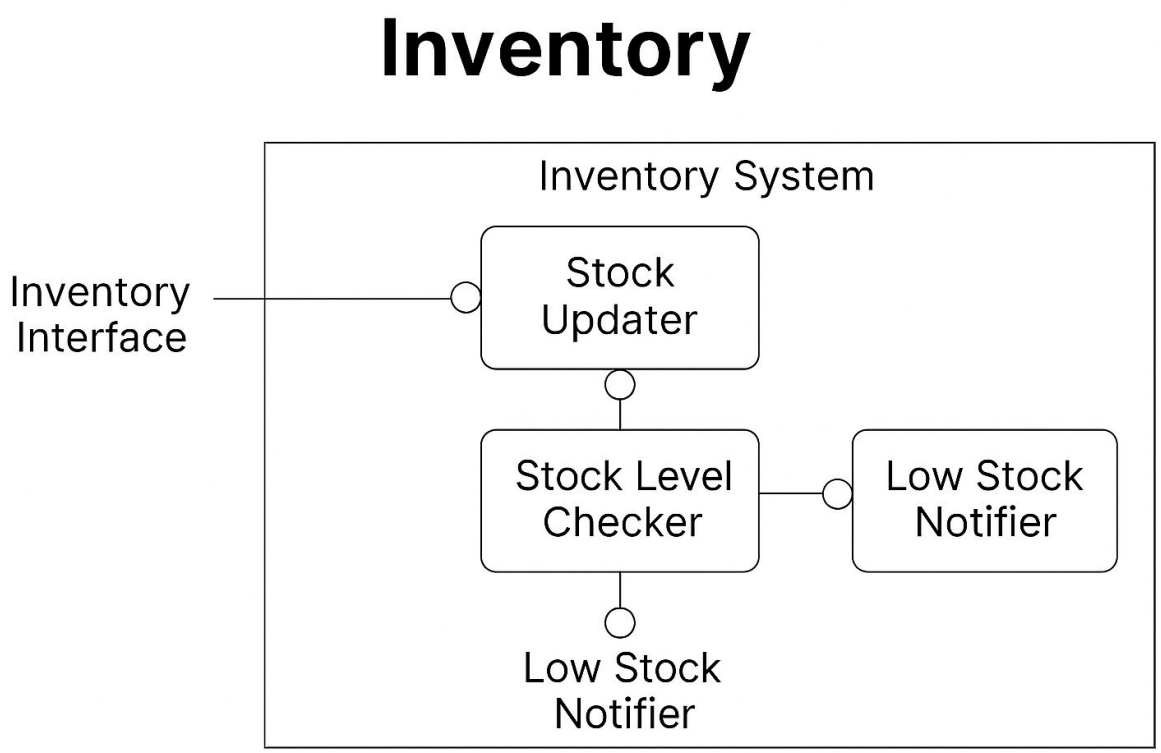


Figure: Inventory

This composite structure diagram shows the internal collaboration between parts of one AgriSoft module.

Orderflow

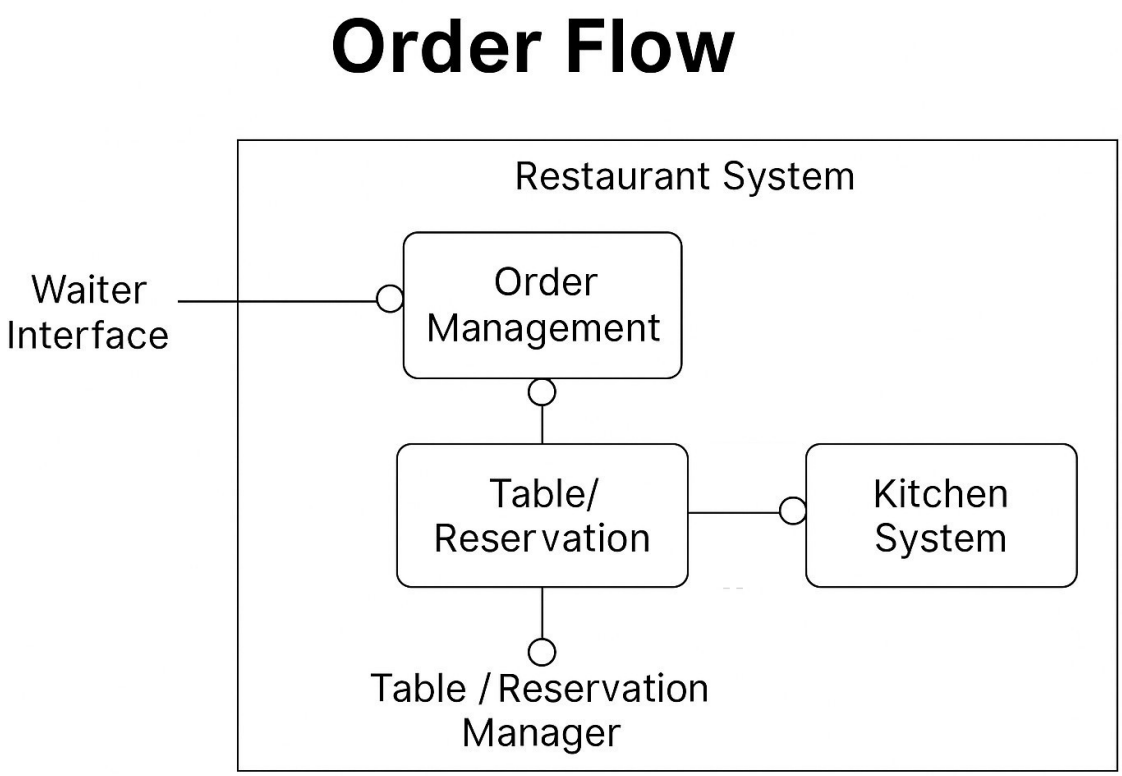


Figure: Orderflow

This composite structure diagram shows the internal collaboration between parts of one AgriSoft module.

Reservation

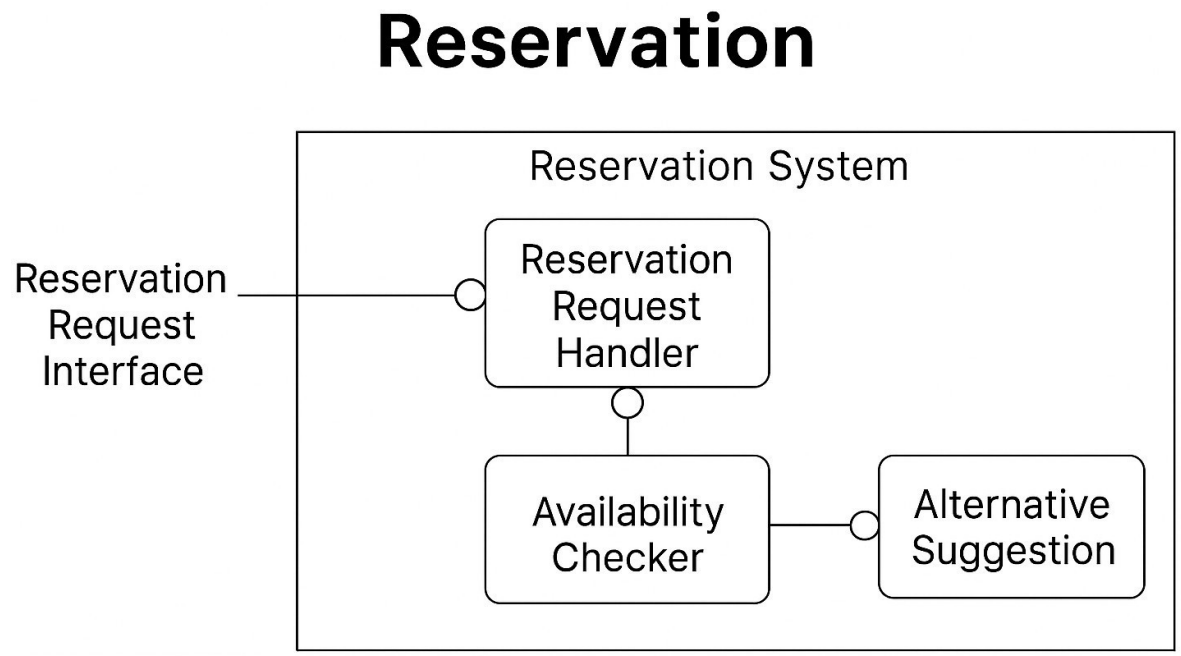


Figure: Reservation

This composite structure diagram shows the internal collaboration between parts of one AgriSoft module.

Productsales

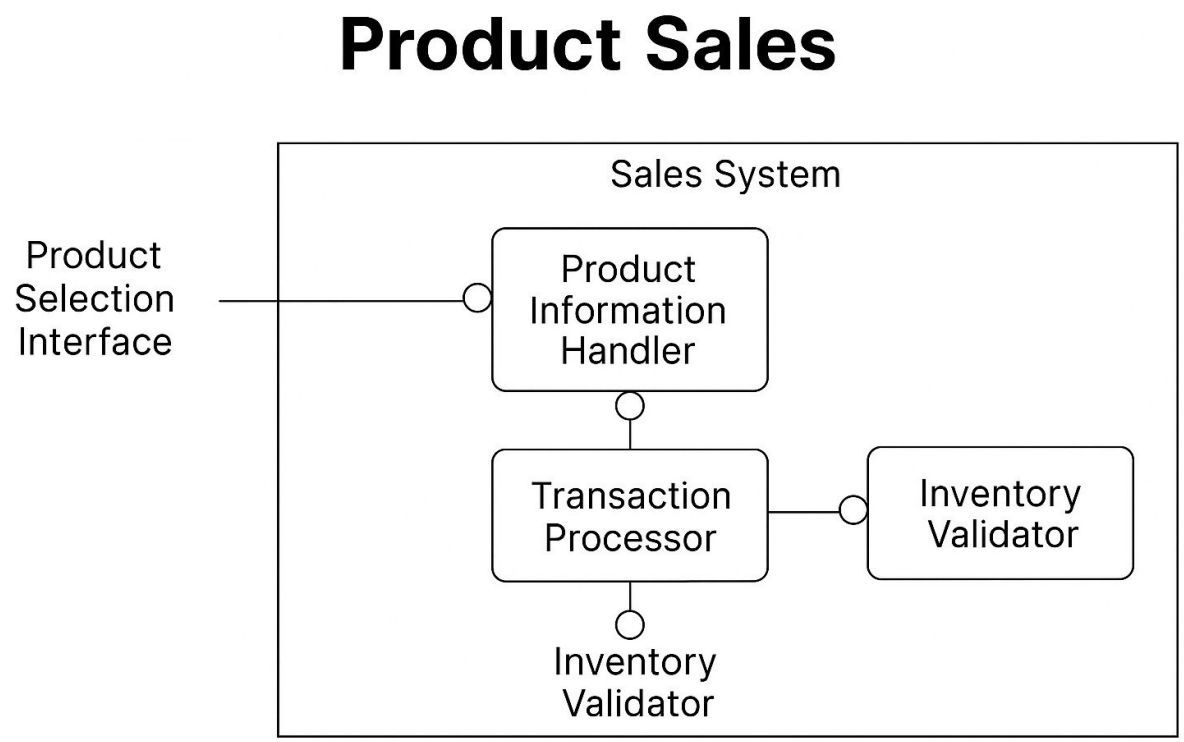


Figure: Productsales

This composite structure diagram shows the internal collaboration between parts of one AgriSoft module.

Reporting

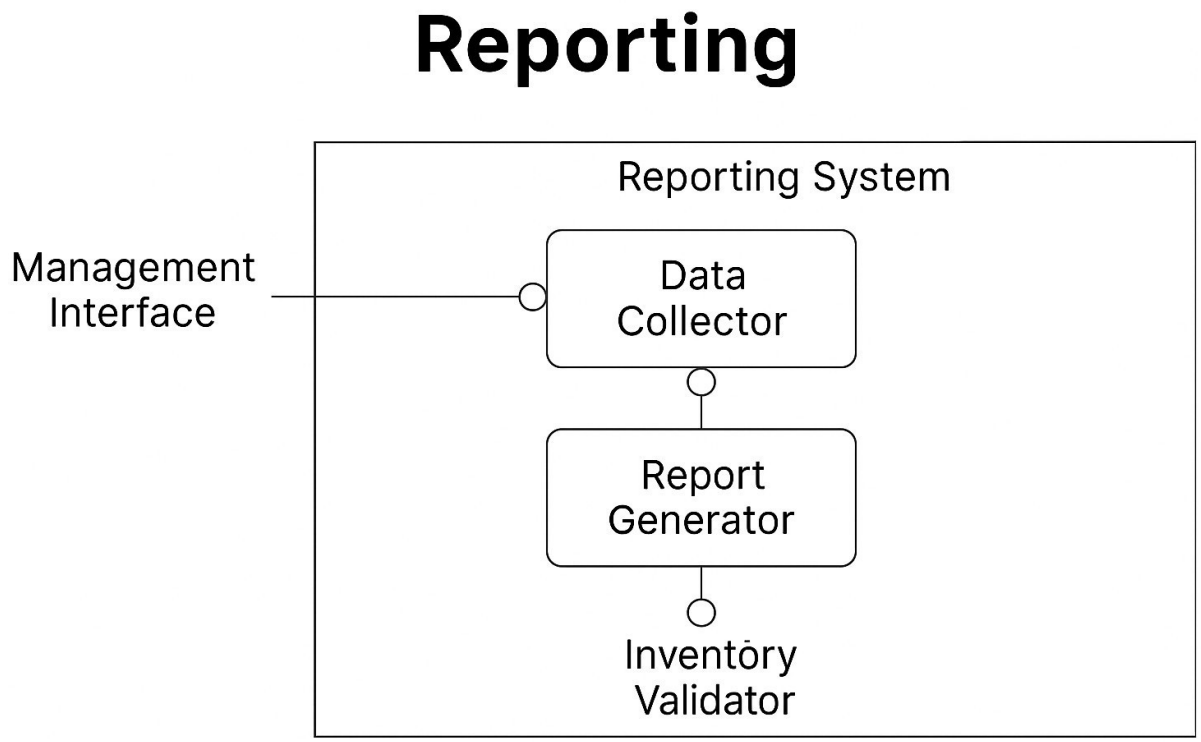


Figure: Reporting

This composite structure diagram shows the internal collaboration between parts of one AgriSoft module.

4.19 Deployment Diagram

The deployment diagram represents the proposed runtime environment. The current project is primarily frontend- based, but the deployment model includes client devices, a web application host, a future API/backend server, and a future database server.

Deployment Diagram Agrisoft

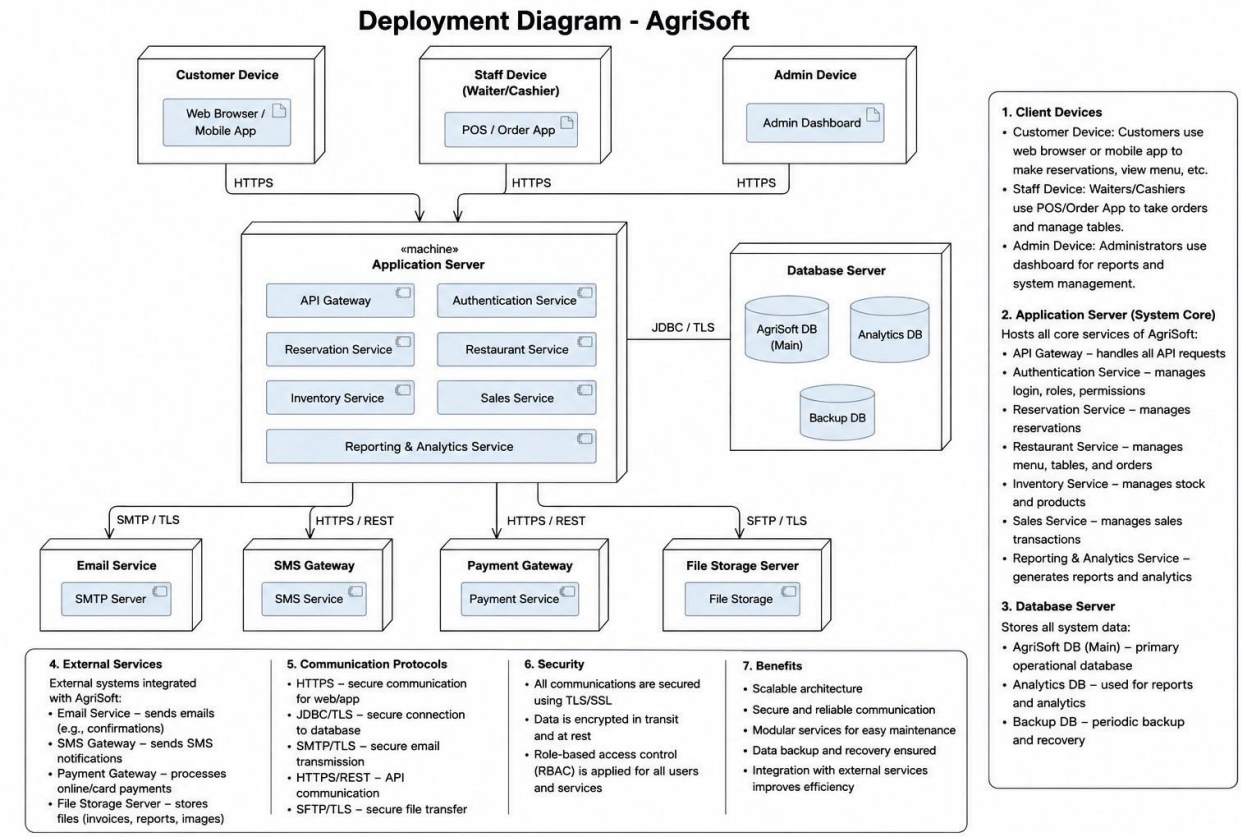


Figure: Deployment Diagram Agrisoft

This deployment diagram represents the proposed production architecture for AgriSoft.

4.20 Component Diagram

Overall Component Diagram

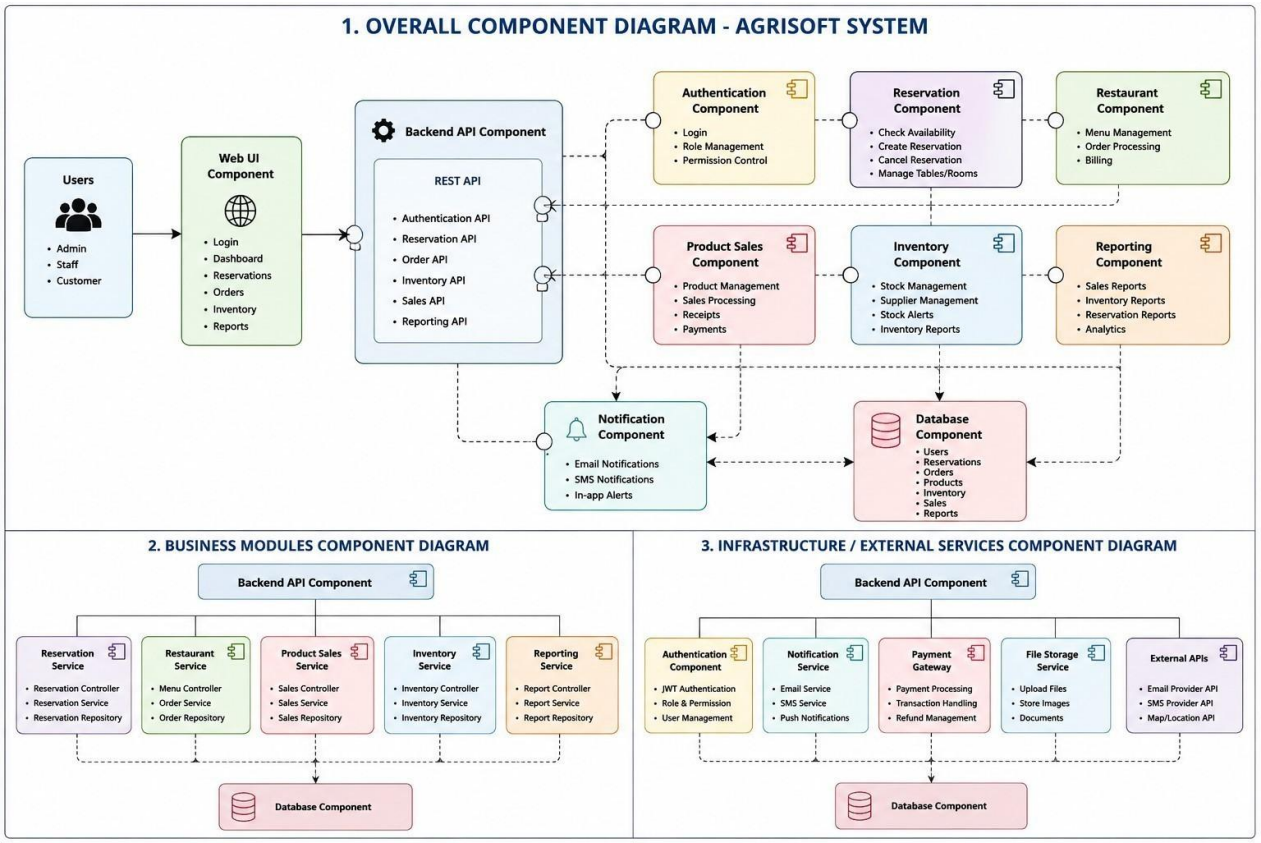


Figure: Overall Component Diagram

This component diagram describes the high-level building blocks of AgriSoft and their dependencies.

5 Design Patterns

Design patterns provide proven solutions to recurring design problems. For AgriSoft, the most suitable patterns are those that support separation of concerns, data access abstraction, simplified interfaces, lifecycle control, and event- driven updates. The selected patterns are MVC, Repository, Facade, State, and Observer.

MVC Pattern

Separates interface, business behavior, and data representation. This supports a clean architecture where routes and components form the view layer, services and hooks support control logic, and data models represent domain data.

Table: MVC Pattern Summary

Aspect	Explanation
Problem solved	MVC Pattern helps organize AgriSoft behavior in a maintainable way.
Participants	UI components, module services, domain models, mock data or future repositories.
Benefit	Improves maintainability, reusability, and academic correctness of the system design.

Repository Pattern

Separates data access from business logic. In the current prototype, mock data acts as a temporary data source; in the future, repositories can connect to a database or API without changing UI components.

Table: Repository Pattern Summary

Aspect	Explanation
Problem solved	Repository Pattern helps organize AgriSoft behavior in a maintainable way.
Participants	UI components, module services, domain models, mock data or future repositories.
Benefit	Improves maintainability, reusability, and academic correctness of the system design.

Facade Pattern

Provides simplified access to complex subsystem behavior. For example, a reservation facade can coordinate validation, availability checking, customer data, and confirmation messages.

Table: Facade Pattern Summary

Aspect	Explanation
Problem solved	Facade Pattern helps organize AgriSoft behavior in a maintainable way.
Participants	UI components, module services, domain models, mock data or future repositories.
Benefit	Improves maintainability, reusability, and academic correctness of the system design.

State Pattern

Represents lifecycle changes such as Reservation Pending, Confirmed, Cancelled, or Completed; Order Created, Sent to Kitchen, Prepared, Served, Paid; and Product Available, Low Stock, Out of Stock.

Table: State Pattern Summary

Aspect	Explanation
Problem solved	State Pattern helps organize AgriSoft behavior in a maintainable way.
Participants	UI components, module services, domain models, mock data or future repositories.
Benefit	Improves maintainability, reusability, and academic correctness of the system design.

Observer Pattern

Supports notification when data changes, such as inventory low-stock alerts, dashboard refreshes after sales, or reservation updates.

Table: Observer Pattern Summary

Aspect	Explanation
Problem solved	Observer Pattern helps organize AgriSoft behavior in a maintainable way.
Participants	UI components, module services, domain models, mock data or future repositories.
Benefit	Improves maintainability, reusability, and academic correctness of the

	system design.

Designpattern 02

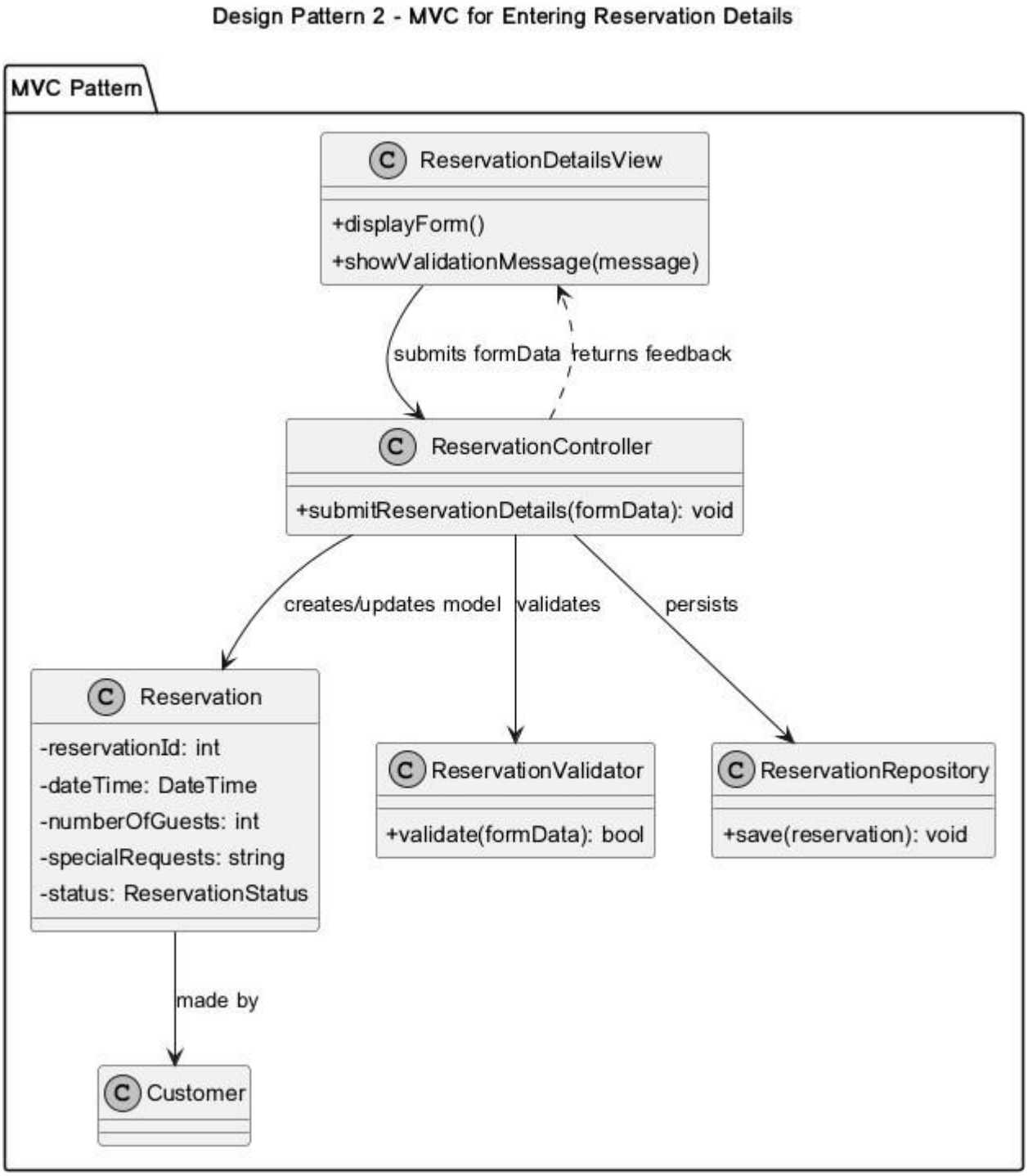


Figure: Designpattern 02

This design pattern diagram is included from the project package and documents a proposed pattern application for AgriSoft.

Designpattern 07

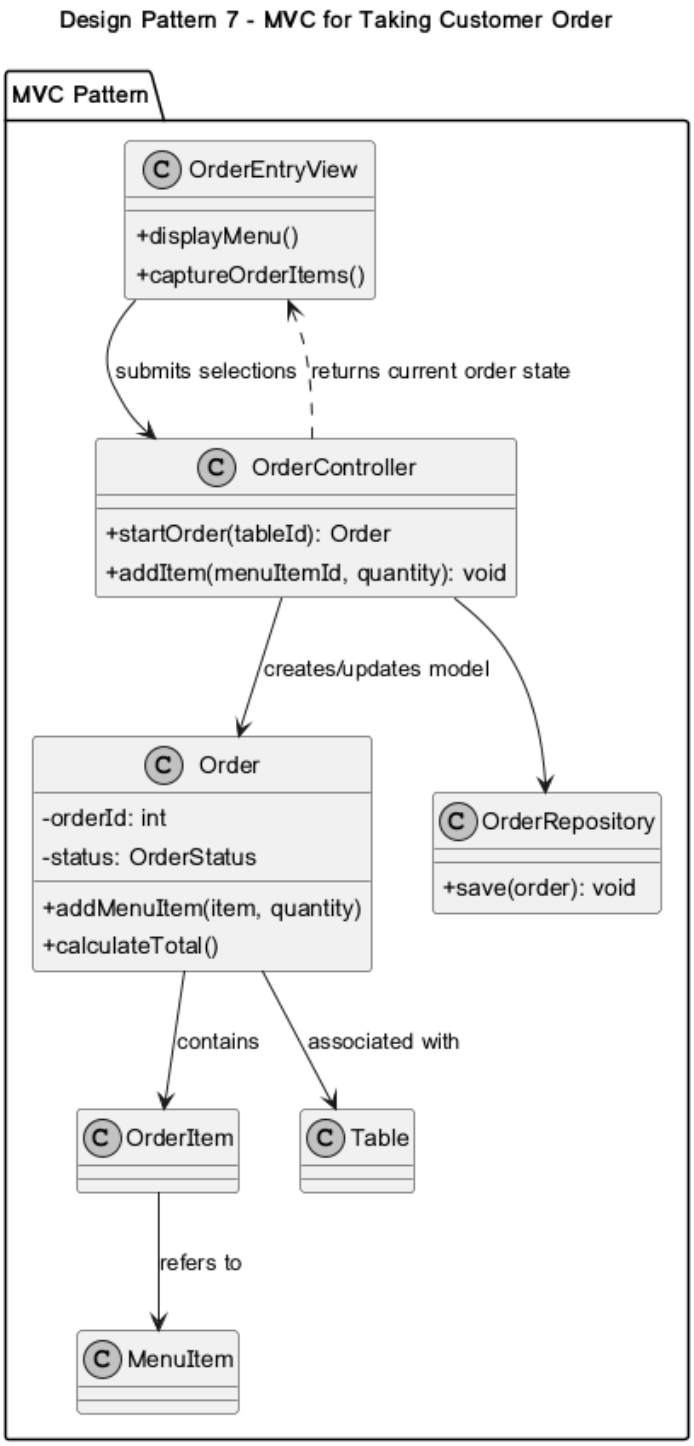


Figure: Designpattern 07

This design pattern diagram is included from the project package and documents a proposed pattern application for AgriSoft.

Designpattern 08

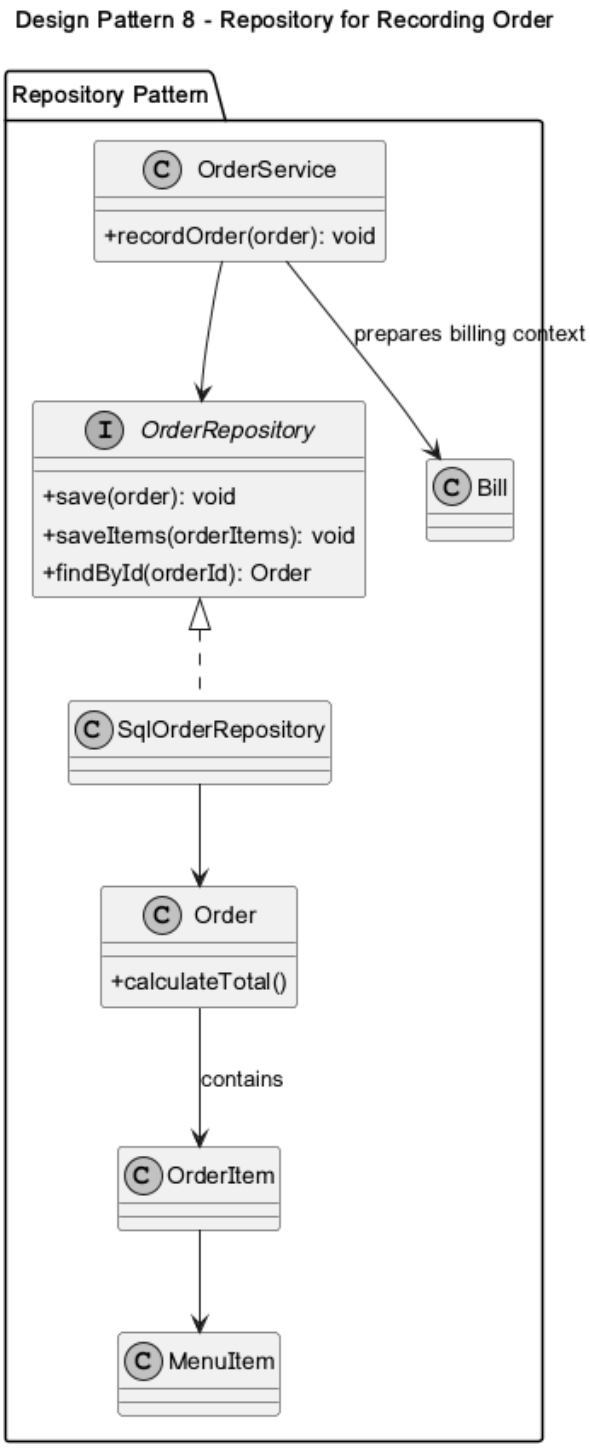


Figure: Designpattern 08

This design pattern diagram is included from the project package and documents a proposed pattern application for AgriSoft.

Designpattern 09

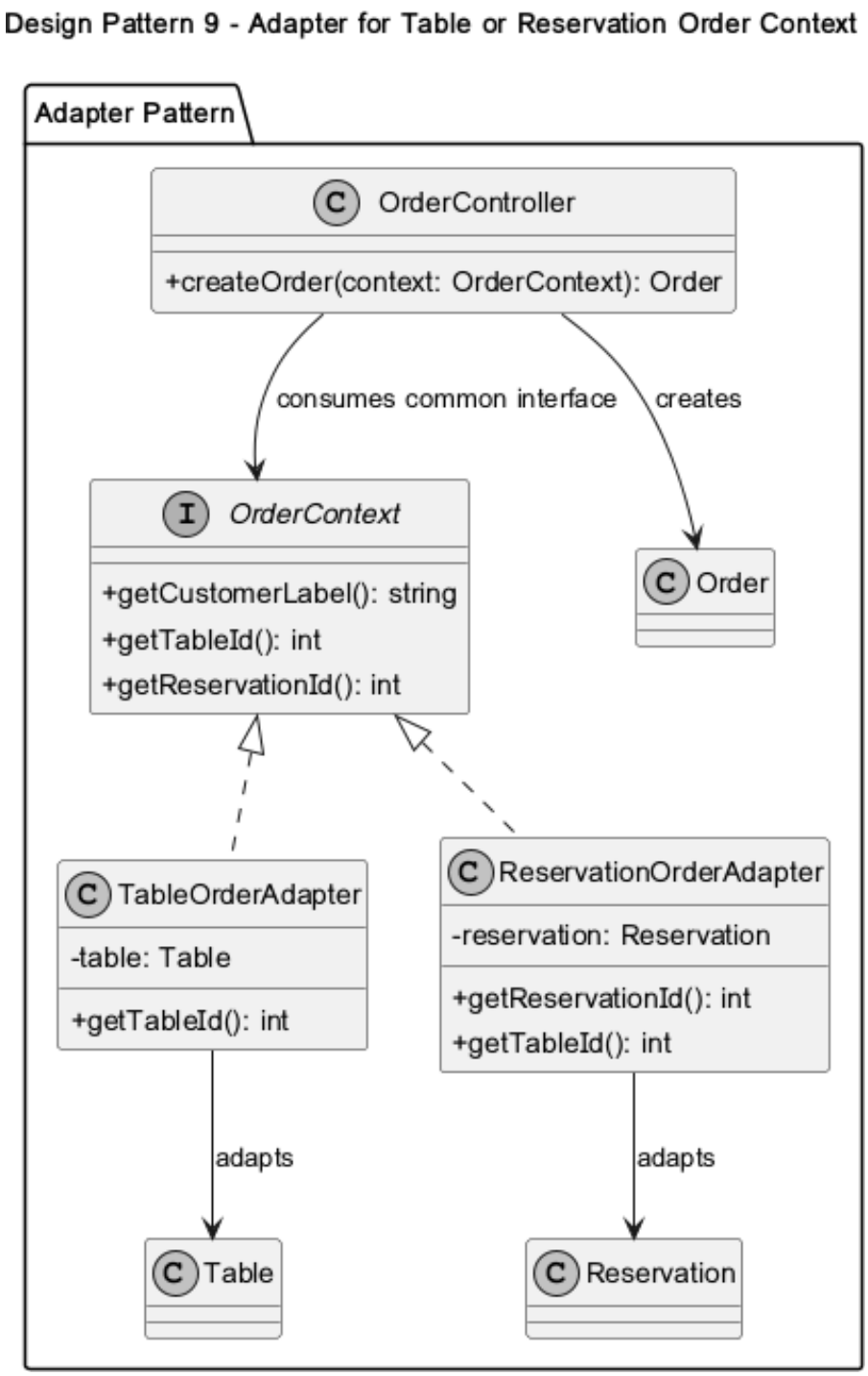


Figure: Designpattern 09

This design pattern diagram is included from the project package and documents a proposed pattern application for AgriSoft.

Designpattern 10

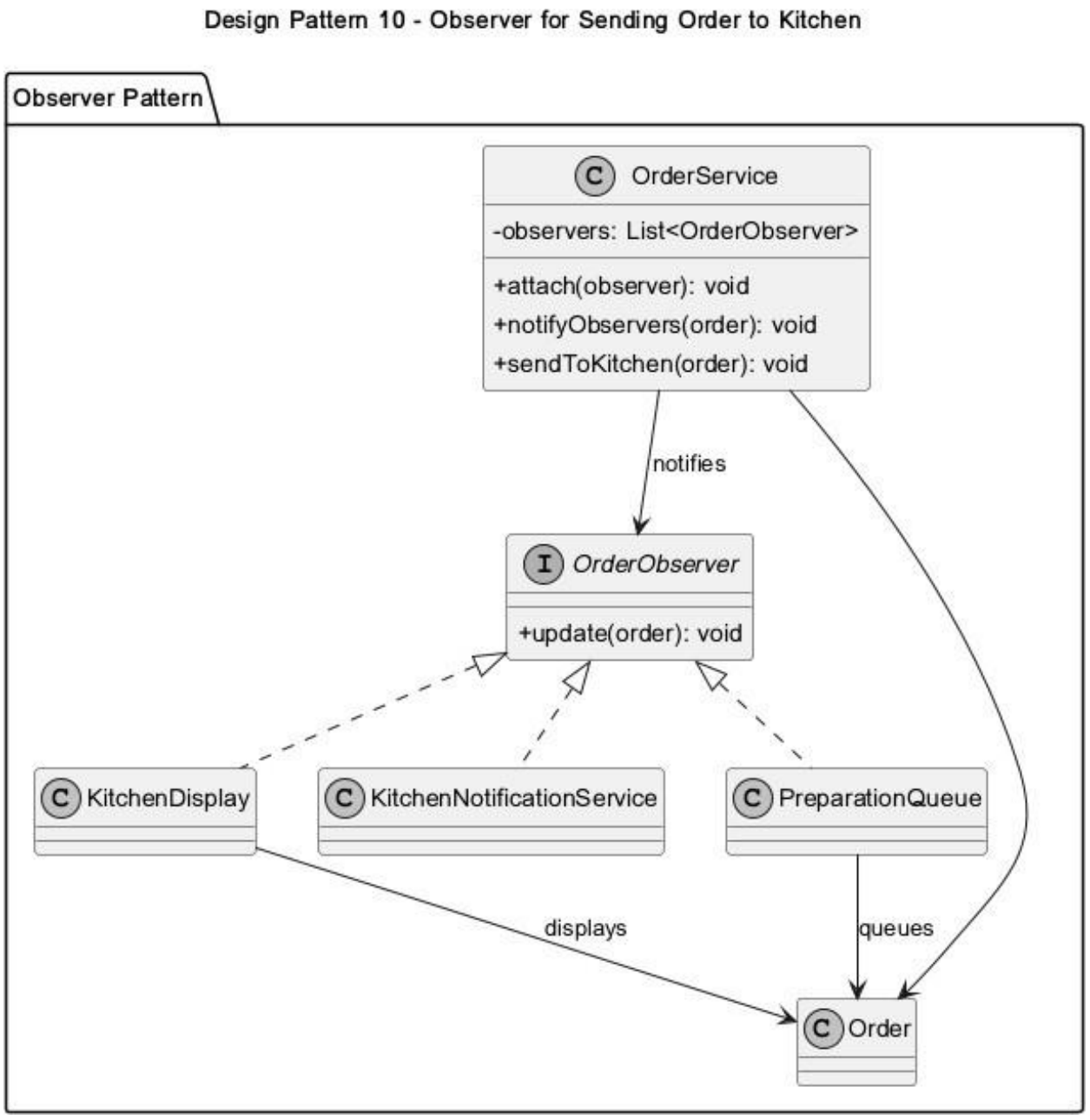


Figure: Designpattern 10

This design pattern diagram is included from the project package and documents a proposed pattern application for AgriSoft.

Designpattern 12

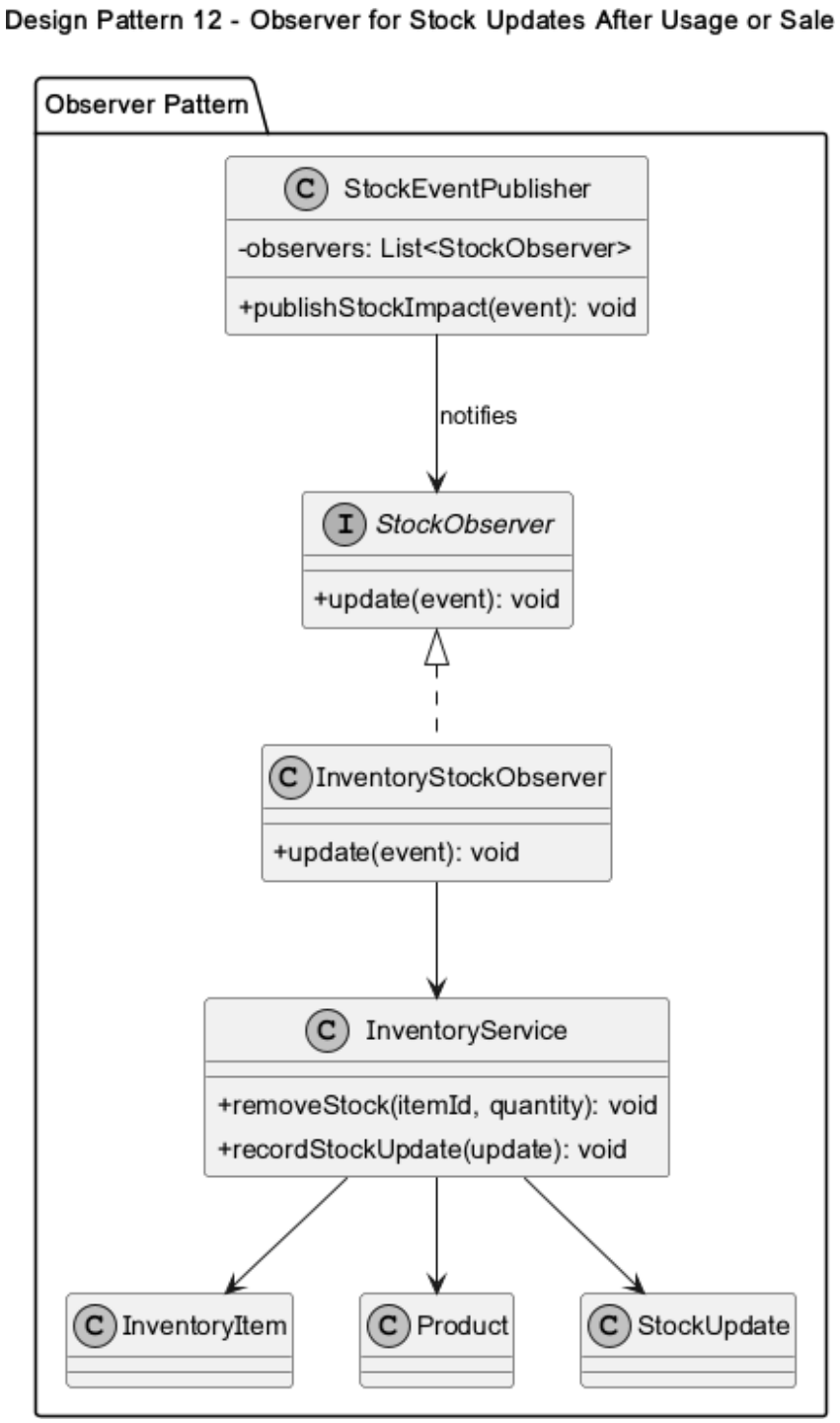


Figure: Designpattern 12

This design pattern diagram is included from the project package and documents a proposed pattern application for AgriSoft.

Designpattern 14

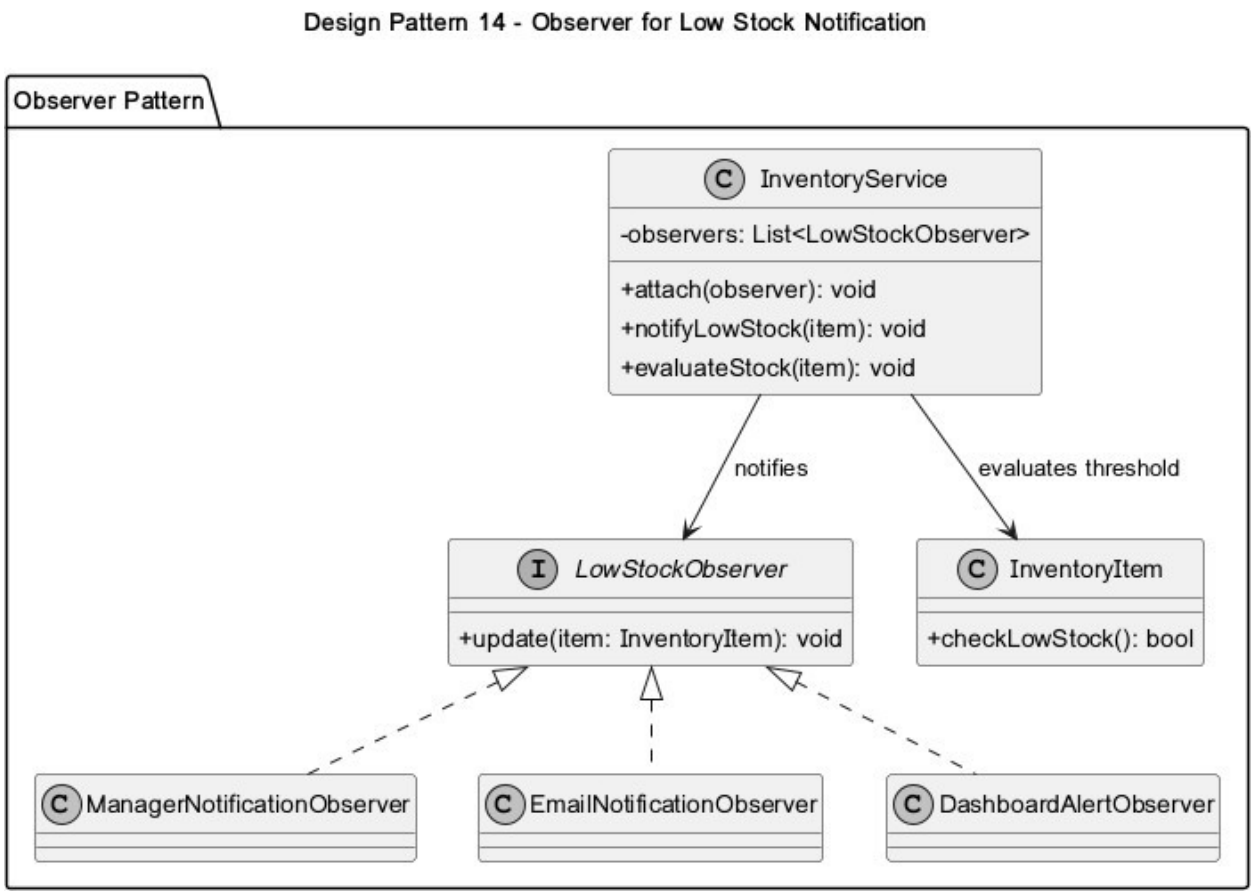


Figure: Designpattern 14

This design pattern diagram is included from the project package and documents a proposed pattern application for AgriSoft.

Designpattern 16

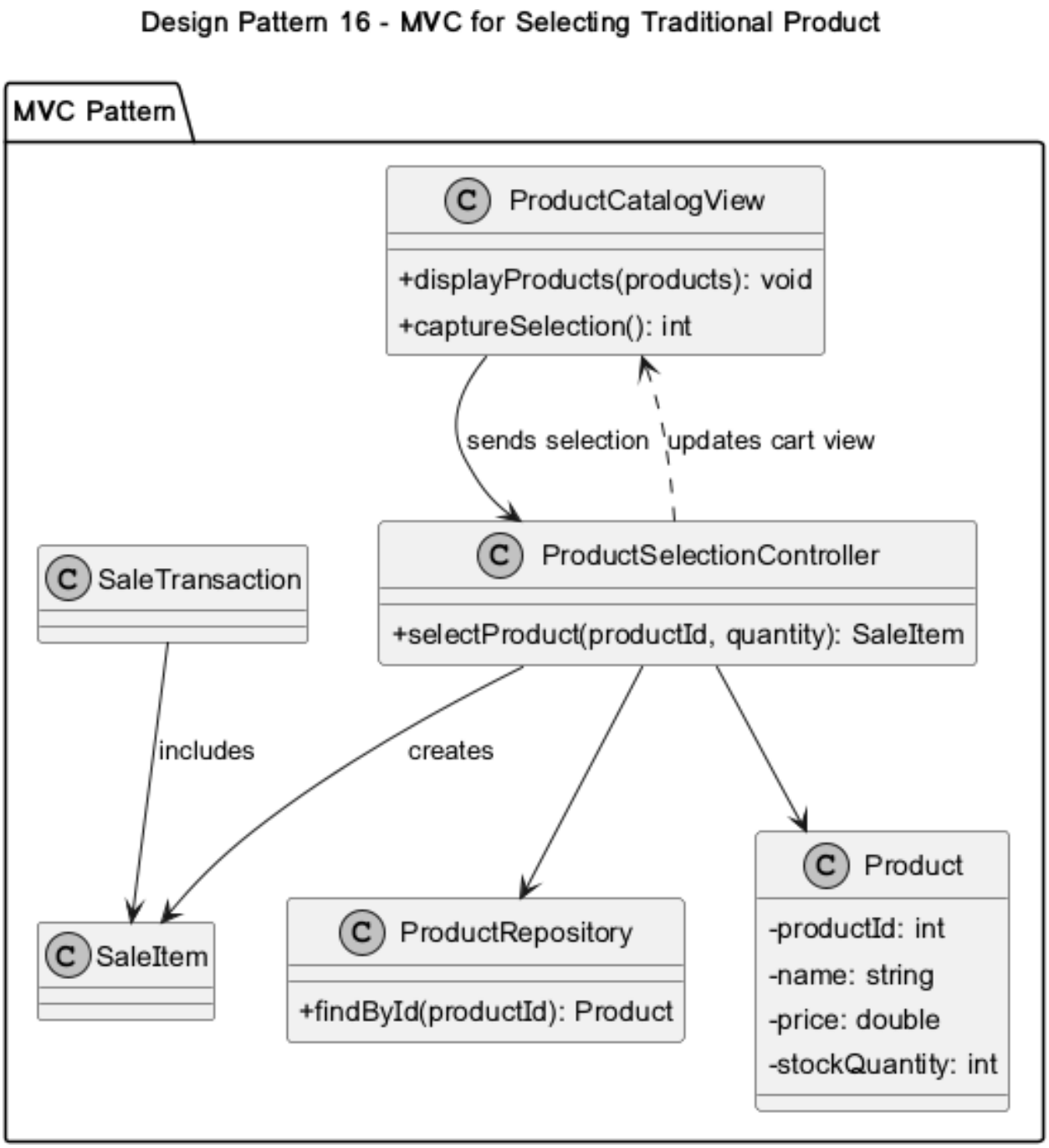


Figure: Designpattern 16

This design pattern diagram is included from the project package and documents a proposed pattern application for AgriSoft.

Designpattern 20

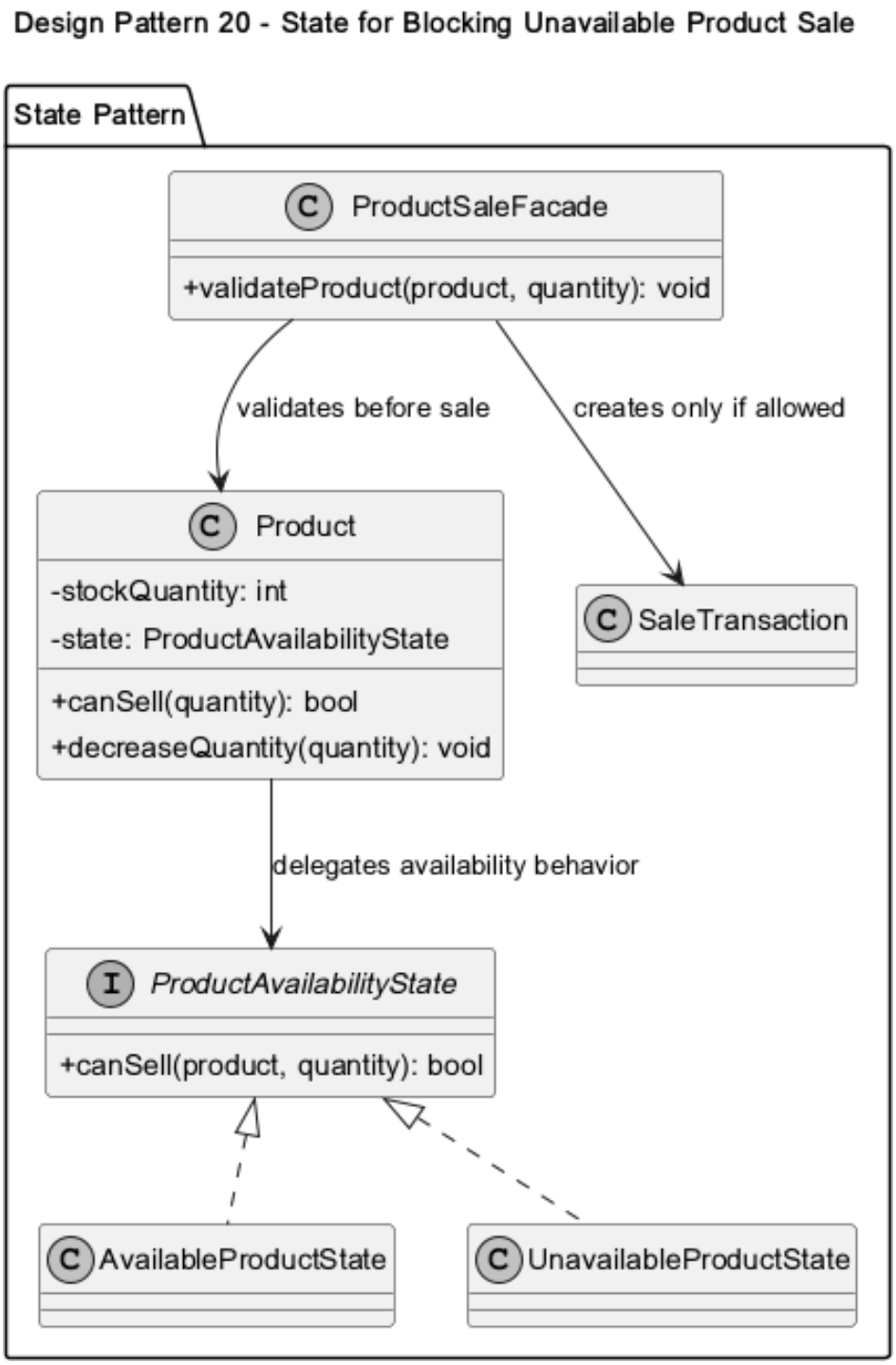


Figure: Designpattern 20

This design pattern diagram is included from the project package and documents a proposed pattern application for AgriSoft.

Designpattern 22

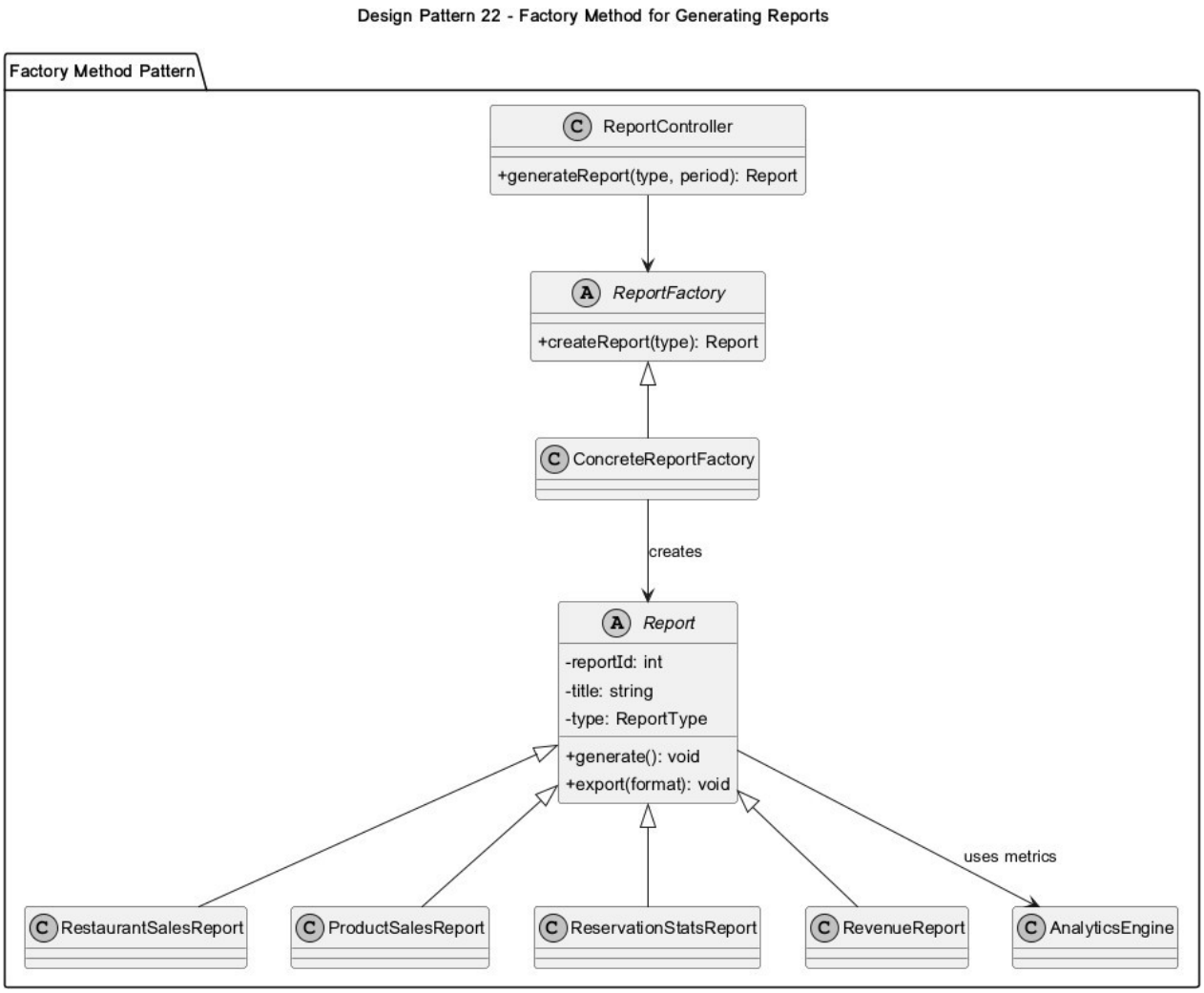


Figure: Designpattern 22

This design pattern diagram is included from the project package and documents a proposed pattern application for AgriSoft.

Designpattern 23

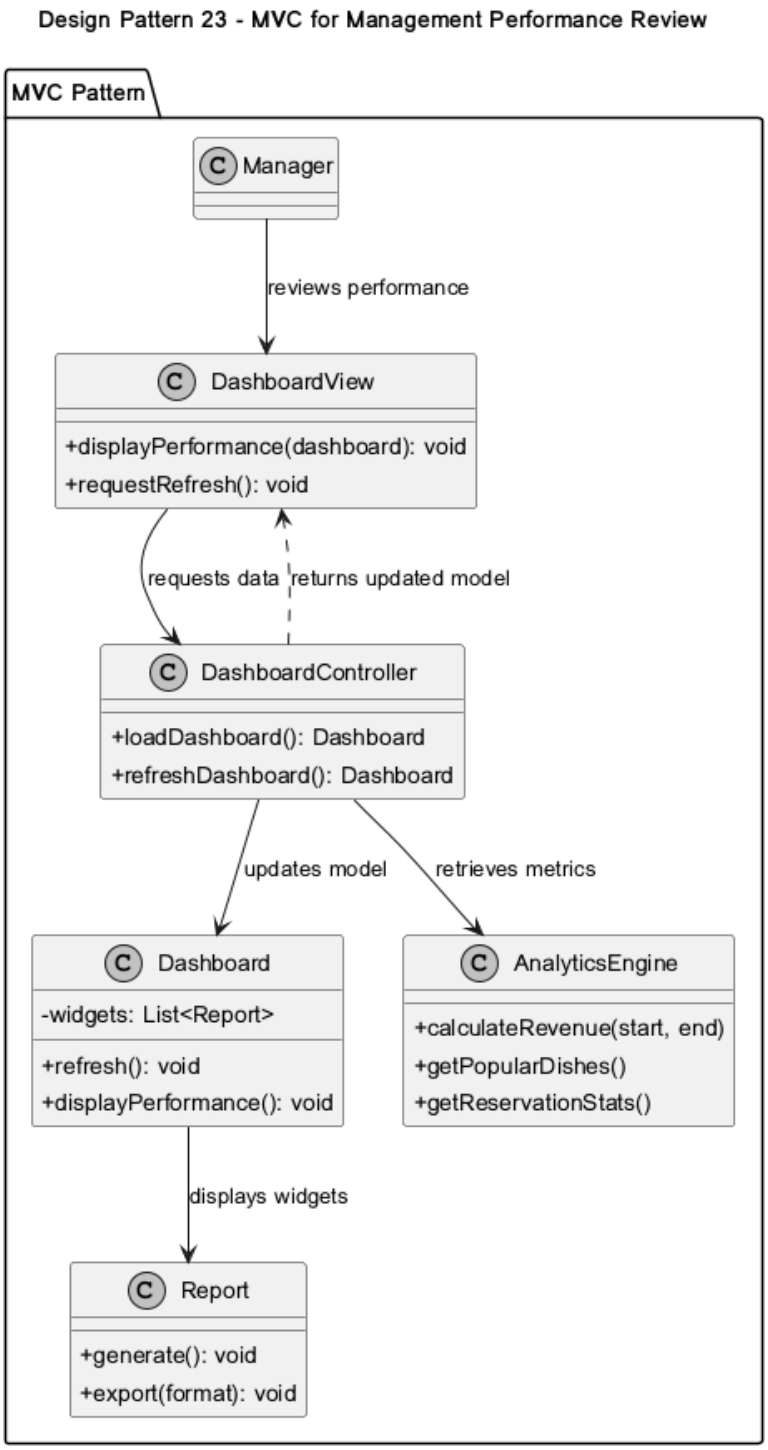


Figure: Designpattern 23

This design pattern diagram is included from the project package and documents a proposed pattern application for AgriSoft.

6 High Fidelity Prototype

The high fidelity prototype represents the visual and interaction design of AgriSoft. The provided screenshots show a polished frontend for agritourism services, including public pages, reservation workflows, dashboard modules, room views, restaurant views, reports, and contact information.

About Us

Figure: About Us

This prototype screen shows the visual design and user interaction flow for AgriSoft.

Contact & Location

Figure: Contact & Location

This prototype screen shows the visual design and user interaction flow for AgriSoft.

Dashboard

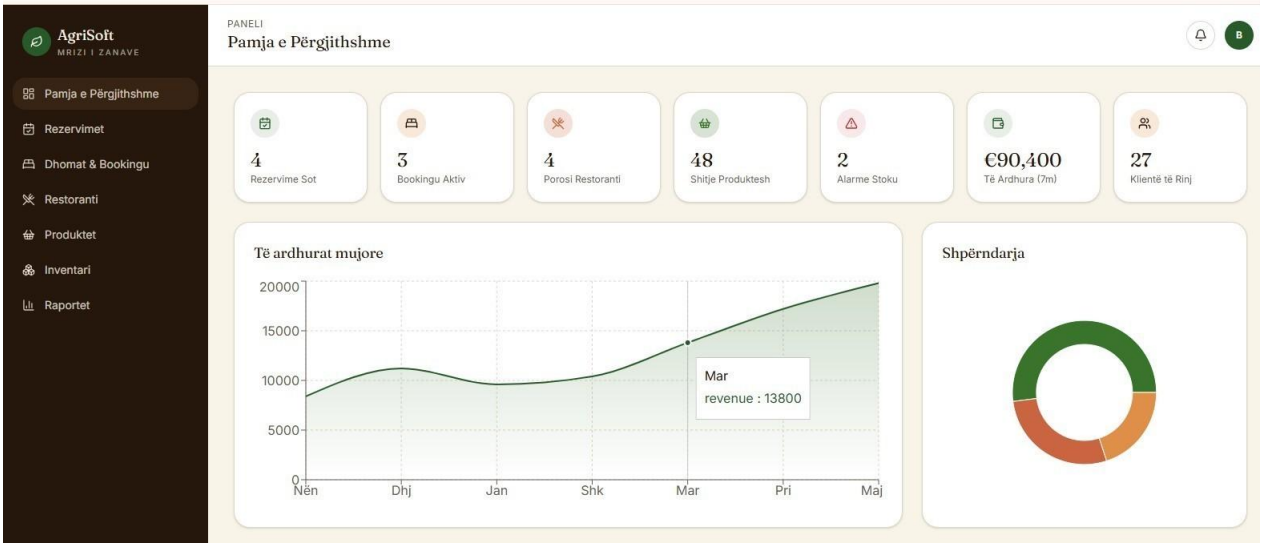


Figure: Dashboard

This prototype screen shows the visual design and user interaction flow for AgriSoft.

Raports

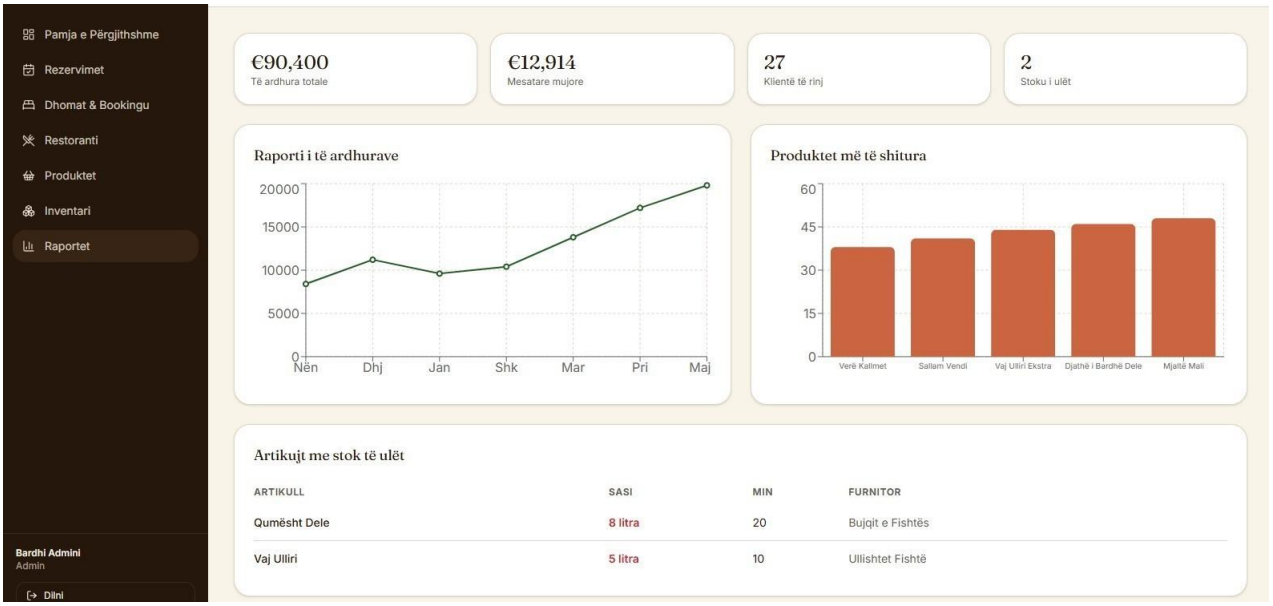


Figure: Raports

This prototype screen shows the visual design and user interaction flow for AgriSoft.

Reservation

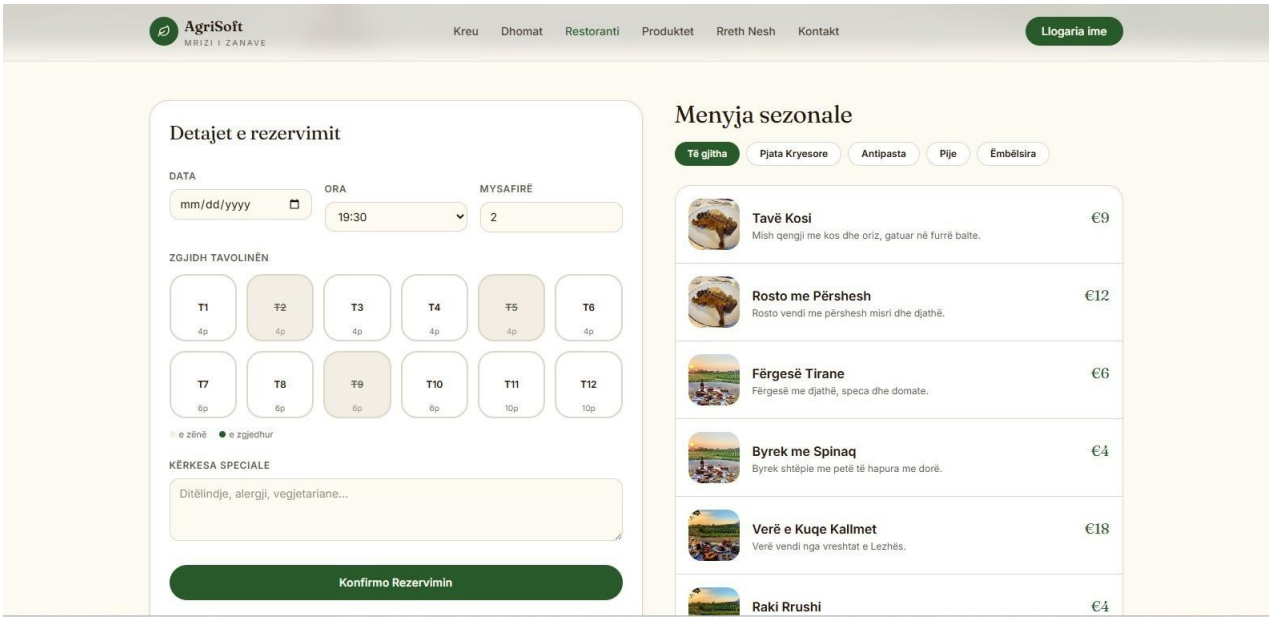


Figure: Reservation

This prototype screen shows the visual design and user interaction flow for AgriSoft.

Reservations

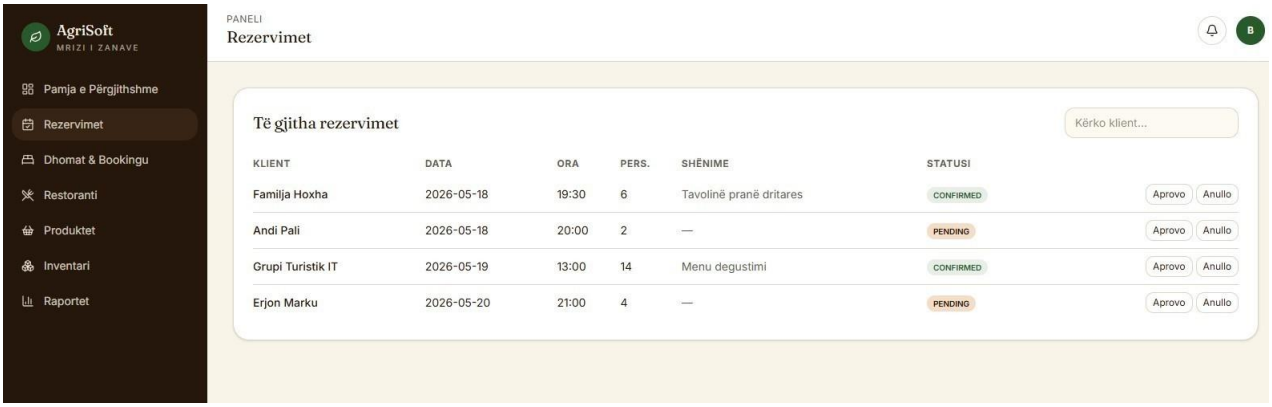


Figure: Reservations

This prototype screen shows the visual design and user interaction flow for AgriSoft.

Restaurant

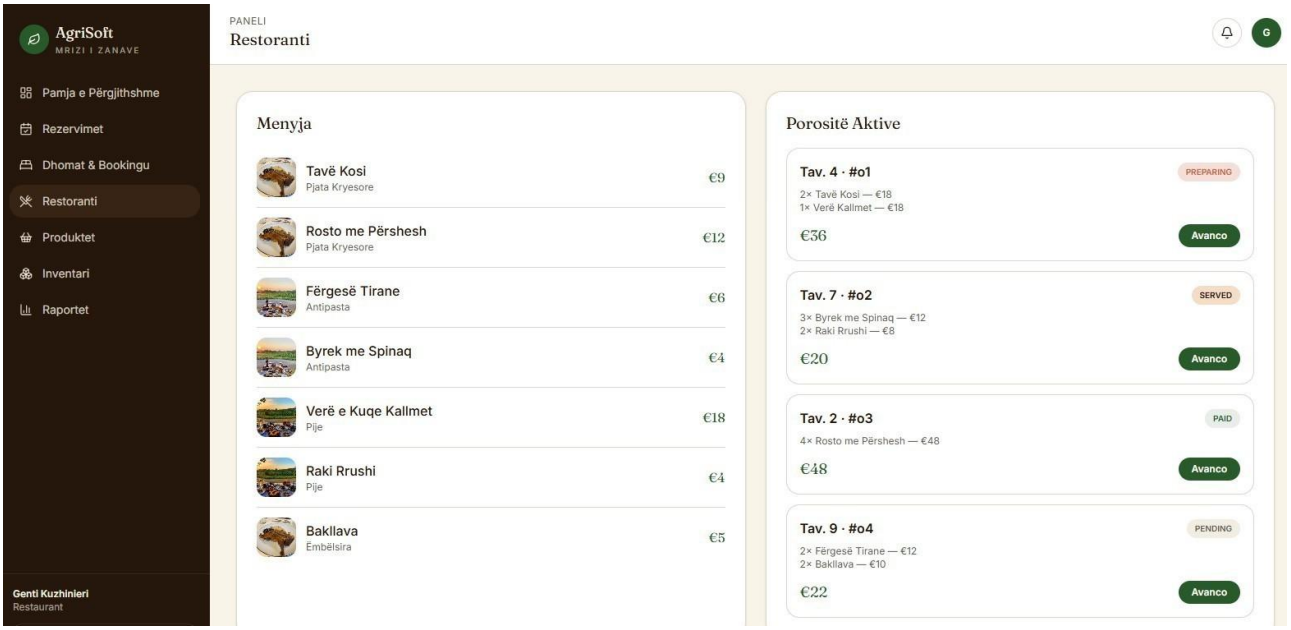


Figure: Restaurant

This prototype screen shows the visual design and user interaction flow for AgriSoft.

Rooms

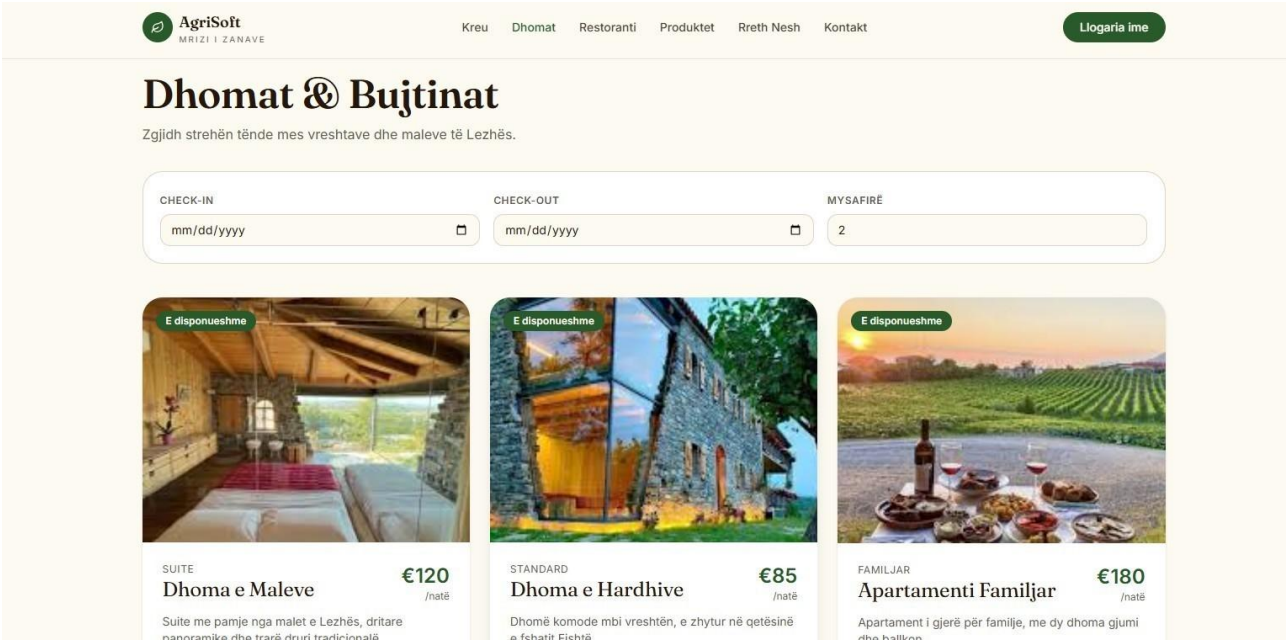


Figure: Rooms

This prototype screen shows the visual design and user interaction flow for AgriSoft.

Welcome Back

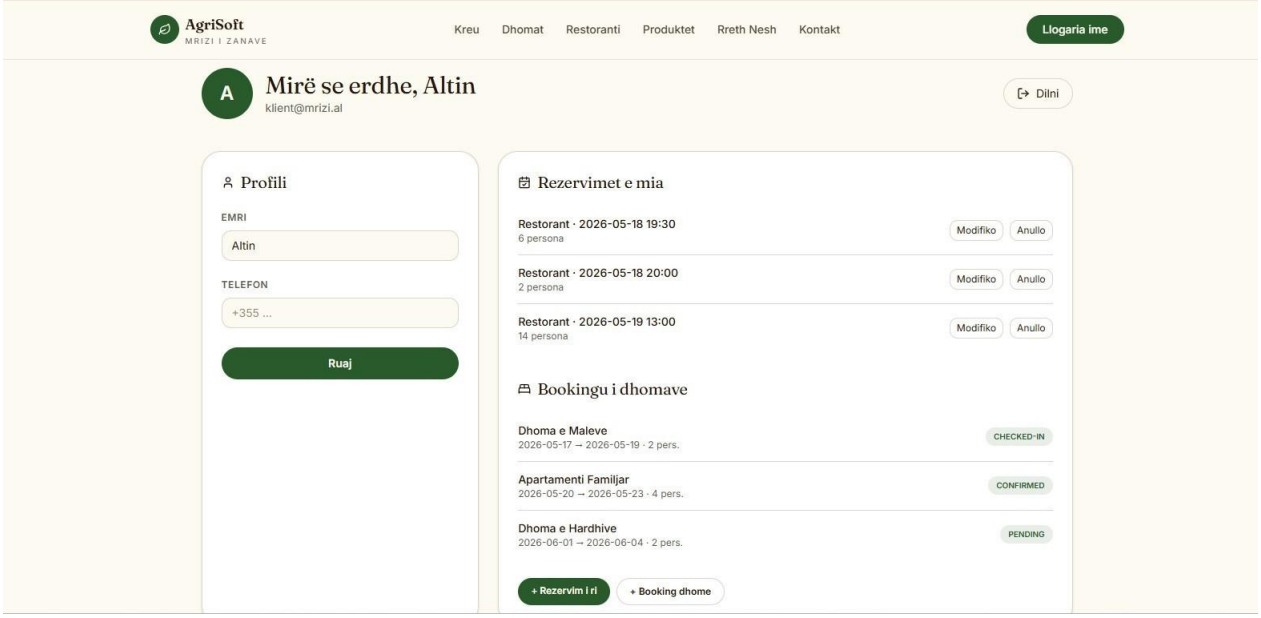


Figure: Welcome Back

This prototype screen shows the visual design and user interaction flow for AgriSoft.

6.7 UI/UX Evaluation

The interface follows a clean and modern design style with strong visual identity. The navigation separates public pages from dashboard functionality. The dashboard screens provide operational focus for managers and staff, while public pages communicate the agritourism experience to visitors. Future improvement should include accessibility testing, responsive testing across devices, and production usability testing with real staff users.

7 Testing

7.1 Testing Strategy

Testing verifies that AgriSoft satisfies its requirements and provides reliable user interaction. Since the current project is a frontend prototype, testing focuses on UI navigation, form validation, mock data behavior, responsive layout, and proposed integration behavior.

7.2 Test Cases

Table 8: Core Test Cases

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-01	Open homepage	Application available	Navigate to /	Homepage loads with navigation and hero content	Passed/To Verify
TC-02	View restaurant page	Application available	Open restaurant page	Restaurant information is displayed	Passed/To Verify
TC-03	View rooms page	Application available	Open rooms page	Room information and visuals are displayed	Passed/To Verify
TC-04	Create reservation	Reservation form visible	Enter reservation data and submit	System accepts valid reservation data or shows confirmation	To Verify
TC-05	Prevent invalid reservation	Reservation form visible	Submit with missing data	System shows validation errors	To Verify
TC-06	Open dashboard	User is authenticated or mock access enabled	Navigate to dashboard	Dashboard layout is displayed	To Verify
TC-07	Inventory view	Dashboard available	Open inventory module	Stock information is visible	To Verify
TC-08	Reports view	Dashboard available	Open reports module	Charts/tables are displayed	To Verify
TC-09	Login form validation	Login page available	Submit invalid credentials	System displays feedback	To Verify
TC-10	Responsive navigation	Browser available	Resize screen	Navigation remains usable	To Verify

Test Case 11

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-11	Reporting	Relevant page is available	Execute a representative reporting workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 12

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-12	Authentication	Relevant page is available	Execute a representative authentication workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 13

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-13	Navigation	Relevant page is available	Execute a representative navigation workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 14

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-14	Reservation	Relevant page is available	Execute a representative reservation workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 15

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-15	Restaurant Order	Relevant page is available	Execute a representative restaurant order workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 16

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-16	Product Sale	Relevant page is available	Execute a representative product sale workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 17

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-17	Inventory	Relevant page is available	Execute a representative inventory workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 18

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-18	Reporting	Relevant page is available	Execute a representative reporting workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 19

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-19	Authentication	Relevant page is available	Execute a representative authentication workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 20

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-20	Navigation	Relevant page is available	Execute a representative navigation workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 21

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-21	Reservation	Relevant page is available	Execute a representative reservation workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 22

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-22	Restaurant Order	Relevant page is available	Execute a representative restaurant order workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 23

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-23	Product Sale	Relevant page is available	Execute a representative product sale workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 24

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-24	Inventory	Relevant page is available	Execute a representative inventory workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 25

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-25	Reporting	Relevant page is available	Execute a representative reporting workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 26

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-26	Authentication	Relevant page is	Execute a	System performs	To Verify

		available	representative authentication workflow and observe feedback	expected action without visual or validation errors	

Test Case 27

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-27	Navigation	Relevant page is available	Execute a representative navigation workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 28

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-28	Reservation	Relevant page is available	Execute a representative reservation workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 29

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-29	Restaurant Order	Relevant page is available	Execute a representative restaurant order workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 30

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-30	Product Sale	Relevant page is available	Execute a representative product sale workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 31

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-31	Inventory	Relevant page is available	Execute a representative inventory workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 32

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-32	Reporting	Relevant page is available	Execute a representative reporting workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 33

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-33	Authentication	Relevant page is available	Execute a representative authentication workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 34

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-34	Navigation	Relevant page is available	Execute a representative navigation workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 35

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-35	Reservation	Relevant page is available	Execute a representative reservation workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 36

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-36	Restaurant Order	Relevant page is available	Execute a representative restaurant order workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 37

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-37	Product Sale	Relevant page is available	Execute a representative product sale workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 38

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-38	Inventory	Relevant page is available	Execute a representative inventory workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 39

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-39	Reporting	Relevant page is available	Execute a representative reporting workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

Test Case 40

Test ID	Feature	Preconditions	Steps	Expected Result	Status
TC-40	Authentication	Relevant page is available	Execute a representative authentication workflow and observe feedback	System performs expected action without visual or validation errors	To Verify

7.3 Test Summary

The test cases cover the main functional modules and the most important non-functional concerns: usability, validation, navigation, responsiveness, and reliability of visible workflows. Integration and database tests should be added after backend implementation.

8 Project Management

8.1 Team Roles

Table 9: Team Roles

Team Member	Role	Main Responsibilities
Bajram Haxhiu	Software Analysis and Documentation	Requirements, diagrams, design documentation, system analysis
Megiljana Hidri	UI/UX and Frontend Support	Prototype review, UI documentation, scenarios, screenshots
Ergi Adhami	Architecture and Testing Support	System architecture, testing, diagrams, technical validation

8.2 Development Methodology

The project follows an iterative academic development process. Requirements are identified first, then transformed into user stories, scenarios, use cases, diagrams, prototype screens, and testing tables. This approach is suitable for Software Analysis and Design because it demonstrates how a software idea becomes a structured model before full production implementation.

8.3 Timeline

Table 10: Project Timeline

Phase	Activities	Output
Planning	Define project domain and objectives	Project idea and scope
Requirements Analysis	Identify functional and non-functional requirements	Requirement tables
Design	Create UML, ERD, DFD, BPMN, and design pattern diagrams	Design model
Implementation Prototype	Develop React/TanStack frontend screens	Working prototype
Testing	Review UI and module workflows	Test cases
Documentation	Prepare final report and GitHub package	Final DOCX/PDF

8.4 GitHub Workflow

The GitHub repository is used to store source code, diagrams, requirements, screenshots, and final documentation. A structured repository helps evaluators verify that the project contains not only implementation files but also Software Analysis and Design deliverables.

8.5 Challenges and Solutions

Table 11: Challenges and Solutions

Challenge	Solution
Connecting prototype to complete documentation	Use requirements and diagrams to bridge implemented screens with proposed architecture
Large number of diagrams	Organize diagrams by type and scenario
Backend not fully implemented	Clearly mark backend/database as proposed future improvements
Maintaining academic quality	Use structured sections, tables, and UML explanations
Keeping document editable	Produce final report in DOCX format with editable text and replaceable images

9 Future Improvements

9.1 Backend Development

A production backend should be implemented to handle API requests, authentication, reservation logic, sales records, inventory updates, and reporting aggregation.

9.2 Database Integration

A relational database such as PostgreSQL or MySQL should store customers, reservations, rooms, products, orders, inventory items, employees, and reports.

9.3 Authentication and Authorization

Role-based access control should separate customer, staff, manager, and administrator permissions. Password security, session management, and secure account recovery should be included.

9.4 Notification System

Email or in-app notifications can alert users about reservation confirmations, low stock, cancelled bookings, and report generation.

9.5 Cloud Deployment

The system can be deployed to a cloud environment with HTTPS, backups, monitoring, and production logging.

Table 12: Future Improvements

Improvement	Priority	Reason
Production backend	High	Required for real transactions and persistence
Database implementation	High	Required for data integrity and reporting
Authentication/RBAC	High	Required for secure staff and manager access
Notifications	Medium	Improves customer and staff communication
Analytics dashboard	Medium	Improves management decision-making
Mobile optimization	Medium	Improves accessibility for staff and customers
Multilingual interface	Low/Medium	Useful for tourism customers

10 Conclusion

AgriSoft demonstrates how Software Analysis and Design principles can be applied to a realistic agritourism management problem. The project transforms the business context of Mrizi i Zanave into a structured system model with requirements, stakeholders, user scenarios, use cases, UML diagrams, data models, design patterns, prototype screens, and testing strategy.

The current implementation provides a modern React/TanStack frontend prototype with strong visual design and module-oriented dashboard pages. The documentation expands this prototype into a complete software specification by defining proposed backend, database, deployment, and security improvements. This separation between implemented features and proposed future work keeps the documentation academically honest and technically useful.

From a course perspective, the project shows the value of requirements engineering, diagram-based reasoning, architectural thinking, and iterative design. The final documentation can be used both as a submission artifact and as a foundation for future development of AgriSoft into a full production-ready agritourism management system.

11 References

116. AgriSoft project ZIP package and GitHub source code materials provided by the project team.
117. UML modeling concepts used for use case, activity, sequence, state, class, object, component, package, composite structure, and deployment diagrams.
118. Software Analysis and Design course materials and academic documentation standards.
119. React, TypeScript, TanStack Router/Start, Vite, TailwindCSS, shadcn/Radix UI, Recharts, and related project technology documentation.
120. Mrizi i Zanave agritourism context as described in the project README.

12 Appendices

12.1 Source Code Notes

The following table summarizes important source code folders and files found in the AgriSoft frontend project.

Table 13: Source Code Structure

Folder/File	Purpose
.gitignore	Project file
.lovable/project.json	Project file
.prettierignore	Project file
.prettierrc	Project file
README.md	Project file
bun.lock	Project file
bunfig.toml	Project file
components.json	Project file
eslint.config.js	Project file
package.json	Dependency and script configuration
src/assets/dish.jpg	Image or visual asset
src/assets/exterior.jpg	Image or visual asset
src/assets/feast.jpg	Image or visual asset
src/assets/room.jpg	Image or visual asset
src/assets/sign.jpg	Image or visual asset
src/assets/staff.jpg	Image or visual asset
src/assets/table-sunset.jpg	Image or visual asset
src/components/site/Footer.tsx	Site layout/navigation component
src/components/site/Navbar.tsx	Site layout/navigation component
src/components/ui/accordion.tsx	Reusable UI component
src/components/ui/alert-dialog.tsx	Reusable UI component
src/components/ui/alert.tsx	Reusable UI component
src/components/ui/aspect-ratio.tsx	Reusable UI component
src/components/ui/avatar.tsx	Reusable UI component
src/components/ui/badge.tsx	Reusable UI component
src/components/ui/breadcrumb.tsx	Reusable UI component
src/components/ui/button.tsx	Reusable UI component
src/components/ui/calendar.tsx	Reusable UI component
src/components/ui/card.tsx	Reusable UI component
src/components/ui/carousel.tsx	Reusable UI component
src/components/ui/chart.tsx	Reusable UI component
src/components/ui/checkbox.tsx	Reusable UI component
src/components/ui/collapsible.tsx	Reusable UI component
src/components/ui/command.tsx	Reusable UI component
src/components/ui/context-menu.tsx	Reusable UI component
src/components/ui/dialog.tsx	Reusable UI component
src/components/ui/drawer.tsx	Reusable UI component
src/components/ui/dropdown-menu.tsx	Reusable UI component
src/components/ui/form.tsx	Reusable UI component
src/components/ui/hover-card.tsx	Reusable UI component
src/components/ui/input-otp.tsx	Reusable UI component
src/components/ui/input.tsx	Reusable UI component
src/components/ui/label.tsx	Reusable UI component
src/components/ui/menubar.tsx	Reusable UI component
src/components/ui/navigation-menu.tsx	Reusable UI component
src/components/ui/pagination.tsx	Reusable UI component
src/components/ui/popover.tsx	Reusable UI component
src/components/ui/progress.tsx	Reusable UI component
src/components/ui/radio-group.tsx	Reusable UI component
src/components/ui/resizable.tsx	Reusable UI component
src/components/ui/scroll-area.tsx	Reusable UI component
src/components/ui/select.tsx	Reusable UI component
src/components/ui/separator.tsx	Reusable UI component
src/components/ui/sheet.tsx	Reusable UI component
src/components/ui/sidebar.tsx	Reusable UI component
src/components/ui/skeleton.tsx	Reusable UI component
src/components/ui/slider.tsx	Reusable UI component
src/components/ui/sonner.tsx	Reusable UI component
src/components/ui/switch.tsx	Reusable UI component
src/components/ui/table.tsx	Reusable UI component
src/components/ui/tabs.tsx	Reusable UI component
src/components/ui/textarea.tsx	Reusable UI component
src/components/ui/toggle-group.tsx	Reusable UI component
src/components/ui/toggle.tsx	Reusable UI component
src/components/ui/tooltip.tsx	Reusable UI component
src/hooks/use-mobile.tsx	Project file

src/lib/auth.tsx	Utility, auth, mock data, or application support file
src/lib/error-capture.ts	Utility, auth, mock data, or application support file
src/lib/error-page.ts	Utility, auth, mock data, or application support file
src/lib/mock-data.ts	Utility, auth, mock data, or application support file
src/lib/utils.ts	Utility, auth, mock data, or application support file
src/routeTree.gen.ts	Project file
src/router.tsx	Project file
src/routes/__root.tsx	Application route/page component
src/routes/about.tsx	Application route/page component
src/routes/account.tsx	Application route/page component
src/routes/contact.tsx	Application route/page component
src/routes/dashboard.index.tsx	Application route/page component
src/routes/dashboard.inventory.tsx	Application route/page component
src/routes/dashboard.products.tsx	Application route/page component

12.2 Technology Stack Notes

Table 14: Technology Stack

Technology	Role in AgriSoft
React	Frontend user interface library
TypeScript	Typed JavaScript for safer development
TanStack Router/Start	Routing and application structure
Vite	Development and build tooling
TailwindCSS	Utility-based styling
Radix UI / shadcn style components	Accessible reusable UI components
Recharts	Dashboard charting and reporting
Zod / React Hook Form	Validation and form handling
Mock Data	Temporary data source before backend integration

12.3 Additional Notes

All diagrams and screenshots in this document are included as images, so they can be moved, replaced, resized, or updated directly in Microsoft Word. The text content is editable, allowing the team to add professor feedback, update page numbers, or expand sections as needed.